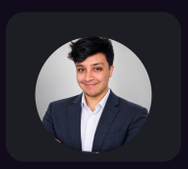
Trust at scale.

Engineering that turns complexity into trust.

A contribution proposal for Nubank. From the app experience to architecture, delivery and leadership decisions.

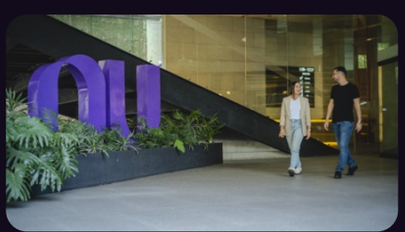
320	20	86	216
PAGES	CASES	CODE BLOCKS	PASSING TESTS



Wesley Vieira

Engineering Manager

Staff Software Engineer



CONTENTS

One idea. Twenty contexts.

Select a chapter or use the footer menu. Bookmarks reveal every subtopic.

Part 1 / 2 | Part 2 / 2

Wesley Vieira · a proposal for Nubank 4-17

Experience, evidence, and the contribution I want to make

Foundations and technologies 18-34

Code, contracts, and the limits of each choice

Architectures connecting the cases 35-38

Boundaries, authority, and recovery

Case 01 / When Pix fails, help fails too 39-51

Idempotency, failure isolation, and outcome recovery

Case 02 / Recovering access when the front camera does not work 52-64

Assisted recovery, decision validity, and digital inclusion

Case 03 / A notification should help someone decide 65-77

Communication contracts, late privacy enforcement, and usefulness

Case 04 / Open Finance must show what is still incomplete 78-90

Provenance, consent, and completeness contracts

Case 05 / The next due date, without searching across months 91-103

Civil-date scheduling, preferences, and job revalidation

Case 06 / Modo Rua: explain permissions without weakening protection 104-116

Contextual policy, permissions, and trust boundaries

Case 07 / One cent can break an entire integration 117-129

Monetary serialization, identity, and statement compatibility

Case 08 / The digital experience also ends with card delivery 130-142

From public evidence to a verifiable contract

Case 09 / Evolve the statement without losing the primary task 143-155

From public evidence to a verifiable contract

Case 10 / A purchase record in context, with the right meaning 156-168

From public evidence to a verifiable contract

CONTENTS

From decisions to delivery.

Select a chapter or use the footer menu. Bookmarks reveal every subtopic.

Part 1 / 2 | Part 2 / 2

Case 11 / Fast payments without trading correctness for speed 169-181
From public evidence to a verifiable contract

Case 12 / Cloud capacity as a product commitment 182-194
From public evidence to a verifiable contract

Case 13 / Ownership that survives reorganization 195-207
From public evidence to a verifiable contract

Case 14 / BDC: speed with an experience contract 208-220
From public evidence to a verifiable contract

Case 15 / Mobile releases guided by the actual customer journey 221-233
App · Observability

Case 16 / Precog: reaching the right specialist 234-246
AI · Customer support

Case 17 / Fraud prevention that responds within the journey 247-259
Security · Machine Learning

Case 18 / The same feature in training and at decision time 260-272
Data · ML quality

Case 19 / Dark Mode: consistency that includes readability 273-285
App · Design System

Case 20 / NuBraille: end-to-end autonomy 286-298
Accessibility · Journey

Leadership, value, and execution 299-313
How to turn hypotheses into decisions and outcomes

Sources and credits 314-320
Public references and visual provenance

OPENING

The work I want to build.

Photograph published by Nubank: a workplace reflecting the company's identity.

I want to contribute to a Nubank where simplicity of experience is supported by technical rigor. My proposal starts with concrete journeys: paying without ambiguity, recovering access securely, understanding a decision, and getting help when an operation leaves the expected path. Each requires product, architecture, and organization to work as one system.

This book makes my candidacy open to examination. It contains twenty studies of public signals, accompanied by code, contracts, architecture diagrams, tests, and execution decisions. Its depth lies in the boundaries: where a guarantee ends, who can decide, how to recover from failure, and what evidence justifies expanding a change.

Wesley Vieira

Application for Engineering Manager or Staff Software Engineer. Independent study; not an official Nubank publication.

Nubank: About Nu

COVER LETTER

Dear Nubank team,

Editorial photograph published by Nubank: professionals in a meeting room.

I want to join Nubank because I am interested in the work between a simple experience and the engineering required to sustain it. For someone using the app, a Pix payment, a dispute, or access recovery is more than a flow. It involves money, time, and trust. I want to help make those journeys understandable when everything works and keep offering a safe next step when something goes wrong.

Today at Itaú, I am responsible for the technical direction of two squads, with more than 15 engineers and more than 20 fraud-prevention systems and services. My background also includes engineering leadership in credit at Bradesco and technical product management at Itaú. These experiences have taught me to bring architecture, business context, and people development together while keeping sight of those who depend on the system.

My practical foundation includes Java/JVM, AWS, messaging, systems integration, and legacy modernization. I also work with data, security, and applied AI. Beyond bringing a list of technologies, I want to contribute the ability to frame problems, discuss alternatives, and build solutions that fit the team's context and operational capacity.

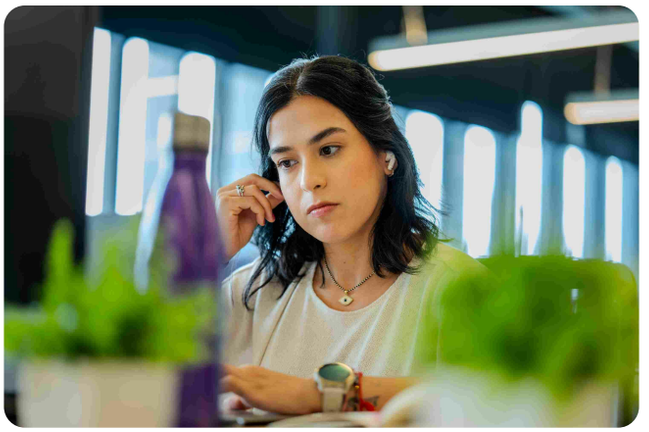
My proposal

Combine depth in financial systems, people development, and accountability for journey outcomes.

Wesley Vieira's portfolio
Nubank: editorial photograph

COVER LETTER

A technical conversation before the interview.



Editorial photograph published by Nubank: a workplace. The image does not depict the candidate.

I prepared this material to make that intention open to examination. I used public information about Nubank and customer reports to organize twenty studies. Each distinguishes what was observed, what remains a hypothesis, and what I would propose validating. I do not know the internal backlog and expect to find existing controls, different priorities, and good reasons to revise my initial ideas.

As an Engineering Manager, I want to help people grow, take ownership, and deliver with clarity. As a Staff Software Engineer, I want to improve technical decisions that cross team boundaries by making contracts, risks, and paths for evolution more explicit. In both roles, my commitment is to collaboration, quality, and results.

This book is a sample of how I would begin a conversation: listen to the pain, make the decision concrete, test the hypothesis, and learn from the result. I would like to build the next version together with the Nubank team.

I do not start from the assumption that Nubank needs to rebuild what it already does well. I start with a willingness to learn from its published capabilities, formulate a useful hypothesis, and test it with the team. The relevant judgment is not how many technologies fit into a diagram; it is how much uncertainty we can remove from the next decision.

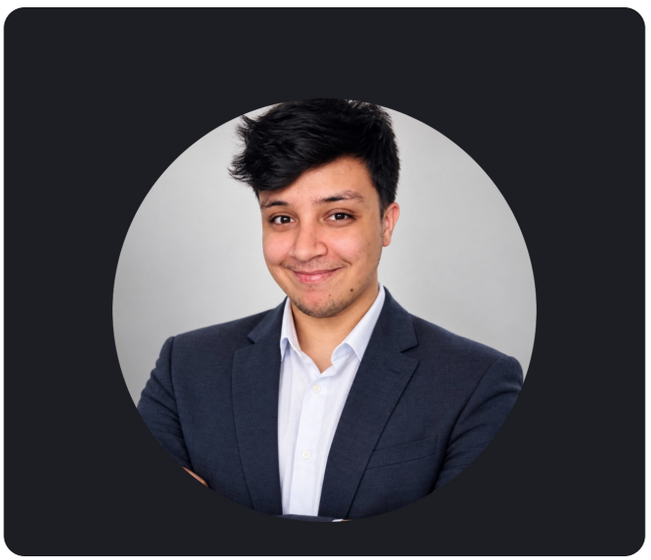
How to read my application

Career evidence supports my experience. The labs demonstrate executable reasoning. The cases show how that experience could serve Nubank's context.

Wesley Vieira's portfolio
Nubank: editorial photograph

INTRODUCTION

I am Wesley Vieira.



Professional photograph available in my portfolio.

I am a Lead Software Engineer working across financial services, fraud prevention, credit, and technical product. Today at Itaú, I lead the technical direction of two squads, with more than fifteen engineers and more than twenty systems and services. My background includes engineering leadership at Bradesco and technical product management at Itaú.

My practical foundation combines Java/JVM, AWS, messaging, systems integration, and legacy modernization. I use that experience to connect software behavior to business needs: what must remain correct, which risks we accept, and how the people operating the solution will recognize a failure.

What I bring to the conversation

Experience with critical financial journeys, the ability to work across team boundaries, and an interest in developing people who make better decisions autonomously.

Background and portfolio

EXPERIENCE

A career with clearly defined scope.

Period / context	Responsibility	Relevant contribution
Itaú · since Jun 2026	Lead Software Engineer	Technical direction of 2 squads and 20+ fraud-prevention systems.
Bradesco · Feb 2025-May 2026	Lead Software Engineer	Architecture and delivery across 3 credit and channel squads.
Itaú · Nov 2023-Oct 2024	Senior Technical Product Manager	Multidisciplinary leadership and protection priorities.

These scopes complement one another. In fraud prevention, technical decisions affect losses and friction. In credit, policies, channels, and integrations must remain coherent through change. In technical product, the challenge is organizing work so that different teams understand the same outcome and can negotiate trade-offs.

Technical direction and people management are distinct responsibilities. The fifteen or more engineers in my current scope are not presented as direct reports. In my previous technical product role, my profile describes leadership of multidisciplinary squads, discovery, priorities, and stakeholder alignment, without a confirmed direct-report count. That precision is part of how I want to lead: clear expectations, explicit responsibility, and evidence that supports the claim.

Modernization

My portfolio describes the evolution of a legacy environment with approximately 1,900 files. The number represents scope; it does not imply a completed migration or audited savings.

Candidate-reported background

CAREER EVIDENCE

Recognition is a collective achievement.

Record of the 21st Walther Moreira Salles Award, available in the candidate's portfolio.

My portfolio records recognition at the 21st Walther Moreira Salles Award in December 2023, associated with the WhatsApp fraud-alert project. The photograph shows a team achievement. For a leadership application, the most relevant aspect is the ability to bring technology, communication, and customer protection together around a real problem.

What I bring from that experience is not a recipe for copying a channel. It is the discipline of asking whether a message arrives in time, whether the person can recognize a legitimate intention, whether the response informs the risk decision, and whether recovery is available when automation does not resolve the issue. Communication also has contracts, observability, and operational ownership.

Evidence and scope

The image documents the recognition presented in the portfolio. On its own, it does not substantiate the financial figures reported by the candidate.

Photographs, recognition, and projects

IMPACT HISTORY

Impact with honest attribution.

Reported outcome	Nature	How to discuss it in the interview
R$ 350 million	Fraud mitigation associated with WhatsApp alerts	Clarify the period, baseline, team contribution, and calculation.
R$ 300 million	Loss mitigation in an ML initiative	Distinguish the counterfactual, prevention, and recovery.
R$ 150 million+	Revenue associated with a credit initiative	Distinguish revenue, margin, and causality.

These figures come from my self-reported background. They are not Nubank results, were not audited in this work, and must not be added together as if they represented a single saving. Initiatives may share periods, populations, or mechanisms of contribution. A good hiring conversation needs to explore the role played, the decision made, and the measurement method.

I would present each project with its prior context, my personal responsibility, contributions from other areas, rejected alternatives, and available evidence. If a measure came from the business, I would explain who produced it and what it does not cover. If a result depended on rollout or a policy change, I would show the sequence rather than attribute everything to software.

Working principle

A strong number should come with an equally strong explanation. The rigor I apply to my application is the same rigor I propose for product decisions.

Self-reported portfolio outcomes

ENGINEERING MANAGER

Why hire me as an EM.

Nubank editorial photograph showing collaboration between professionals.

As an Engineering Manager, my contribution is to create an environment where people understand the problem, have autonomy proportionate to risk, and can complete quality work. This requires more than tracking delivery: capacity must be visible, overload recognized, technical leadership developed, and dependencies resolved before they become permanent delays.

My experience in technical product and engineering lets me connect business objectives to implementation decisions without making myself the center of everything. I want to work with strong tech leads, establish expectations by level, offer specific feedback, and reserve space for learning. Management cadence should help the team decide; if it only produces reports, it needs to change.

Responsibility	Evidence in this book	Outcome to pursue
People	Delegation, mentoring, and capacity	Greater autonomy with accountability.
Execution	Pilots, stop criteria, and dependencies	Less work started without completion.
Product	Journey and value metrics	Verifiable improvement for customers.

Engineering Manager: official job description

STAFF SOFTWARE ENGINEER

Why hire me as a Staff engineer.

As a Staff Software Engineer, I want to improve decisions that cross team boundaries: event semantics, data authority, transaction boundaries, migration strategy, and how a journey is observed. My role is to make a problem clear enough for others to act safely without needing me in every review.

My foundation in Java/JVM and financial systems transfers to the functional reasoning demonstrated in this book. At the same time, I do not present my study of Clojure or Dart as professional experience already acquired at Nubank. I show original code, executed tests, and integration questions so that the technical discussion has something concrete to examine.

Scope	Typical decision	Artifact I leave behind
Domain	Which state is authoritative?	Invariants and a state machine.
Across teams	Who changes the contract, and how?	ADR, version, and compatibility plan.
Operations	How do we detect and recover?	SLO, failure test, and procedure.

The official Staff role consulted mentions Clojure, Kafka, AWS, Datomic/DynamoDB, Prometheus, and Kubernetes. The chapters use this context as a starting point. Complementary tools appear where they help test a hypothesis; they are not presented as universal company requirements.

Staff Software Engineer: official job description

RESEARCH METHOD

How to recognize each type of evidence.

Label	What it means	What it does not mean
OBSERVED	An identified public source with a link and context.	A reproduced incident or estimated frequency.
IMPLEMENTED	A local lab with executed code and tests.	Integration with real Nubank systems.
PROPOSED	A contract, design, or decision developed in this book.	An approved roadmap or internal architecture.
MODELED	A synthetic scenario with explicit assumptions.	Revenue, savings, or an SLA measured at the company.

The first ten cases address public customer reports or reviews. They are investigation signals, not a representative sample of the entire customer base. Some reports come from the same author; app-store review cases share the same source page. The book does not convert diversity of problems into a statistical failure rate.

The next ten cases begin with Nubank publications on technology, product, and accessibility. Their starting point may be a successful capability. The proposal is to discuss how to reason about extension, operations, and limits. When a source reports an improvement, the figure remains attributed to the original publication and the context it describes.

Access date

October 23, 2026. Public pages, job descriptions, and reviews may change after this edition.

PRODUCT AND EVIDENCE

The app: public promise and experience.

Official promotional material for the Nubank app on Google Play.

The public promise of simplicity helps define the unit of analysis: a person trying to complete a financial task. In this book, the app is more than a presentation layer. It participates in states, recovery decisions, accessibility, risk communication, and the perception of time.

Official images show how the product presents itself; they are not screenshots of an account tested by the author. Customer screenshots, when used in cases, remain identified as material published by third parties. No conceptual screen is presented as evidence that a feature exists or is missing from the app.

Nubank on Google Play

READING MAP

Twenty questions, one discipline.

The first half of the book begins where experience becomes ambiguous: a Pix payment without confirmation, a camera that blocks recovery, an incomplete notification, an integration that appears to omit a balance. The second follows Nubank's public capabilities: performance, platforms, observability, support, models, and inclusion.

Each study moves from a signal to a hypothesis, draws boundaries, implements a core decision, and defines how to verify its behavior. The labs are deliberately smaller than production systems. This makes it possible to test a property without hiding the discussion in thousands of infrastructure lines. The chapter then makes explicit the additional work needed for integration, distributed concurrency, and operations.

Path	Cases	Focus
Trust in the journey	01-07	Payments, access, context, and data.
Operations and platform	08-14	Delivery, documents, performance, and contracts.
Intelligence and inclusion	15-20	Observability, support, ML, and accessibility.

The question running through the book

What evidence lets us claim that the experience improved without moving risk to another customer, team, or stage?

TECHNOLOGIES

Documented stack, justified choices.

clojure	kafka kafka	aws aws	Datomic datomic
prometheus	kubernetes	flutter	dart

The official Staff job description consulted highlights Clojure, Kafka, AWS, Datomic/DynamoDB, Prometheus, and Kubernetes. Nubank's technical publications also document Flutter and the importance of standardized choices. This context guides the examples' language and the structure of the proposals.

Java/JVM provides a bridge to my professional experience. Python supports reproducible labs with minimal setup. PostgreSQL, Redis, OpenTelemetry, Grafana, Terraform, and TypeScript appear as options or teaching tools, with their reasons and limits made explicit. A logo identifies the technology being discussed, not a claim of universal use at Nubank.

This is not a statistical ranking of the "most requested" technologies. It is alignment with identified official sources. In a real team, I would confirm versions, canonical libraries, internal platforms, and constraints before introducing components. Technical variety creates value only when it addresses a concrete need.

Stack in the official job description
Nubank: The value of canonicity

ADAPTATION PLAN

From my experience to the team's stack.

Starting experience	Application in context	Evidence I would seek
Java/JVM and AWS	Clojure services and cloud operations	Paired reviews of code and incidents.
Messaging and integration	Kafka contracts and consumers	Controlled replays and compatibility.
Fraud and credit	Authority, risk, and money	Boundary cases and policy review.
Product and management	Experiments and execution	Baseline, owner, and decision criterion.

Joining a new organization means learning how it makes decisions. I would start with one complete flow: from the gesture in the app to the service, event, persistence, and support. From there, I would identify what is standard, where exceptions exist, and which commitments were made to sustain current operations.

The functional lab included in this edition demonstrates that approach. It runs in Clojure and separates state transformation from external effects. This does not replace experience with the internal platform; it narrows the distance between saying I learn a language and showing an artifact that can be reviewed. The same standard applies to management: explain how I would learn from the team before promising reorganization.

The first delivery I would aim for

A small, observable, reversible improvement built with people who know the domain. The initial objective is to earn context and trust through the work.

FOUNDATIONS · CLOJURE

Clojure: decisions as data.

The functional core receives one state and returns another. An invalid transition produces an explicit error; it does not call a network, queue, or database. This separation allows the contract to be discussed before infrastructure is chosen. Within the model, versioning prevents an old intent from changing a newer state.

CLOJURE / ORIGINAL PROPOSAL

```clojure
(def transitions
  {:received #{:pending :rejected}
   :pending #{:settled :rejected}
   :settled #{}
   :rejected #{}})

(defn advance [db key expected-version next-status]
  (let [{:keys [status version] :as current} (get db key)]
    (when-not current
      (throw (ex-info "unknown operation" {:reason :not-found})))
    (when-not (= expected-version version)
      (throw (ex-info "stale writer" {:reason :version-conflict})))
    (when-not (contains? (get transitions status #{}) next-status)
      (throw (ex-info "illegal transition"
                      {:reason :illegal-transition})))
    (assoc db key (assoc current :status next-status
                         :version (inc version)))))
```

Excerpt from decision_core.clj; local execution verified. In-memory model without networking.

The excerpt is part of decision_core.clj, executed in Clojure 1.12.0 on Java 17. The complete file has 11 tests and 22 assertions. Value immutability provides neither durable persistence nor distributed exclusion: when integrating a database, version comparison and writing must share the same atomic boundary.

Official technology documentation

FOUNDATIONS · CLOJURE

Clojure: repetition does not create another intent.

The idempotency key identifies intent; the fingerprint identifies its relevant content. Repeating the same key with the same value preserves state. Reusing it with a different value is a conflict, not a new purchase. In the synthetic example, key scope is the in-memory database; production would need customer, operation, and retention policy.

CLOJURE / ORIGINAL PROPOSAL

```clojure
(defn register [db {:keys [key amount currency] :as cmd}]
  (when-not (valid-command? cmd)
    (throw (ex-info "invalid command"
                    {:reason :invalid-command})))
  (let [fingerprint [amount currency]
        existing (get db key)]
    (cond
      (nil? existing)
      (assoc db key {:fingerprint fingerprint
                     :status :received :version 0})
      (= fingerprint (:fingerprint existing)) db
      :else
      (throw (ex-info "key reused with different payload"
                      {:reason :key-conflict})))))

;; Em um único processo: atualização atômica do modelo.
;; Nunca adicionar efeitos externos à função de swap!.
(def state (atom {}))
(swap! state register
       {:key "synthetic-001" :amount 1250 :currency :BRL})
```

Excerpt from the local lab; valid-command? is defined in the complete file. Concurrency test executed.

The concurrency test submits eighty identical registrations against an atom and verifies one entry. This exercises local atomicity; it does not demonstrate exactly-once processing between services. Production still needs authenticated identity, persistence, expiration control, reconciliation, and a consistent response when the connection drops after commit.

Official technology documentation

FOUNDATIONS · DATOMIC

Datomic: identity and decision history.

The model separates operation identity from state evolution. The identity constraint prevents the same operation being represented as two entities. Version compare-and-swap accompanies the status change in the transaction. This is an educational proposal, not a description of Nubank's schema.

CLOJURE / ORIGINAL PROPOSAL

```clojure
;; Schema proposto para um exercício de domínio.
[{:db/ident :operation/id
  :db/valueType :db.type/uuid
  :db/cardinality :db.cardinality/one
  :db/unique :db.unique/identity}
 {:db/ident :operation/status
  :db/valueType :db.type/keyword
  :db/cardinality :db.cardinality/one}
 {:db/ident :operation/version
  :db/valueType :db.type/long
  :db/cardinality :db.cardinality/one}]

;; Uma transação propõe a mudança de versão e status.
;; entity-id e expected-version são obtidos pelo adapter.
[[:db/cas entity-id :operation/version
  expected-version (inc expected-version)]
 [:db/add entity-id :operation/status :operation/pending]]
```

Independent reference example; not executed against real infrastructure.

A correct version alone does not guarantee a valid business transition. The adapter must validate the state machine and ensure the observed condition belongs to the authorized transaction. History helps explain decisions but requires retention policy and care not to persist unnecessary sensitive data. Exact semantics depend on the edition and API the team adopts.

Official technology documentation

FOUNDATIONS · KAFKA

Kafka: contract before topic.

The proposed envelope carries event identity, aggregate identity, and domain version. Each answers a different question: have I processed this delivery, which operation does it belong to, and where does it fit in the sequence? The consumer must not confuse a transport offset with a business version.

JSON / ORIGINAL PROPOSAL

```json
{
  "event_id": "synthetic-event-001",
  "event_type": "operation.status_changed",
  "schema_version": 1,
  "aggregate_id": "synthetic-operation-001",
  "aggregate_version": 2,
  "occurred_at": "2026-10-01T12:00:00Z",
  "producer": "proposed-operation-service",
  "correlation_id": "synthetic-correlation-001",
  "payload": {
    "previous_status": "pending",
    "status": "settled",
    "currency": "BRL",
    "amount_minor": 1250
  }
}
```

Proposed JSON contract with synthetic identifiers; not an internal Nubank event.

The partition key must reflect the unit for which order matters. Reprocessing a message requires idempotency in the effect, not just the producer. A Kafka transaction does not automatically turn an external call or a write to another database into a single global transaction. The relevant test interrupts the consumer between persistence and acknowledgment and observes recovery.

Official technology documentation

FOUNDATIONS · AWS

AWS / DynamoDB: arbitrate the first write.

A conditional reservation makes key creation a competition resolved by storage. A service must not perform query-then-insert as two independent operations and assume exclusivity. Scope prevents a customer's key from being confused with another intent.

PYTHON / ORIGINAL PROPOSAL

```python
# Exemplo de adapter com boto3; cliente e tabela são externos.
# Nomes e valores são sintéticos.
def reserve(table, scope, request_key, fingerprint):
    pk = f"IDEMPOTENCY#{scope}#{request_key}"
    return table.put_item(
        Item={
            "pk": pk,
            "fingerprint": fingerprint,
            "status": "reserved",
            "version": 0,
        },
        ConditionExpression="attribute_not_exists(pk)",
    )

# Uma ConditionalCheckFailedException exige consultar
# o registro: replay compatível ou conflito de payload.
# Não converter todo erro em 'já processado'.
# A expiração da chave precisa respeitar a janela de replay.
```

Independent reference example; not executed against real infrastructure.

Reservation is not settlement. If the process crashes after reserving and before submitting the operation, pending work must be distinguished from completed work. Reserved state needs recovery with ownership and a time bound. Expiration policy must not allow a late retry to repeat a financial effect already performed.

Official technology documentation

FOUNDATIONS · KUBERNETES

Kubernetes: availability during change.

The example reserves capacity for one additional replica while gradually removing the previous version. Readiness should indicate whether an instance can receive work; it must not fluctuate with every shared-dependency failure and remove the entire service simultaneously. The numbers are educational assumptions.

YAML / ORIGINAL PROPOSAL

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: proposed-journey-reader
spec:
  replicas: 3
  selector:
    matchLabels: {app: journey-reader}
  strategy:
    rollingUpdate: {maxUnavailable: 0, maxSurge: 1}
  template:
    metadata:
      labels: {app: journey-reader}
    spec:
      containers:
        - name: reader
          image: example.invalid/reader:replace-with-reviewed-digest
          ports: [{containerPort: 8080}]
          readinessProbe:
            httpGet: {path: /ready, port: 8080}
            periodSeconds: 5
          resources:
            requests: {cpu: "250m", memory: "256Mi"}
            limits: {memory: "512Mi"}
```

Reference manifest, deliberately without an executable image. Not applied to a cluster.

Before applying it, a reviewed image, service identity, network policies, graceful shutdown, concurrency limits, and distribution across zones would be required. Three replicas in the same failure domain do not fail independently. Rollout must observe journey completion and saturation as well as count ready pods.

Official technology documentation

FOUNDATIONS · FLUTTER

Flutter: state drives the interface.

The component receives domain state; it does not infer completion from an absent spinner. A pending message must remain true when the network fluctuates. The design avoids offering retry while the financial outcome remains unknown; the next action depends on the reconciliation contract.

DART / ORIGINAL PROPOSAL

```dart
// Fragmento didático: imports e modelos do app são omitidos.
Widget statusPanel(JourneyState state) {
  final String label;
  switch (state) {
    case JourneyState.pending:
      label = 'Estamos confirmando a operação';
      break;
    case JourneyState.settled:
      label = 'Operação concluída';
      break;
    case JourneyState.rejected:
      label = 'Operação não concluída';
      break;
  }
  return Semantics(
    liveRegion: true,
    child: Padding(
      padding: const EdgeInsets.all(16),
      child: Text(label),
    ),
  );
}
```

Reference Flutter/Dart excerpt; not compiled in this edition.

The example needs widget, screen-reader, text-scaling, and real-navigation tests. A live region can announce a change without moving focus, but excessive announcements also harm the experience. Final wording should be developed with Design and Content, including expected time, a safe action, and a help channel when the delay is no longer normal.

Official technology documentation

FOUNDATIONS · DART

Dart: states that do not contradict one another.

A collection of booleans can represent impossible combinations: loading and completed, no receipt and available. A closed set of states makes these possibilities more visible. Here, a confirmed receipt requires its own identifier; query unavailability is not presented as an operation failure.

DART / ORIGINAL PROPOSAL

```dart
sealed class ReceiptState {
  const ReceiptState();
}
final class AwaitingConfirmation extends ReceiptState {
  const AwaitingConfirmation(this.operationId);
  final String operationId;
}
final class Confirmed extends ReceiptState {
  const Confirmed(this.operationId, this.receiptId);
  final String operationId;
  final String receiptId;
}
final class Unavailable extends ReceiptState {
  const Unavailable(this.operationId);
  final String operationId;
}

String actionLabel(ReceiptState state) => switch (state) {
  AwaitingConfirmation() => 'Consultar andamento',
  Confirmed() => 'Abrir comprovante',
  Unavailable() => 'Tentar consultar novamente',
};
```

Educational Dart 3 example; not compiled in this edition.

Typing helps implementation, but the server remains authoritative for financial state. The client must handle old contract versions and unknown responses. Fallback should preserve confirmed data without inventing a new success state. The proposal must be validated against the team's Dart version and modeling conventions.

Official technology documentation

FOUNDATIONS · PROMETHEUS

Prometheus: measure a promise.

Numerator and denominator must observe the same population. If abandoned journeys disappear from instrumentation, latency can look better while the experience gets worse. The measurement contract must define start, completion, timeout, legitimate cancellation, and missing events.

PROMQL / ORIGINAL PROPOSAL

```promql
# Proporção de jornadas com resultado confirmado em até 2s.
# Métricas e rótulos propostos; janela ilustrativa.
sum(rate(journey_confirmation_seconds_bucket{
  journey="payment", le="2"
}[30m]))
/
sum(rate(journey_confirmation_seconds_count{
  journey="payment"
}[30m]))

# Percentil de cauda: analisar por versão do app/serviço.
histogram_quantile(
  0.99,
  sum by (le, release) (
    rate(journey_confirmation_seconds_bucket[5m])
  )
)
```

Proposed PromQL queries; not executed on a Prometheus server.

Aggregated percentiles require care with buckets and distributions. A percentile alone reveals neither volume nor success rate; it must be read alongside counts, errors, and segmentation. Labels need bounded cardinality. Account, operation, and customer identifiers belong in controlled diagnostic trails, not time-series labels.

Official technology documentation

FOUNDATIONS · OPENTELEMETRY

OpenTelemetry: follow causality.

A trace should explain relationships between operations, not merely enumerate calls. Payment confirmation can cross synchronous and asynchronous stages, retries, and reconciliation. Sampling must retain enough failure and high-latency examples for investigation.

PYTHON / ORIGINAL PROPOSAL

```python
# Fragmento de instrumentação; tracer fornecido pelo runtime.
def confirm(command, repository, tracer):
    with tracer.start_as_current_span("journey.confirm") as span:
        span.set_attribute("journey.kind", "payment")
        span.set_attribute("contract.version", 1)
        try:
            result = repository.confirm(command)
            span.set_attribute("journey.outcome", result.kind)
            return result
        except TimeoutError:
            span.set_attribute("journey.outcome", "unknown")
            raise

# Não registrar CPF, token, saldo ou payload integral.
# Propagar contexto entre produtores e consumidores.
# Para processamento assíncrono, modelar links/parentesco
# de acordo com a semântica real do trabalho.
```

Independent reference example; not executed against real infrastructure.

A trace does not replace a durable financial record. It may be sampled, delayed, or missing precisely during an incident. Diagnostics should associate an operation with a trace through controlled access while keeping personal data out of common attributes. The goal is shorter investigations without creating another indiscriminate repository of sensitive information.

Official technology documentation

FOUNDATIONS · JAVA

Java / JVM: explicit budgets.

A timeout for each dependency can add up to more time than the journey allows. A shared budget forces every stage to account for time already consumed. A monotonic clock suits duration within a process; distributed timestamps require another contract and tolerance for clock differences.

JAVA / ORIGINAL PROPOSAL

```java
// Fragmento autoral: todas as durações são ilustrativas.
record Budget(long deadlineNanos) {
    static Budget start(java.time.Duration timeout) {
        return new Budget(System.nanoTime() + timeout.toNanos());
    }
    long remainingNanos() {
        return Math.max(0L, deadlineNanos - System.nanoTime());
    }
    boolean exhausted() {
        return remainingNanos() == 0L;
    }
}

// O adapter repassa apenas o orçamento restante.
// O relógio monotônico mede intervalo dentro do processo.
// Não serializar deadlineNanos para outro host.
var budget = Budget.start(java.time.Duration.ofMillis(800));
if (budget.exhausted()) {
    throw new java.util.concurrent.TimeoutException();
}
```

Reference Java excerpt; not compiled as a program in this edition.

Canceling a wait does not undo a remote effect. After a timeout, the outcome may remain unknown and require reconciliation. This is where my JVM experience connects to the financial domain: controlling threads and connections is necessary, but retry decisions depend on idempotency, authority, and state. The example is not a complete HTTP client.

Official technology documentation

FOUNDATIONS · PYTHON

Python: an experiment must be allowed to fail.

Python can quickly make a behavioral hypothesis concrete. A useful test describes an observable consequence, such as blocking expansion when the sample is insufficient. The code example is a deterministic gate, not a statistical significance test. The threshold must come from an approved journey policy.

PYTHON / ORIGINAL PROPOSAL

```python
import unittest

def can_release(samples, failures, min_samples=1000):
    if type(samples) is not int or type(failures) is not int:
        raise ValueError("counts must be integers")
    if type(min_samples) is not int or min_samples <= 0:
        raise ValueError("invalid sample policy")
    if not 0 <= failures <= samples:
        raise ValueError("invalid counts")
    if samples < min_samples:
        return False
    return failures / samples <= 0.001

class ReleaseGateTests(unittest.TestCase):
    def test_small_sample_does_not_pass(self):
        self.assertFalse(can_release(50, 0))

    def test_error_budget_is_respected(self):
        self.assertTrue(can_release(2000, 2))
        self.assertFalse(can_release(2000, 3))

    def test_invalid_evidence_is_rejected(self):
        with self.assertRaises(ValueError):
            can_release(2000, 2001)
```

Educational example separate from the twenty executed labs; synthetic threshold.

The twenty case labs use synthetic data and local tests. They help detect contract regressions, illegal states, and representation errors. They do not measure Nubank's infrastructure, estimate real load, or certify production controls. Each chapter records exactly what the lab covers and what requires additional validation.

Official technology documentation

FOUNDATIONS · POSTGRESQL

PostgreSQL: integrity in storage.

The schema makes certain invariants impossible to ignore accidentally: positive amount, supported currency, and a unique key per scope. The update conditions the transition on state and version. If no row returns, the service must distinguish conflict, replay, and a nonexistent identifier.

SQL / ORIGINAL PROPOSAL

```sql
CREATE TABLE proposed_operation (
  operation_id UUID PRIMARY KEY,
  customer_scope TEXT NOT NULL,
  request_key TEXT NOT NULL,
  amount_minor BIGINT NOT NULL CHECK (amount_minor > 0),
  currency CHAR(3) NOT NULL CHECK (currency = 'BRL'),
  version BIGINT NOT NULL DEFAULT 0 CHECK (version >= 0),
  status TEXT NOT NULL CHECK (
    status IN ('received','pending','settled','rejected')
  ),
  UNIQUE (customer_scope, request_key)
);

UPDATE proposed_operation
SET status = 'pending', version = version + 1
WHERE operation_id = $1
  AND version = $2
  AND status = 'received'
RETURNING operation_id, version, status;
```

Independent reference example; not executed against real infrastructure.

PostgreSQL is an educational option in this book, not an inference about every Nubank domain's database. A status constraint limits vocabulary but does not express the complete state machine. Transaction, indexes, isolation, and outbox strategy must be evaluated together so persistence and publication recover coherently from failures.

Official technology documentation

FOUNDATIONS · REDIS

Redis: cache with domain validity.

Cache policy starts with a product question: how much delay can this data tolerate? An informational projection may accept brief validity; financial authorization may require the authoritative source. The same tool can be appropriate for one path and unsuitable for another.

PYTHON / ORIGINAL PROPOSAL

```python
# Interface proposta; não é um cliente Redis completo.
def read_projection(key, cache, source, clock):
    cached = cache.get(key)
    if cached and cached["valid_until"] > clock():
        return {
            "data": cached["data"],
            "origin": "cache",
            "version": cached["version"],
        }
    current = source.read(key)
    cache.set(key, {
        "data": current.data,
        "version": current.version,
        "valid_until": clock() + 15,
    }, ttl_seconds=15)
    return {"data": current.data, "origin": "source",
            "version": current.version}
```

Independent reference example; not executed against real infrastructure.

TTL bounds duration but does not prevent a late write from reinstalling an older version. Distributed implementation needs version comparison, invalidation, or another workload-appropriate strategy. The example deliberately omits that mechanism and identifies it as remaining work. Redis is a proposed alternative, not a claim about Nubank's internal design.

Official technology documentation

FOUNDATIONS · TERRAFORM

Terraform: reviewable changes.

Infrastructure as code makes decisions available for review: ownership, recovery, resource identity, and protection against accidental removal. Someone capable of assessing impact must interpret the plan; it should not be approved merely because the file passed formatting.

HCL / ORIGINAL PROPOSAL

```hcl
# Fragmento de referência. Provider e backend são externos.
resource "aws_dynamodb_table" "operation_keys" {
  name         = "proposed-operation-keys"
  billing_mode = "PAY_PER_REQUEST"
  hash_key     = "pk"

  attribute {
    name = "pk"
    type = "S"
  }
  point_in_time_recovery {
    enabled = true
  }
  server_side_encryption {
    enabled = true
  }
  lifecycle {
    prevent_destroy = true
  }
  tags = {
    owner       = "proposed-journey-team"
    environment = "example"
  }
}
```

Independent reference example; not executed against real infrastructure.

prevent_destroy replaces neither access control nor backup policy. Removing the configuration block or changing the management strategy requires additional care. The example was not applied and includes no remote backend, provider version, managed key, or IAM policy. These omissions are explicit so an editorial excerpt does not look production-ready.

Official technology documentation

FOUNDATIONS · TYPESCRIPT

TypeScript: validate the boundary.

A static type does not validate JSON received at runtime. The proposed parser treats the boundary as unknown and rejects states outside the contract. In a product, rejection must become a safe experience with an alternative query or incompatibility message, without inventing financial confirmation.

TYPESCRIPT / ORIGINAL PROPOSAL

```ts
type Status = "pending" | "settled" | "rejected";
type Projection = { version: number; status: Status };

function parseProjection(input: unknown): Projection {
  if (typeof input !== "object" || input === null) {
    throw new Error("invalid object");
  }
  const x = input as Record<string, unknown>;
  if (typeof x.version !== "number" ||
      !Number.isSafeInteger(x.version) || x.version < 0) {
    throw new Error("invalid version");
  }
  if (typeof x.status !== "string" ||
      !["pending", "settled", "rejected"].includes(x.status)) {
    throw new Error("unsupported status");
  }
  return { version: x.version as number,
           status: x.status as Status };
}
```

Independent reference example; not executed against real infrastructure.

TypeScript is used here for contracts and visualization, not as a general Nubank requirement. For a real SDK, I would generate types from a versioned schema, maintain fixtures for old consumers, and define handling of additional fields. Safe evolution expands the contract without forcing every client to update simultaneously.

Official technology documentation

FOUNDATIONS · GRAFANA

Grafana: a dashboard that guides action.

The dashboard starts with the decision someone needs to make. Confirmed completion, unknown state, and reconciliation age describe different aspects of trust. CPU and memory aid diagnosis but do not independently demonstrate that the customer completed the task.

JSON / ORIGINAL PROPOSAL

```json
{
  "title": "Proposta: confiança na jornada",
  "panels": [
    {"title": "Conclusão confirmada", "unit": "percent"},
    {"title": "Estado desconhecido", "unit": "short"},
    {"title": "Tempo de confirmação P99", "unit": "ms"},
    {"title": "Reconciliação mais antiga", "unit": "s"},
    {"title": "Contatos por 1.000 jornadas", "unit": "short"}
  ],
  "dimensions": ["journey", "release", "cohort"],
  "decision": {
    "owner": "proposed-journey-team",
    "action": "hold rollout when guardrails fail",
    "runbook": "referência local do serviço"
  }
}
```

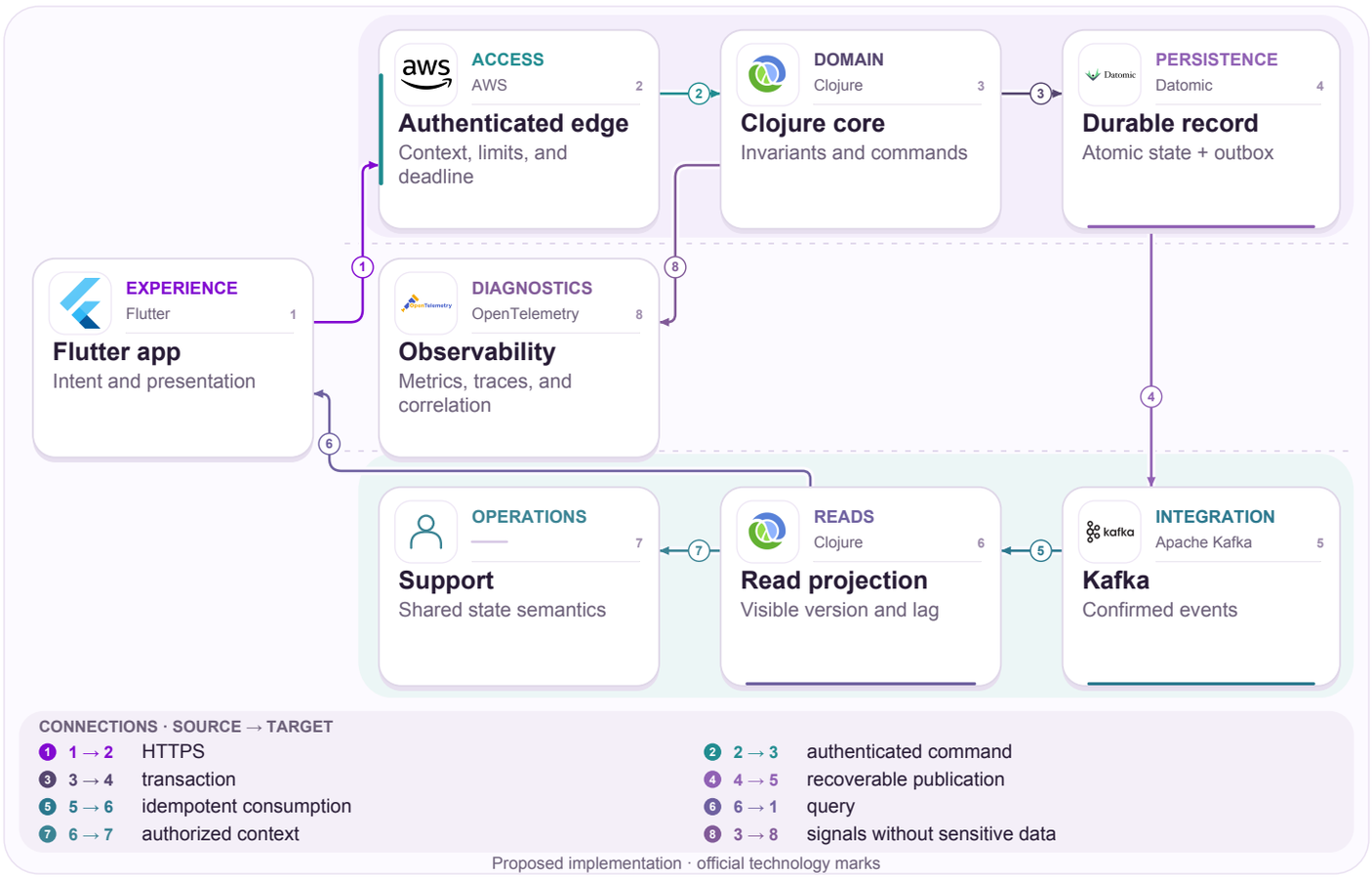
Conceptual dashboard specification; not Grafana's import schema.

The JSON is an editorial specification, not an importable Grafana dashboard. Each indicator needs a definition, denominator, window, owner, and connection to the response procedure. Red without an agreed action creates noise. A good dashboard also displays missing data and small samples so telemetry silence is not interpreted as success.

Official technology documentation

CROSS-CUTTING ARCHITECTURE · EXTERNAL PROPOSAL

One journey, two authorities.



Independent reference design. Does not represent Nubank's internal architecture.

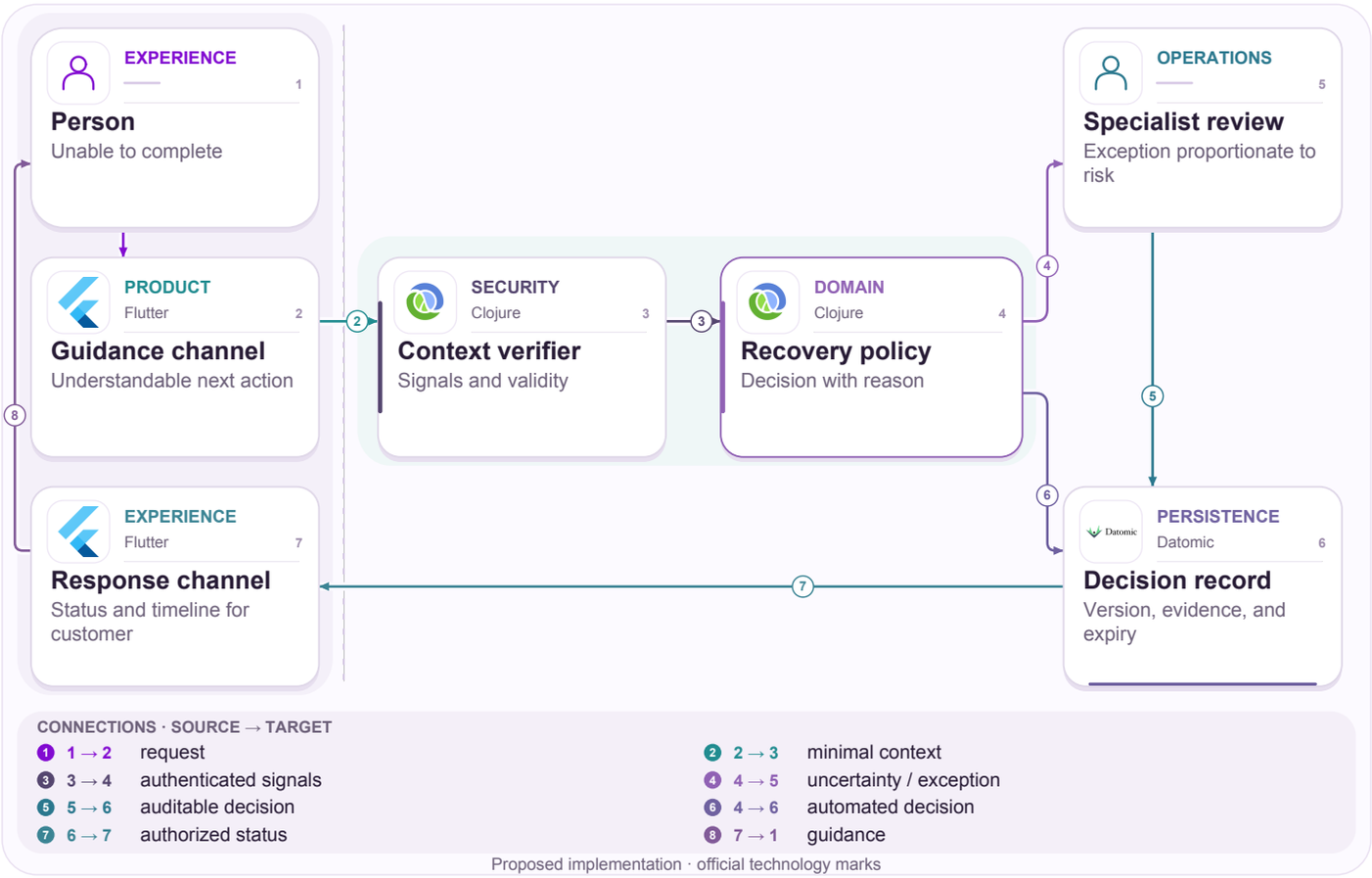
The architecture separates the authority that changes state from the authority that presents it. The domain core and its storage decide whether an operation was accepted. The projection makes reading easier, but carries version and lag. A screen cannot turn a missing update into rejection, and support cannot promise reversal without consulting the state responsible for the effect.

The outbox represents publication intent within the same boundary as the domain write. A publisher may repeat messages, so the consumer must deduplicate effects. The design allows failures between stages and makes recovery part of the contract. The choice between Datomic, DynamoDB, or another persistence option depends on the domain and existing platform; all three are not added simultaneously for aesthetic reasons.

The Clojure lab demonstrates only the in-memory decision. The case labs explore specific properties. Distributed integration, authorization, retention, and capacity still require validation in the team's context.

CROSS-CUTTING ARCHITECTURE · EXTERNAL PROPOSAL

Recovery as part of the product.



Independent reference design. Does not represent Nubank's internal architecture.

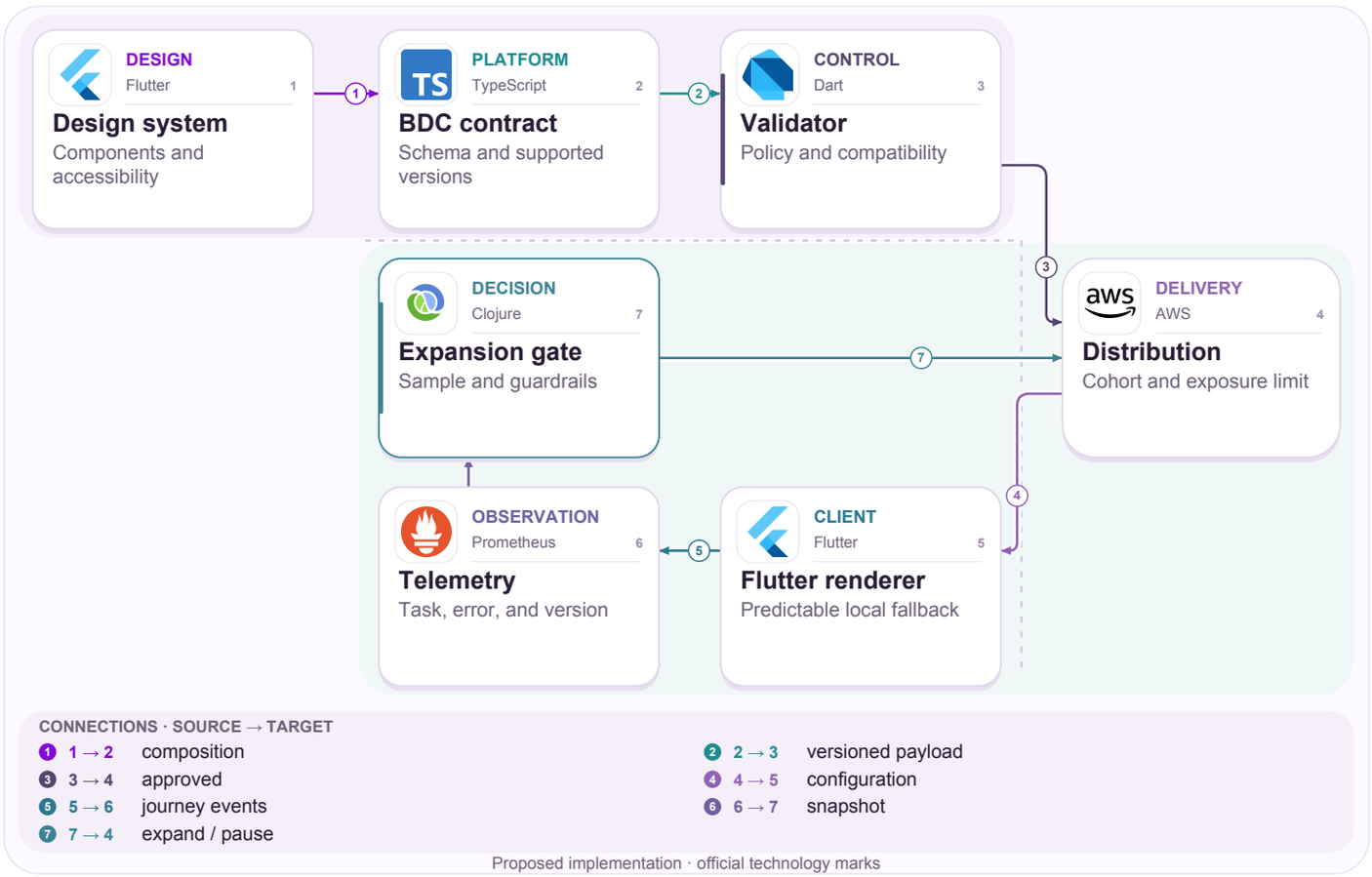
Recovery should not be a collection of ownerless exceptions. The flow must distinguish technical inability, suspected fraud, and insufficient evidence. Each may require a different path. Human review belongs where it adds judgment, not as an unlimited destination for every integration error.

The record associates decision, policy, and validity. An old signal must not authorize a sensitive action indefinitely. When an alternative is denied, the product must explain the next available step without disclosing details that help bypass controls. This language requires collaboration between Security, Product, Content, and Support.

Success combines legitimate resolution, recovery time, and protection. Reducing a queue by rejecting more people can improve an operational metric while worsening the outcome. As EM, I would ensure capacity and responsibilities; as Staff, I would define authority, expiration, and review mechanisms.

CROSS-CUTTING ARCHITECTURE · EXTERNAL PROPOSAL

An experience platform with contracts.



Independent reference design. Does not represent Nubank's internal architecture.

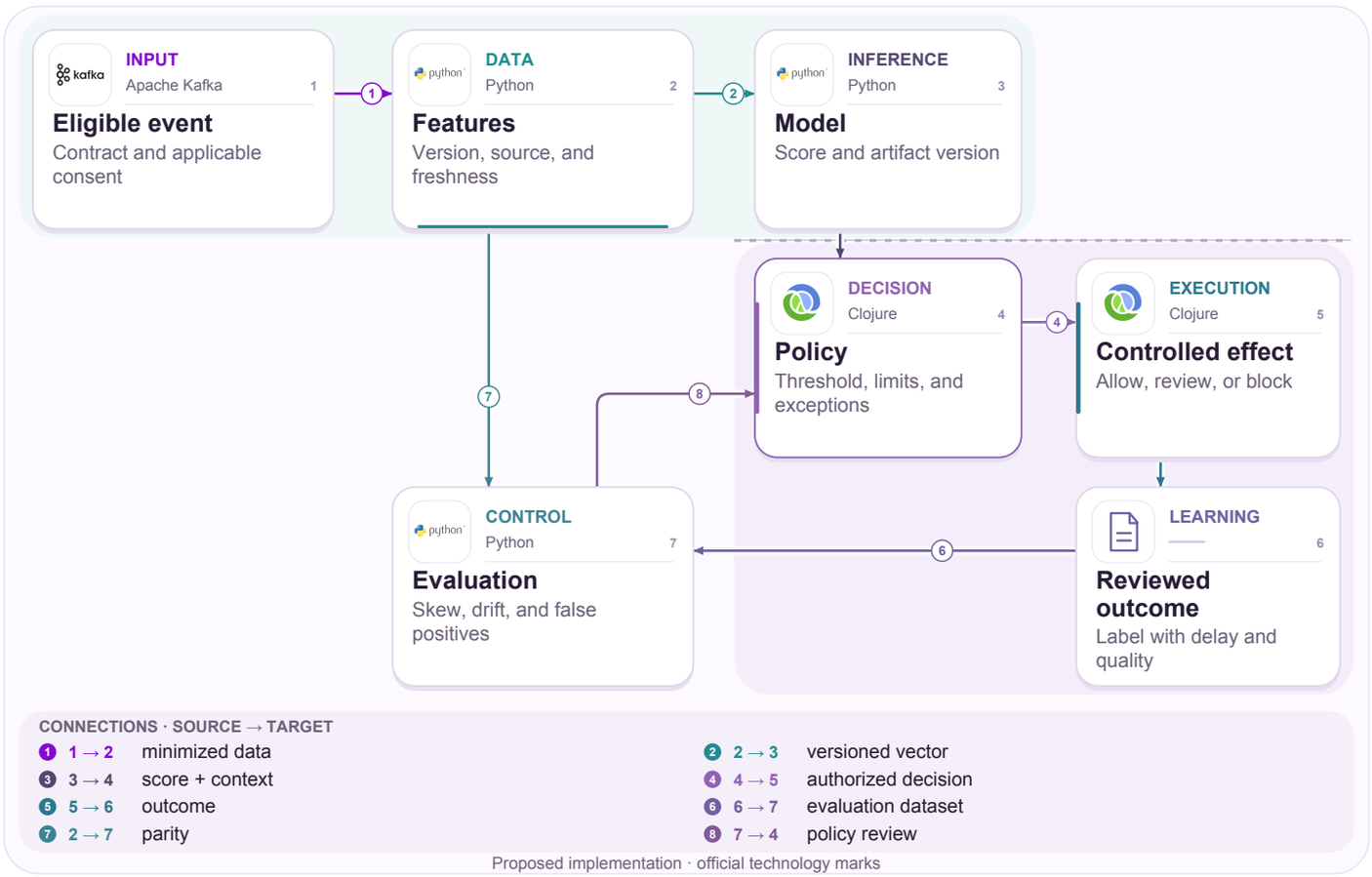
A server-driven interface platform increases composition speed, but also increases the importance of its contract. The installed client may lag behind the server version. The validator must understand the renderer's minimum capability and prevent a configuration requiring nonexistent behavior.

Fallback is a product component: it must preserve the task and accessibility, not merely prevent crashes. Telemetry connects configuration, app version, and journey outcome. The gate requires sufficient evidence before expanding exposure. The decision retains an owner; it is not delegated to one context-free metric.

Nubank publishes work on BDC; this design is an educational proposal for discussing governance and evolution. I would investigate existing controls and seek the smallest necessary extension. The platform creates value when it reduces consumer-team effort while maintaining quality and recovery capability.

CROSS-CUTTING ARCHITECTURE · EXTERNAL PROPOSAL

ML: separate prediction from decision.



Independent reference design. Does not represent Nubank's internal architecture.

A score is not a business decision. Policy combines prediction with limits, context, and review paths. Separating these responsibilities allows a threshold to change without pretending the model was retrained and supports investigation into whether the problem lies in data, inference, or use of the result.

Features need identity and time: what was available when a decision was made may differ from what we know today. Evaluating with future information creates an impression of quality that would not exist in production. The training-serving contract must prevent this leakage and measure relevant differences.

Ground truth may arrive late and remain incomplete. Experiments must preserve version trails and treat false positives as legitimate people harmed by a decision. The proposed value is to improve protection and recovery together, with specialists responsible for policy and exposure criteria.

EXTERNAL ANALYSIS AND PROPOSAL

The problem begins when the state becomes invisible

01

Pix fora do AR novamente?

Original source screenshot, accessed 23 October 2026. An individual report is not a reproduced incident.

The public post dated July 15, 2026 describes access to the app followed by errors in transfers, Pix payments, and the support channel. This raises an engineering question, not a diagnosis: how can guidance remain available when the primary operation fails? The report does not establish that the entire customer base was affected, that an incorrect debit occurred, or that the components share infrastructure. The analysis respects that boundary and proposes an investigation that can be verified.

I would distinguish three problems that can look identical on screen: unavailability before a command is accepted, an unknown outcome after acceptance, and unavailable status information. The first permits another attempt when safe; the second requires a query or reconciliation; the third requires a message that preserves the uncertainty. A single “try again” button can conceal these differences and lead customers to create unnecessary new operations.

The product objective would be to help people understand whether their money moved, what they can do next, and how to get help. The corresponding technical objective would be consistency between operation identity, the financial decision, and communication. This chapter’s lab tests the local part of that promise: a committed debit has a durable receipt and does not become another debit when the response is lost.

NuCommunity - Is Pix down again?

EXTERNAL ANALYSIS AND PROPOSAL

An operation is an entity; an attempt is an event

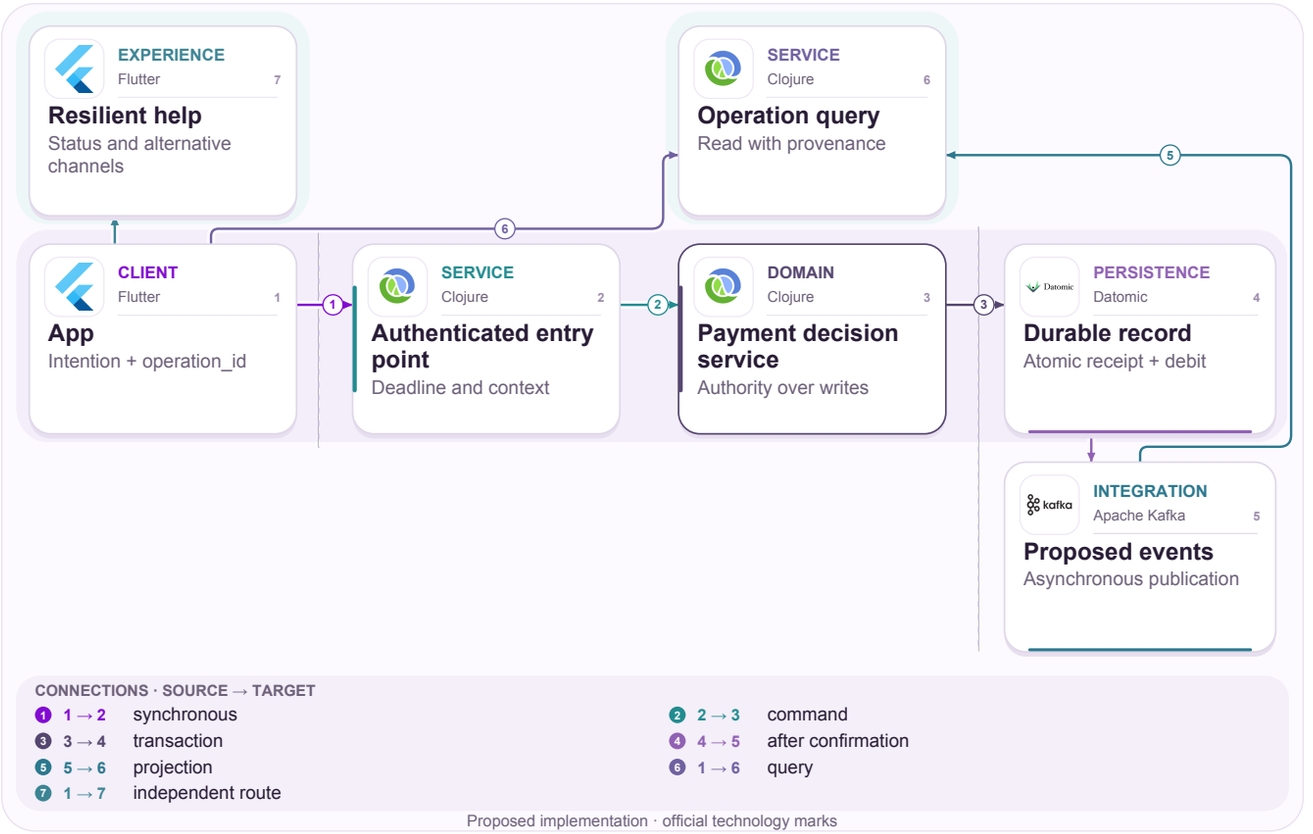
The proposed domain distinguishes operation_id from request_id. An operation represents a stable intention: an account, a beneficiary, and an amount. An attempt represents a transport interaction, with its own timestamp, deadline, and network outcome. Confusing the two identifiers makes every retry look like a new intention. Under the suggested contract, the app retains operation_id while establishing the outcome of that same intention; changing the amount or beneficiary requires a new confirmation and another identifier.

Useful states would include received, rejected, confirmed, and outcome awaiting clarification. “Timeout” describes the attempt and must not, by itself, mark the operation as rejected. A confirmed state must reference a durable decision. An unavailable query must not erase a previously known confirmation either. For the interface, this means retaining the receipt and context through connectivity changes while distinguishing local information from confirmation obtained from the service.

The invariants are concrete: a nonnegative balance, one operation identity bound to a single payload, and agreement between the debit, ledger entry, and receipt. Session recovery adds another condition: an old session cannot issue new commands after the revocation barrier. These rules can be tested without knowing the bank’s internal architecture and allow candidate implementations to be evaluated before discussing language or infrastructure.

PROPOSED ARCHITECTURE • EXPLICIT BOUNDARIES

Separate the financial path from the guidance path



Original proposed design; it does not represent Nubank's internal architecture.

The design proposes two observable paths: executing a financial intention and clarifying an existing one. Authority over the debit remains with the decision service and its transactional record. A query projection can help scale reads, but it must disclose lag and cannot invent a confirmation. If it is stale, the query must consult the authoritative source or communicate that a reliable answer is not yet available.

Help appears as a separate domain because its minimum useful content should survive degradation of the financial path. This does not automatically require another cluster: shared dependencies, authentication, configuration, and capacity limits would need to be identified first. A static guidance page may address part of the problem while account information remains protected.

Kafka is an option for propagating confirmed events; it does not make an external write part of a distributed atomic transaction. The proposed implementation would need recoverable publication, an idempotent consumer, and reconciliation. The lab demonstrates only the local transaction and makes that boundary explicit.

ENGINEERING EXPLAINED

The existing receipt takes precedence over the retry

PYTHON / ORIGINAL PROPOSAL

```python
def pay(self, *, account_id, operation_id, amount_cents,
        payee, recovery_epoch, fault=None):
    # Recorte: validação e payload/digest estão no arquivo completo.
    with self._write() as connection:
        account = connection.execute(
            "SELECT balance_cents, recovery_epoch FROM accounts "
            "WHERE account_id=?", (account_id,)
        ).fetchone()
        if account is None:
            raise UnknownAccount(account_id)
        # A sessão é validada antes inclusive de retornar um recibo já existente.
        if recovery_epoch != account["recovery_epoch"]:
            raise StaleSession("Comando de sessão anterior à recuperação")
        previous = connection.execute(
            "SELECT payload_json, payload_hash, receipt_json "
            "FROM payments WHERE operation_id=?",
            (operation_id,),
        ).fetchone()
        if previous is not None:
            # Compara também o conteúdo canônico, sem depender apenas do hash.
            if (previous["payload_hash"] != digest
                    or previous["payload_json"] != payload):
                raise IdempotencyConflict(
                    "operation_id reutilizado com outro payload")
            return json.loads(previous["receipt_json"])
        if account["balance_cents"] < amount_cents:
            raise InsufficientFunds("Saldo insuficiente no cenário sintético")
```

Excerpt from the pay method; setup and writes omitted. The complete file was executed and verified.

The excerpt checks the session generation before revealing any outcome. It then looks up the operation identity and compares the canonical content as well as its hash. A valid retry returns the stored receipt; reusing the identity with another payload creates a conflict. Only a previously unseen intention reaches the balance check.

This order matters: the balance after the first purchase may be lower than the original amount, but that should not turn a replay into “insufficient funds.” In the SQLite model, the block executes inside BEGIN IMMEDIATE. Serialization is a local teaching choice, not a recommendation for distributed scale.

Python - sqlite3

ENGINEERING EXPLAINED

Clojure: separate pure decisions from transactional effects

CLOJURE / ORIGINAL PROPOSAL

```clojure
(defn payment-decision [account previous command]
  ;; Contrato proposto; não é código interno do Nubank.
  (let [{:keys [epoch balance-cents]} account
        {:keys [session-epoch payload]} command
        amount (:amount-cents payload)]
    (cond
      (not= epoch session-epoch)
      {:type :reject :reason :stale-session}

      (and previous (not= payload (:payload previous)))
      {:type :reject :reason :idempotency-conflict}

      previous
      {:type :replay :receipt (:receipt previous)}

      (or (not (integer? amount)) (not (pos? amount)))
      {:type :reject :reason :invalid-amount}

      (< balance-cents amount)
      {:type :reject :reason :insufficient-funds}

      :else
      {:type :commit
       :new-balance (- balance-cents amount)
       :payload payload})))
```

Proposed teaching function; this Clojure excerpt was not executed.

The proposed function returns a decision as data. It does not write, call services, or guarantee concurrency safety on its own. Its purpose is to make the rules readable and allow combinations of state to be explored without a network dependency. The persistence layer must execute the decision against a valid view and confirm its preconditions atomically.

In a Datomic implementation, I would examine unique identity, version comparison, and the appropriate transaction before mapping this result to transaction data. Reading a balance and writing later without protection would retain the race, even with a correct pure function. The example is illustrative and was not executed in Clojure.

Datomic - Transaction Data

EXTERNAL ANALYSIS AND PROPOSAL

The persistence contract: what must survive the process

The idempotency record cannot live only in the memory of the process that received the first attempt. Restarts, scaling, and instance replacement would make retries lose their history. In the local model, the payments table contains a unique operation_id, canonical content, and a receipt. The ledger table requires a unique operation identity and references the payment. The balance update occurs in the same transaction as both inserts, forming a single recovery unit.

The contract must specify how long that identity remains recognized. A short window may allow a disconnected client to resubmit an old intention as new; indefinite retention increases costs and data handling obligations. I would choose the window using retry data and operational needs, retaining enough identification to investigate disputes. This book does not establish an actual retention period for Nubank.

The receipt records the historical confirmation, including the balance observed after that decision. It must not be used as the current balance. A real integration would also require an external participant identity and settlement states that the lab does not implement. Guaranteeing a local entry does not guarantee that a remote system received the message exactly once. That distinction belongs in the contract, the runbook, and the explanation provided to the customer.

ENGINEERING EXPLAINED

Concurrency must challenge the same invariant

PYTHON / ORIGINAL PROPOSAL

```python
def test_concurrent_distinct_payments_cannot_overdraw(self):
    barrier = threading.Barrier(2)
    def attempt(operation_id):
        barrier.wait(timeout=10)
        try:
            return self.bank.pay(**self.command(operation_id=operation_id))
        except InsufficientFunds:
            return "insufficient"
    with ThreadPoolExecutor(max_workers=2) as pool:
        results = list(pool.map(attempt, ("parallel-a", "parallel-b")))
    self.assertEqual(sum(isinstance(value, dict) for value in results), 1)
    self.assertEqual(results.count("insufficient"), 1)
    self.assert_invariants(balance=300, payments=1)

def test_concurrent_same_operation_has_one_receipt_and_one_debit(self):
    barrier = threading.Barrier(8)
    def retry(_):
        barrier.wait(timeout=10)
        return self.bank.pay(**self.command())
    with ThreadPoolExecutor(max_workers=8) as pool:
        results = list(pool.map(retry, range(8)))
    self.assertTrue(all(receipt == results[0] for receipt in results))
    self.assert_invariants(balance=300, payments=1)
```

Excerpt from the synthetic lab; execution verified in the complete file.

The first test releases two threads simultaneously and uses independent connections to compete for a balance that can fund only one purchase. The acceptable result is exactly one confirmation and one rejection for insufficient funds. In the second test, eight attempts represent the same intention and must converge on a single receipt.

The barrier increases the likelihood of overlap; it does not enumerate every possible interleaving. The 1,000-cent balance and 700-cent payments are synthetic fixtures. Subsequent checks verify the balance, record counts, and total debits. The test therefore observes financial effects, not just return codes.

ENGINEERING EXPLAINED

Killing a process is different from raising an exception

PYTHON / ORIGINAL PROPOSAL

```python
def run_crash_worker(self, fault: str, expected_exit: int):
    result = subprocess.run(
        [sys.executable, str(Path(__file__).resolve()),
         "--crash-worker", str(self.database), fault],
        capture_output=True, text=True, timeout=20,
    )
    self.assertEqual(result.returncode, expected_exit, result.stderr)
    self.bank = SyntheticPayments(self.database)

def test_process_crash_before_commit_rolls_back_and_resume_executes_once(self):
    self.run_crash_worker("crash_before_commit", 72)
    self.assert_invariants(balance=1000, payments=0)
    receipt = self.bank.pay(**self.command())
    self.assertEqual(self.bank.pay(**self.command()), receipt)
    self.assert_invariants(balance=300, payments=1)

def test_process_crash_after_commit_preserves_receipt_and_resume_replays(self):
    self.run_crash_worker("crash_after_commit", 73)
    self.assert_invariants(balance=300, payments=1)
    stored = json.loads(self.bank.inspect("alice")["payments"][0]["receipt_json"])
    self.assertEqual(self.bank.pay(**self.command()), stored)
    self.assert_invariants(balance=300, payments=1)
```

Excerpt from the synthetic lab; execution verified in the complete file.

The tests start another process and terminate it with os._exit at defined points. Before the commit, reopening must find the balance unchanged; after the commit, it must find the persisted receipt. This distinction exercises recovery without relying on the finally block of an ordinary exception.

The complete file passed 12 tests, including these scenarios and a timeout after confirmation. The result does not prove resilience to power loss, disk corruption, or failure of an external provider. The evidence is narrower and more useful: in this local model, terminating the process at these two points does not create a duplicate debit on resumption.

EXTERNAL ANALYSIS AND PROPOSAL

Observe uncertainty, not just HTTP success

I would propose three families of signals. The first measures attempts: duration, transport errors, and saturation. The second measures operations: confirmations, identity conflicts, and time to resolve an unknown outcome. The third measures experience: successful queries, accessible help, and contacts associated with the incident. A fast HTTP response does not compensate for an operation that remains unresolved, and a transport error does not necessarily mean the financial decision was lost.

Proposed metrics could include payment_attempts_total with a bounded set of outcomes, payment_resolution_seconds by journey, and payment_unknown_operations as an aggregate count. Account identifiers, operation_id, and beneficiaries would not become Prometheus labels. Where needed and authorized, they belong in controlled investigations with appropriate correlation and retention. This decision avoids explosive cardinality and reduces unnecessary data exposure.

The incident dashboard would start with observable impact: how many intentions await clarification, is that number growing continuously, and is the help channel responding? Alerts should have an owner and an actionable response, such as reducing new admissions, initiating reconciliation, or publishing guidance. I would track percentiles and app version cohorts, but would not combine reconciliation queue time with command latency. They are different problems and need separate limits.

Prometheus - Metric and label naming

EXTERNAL ANALYSIS AND PROPOSAL

Release with a rollback path that preserves receipts

Rollout would begin with instrumentation and comparative reads, without changing the financial decision. A set of synthetic intentions would exercise timeouts and queries while the team checked whether identifiers travel through every layer. The first visible change could be a clearer state explanation and help route, provided the information was accurate. This can reduce confusion before replacing critical write mechanisms.

For the transactional component, I would require compatibility between versions during migration. The old version must recognize an operation written by the new one or be prevented from executing it again. An interface feature flag does not solve that compatibility problem. A safe strategy would maintain a shared record and stable read contracts, with clear criteria for halting expansion if unexpected conflicts, unresolved operations, or balance discrepancies increased.

In Kubernetes, traffic distribution and rollback would be operational controls, not guarantees of domain correctness. Reverting a binary must not erase idempotency records or restore an incompatible schema. I would prepare an exercise in which the new process commits, fails before responding, and the previous process handles the retry. Rollout would advance only after receipt continuity had been demonstrated and support could explain that path.

EXTERNAL ANALYSIS AND PROPOSAL

Limit repeated work without hiding the outcome

A retry storm can turn partial unavailability into saturation. I would use explicit deadlines, attempt budgets, and jittered backoff for repeated queries, separating these controls from financial identity. The client may wait longer before querying, but must not generate another operation merely because its network budget expired. A conflict response must also be distinguishable from a transient error, since repeating the same invalid payload does not help.

The query endpoint requires authorization appropriate to the sensitivity of the receipt. Knowing an operation_id cannot be sufficient to read another account's data. The lab's session generation demonstrates the ordering of the barrier, but the program receives parameters directly and does not implement real authentication. In a service, identity and generation would come from verified credentials, and queries would be scoped to the account holder.

For performance, I would measure contention before choosing a partitioning scheme. The lab serializes writers in the SQLite file, simplifying the local proof while limiting throughput. A distributed design would need to choose its consistency boundary and prevent unsafe concurrency on the same account. The Staff-level objective would be to remove unnecessary work from the critical path without moving financial authority into a potentially stale projection.

EXTERNAL ANALYSIS AND PROPOSAL

ADR: durable receipts and recoverable publication

The proposed decision is to make local confirmation a durable fact and use that identity to answer retries. Keeping only a response cache would be simpler initially, but would leave a gap when the cache expired or an instance restarted. Blocking every retry would prevent some duplicates, yet would also prevent recovery of lost responses and transfer uncertainty to support.

To distribute the outcome, I would compare recoverable publication associated with the transaction, reading a change log, and synchronous integration with the consumer. The choice would depend on storage guarantees and available operational capabilities. In every option, the consumer must recognize the event identity and make its effect safe to repeat. A proposed Kafka topic must not be presented as proof of a single effect in external services.

The accepted consequence is greater attention to states, retention, and reconciliation. The intended benefit is a deterministic explanation for the customer and the incident investigator. I would revisit this decision if the settlement boundary required separate reservation and confirmation, if multiple currencies were involved, or if throughput pressure changed the unit of serialization. The ADR would record these conditions and the tests that must continue passing after any migration.

EXTERNAL ANALYSIS AND PROPOSAL

How I would lead delivery and demonstrate value

I would organize the initiative around a journey rather than a component list. A payments owner would define financial truth; mobile would validate how an intention survives connectivity changes; support would define the information needed to resolve a case; and platform would support isolation and operations. My role as an EM would be to turn those dependencies into verifiable deliverables and prevent responsibility for uncertainty from falling between teams.

The first review would require failure examples and permitted states. The second would demonstrate concurrency and recovery tests. The third would evaluate a simulation with support and communications, including how to explain an outcome that is still unknown. As a Staff engineer, I would review the transactional boundary, retry contracts, and migration decisions, documenting what the lab proves and what remains outside its scope. None of this requires attributing an unobserved architecture to the bank.

Value would be estimated as avoided contacts multiplied by marginal resolution cost, minus maintenance and infrastructure costs. Time to clarification and perceived trust would remain outcomes in their own right, without automatically converting them into revenue. The interview question I would bring is concrete: “Which component can assert that an intention was confirmed when the original response was lost, and how do we demonstrate that every client version can recover that truth?”

EXTERNAL ANALYSIS AND PROPOSAL

Security also needs a legitimate recovery path

02

Sem acesso a minha conta , pede reconhecimento facial

Original source screenshot, accessed 23 October 2026. An individual report is not a reproduced incident.

The NuCommunity participant reports that their front camera was broken, their rear camera worked, and they could not activate the account through facial recognition. The public excerpt does not reveal the outcome or any alternatives subsequently offered. It therefore does not establish a biometric failure. It provides a concrete case for examining how a security requirement behaves when a device lacks the expected capability.

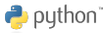
A well-designed recovery process must protect two people: the legitimate account holder who has lost access and the account holder someone is trying to impersonate. Simplifying the path by ignoring identity evidence might resolve the first contact while creating a larger risk. My proposal would make exception handling predictable through capability checks, assisted routing, and trackable status while preserving the risk assessment required to restore access.

The engineering problem extends beyond the camera. It includes repeated attempts, deadlines, human review, device changes, and commands issued by old sessions. This chapter models those boundaries with a persistent state machine and a recovery generation. The lab does not perform liveness detection, face matching, or fraud assessment: it demonstrates how to prevent skipped steps, reuse of an expired decision, and repeated restoration of the same case. This boundary makes the work reviewable and avoids presenting a simulation as an identity system.

NuCommunity - Unable to access my account; facial recognition required

EXTERNAL ANALYSIS AND PROPOSAL

Device capability is not account-holder identity



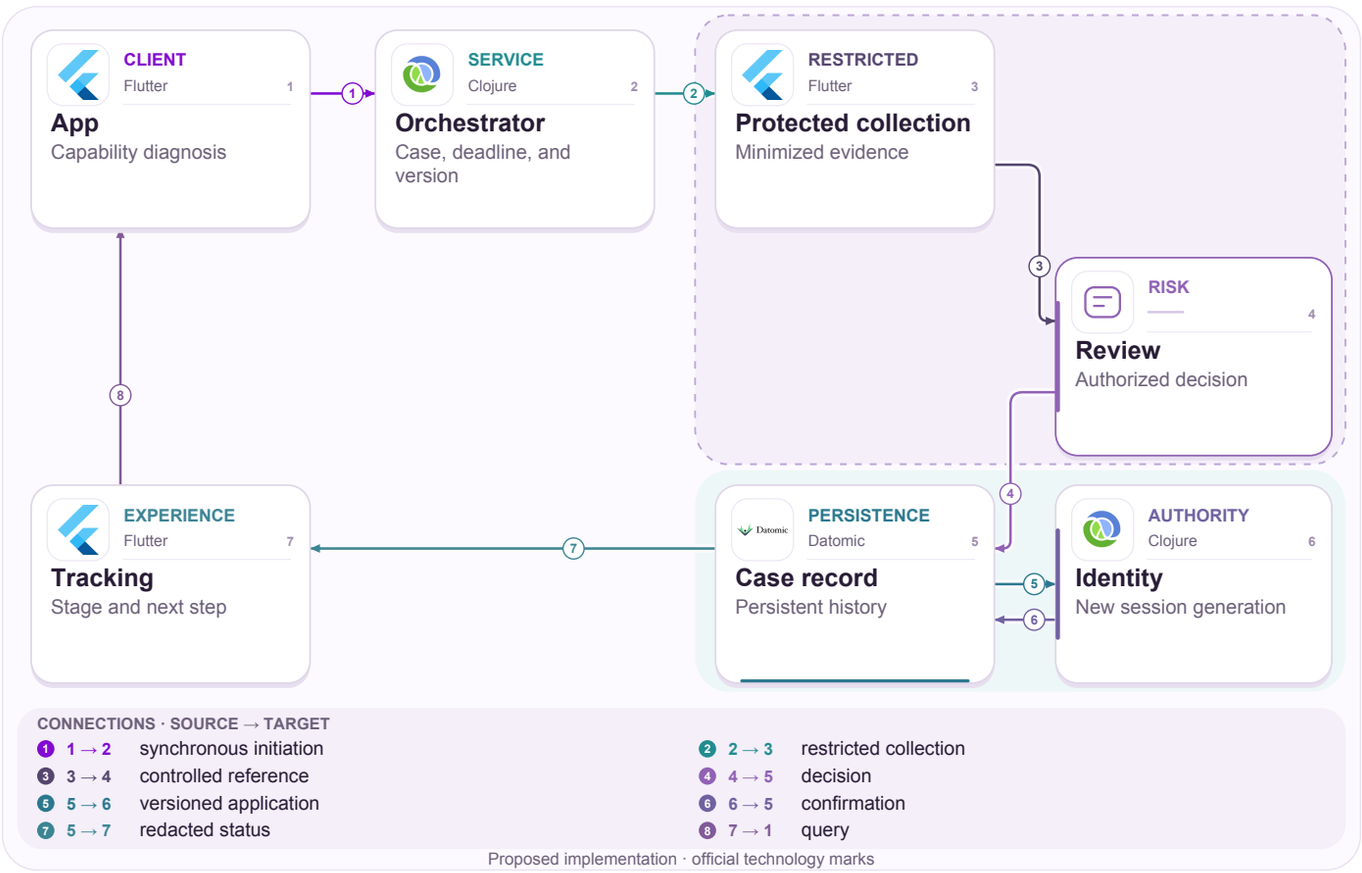
I would separate technical diagnosis from identity assessment. The first determines whether a camera exists, permission has been granted, and capture can begin. The second decides whether the available evidence is sufficient for a sensitive action. An unavailable camera may route the case to another journey; it cannot, on its own, authorize access. This distinction prevents a device error code from inadvertently becoming a security exception.

The proposed states would be diagnosis, review, verified, restored, expired, and closed. Each transition has preconditions, an owner, and a version. History would preserve why a step occurred without requiring biometric images to circulate through every service. “Verified” means an authorized assessment has completed its step; “restored” means the access change has taken effect. Separating them makes it possible to detect an approval still awaiting application.

The local model uses two distinct reviewers to make separation of responsibility explicit. This is a synthetic policy, not a universal recommendation for manual operations. In production, the mix of automation and review would depend on risk and governance. The transferable principle is that applying a decision must check its scope, validity, and version at the moment access is restored, including when messages arrive late.

PROPOSED ARCHITECTURE • EXPLICIT BOUNDARIES

Recovery with a boundary between evidence and authorization



Original proposed design; it does not represent Nubank's internal architecture.

The proposal keeps sensitive evidence within a restricted boundary and uses references to track the case. Support needs to know the stage and next action, but not every operator needs access to biometric material. Restoration goes through the identity authority, which records the session generation and confirms application. The tracking screen receives only the data needed to guide the account holder.

The delicate point is the gap between approval and execution. If a decision expires or is revoked while awaiting processing, the consumer must reject it. A duplicate message must recover the existing result. The case version acts as a precondition, while the session generation prevents old commands from remaining valid after recovery.

A real implementation would require reviewer authentication, role-based authorization, protected storage, and access auditing. The lab demonstrates transitions with fictional names and SQLite; it does not provide those identity or operational guarantees.

ENGINEERING EXPLAINED

A transition validates version, deadline, and actor

PYTHON / ORIGINAL PROPOSAL

```python
def transition(self, case_id, expected, action, actor, now):
    if type(now) is not int or type(expected) is not int:
        raise ValueError("integer_clock_and_revision_required")
    with sqlite3.connect(self.path, isolation_level=None) as db:
        db.execute("BEGIN IMMEDIATE")
        row = db.execute("SELECT * FROM cases WHERE id=?", (case_id,)).fetchone()
        if row is None:
            raise KeyError(case_id)
        _, state, revision, expires, owner, verifier, epoch = row
        if expected != revision:
            raise ValueError("stale_revision")
        if now >= expires:
            raise ValueError("expired")
        if action == "verify" and state == "review" and actor != owner:
            state, verifier = "verified", actor
        elif (action == "restore" and state == "verified"
              and actor not in (owner, verifier)):
            state, epoch = "restored", epoch + 1
        else:
            raise ValueError("invalid_transition_or_actor")
        db.execute("UPDATE cases SET state=?, revision=?, verifier=?, epoch=? "
                   "WHERE id=?", (state, revision + 1, verifier, epoch, case_id))
        db.commit()
    return self.read(case_id)
```

Excerpt from the synthetic lab; execution verified in the complete file.

The read and update take place inside a write transaction. The expected revision prevents two operators from applying decisions against the same version. The deadline is checked before every transition, including restoration. The actor must be different from the requester and, for final application, also different from the previous reviewer.

In the lab, these actors are strings supplied by the test, so they do not constitute authentication. The value lies in the domain rule, which remains visible and verifiable. In a real implementation, the actor and permissions would be derived from a trusted credential, and each decision would leave an appropriate audit record. The function does not validate biometrics.

ENGINEERING EXPLAINED

Datomic: version as an explicit precondition

CLOJURE / ORIGINAL PROPOSAL

```clojure
(defn restore-tx [case-id expected-version now decision]
  ;; Esboço proposto de dados transacionais.
  ;; O chamador autenticado precisa validar a autorização.
  (when-not (= :verified (:state decision))
    (throw (ex-info "case-not-verified" {})))
  (when (>= now (:expires-at decision))
    (throw (ex-info "decision-expired" {})))
  (let [new-version (inc expected-version)
        identity-id (:identity-id decision)
        old-epoch (:recovery-epoch decision)]
    [[:db/cas case-id
      :recovery/version expected-version new-version]
     [:db/cas case-id
      :recovery/state :verified :restored]
     [:db/cas identity-id
      :identity/recovery-epoch old-epoch (inc old-epoch)]
     {:db/id "audit-event"
      :audit/kind :recovery-restored
      :audit/case case-id
      :audit/decision-id (:decision-id decision)
      :audit/actor-id (:actor-id decision)
      :audit/at now}]))
```

Datomic/Clojure sketch with proposed attributes; not executed.

The sketch uses transactional comparisons to prevent a decision from being applied after the version, state, or generation has changed. The data format and attributes are proposed. The aim is to discuss atomicity between the case change, identity change, and audit record without claiming that Nubank uses this schema.

Checking a deadline before submitting the transaction is insufficient if the wait is long. The final implementation would need to guarantee validity at the point of application, according to the capabilities and transaction model selected. The excerpt was not executed in Datomic; this gap is a review requirement, not an implicit guarantee.

Datomic - Transaction Data

EXTERNAL ANALYSIS AND PROPOSAL

Evidence has a different lifecycle from the case

A case identifier may be needed for support longer than an image collected for a specific verification. I would model references and retention policies separately, avoiding copies of sensitive material in logs, general-purpose events, or tracking tools. A completed-stage event should carry the decision, its provenance, and the authorized scope, not an indiscriminately attached bundle of personal documents.

The proposed contract would include case_id, revision, status, expires_at, and reason_category with a controlled vocabulary. Customer-facing details would differ from investigation details. An understandable category such as “we need to confirm some information” can provide guidance without exposing thresholds or signals used to detect abuse. The contract must not promise a deadline operations cannot yet meet; it must distinguish an estimate, a commitment, and the absence of a forecast.

Reopening a case also requires an explicit decision. Reusing an old approval may be unsafe after a device change or a change in risk. Creating a new revision preserves the link to history without treating previous evidence as permanent authorization. In the lab, expiration rejects new transitions; there is no automatic deletion or regulatory policy. Those capabilities would be defined with identity, privacy, and support owners before any use with real people.

ENGINEERING EXPLAINED

Test the rule a shortcut would try to bypass

PYTHON / ORIGINAL PROPOSAL

```python
# Validação dos limites de autoridade e validade.
def test_restore_requires_two_distinct_reviewers(self):
    verified = self.store.transition("case", 0, "verify", "reviewer-a", 101)
    self.assertEqual(verified["state"], "verified")
    with self.assertRaises(ValueError):
        self.store.transition("case", 1, "restore", "reviewer-a", 102)
    result = self.store.transition("case", 1, "restore", "reviewer-b", 102)
    self.assertEqual((result["state"], result["epoch"]), ("restored", 1))

def test_owner_cannot_verify(self):
    with self.assertRaises(ValueError):
        self.store.transition("case", 0, "verify", "customer", 101)
    self.assertEqual(self.store.read("case")["revision"], 0)

def test_expired_case_cannot_restore(self):
    self.store.transition("case", 0, "verify", "a", 101)
    with self.assertRaisesRegex(ValueError, "expired"):
        self.store.transition("case", 1, "restore", "b", 400)
```

Excerpt from the synthetic lab; execution verified in the complete file.

The tests challenge three shortcuts: the same reviewer approving and restoring access, the requester verifying their own identity, and a decision being applied after its deadline. Each rejection must preserve the previous state. The test does not ask whether a screen displayed a message; it verifies that the prohibited mutation did not happen.

These restrictions are governance fixtures for the simulation. A real team could adopt another approval policy, provided the rule was explicit and tested. The important question in a Staff-level review is who has authority over each transition and how that authority remains valid under repetition and delay.

ENGINEERING EXPLAINED

Resuming a case must not repeat restoration

PYTHON / ORIGINAL PROPOSAL

```python
def test_stale_revision_cannot_repeat_verification(self):
    self.store.transition("case", 0, "verify", "a", 101)
    with self.assertRaisesRegex(ValueError, "stale_revision"):
        self.store.transition("case", 0, "verify", "b", 102)

def test_reopen_preserves_case_and_epoch(self):
    self.store.transition("case", 0, "verify", "a", 101)
    reopened = Recovery(self.path)
    self.assertEqual(reopened.read("case")["revision"], 1)
    reopened.transition("case", 1, "restore", "b", 102)
    self.assertEqual(Recovery(self.path).read("case")["epoch"], 1)

def test_cannot_skip_review_or_restore_twice(self):
    with self.assertRaises(ValueError):
        self.store.transition("case", 0, "restore", "b", 101)
    self.store.transition("case", 0, "verify", "a", 101)
    self.store.transition("case", 1, "restore", "b", 102)
    with self.assertRaises(ValueError):
        self.store.transition("case", 2, "restore", "c", 103)
    self.assertEqual(self.store.read("case")["epoch"], 1)
```

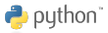
Excerpt from the synthetic lab; execution verified in the complete file.

Reopening the Recovery object uses the same persisted file, demonstrating that the version and generation do not depend on the previous instance’s memory. The other tests reject a stale revision, an attempt to skip verification, and a second restoration.

All seven tests in the file passed, including checks for an invalid clock and revision type. They cover the local machine and its preconditions, but do not exercise concurrency with authenticated operators, real queues, or identity services. Nor do they measure biometric quality. The next implementation experiment would inject duplicate delivery and revocation between approval and application while retaining the same criterion: no access change without a valid decision.

EXTERNAL ANALYSIS AND PROPOSAL

Measure the queue without exposing its members



The central metric would be time to completed legitimate recovery, broken down by stage and technical reason for routing. An average alone would hide people stuck for days while simple cases finish quickly. I would track the tail of waiting times, the age of the oldest case, and repeat-contact rate, as well as the proportion of requests returning to the same stage without useful new information.

A second dimension measures protection: restorations later disputed, rejected attempts to use expired decisions, and blocked commands from old generations. These numbers must not reward a team for denying everything, nor should speed encourage approval without sufficient evidence. The dashboard must show both sides of the decision and support qualitative sample reviews with controlled access to details.

I would avoid identifying people, specific devices, or reviewers in aggregate metric labels. Individual investigations would remain in audit trails with a defined purpose and permissions. For accessibility, useful segmentation would follow the observed barrier, such as an unavailable camera or reading difficulty, without turning collection into intrusive classification. The intended result is to locate the operational and technical bottleneck with enough information to act while retaining the minimum necessary information about each account holder.

EXTERNAL ANALYSIS AND PROPOSAL

Launch exception paths with support prepared

The first delivery would not be a new identity algorithm. It would detect the technical failure earlier and prevent a futile sequence of capture attempts. A message could explain that the component is unavailable and route the person to the existing assisted path. This behavior should be validated with people using broken cameras, denied permissions, and limited-capability devices, including use with assistive technology.

Next, I would introduce case tracking to a controlled cohort. Support would need access to the same state the customer sees, differing only in the permitted level of detail. A migration that creates more states without preparing operators can increase transfers between teams. Each new state would therefore have an operational action, an owner, and an understandable exit criterion.

Automated restoration or identity integration would follow only after authorization, expiration, and revocation had been validated in a test environment. Rollback would keep open cases queryable and prevent reapplication of already confirmed decisions. I would define an operational intake limit so that the new journey did not collect requests faster than the team could review them. The experience must be sustainable for both the customer and the person resolving the exception.

EXTERNAL ANALYSIS AND PROPOSAL

Accessibility is part of the security surface

Ambiguous instructions can make people repeat movements, seek unsuitable environments, or share information through the wrong channels. I would treat journey clarity as a security control: explain what will be requested, where information is sent, and what happens next. That does not mean revealing fraud rules. It means reducing the gap between a legitimate action and an improvised workaround that exposes the customer.

In the proposed Flutter interface, states would need accessible semantics, stable focus order, and announcements that do not rely exclusively on color or animation. A camera failure must not disappear so quickly that a screen reader cannot announce it. Validation would include enlarged text and app interruption, because recovery often takes place under less comfortable conditions than an ideal onboarding flow.

For performance, I would reduce repeated collection and loading steps that do not change the decision. The client could query status at a moderate frequency while relevant updates were propagated recoverably. The priority is not approval at any cost, but removing waits that add no verification. If a stage takes time because it depends on a person, the app should acknowledge that and provide accurate tracking rather than an endless spinner that looks like a technical failure.

Flutter - Accessibility

EXTERNAL ANALYSIS AND PROPOSAL

ADR: an explicit recovery process with limited authorization

The proposed decision is to represent recovery as a domain process separate from everyday login. One alternative would embed every exception in the same capture flow. That would simplify initial navigation, but make it difficult to understand attempts that span support, device replacement, and review. Another alternative would resolve everything manually through messages, providing considerable flexibility but little consistency in state, authorization, or tracking.

I would choose an explicit process because it makes the validity of each decision clear and limits what each step authorizes. Verifying a document need not mean enabling every financial operation. Restored access may require further risk checks under the applicable policy without turning the state machine into a universal shortcut. Those policies would be product and security decisions, not deductions from the public report.

The accepted cost is maintaining contracts between identity, review, and support. The design must also avoid bureaucracy: states should exist only when they change responsibility, authorization, or useful information. I would revisit the ADR if most exceptions could be resolved through reliable local diagnosis, if manual operations became the dominant bottleneck, or if new device capabilities reduced the need for collection. The choice would be reviewed against evidence of legitimate resolution and protection, not the number of steps.

EXTERNAL ANALYSIS AND PROPOSAL

EM execution: the capacity to resolve cases, not merely receive them

As an EM, I would begin with a workshop on real cases already authorized for analysis, bringing together support, identity, risk, mobile, and accessibility. The output would be a matrix of barriers, owners, and existing paths. The broken-camera report would inform one category without being used to claim prevalence. I would separate improvements to clarity, process changes, and authorization changes because they have different risks and dependencies.

The plan would have small deliverables: understandable diagnosis, routing with context, shared state, and transactional integration with identity. Each would require a demonstration under failure, an operational review, and the capacity to track outcomes. As a Staff engineer, I would focus on decision validity across boundaries, session generation, and persistent transitions. The lab is an artifact for technical discussion, not a substitute for security review.

Value could be calculated through support minutes avoided, fewer repeat contacts, and shorter periods without access for legitimate account holders. Any retention gain would require an appropriate comparison, and fraud costs or additional review costs would be deducted. The leadership question would be: “Are we removing a barrier that adds no security, or simply shifting work to another team?” The answer must be visible in case resolution and protection metrics, not just screen conversion.

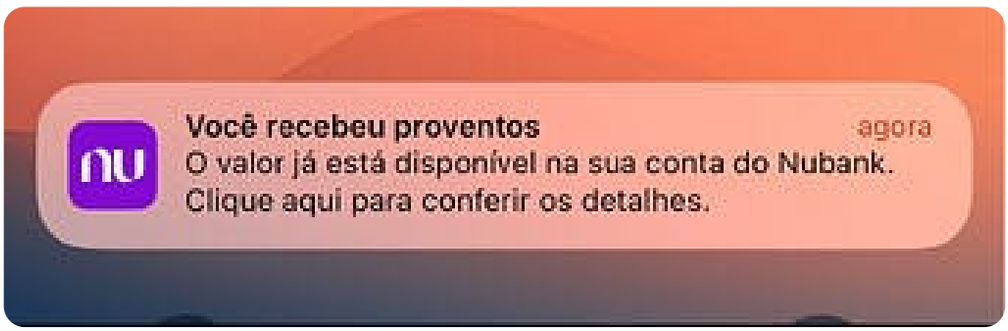
EXTERNAL ANALYSIS AND PROPOSAL

Enough information, deliberate disclosure

03

Notificações Push do App Nubank esquece do valor (sugestões)

Original source screenshot, accessed 23 October 2026. An individual report is not a reproduced incident.



Screenshot published by a customer on NuCommunity, kept in Portuguese. Context and limitations are available in the source.

The public suggestion collects examples of investment income, cashback, and bill notifications that, according to the author, omitted amounts. The attached images provide context for the complaint, but do not reveal all privacy preferences or the version currently available. The opportunity is not to conclude that every notification should display money. It is to build a content policy that helps people decide while respecting the disclosure context they have chosen.

I would begin by asking what task the message enables someone to complete. An alert may confirm that something happened, request an action, or highlight a deadline. Each purpose needs different fields. An amount may prevent an unnecessary app opening; on a locked screen visible to other people, the same amount may disclose information the account holder would prefer to keep private. The experience must make this choice understandable rather than accidental.

This chapter’s proposal combines an event contract, a message catalog, versioned preferences, and late enforcement of the privacy policy. The lab queues synthetic facts, changes the preference before sending, and verifies the final content. It does not connect to push providers. Its purpose is to show that the moment at which a policy is evaluated is part of system correctness, just as much as the sentence displayed on screen.

NuCommunity - Nubank app push notifications omit the amount (suggestions)

EXTERNAL ANALYSIS AND PROPOSAL

A financial fact is not notification copy

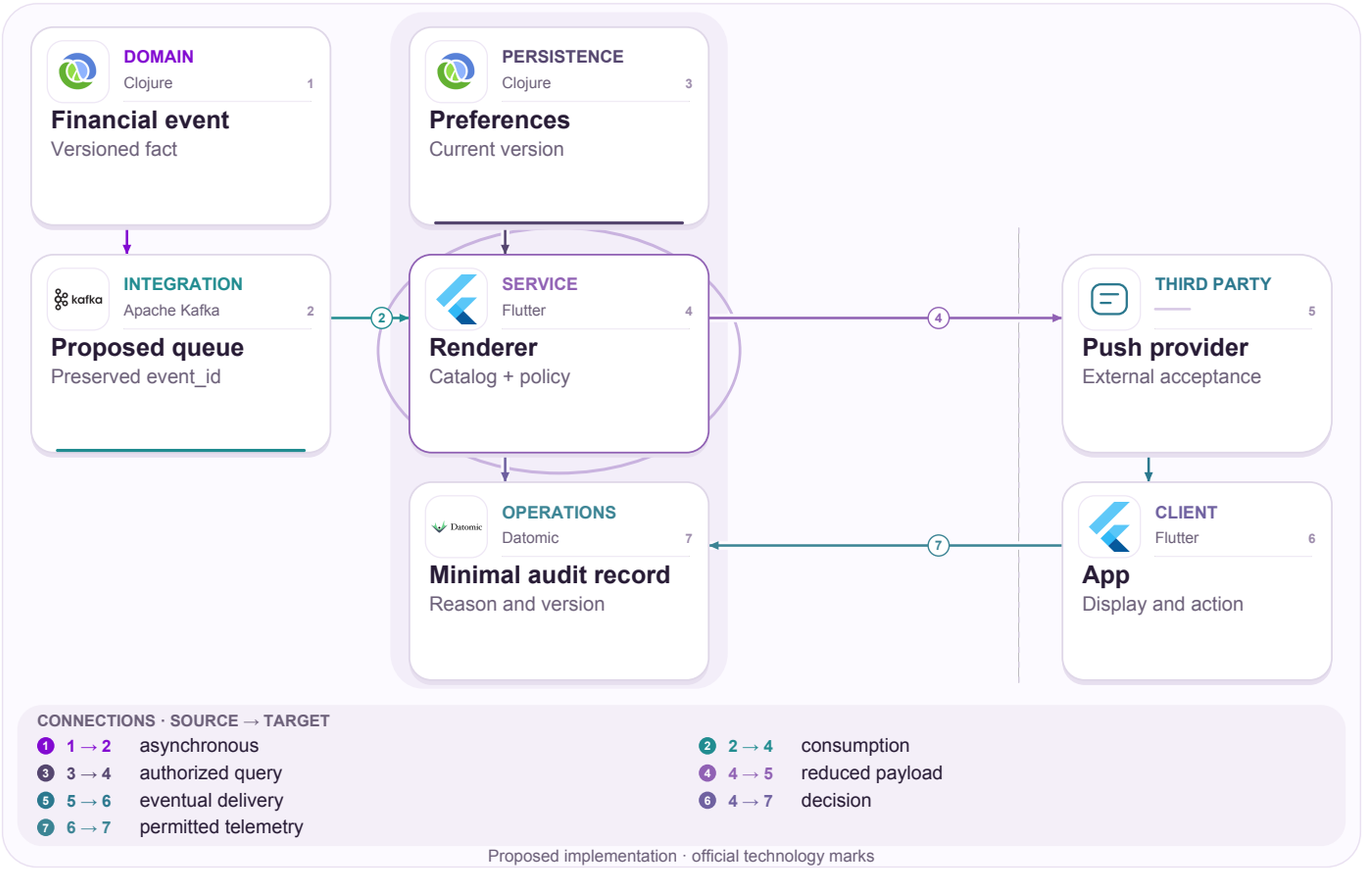
A domain event should describe what happened without embedding a ready-made sentence for a particular language or device. The communication catalog interprets that fact, decides whether there is a relevant message, and applies the presentation policy. Separating these responsibilities allows copy to be corrected without changing event semantics and privacy to evolve without requiring every financial producer to understand each mobile platform's particularities.

The proposed contract contains event_id, schema version, event type, account holder, and only the necessary data. Preferences have a different lifecycle: they can change while a message waits in the queue. Storing final copy with an exposed amount when the event is produced would therefore be insufficient. The proposal reads the current policy close to delivery and includes its version in the operational evidence.

There are at least four communication states: eligible, suppressed, accepted by the provider, and displayed or opened when that information is available. Acceptance does not prove reading, and the absence of an open event does not prove failure. The lab models only local acceptance and suppression. Product decisions must avoid treating "sent" as certainty that the customer learned about a financial fact or had an effective opportunity to act.

PROPOSED ARCHITECTURE • EXPLICIT BOUNDARIES

Render close to delivery and record the decision



Original proposed design; it does not represent Nubank's internal architecture.

The renderer is the boundary where a financial fact becomes text potentially visible outside the app. It needs the current preference and catalog version. If the preference query is unavailable, the proposed policy falls back to content without an amount or suppresses the message, depending on its purpose. Consent is not assumed because the preferences service is down.

The queue preserves event identity and version; duplicates must not generate repeated messages indefinitely. The audit record captures the communication decision and its policy parameters, avoiding storage of the sensitive body when a controlled reference is enough. The external provider remains an important boundary: after it accepts a payload, withdrawing it may no longer be possible.

Kafka would be an asynchronous transport option, but would not replace deduplication of the effect at the provider boundary. The lab keeps its queue and deduplication in memory to isolate content rules, without claiming durability or actual delivery.

ENGINEERING EXPLAINED

Apply privacy at the last controllable point

PYTHON / ORIGINAL PROPOSAL

```python
def deliver(self, event_id):
    if event_id in self.sent:
        return self.sent[event_id]
    event = json.loads(self.events[event_id])
    pref = self.preferences.get(event["user"],
                                dict(show_values=False, enabled=True, version=0))
    if not pref["enabled"]:
        return {"status": "suppressed"}
    # A política é aplicada agora, não no instante de enfileirar.
    message = {"title": "Atualização financeira",
               "body": "Consulte os detalhes no aplicativo.",
               "preference_version": pref["version"]}
    if pref["show_values"]:
        whole, cents = divmod(event["amount_cents"], 100)
        message["body"] = f"Crédito disponível: R$ {whole},{cents:02d}."
    # Aceitação local pelo provedor simulado, sem garantia de leitura.
    self.sent[event_id] = {"status": "accepted", "message": message}
    return self.sent[event_id]
```

Excerpt from the synthetic lab; execution verified in the complete file.

The method first recognizes a delivery already accepted in the model and avoids repeating it. For a pending message, it reads the current preference. The default hides the amount; disclosure requires explicit enablement. Monetary formatting uses divmod on integer cents, without binary floating-point rounding.

The “accepted” result is only the simulated provider’s response. After that point, changing the preference cannot recall a message already delivered, and the lab makes no such promise. A real implementation would also need to bound the interval between reading the preference and sending, handle an unavailable read, and document the revocation boundary.

ENGINEERING EXPLAINED

Clojure: a message catalog as data

CLOJURE / ORIGINAL PROPOSAL

```clojure
(def templates
  {:income {:title "Atualização financeira"
            :private-body "Consulte os detalhes no aplicativo."}
   :cashback {:title "Cashback disponível"
              :private-body "Veja a atualização no aplicativo."}})

(defn cents-label [cents]
  (str "R$ " (quot cents 100) ","
       (format "%02d" (mod cents 100))))

(defn render-message [event preference]
  ;; Catálogo e contrato propostos; sem envio neste exemplo.
  (when-not (= 1 (:schema-version event))
    (throw (ex-info "unsupported-schema" {})))
  (when-not (true? (:enabled? preference))
    (throw (ex-info "communication-suppressed" {})))
  (let [template (get templates (:kind event))
        amount (:amount-cents event)]
    (when-not (and template (integer? amount) (<= 0 amount))
      (throw (ex-info "invalid-event" {})))
    {:title (:title template)
     :body (if (true? (:show-values? preference))
             (str "Crédito disponível: " (cents-label amount))
             (:private-body template))
     :policy-version (:version preference)}))
```

Proposed catalog and function; Clojure not executed.

The example makes the catalog inspectable and keeps the privacy decision in a function separate from transport. This facilitates tests of wording, amounts, and preference combinations. The field names and sentences are proposals in this book, not actual Nubank contracts.

A production catalog would need governance for translation, accessibility, and purpose, alongside disclosure review. The function does not resolve the race between revocation and sending, nor does it replace authorization to query account-holder information. It must not receive unvalidated monetary values from arbitrary sources either. This Clojure excerpt is educational and was neither compiled nor executed.

EXTERNAL ANALYSIS AND PROPOSAL

Version content without breaking older consumers

The event schema and message version solve different problems. The former defines fields and meaning; the latter identifies how those fields were presented. A title change may be compatible with the same event, whereas introducing a new financial state may require a consumer update. I would record these versions separately so that investigations do not have to reconstruct communication from the code running in production today.

In transport, event_id must remain stable when a producer repeats publication. Deduplication must detect the same identifier reappearing with different content, since that signals a contract error. The lab rejects this reuse. For a real persistent queue, I would define retention, partitioning, and handling of unrecognized events, ensuring that a producer update could not silently suppress critical messages without alerting the owner.

Preferences also need versions to explain which policy was applied. That does not authorize storing a complete history of personal information inside the event. A minimal operational record can reference a policy and a decision, with restricted access to details when necessary. In the customer's settings, the configuration must have a stable meaning: "hide amounts in notifications" must not accidentally hide only some categories. Any exceptions should be explicit and justified.

ENGINEERING EXPLAINED

Reject ambiguity before generating copy

PYTHON / ORIGINAL PROPOSAL

```python
def configure(self, user, show_values=False, enabled=True):
    if type(show_values) is not bool or type(enabled) is not bool:
        raise ValueError("boolean_preferences_required")
    old = self.preferences.get(user, {"version": 0})
    self.preferences[user] = dict(show_values=show_values, enabled=enabled,
                                  version=old["version"] + 1)

def enqueue(self, event):
    if set(event) != {"id", "user", "version", "kind", "amount_cents"}:
        raise ValueError("schema")
    if (type(event["version"]) is not int or event["version"] != 1
            or event["kind"] not in ("cashback", "income")):
        raise ValueError("unsupported")
    cents = event["amount_cents"]
    if type(cents) is not int or cents < 0:
        raise ValueError("money")
    payload = json.dumps(event, sort_keys=True)
    old = self.events.get(event["id"])
    if old is not None and old != payload:
        raise ValueError("id_conflict")
    self.events[event["id"]] = payload
```

Excerpt from the synthetic lab; execution verified in the complete file.

Input validation checks the set of fields, integer version, category, and monetary type. Reusing event_id with different content creates a conflict. Preferences require actual booleans: the string false cannot be interpreted as permission to expose amounts. A valid configuration increments the preference version, making it possible to recognize that a communication decision occurred under a different policy.

The model is deliberately strict: a producer adding fields must negotiate the contract instead of relying on silent acceptance. That choice suits the lab; a real platform could adopt extensible compatibility with well-defined rules. The in-memory queue does not survive a restart, so the tests on this page verify semantics, not transport reliability.

ENGINEERING EXPLAINED

A preference changes between production and delivery

PYTHON / ORIGINAL PROPOSAL

```python
def test_privacy_changed_while_queued_is_respected(self):
    self.n.configure("u", show_values=True)
    self.n.enqueue(self.event)
    self.n.configure("u", show_values=False)
    self.assertNotIn("123", str(self.n.deliver("e1")))

def test_opt_out_suppresses_delivery(self):
    self.n.enqueue(self.event)
    self.n.configure("u", enabled=False)
    self.assertEqual(self.n.deliver("e1"), {"status": "suppressed"})
    self.assertEqual(len(self.n.sent), 0)

def test_replay_is_identical(self):
    self.n.enqueue(self.event)
    first = self.n.deliver("e1")
    self.n.enqueue(self.event)
    self.assertEqual(self.n.deliver("e1"), first)
    self.assertEqual(len(self.n.sent), 1)

def test_payload_conflict_is_rejected(self):
    self.n.enqueue(self.event)
    with self.assertRaises(ValueError):
        self.n.enqueue(dict(self.event, amount_cents=999))
```

Excerpt from the synthetic lab; execution verified in the complete file.

The privacy test changes the preference after the event is queued. That ordering reproduces a window an isolated screen test would miss. Another scenario disables communication and verifies that the message does not enter the set of accepted deliveries. Retry and conflict tests complete the protection without depending on an external provider.

The file passed eight tests, including rejection of preferences and versions with incorrect types. The evidence does not cover lock-screen presentation, screen readers, or operating-system behavior. Those layers would need device tests and research. The lab demonstrates only that the local renderer respects the preference it reads before its simulated delivery.

EXTERNAL ANALYSIS AND PROPOSAL

Measure usefulness without optimizing for curiosity

Open rate is an ambiguous signal for financial notifications. A clear message may reduce opens because it already answered the question; an incomplete message may increase them because it forces the customer to investigate. I would not use opens as a standalone objective. Evaluation would begin with comprehension: can the person identify what happened, know whether they need to act, and recognize where to find details without misinterpreting the situation?

The proposed operational metrics would separate eligible events, messages suppressed by preference, provider acceptance, and rendering failures. Increased suppression may be the correct behavior after a preference change, not a regression. Alerts would look for unexpected deviations by category and version using a low-cardinality vocabulary. Financial amounts, recipients, and full message bodies would not become metric labels.

In a product experiment, I would measure queries motivated solely by missing information, contacts associated with the message, and adherence to preferences. Participants could explain the message without executing an operation, reducing test risk. The experience must also be evaluated on locked screens, with enlarged fonts, and with screen readers. Success would mean sufficient information and deliberate disclosure, not maximizing message volume or time spent in the app.

EXTERNAL ANALYSIS AND PROPOSAL

Migrate a catalog without overwhelming the customer

Rollout would begin with an inventory of notification types, their producers, available fields, and purposes. Before changing copy, I would identify transactional messages, reminders, and optional communications. This classification informs preferences, fallback, and urgency. The public post is a starting point for a few categories, not a justification for indiscriminate changes to every app message.

I would validate the new renderer in comparison mode: generate the candidate version without sending it, then check field and policy differences using synthetic data or properly authorized, minimized data. A small cohort would then receive a single revised category. Per-recipient and per-event limits would prevent an integration failure from causing mass repetition. Support would have examples of old and new messages to recognize questions arising during the transition.

Rollback must preserve event_id and delivery records. Reprocessing the backlog with the previous catalog must not resend everything already accepted. Privacy changes must not be silently reversed when the binary is rolled back either. I would treat preferences as a persistent customer contract independent of renderer version. Expansion would depend on evidence of comprehension and the absence of unexpected disclosure, not technical stability alone.

EXTERNAL ANALYSIS AND PROPOSAL

The message body crosses a new boundary

Once text leaves the app, it may appear on a locked screen, a watch, system history, and provider infrastructure. Minimization must therefore happen before sending, not merely through a visual layer inside the app. Hiding an amount on an internal screen does not remove it from a payload already transmitted. The proposal would keep sensitive content out of the body when the preference does not authorize its disclosure.

Action links also need careful design. An embedded identifier cannot act as a credential for reading a transaction. When opening the message, the app must authenticate the required context and resolve the destination with authorization. I would avoid personal information in URLs and check link behavior after logout, device replacement, or session recovery. An expired destination should provide guidance without exposing old data.

On cost, reading preferences for every delivery may add latency and load. Choosing between a direct read and a cache requires an explicit staleness limit and a conservative policy when that limit cannot be guaranteed. Optimizing this path without modeling revocation could make the system fast and incorrect. Performance would be evaluated alongside preference propagation time, keeping both objectives visible.

EXTERNAL ANALYSIS AND PROPOSAL

ADR: centralized rendering with distributed responsibility

The proposed decision centralizes cross-cutting content and privacy rules while keeping ownership of event meaning in the producing domain. Allowing every team to write its own push messages makes local delivery easier but tends to create variations in wording, fields, and policy. Making a central service decide every financial fact as well would concentrate too much responsibility and weaken domain clarity.

I would choose a governed catalog and per-event contracts. The producing team would guarantee semantics and identity; the communication platform would guarantee rendering, policy, and provider integration; product and design would guarantee comprehension and accessibility. This division would have contract tests on both sides. An unknown message should not automatically become generic copy if doing so concealed an important customer action.

The accepted cost is additional coordination when introducing new event types. The intended benefit is consistent privacy and content changes without reimplementing the rule in every domain. I would revisit the ADR if the catalog became a delivery bottleneck or categories with different guarantees required operational isolation. The solution is not one template for everything: it is a common mechanism for applying explicit choices, with clear ownership of every message and its purpose.

EXTERNAL ANALYSIS AND PROPOSAL

EM and Staff: govern communication as a product

As an EM, I would form a small group spanning communications, mobile, the financial domain, and privacy to review the categories with the greatest friction. The initial delivery would be a catalog with an owner, purpose, required fields, and defined behavior when data is missing. Every change would include authorized real or synthetic examples and a comprehension test. This prevents the project from becoming a sequence of copy edits that never addresses the source of inconsistencies.

As a Staff engineer, I would review event identity, the preference enforcement boundary, and the distinction between provider acceptance and the perceived effect. I would also require a reprocessing strategy that preserves deduplication. Technical contribution becomes especially valuable when several teams publish events and one contract failure can affect communications with different purposes. The lab provides a small foundation for discussing these invariants without claiming knowledge of internal infrastructure.

Value would be expressed through time saved on queries, fewer contacts, and comprehension, including operational and review costs. An increase in opens would not be presented as a benefit if incomplete messages caused it. My interview question would be: “How do we prove that the preference applied to a message was the correct one at the last moment when we could still control its disclosure?” The answer connects product, engineering, and operational responsibility.

EXTERNAL ANALYSIS AND PROPOSAL

A subtotal must acknowledge what it does not know

04

Open Finance: Cofrinhos do Mercado Pago não aparecem no Nubank

Original source screenshot, accessed 23 October 2026. An individual report is not a reproduced incident.

The report dated October 3, 2026 distinguishes two observations: balances and transactions from another institution appeared in Nubank, but Cofrinhos positions did not appear in the investment aggregator. The participant does not claim certainty about the cause. The post therefore cannot establish a Nubank defect or an obligation to provide that resource. The evidence supports investigating coverage, consent, availability, and data interpretation.

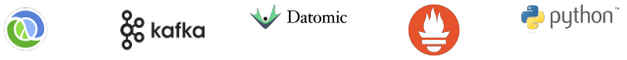
Financial aggregation has two products: the values displayed and an explanation of how complete that view is. Adding only what arrived may be mathematically correct yet semantically misleading if the result appears to represent all assets. My proposal would maintain state by institution and resource, including provenance, freshness, and the reason for absence. Not receiving a position must not automatically be interpreted as a zero-value position.

This chapter's lab models that distinction with complete, unsupported, and unavailable states, alongside missing or stale resources. When there are gaps, the sum is explicitly a known subtotal. There is no connection to real banking APIs. The demonstration makes an important product decision verifiable: partial information can remain useful, provided the app does not convey more certainty than the available evidence supports.

NuCommunity - Open Finance: Mercado Pago Cofrinhos do not appear in Nubank

EXTERNAL ANALYSIS AND PROPOSAL

Completeness is a property of the requested set



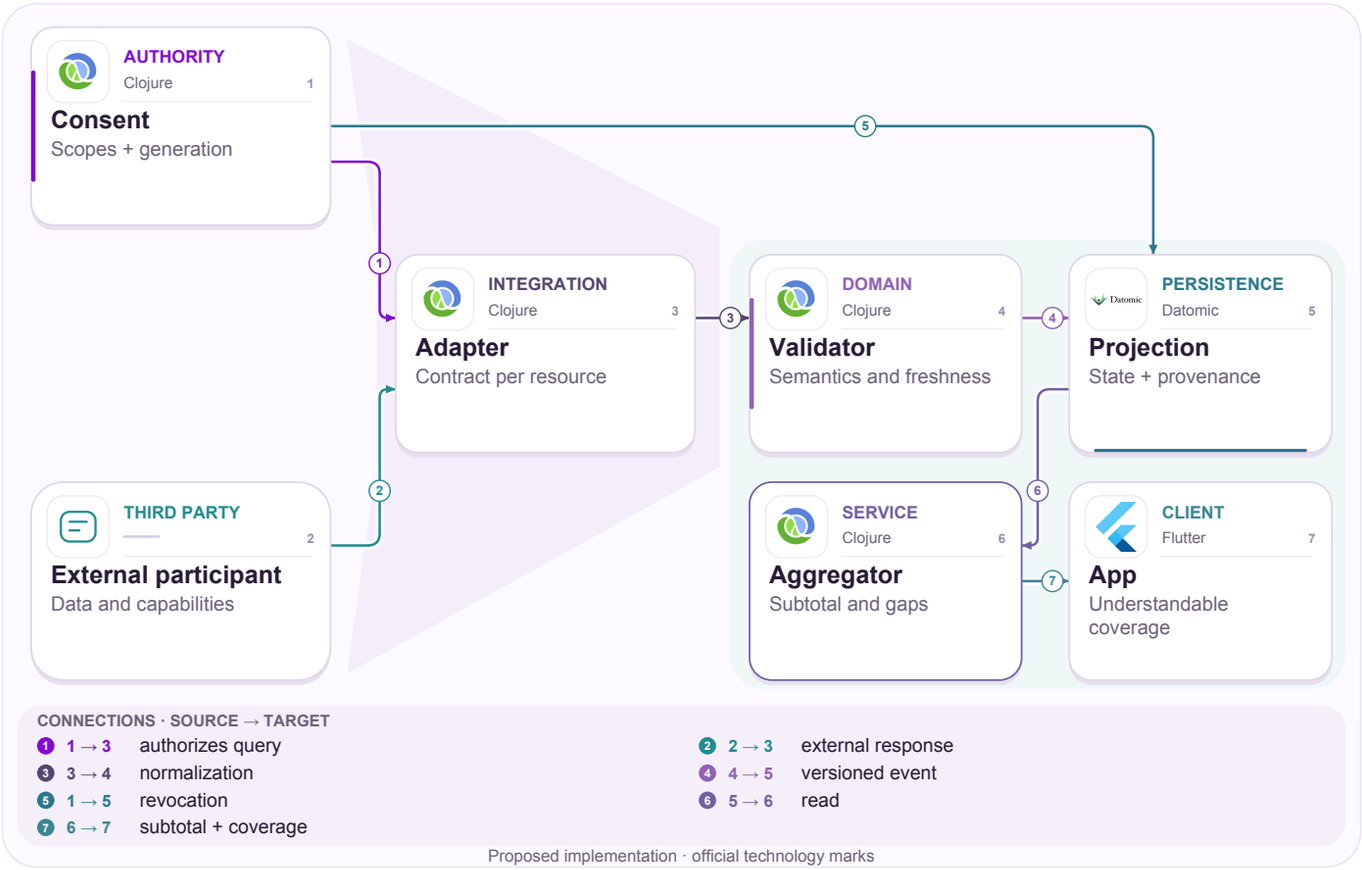
A complete resource does not make a complete portfolio. The answer depends on the resources expected for that connection and the scope authorized by the customer. If the current account was synchronized but investments were not queried, the system must preserve that distinction. I would represent a coverage matrix combining participant, product type, consent, and integration capability, avoiding a single “connected” indicator that conceals different situations.

Zero means a valid source reported no value for that resource. Unsupported means the resource is outside the capability being considered. Unavailable means the capability exists, but the query did not return a usable answer at that moment. Missing means there is not yet sufficient observation. These states may lead to different actions: explaining coverage, waiting, retrying a query, or reviewing consent without forcing the customer to disconnect everything.

Freshness must also be defined. A position valid yesterday may be useful as history and unsuitable for a total presented as current. The proposal stores observed_at and an orderable version per resource, with an age limit determined by purpose. The lab excludes values outside that limit from the subtotal and records the gap. A real experience could show the last known value separately, provided its age and meaning were unambiguous.

PROPOSED ARCHITECTURE • EXPLICIT BOUNDARIES

Provenance by resource, not a generic connection badge



Original proposed design; it does not represent Nubank's internal architecture.

Normalization happens per resource to preserve differences between external responses. The adapter does not decide that absence means zero; it supplies data and context for validation. The projection retains the result together with its source, version, and observation time. The aggregator combines only values that satisfy the freshness policy and returns gaps as part of the contract, not as a detail hidden in logs.

Consent spans the entire flow. A response already in transit before revocation must be recognized as belonging to the old generation and must not silently repopulate the view. The lab uses consent_epoch to demonstrate that barrier, but does not implement actual consent protocols.

The design makes it possible to investigate where information disappeared: capability, authorization, response, transformation, or presentation. This separation helps teams work with partners without assigning blame prematurely. The interface receives enough information to explain the boundary and offer an action proportionate to the observed gap.

ENGINEERING EXPLAINED

Add known values without hiding gaps

PYTHON / ORIGINAL PROPOSAL

```python
# max_age mede a idade em segundos simulados.
# Uma lacuna não recebe um valor zero substituto.
# A interface deve preservar complete e gaps.
def summarize(self, now, max_age=60):
    total, gaps = 0, {}
    if not self.active:
        return dict(cents=None, complete=False, gaps={"consent": "revoked"})
    for key in sorted(self.required):
        item = self.resources.get(key)
        if item is None:
            gaps[key] = "missing"
        elif item.status != "complete":
            gaps[key] = item.status
        elif not 0 <= now - item.observed_at <= max_age:
            gaps[key] = "stale_or_future"
        else:
            total += item.cents
    return dict(cents=total, complete=not gaps, gaps=gaps)
```

Excerpt from the synthetic lab; execution verified in the complete file.

The function walks the expected set and assigns a reason to each resource that cannot contribute. A valid zero participates normally in the sum; a missing position is not replaced with zero. The complete field depends on there being no gaps, and revocation returns an unavailable value rather than an apparently empty total.

A zero subtotal with complete=false needs explicit presentation: “known subtotal,” accompanied by the gaps. The lab does not implement the interface. It also uses a uniform age limit to simplify testing; in a real product, the freshness policy would be defined by resource type and purpose.

ENGINEERING EXPLAINED

Clojure: partial results as first-class data

CLOJURE / ORIGINAL PROPOSAL

```clojure
(defn usable? [now max-age resource]
  (and (= :complete (:status resource))
       (integer? (:cents resource))
       (<= 0 (- now (:observed-at resource)) max-age)))

(defn aggregate [expected resources now max-age]
  ;; Contrato proposto: total conhecido sempre vem com cobertura.
  (reduce
    (fn [acc key]
      (let [resource (get resources key)]
        (if (usable? now max-age resource)
          (update acc :known-cents + (:cents resource))
          (assoc-in acc [:gaps key]
            (cond
              (nil? resource) :missing
              (not= :complete (:status resource))
              (:status resource)
              :else :stale-or-future)))))
    {:known-cents 0 :gaps {}}
    expected))

(defn presentation [result]
  (assoc result :label
    (if (empty? (:gaps result))
      :covered-total :known-subtotal)))
```

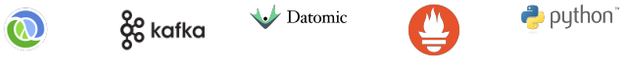
Proposed teaching contract and functions; Clojure not executed.

The proposed structure carries gaps alongside the subtotal. This reduces the likelihood of an intermediate layer discarding the context and presenting only a number. The presentation function selects a label without changing the sum, making it clear that completeness is a semantic decision distinct from arithmetic.

The excerpt assumes previously validated resources and does not implement consent, persistence, or event ordering. It was not executed in Clojure. Its role in this chapter is to explore the contract between aggregation and presentation. A real implementation would need to test older consumers to ensure they do not ignore gaps and display known-cents as complete assets.

EXTERNAL ANALYSIS AND PROPOSAL

A coverage contract requires a shared vocabulary



The proposed response would contain known_cents, coverage_complete, gaps, and freshness references for each resource. Currency would also be mandatory: adding different currencies without an explicit conversion would produce a meaningless number. The lab assumes a single unit in cents and does not handle foreign exchange. This restriction is stated in scope to prevent the example's simplicity from being mistaken for a complete asset aggregator.

Each gap would need a stable reason that the app can present and support can investigate. A participant's technical codes should not flow directly into customer-facing copy. The adapter would map them into domain categories while retaining diagnostic details in a controlled channel. When the cause is uncertain, the category must acknowledge that uncertainty instead of automatically blaming the user or the external institution.

Contract evolution also includes new resources. The expected set must be versioned or derived from a known capability, because adding a category may turn a previously complete view into a partial one. That may be correct, but it needs explanation. I would treat the coverage matrix as an operational product with an owner, tests, and review, not an informal exception list maintained only by support.

ENGINEERING EXPLAINED

Reject old events and preserve meaning

PYTHON / ORIGINAL PROPOSAL

```python
# A geração acompanha o recurso desde a solicitação.
def apply(self, key, resource):
    if not self.active or resource.consent_epoch != self.consent_epoch:
        return False
    if type(resource.sequence) is not int or resource.sequence < 0:
        raise ValueError("integer_sequence_required")
    if key not in self.required:
        raise ValueError("unknown_resource")
    if resource.status not in ("complete", "unsupported", "unavailable"):
        raise ValueError("status")
    if resource.status == "complete":
        if type(resource.cents) is not int or resource.cents < 0:
            raise ValueError("amount")
    elif resource.cents is not None:
        raise ValueError("unknown_is_not_zero")
    old = self.resources.get(key)
    if old and resource.sequence <= old.sequence:
        return False
    self.resources[key] = resource
    return True
```

Excerpt from the synthetic lab; execution verified in the complete file.

The method applies the consent barrier before considering the content. It then validates the resource and requires unknown states to carry cents=None. Per-resource versions prevent a delayed response from replacing a newer observation. The boolean result indicates whether the projection changed.

The lab ignores equal or older versions. A real implementation should investigate the same version arriving with different content, for example through a fingerprint comparison, rather than treating the anomaly as a normal duplicate. The sequence must also come from a trusted contract or be assigned at the correct ingestion point; arrival time alone does not necessarily establish business order.

ENGINEERING EXPLAINED

Four tests against false precision

PYTHON / ORIGINAL PROPOSAL

```python
def test_partial_is_not_complete(self):
    self.p.apply("account", self.resource())
    self.assertEqual(self.p.summarize(110),
                     dict(cents=100, complete=False,
                          gaps={"investment": "missing"}))

def test_zero_is_a_real_complete_position(self):
    self.p.apply("account", self.resource())
    self.p.apply("investment", self.resource(0))
    self.assertTrue(self.p.summarize(110)["complete"])

def test_unsupported_requires_none(self):
    with self.assertRaises(ValueError):
        self.p.apply("investment", self.resource(0, status="unsupported"))
    self.p.apply("investment", self.resource(None, status="unsupported"))
    self.assertEqual(self.p.summarize(110)["gaps"]["investment"], "unsupported")

def test_old_or_duplicate_events_do_not_overwrite(self):
    self.p.apply("account", self.resource(200, sequence=2))
    self.assertFalse(self.p.apply("account", self.resource(100)))
    self.assertFalse(self.p.apply("account", self.resource(300, sequence=2)))
    self.assertEqual(self.p.summarize(110)["cents"], 200)
```

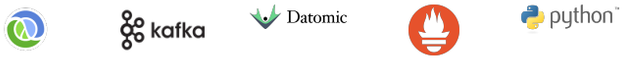
Excerpt from the synthetic lab; execution verified in the complete file.

The scenarios separate four situations that may look similar: only some resources have arrived, a resource reported zero, a resource is unsupported, and an old version arrived after a newer one. The sum alone cannot distinguish them, so the tests also check coverage and the reason for each gap.

The complete file passed eight tests, covering an invalid sequence, expired observation, future timestamp, and revocation with an event in transit. No real Open Finance query was made. The evidence demonstrates the local contract and provides fixtures for a future integration, which would need to validate scopes, schemas, and the behavior of authorized participants.

EXTERNAL ANALYSIS AND PROPOSAL

Observability that locates the incomplete boundary



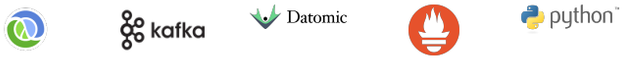
The proposed dashboard would start with the proportion of connections with complete coverage and the distribution of gap reasons. It would then show observation age, validation failures, and responses discarded because of consent generation. A technically active connection with only some resources synchronized should not be counted in the same way as a complete, current view. The metric must reflect the contract delivered to the customer.

I would segment by participant and resource class where cardinality is controlled, avoiding identification of individual accounts. A decline concentrated in one resource may indicate a contract change; widespread failures may point to shared infrastructure or an internal policy. These signals guide investigation but do not establish a cause without evidence. The runbook would check authorization, availability, received response, transformation, and presentation in that order.

I would also measure repeated reconnections, because asking users to renew consent may be the wrong solution for an unsupported resource. The objective is to reduce futile actions and diagnosis time. Alerts must distinguish expected data absence from a coverage regression. A new product that is not yet supported should not trigger the same incident as the interruption of a previously working integration.

EXTERNAL ANALYSIS AND PROPOSAL

Launch transparency before promising broader coverage



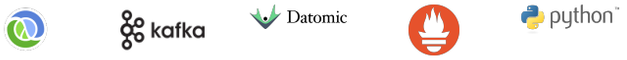
The first delivery could improve the subtotal label and show the last update without adding new participants or products. That would require confirming that provenance data already exists or instrumenting its collection safely. Displaying an arbitrary date merely to fill the screen would create another false certainty. The displayed information must correspond to observation, processing, or query time, with a defined meaning.

Next, I would validate the coverage matrix with authorized test accounts and known scenarios. Each participant-resource combination would have cases for a positive value, zero, missing permission, unavailability, and a delayed response. Gradual rollout would allow comparison of old and new classifications, checking that consumers preserve the meaning of partial. A change in the apparent total would need careful explanation so it did not look like money had disappeared.

Rollback must not restore an incomplete number as though it were complete. A presentation flag can change how the explanation is shown, but must not erase coverage semantics. I would preserve that invariant in the contract. Support for new resources would expand through a separate project with its own integration dependencies and acceptance criteria, keeping transparency distinct from promises of capability not yet demonstrated.

EXTERNAL ANALYSIS AND PROPOSAL

Revoked consent and a delayed response



An integration may be querying data when the customer revokes consent. If the system checks state only when the request begins, a delayed response can repopulate a projection that was just cleared. The proposed generation handles the logical part of that problem: the response carries the context under which it was requested, and the update handler checks that the context is still valid before publishing the change.

The lab clears resources on revocation and rejects events from an old generation. A real implementation would need to define handling of copies, caches, history, and applicable retention obligations with privacy and security owners. The example does not establish legal rules. It makes a technical condition explicit: the decision to display data must respect the authorization valid at the point of use, not only at the start of asynchronous work.

For performance, I would avoid querying every institution on each app opening. Projections can provide fast reads if they carry provenance and freshness limits. The cost is handling partial states and delays. The benefit does not justify hiding those states. The architecture question would be which resources tolerate which age for each purpose, preserving a useful experience without presenting an old observation as the current state.

EXTERNAL ANALYSIS AND PROPOSAL

ADR: explicit completeness instead of filling gaps with zero

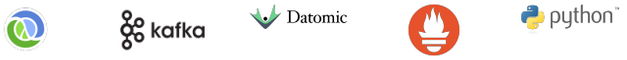
The proposed decision keeps absence and zero semantically distinct. Filling everything with zero simplifies the sum and the interface but destroys coverage information. Hiding the whole aggregator whenever one resource is missing avoids false precision, yet removes the usefulness of valid data and may make the experience unstable. I would choose a known subtotal accompanied by gaps and freshness information, with tested presentation.

This decision requires consumers to treat partial state as a normal condition. Adding an optional field and hoping every screen uses it correctly is not enough. Contracts and integration tests must demonstrate that a partial value does not appear under a complete-total label. It is also necessary to define when the experience stops being useful, for example when no relevant resource is current, so it can show unavailability rather than an empty subtotal without context.

The accepted cost is greater complexity in the domain and its communication. The intended return is trust and faster diagnosis. I would revisit the ADR if a stronger coverage guarantee became available or research showed that the chosen vocabulary confused customers. I would change presentation and contract together while preserving the fundamental distinction between what was observed, what was not observed, and what is outside scope.

EXTERNAL ANALYSIS AND PROPOSAL

Leadership across institutions, product, and support



As an EM, I would structure the work into two tracks: quality of explanation and broader coverage. The first can progress through state and provenance; the second depends on participant contracts and capabilities. Combining them creates promises the team does not control. An integration owner would maintain the resource matrix while product and support validated whether gap reasons can guide people without unnecessary reconnections.

As a Staff engineer, I would review the consent boundary, update ordering, and subtotal semantics. Investigation would begin with the evidence received, not an assumed defect at the partner or Nubank. This approach improves technical discussion and makes it easier to demonstrate which layer needs to change. The lab provides small examples every team can reproduce without access to real financial data.

Value would be estimated through fewer support requests, avoided reconnections, and diagnosis time. Retention or incremental activation would enter the assessment only after an appropriate experiment. My interview question would be: “When we display an aggregate number, what completeness contract does that number carry, and how does the customer understand its boundaries?” A good answer combines data modeling, experience, integration, and operational responsibility.

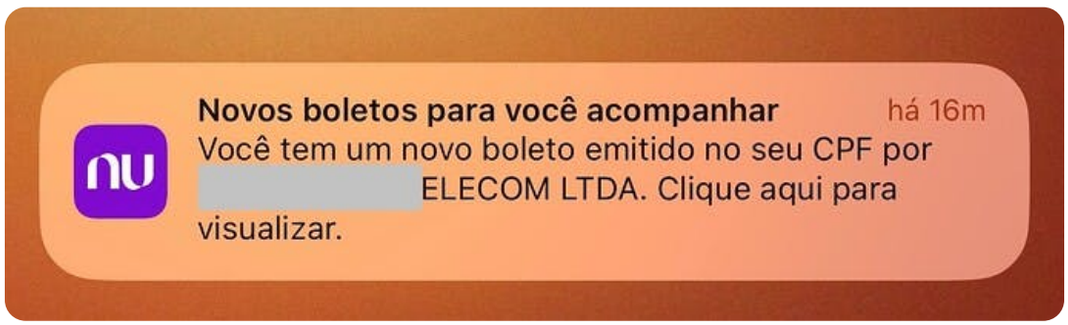
EXTERNAL ANALYSIS AND PROPOSAL

Organize the next payment decision

05

Assistente de pagamentos do Nubank precisa de melhorias

Original source screenshot, accessed 23 October 2026. An individual report is not a reproduced incident.



Screenshot published by a customer on NuCommunity, kept in Portuguese. Context and limitations are available in the source.

The NuCommunity post raises concerns about reminder timing, identifying the next due date, and navigating between months. It describes one participant's experience and includes attached screenshots; it does not establish the product's current state or demonstrate that a notification caused a late payment. The product question is more specific: does the assistant help someone recognize their next obligation without reconstructing a mental schedule from several screens?

I would treat viewing due dates and executing payments as different journeys. The first can reduce effort through ordering, wording, and state updates. The second involves authorization, funds, confirmation, and financial consequences. Adding a better list does not authorize payment automation, and increasing reminder volume does not guarantee organization. The project must preserve that boundary to improve the experience without introducing unexpected actions.

The proposal organizes obligations by civil date, makes the next due date explicit, and applies time preferences to reminders. The lab tests month changes, payment before sending, duplicates, and time-zone rules, including ambiguous or nonexistent times in another country. These scenarios do not describe confirmed Nubank problems; they are engineering conditions a scheduling solution must resolve predictably before expansion.

NuCommunity - Nubank's payment assistant needs improvements

EXTERNAL ANALYSIS AND PROPOSAL

A due date is a civil date; sending happens at an instant

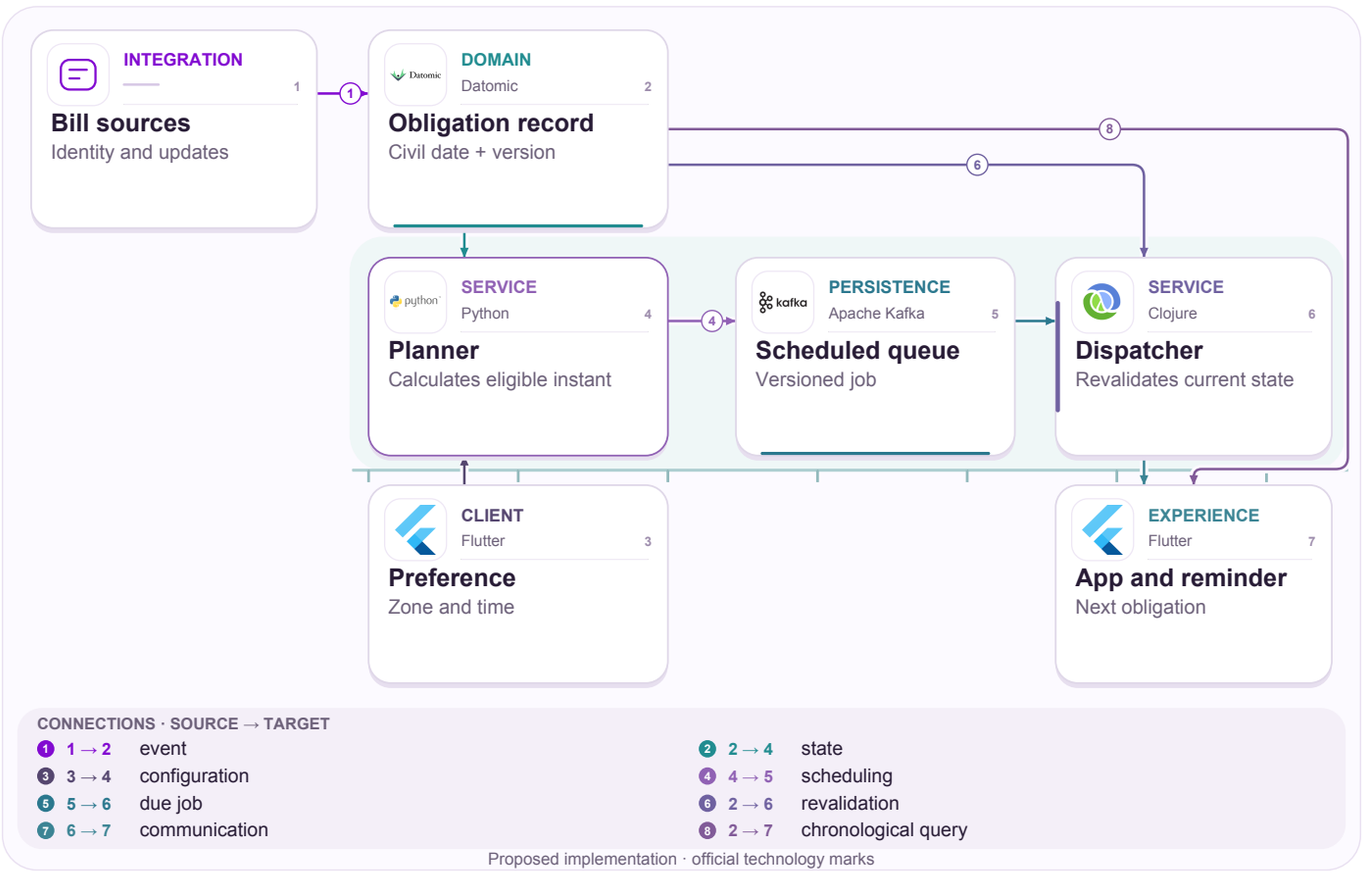
A bill due on the first of November has a business date. A reminder sent at nine in the morning has an instant that depends on the selected time zone. Storing both as though they were the same timestamp creates ambiguity: converting midnight UTC into another zone can shift the date the customer understands as the due date. I would keep due_date as a civil date and reminder_at as an instant calculated from an explicit preference and zone.

An obligation also has an identity and version. If its amount, due date, or state changes, a previously generated job may become stale. The dispatcher must compare the current version before sending. A paid state must not depend solely on an interface click; it needs to indicate the source and certainty of the available confirmation. When a bill is paid elsewhere and that has not yet been observed, the message should avoid asserting a debt as a definitive fact.

The lab uses increasing versions and invalidates a reminder when the obligation has been updated or paid. It does not integrate DDA, a banking calendar, holidays, or settlement. This reduction makes temporal discipline testable without mixing in external rules. The proposed interface could show the “next known due date” and allow details to be checked, keeping the information source and the remaining action clear.

PROPOSED ARCHITECTURE • EXPLICIT BOUNDARIES

A schedule with explicit clocks and versions



Original proposed design; it does not represent Nubank's internal architecture.

The planner turns a business date and a preference into a sending instant. The queue retains the obligation version used in that calculation. When the job becomes due, the dispatcher checks the current state: a paid, changed, or canceled bill makes the old work obsolete. The chronological query path does not depend on executing a payment or sending a message.

This proposed architecture accepts that a queue may deliver late or more than once. Reminder identity prevents local duplication, while a validity window prevents a message from arriving the following day after losing its context. Delay policy should vary by purpose; the lab chooses to expire a reminder when the civil date changes.

The test clock is passed as a parameter, allowing date boundaries to be reproduced without waiting for real time. Production operations would need to consider persistence, clock synchronization, preference changes, and notification-provider behavior.

ENGINEERING EXPLAINED

Resolve times that do not exist on the local clock

PYTHON / ORIGINAL PROPOSAL

```python
# day é uma data civil já validada.
# zone usa regras IANA, não um offset fixo.
# A ida e volta detecta horários inexistentes.
# fold considera as duas ocorrências de uma dobra.
def local_instant(day, hour, minute, zone):
    # Política proposta: primeiro instante válido; na dobra, primeiro UTC.
    naive = datetime.combine(day, time(hour, minute))
    for shift in range(181):
        candidate = naive + timedelta(minutes=shift)
        valid = []
        for fold in (0, 1):
            instant = candidate.replace(tzinfo=zone, fold=fold).astimezone(UTC)
            roundtrip = instant.astimezone(zone).replace(tzinfo=None)
            if roundtrip == candidate:
                valid.append(instant)
        if valid:
            return min(valid)
    raise ValueError("no_valid_instant")
```

Excerpt from the synthetic lab; execution verified in the complete file.

The function constructs candidates for the preferred time and checks the round trip between the local zone and UTC. A nonexistent time during a zone transition does not return to the same civil value, so the search advances to the first valid minute. When two occurrences exist, the synthetic policy chooses the earlier UTC instant.

This policy should be an explicit product decision, not an accidental library behavior. The code uses the standard library's zoneinfo and the system's available time-zone database. Tests include Berlin only to exercise transitions; this does not suggest that the Brazilian report involved daylight saving time.

Python - zoneinfo

ENGINEERING EXPLAINED

Dart: a list ordered by due date need not execute anything

DART / ORIGINAL PROPOSAL

```dart
class CivilDate implements Comparable<CivilDate> {
  final int year, month, day;
  const CivilDate(this.year, this.month, this.day);
  int get orderKey => year * 10000 + month * 100 + day;
  @override
  int compareTo(CivilDate other) =>
      orderKey.compareTo(other.orderKey);
}

class BillView {
  final String id;
  final CivilDate due;
  final int cents;
  final bool paid;
  const BillView(this.id, this.due, this.cents, this.paid);
}

List<BillView> upcoming(List<BillView> source) {
  // Datas já validadas na fronteira do contrato proposto.
  final visible = source.where((bill) => !bill.paid).toList();
  visible.sort((a, b) {
    final byDate = a.due.compareTo(b.due);
    return byDate != 0 ? byDate : a.id.compareTo(b.id);
  });
  return List.unmodifiable(visible);
}
```

Proposed presentation model; Dart/Flutter not executed.

The presentation model uses a civil date, preserves money in cents, and orders items with a stable tie-breaker. It does not turn a due date into a UTC time and contains no payment authorization. The resulting list is immutable to reduce unexpected effects between interface components.

The example assumes valid dates received through the contract; the simplified constructor does not validate the calendar. In Flutter, the screen would still need accessible semantics, understandable groups, and loading or refresh states. This is a teaching proposal in Dart, not an executed implementation. The Python lab tests scheduling and sending logic, not this widget or mobile model.

EXTERNAL ANALYSIS AND PROPOSAL

Persist the reminder’s intention, not just a time

A reminder job needs a reference to the obligation, its version, its purpose, and the policy that produced the instant. Storing only “send at nine” makes it difficult to know whether the work remains valid after a due-date change. I would propose an identity composed of obligation, version, and reminder type, with a separate delivery-outcome record. Confirmation of sending does not change the bill’s financial state.

Changing a time preference requires a policy for jobs already created. Recalculating them may be correct if it does not duplicate a previously accepted reminder. Ignoring them until the next cycle may be surprising. The choice should be explained to customers when relevant and applied with a configuration generation. The lab simplifies this dimension by receiving the time at dispatch; a persistent implementation would need versioned preferences and replanning.

A due date may also carry a source and confidence level. Information entered manually, detected automatically, or updated by a partner should not be indistinguishable during an investigation. The interface need not display every technical detail, but the system should preserve them. This makes discrepancies explainable without blaming customers for information that changed after entry, and without promising that a schedule replaces checking the billing document.

ENGINEERING EXPLAINED

Before sending, check whether the job still makes sense

PYTHON / ORIGINAL PROPOSAL

```python
# Validar a versão antes de registrar uma entrega.
def dispatch(self, bill_id, expected_version, now, hour=9, minute=0):
    if now.tzinfo is None:
        raise ValueError("timezone_required")
    bill = self.bills[bill_id]
    if bill["version"] != expected_version or bill["paid"]:
        return "obsolete"
    target = local_instant(bill["due"], hour, minute, self.zone)
    key = (bill_id, expected_version, bill["due"].isoformat())
    if key in self.sent:
        return "duplicate"
    if now < target:
        return "not_due"
    # Não entregar lembrete que ficou preso até outro dia civil.
    if now.astimezone(self.zone).date() != bill["due"]:
        return "expired"
    self.sent.add(key)
    return "sent_locally"
```

Excerpt from the synthetic lab; execution verified in the complete file.

The dispatcher rejects clocks without a time zone, compares versions, and checks whether the bill has already been paid. It then calculates the eligible instant and recognizes a repeated reminder. In the model, a job arriving on a different civil date expires, avoiding urgency messaging that has lost its context.

The sent set remains in memory, so the demonstrated deduplication does not survive a restart or guarantee a single effect at an external provider. For production, I would persist the decision and address the gap between external acceptance and local confirmation. The test isolates temporal semantics so this limitation remains visible rather than being hidden by infrastructure.

ENGINEERING EXPLAINED

The calendar belongs in the test suite

PYTHON / ORIGINAL PROPOSAL

```python
def test_orders_across_month_boundaries(self):
    self.p.upsert("a", date(2026, 10, 31), 999, 1)
    self.assertEqual([key for key, _ in self.p.upcoming()], ["a", "b"])

def test_paid_before_dispatch_suppresses_reminder(self):
    self.p.upsert("b", date(2026, 11, 1), 1234, 2, paid=True)
    self.assertEqual(self.p.dispatch(
        "b", 1, datetime(2026, 11, 1, 12, tzinfo=UTC)),
                     "obsolete")
    self.assertEqual(self.p.upcoming(), [])

def test_gap_advances_to_first_valid_minute(self):
    zone = ZoneInfo("Europe/Berlin")
    value = local_instant(date(2026, 3, 29), 2, 30, zone)
    self.assertEqual(value.astimezone(zone).strftime("%H:%M"), "03:00")

def test_fold_selects_first_occurrence(self):
    zone = ZoneInfo("Europe/Berlin")
    value = local_instant(date(2026, 10, 25), 2, 30, zone)
    self.assertEqual(value, datetime(2026, 10, 25, 0, 30, tzinfo=UTC))
```

Excerpt from the synthetic lab; execution verified in the complete file.

The tests check a list spanning months, a payment observed before sending, and both forms of temporal ambiguity: a nonexistent time and a repeated time. Dates were selected as fixtures and have no connection to customer bills. The expected result makes the calendar policy reviewable.

The file passed nine tests, covering an invalid version, deduplication, expiration of a delayed reminder, and rejection of a clock without a zone. There was no integration with a banking calendar or DDA. In a real delivery, holidays, document rules, and differences between the due date and clearing would require additional contracts and tests without silently changing the date visible to the customer.

EXTERNAL ANALYSIS AND PROPOSAL

Measure the task of finding the next obligation

The primary experience metric would be success in identifying the next relevant bill with the correct amount and due date. I would measure time and interpretation errors on concrete tasks, including month changes and no bills in the current period. Fewer clicks are useful only if the person still understands what they are seeing. A shortcut that reduces navigation but increases due-date confusion does not meet the objective.

Operationally, I would separate planned, sent, update-suppressed, repeated, and delay-expired jobs. More suppressions because of payment may be positive; more expirations may indicate queue delay. Job age and the difference between planned and actual dispatch times help locate the problem. I would also observe messages sent outside preferences, classified by reason, without using personal identifiers as labels.

The effect on late payments requires causal care. People may pay late because of insufficient funds, their own choices, or incorrect information, not merely because they forgot the due date. An experiment should distinguish those conditions before assigning economic value to the assistant. The initial proposal can be validated through organization and comprehension, leaving reduced fees or delinquency as hypotheses to measure rather than promises derived from a visual improvement.

EXTERNAL ANALYSIS AND PROPOSAL

Separate visual migration from scheduling changes

I would first launch the chronological view using the same known obligations while preserving access to month views and details. Research would compare the task of finding the next obligation, not merely aesthetic preference. People with many bills, recurring bills, and bills spread across months would need to participate in validation. The new presentation must not change existing due dates, amounts, or authorizations.

Reminder changes would have a separate rollout. Before sending, the candidate planner would produce decisions in comparison mode to check time, zone, and suppression after payment. The team would review cases where the candidate differs from existing behavior without overwhelming customers with two message versions. A controlled cohort would make inappropriate timing and repeat contacts measurable, with message limits and a clear stop mechanism.

Rollback would preserve delivery records and obligation versions. Reverting the scheduler must not make the backlog reappear as new work. Customer-selected preferences must also remain in force even if the previous presentation lacks some controls. The migration plan would include read compatibility and support guidance explaining why a reminder was sent, suppressed, or expired at each stage.

EXTERNAL ANALYSIS AND PROPOSAL

A useful schedule must remain readable under pressure

People may check due dates while commuting, working, or experiencing poor connectivity. I would prioritize amount, date, and state, keeping charts and secondary explanations from competing with the immediate decision. A sorted list must preserve context when it updates: if a bill disappears because it was marked paid, the interface must avoid shifting focus so that the next action lands on another item without the user noticing.

For accessibility, relative and absolute dates can complement each other. “Tomorrow” is quick to read, while an explicit date prevents ambiguity when revisiting a screenshot or delayed message. Urgency colors need equivalent text, and amounts should be announced understandably. A glass-and-transparency design would be acceptable only if contrast, focus, and readability remained clear across different backgrounds and motion settings.

For performance, I would use pagination or incremental loading when volume required it while retaining a fast query for the next due date. The server can offer an ordered view, but the contract must explain filters and coverage. A partial list must not be presented as evidence that no other bills exist. The assistant’s quality depends on organization and honesty about what has loaded, not just smooth animation.

EXTERNAL ANALYSIS AND PROPOSAL

ADR: civil dates in the domain, instants in operations

The proposed decision keeps due dates as civil dates and calculates notification instants using an explicit zone and preference. Storing everything as UTC timestamps simplifies some queue mechanisms but shifts a business interpretation into implicit conversions. Using only the device's local time makes consistency harder when people change devices, travel, or go offline. I would choose a contract that preserves both representations and their purposes.

The system also needs a policy for clock transitions. The lab advances to the first valid minute and chooses the first occurrence when a time repeats. That policy is demonstrative and should be reviewed for the product. For financial reminders, predictability and avoiding duplicates matter more than an apparently elegant conversion nobody can explain to a customer or support agent.

The accepted cost is storing more context and maintaining calendar tests. The benefit is preventing time-zone changes from altering the meaning of a due date. I would revisit the ADR if the app expanded to more countries, obligations had specific contractual times, or preferences needed to follow travel automatically. These changes require product and data decisions, not merely switching date libraries.

EXTERNAL ANALYSIS AND PROPOSAL

EM: deliver organization, not a collection of features

As an EM, I would define the main task with product and research: recognize the next obligation and know which action is available. I would then separate the query delivery, reminder review, and any payment evolution. This reduces dependencies and prevents an organizational improvement from becoming a broad automation project without a clear hypothesis. Support would contribute contact reasons and examples of misinterpretation.

As a Staff engineer, I would review the temporal model, obligation identity, and job revalidation. Technical discussions would include queue failures, payments observed after scheduling, and preferences changed during the wait. The lab's nine tests serve as executable examples of those decisions. For a real implementation, I would add persistence and integration tests only after settling the semantics, rather than using infrastructure to hide unanswered domain questions.

Initial value would be time saved per planning session and fewer contacts about locating bills or reminder timing. Financial benefits associated with late payments would require causal measurement. My interview question would be: "When an obligation changes between planning and notification, which version has authority, and how does the customer perceive that change?" The answer connects calendars, consistency, experience, and responsible execution.

EXTERNAL ANALYSIS AND PROPOSAL

Explain why protection remains active

06

Modo Rua precisa de um pequeno ajuste né

Original source screenshot, accessed 23 October 2026. An individual report is not a reproduced incident.

Screenshot published by a customer on NuCommunity, kept in Portuguese. Context and limitations are available in the source.

The participant reports that Modo Rua continued requesting biometrics while connected to a registered Wi-Fi network when location permission had not been granted. The post does not provide a platform diagnosis or establish that its interpretation of GPS or battery use is correct. The case is useful for examining the gap between the promise perceived by the customer and the technical signals needed to confirm context.

I would not treat friction as a reason to remove protection. The proposal would present understandable states: context confirmed, context unconfirmed, permission required, or information expired. This can explain why the conservative rule remains in effect without declaring a network safe merely because its name looks familiar. Customers need to know what action they can take and which protection remains, even if they choose not to grant permission.

The lab uses a fictional HMAC-signed contextual proof to test validity, device binding, and generation. It does not detect Wi-Fi, attest network security, or authenticate people. The simulation demonstrates one invariant: a missing, tampered, or expired signal must not be promoted to trust. A decision about a particular challenge is also not authorization to move money.

NuCommunity - Modo Rua needs a small adjustment

EXTERNAL ANALYSIS AND PROPOSAL

Observed context, granted trust, and authorized action

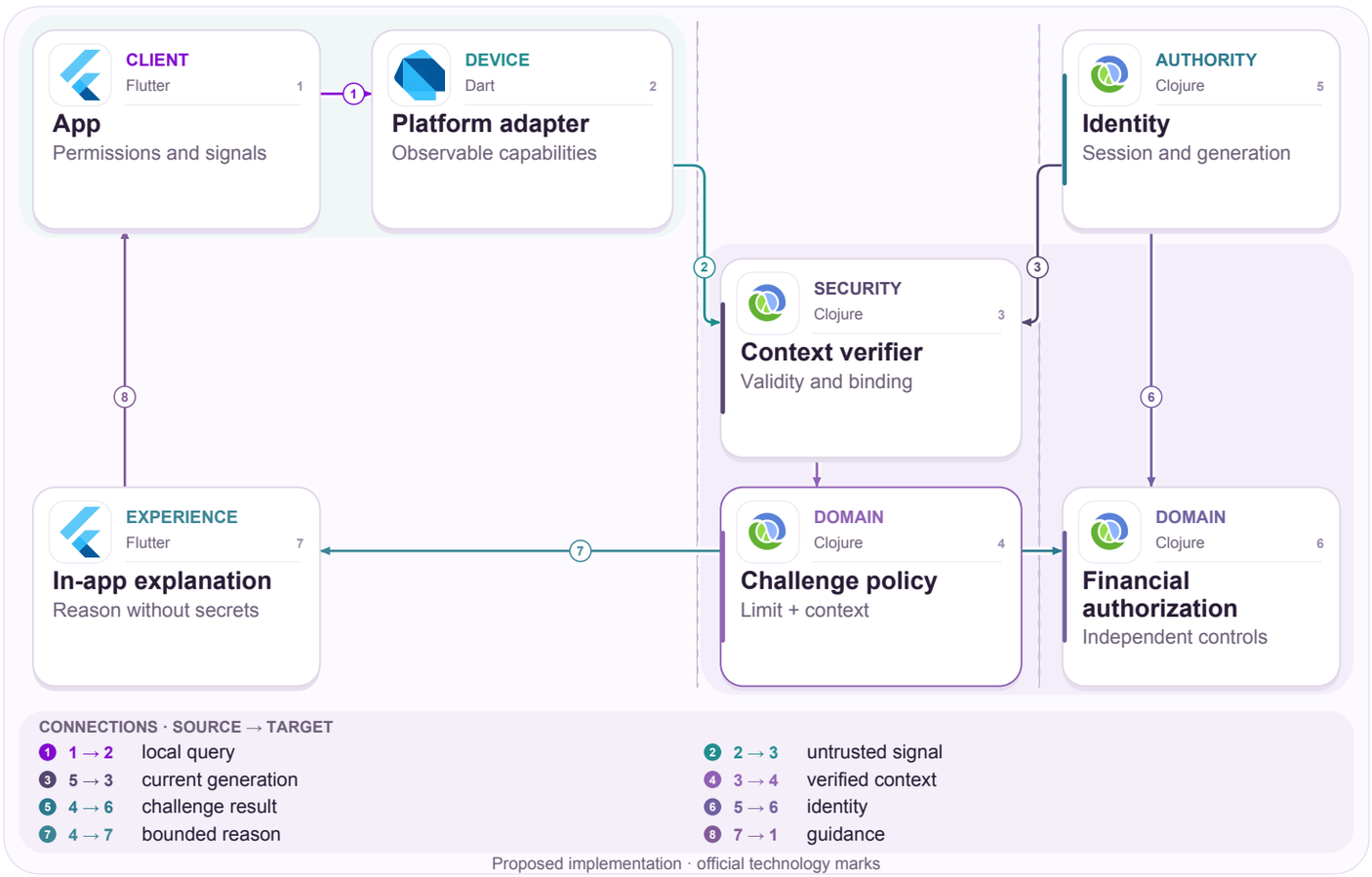
Three concepts must remain separate. Observed context is the set of signals available on the device. Granted trust is an authorized interpretation of those signals under a policy. Authorized action is the final decision about a financial operation. A recognized Wi-Fi network may inform a security challenge, but must not replace balance, identity, limits, or other controls. Mixing these concepts gives a visual indicator more power than it should have.

The proposed decision contract returns context_trusted, challenge, and reason. The interface uses reason to explain the experience, while financial authorization remains at another boundary. The lab's contextual proof contains a device, fictional network, generation, and validity interval. Even with a valid signature, it is accepted only for the corresponding device and an approved identifier. The network's public name is not sufficient proof.

When permission is missing, the system retains the restrictive state and presents the specific reason. When the proof expires, the solution must obtain a new observation or retain the challenge. Reusing trust indefinitely merely to avoid latency is unacceptable. These are modeling proposals, not a description of Modo Rua's implementation. The case is useful because it makes agreement between security and explanation explicit.

PROPOSED ARCHITECTURE • EXPLICIT BOUNDARIES

The interface explains; policy decides; payments authorize



Original proposed design; it does not represent Nubank's internal architecture.

The diagram keeps device data outside the trust boundary until evaluated. The adapter must distinguish missing permission, missing capability, and a valid observation. The verifier checks binding, generation, and freshness before supplying context to the policy. Financial authorization receives the challenge result as one input without delegating the entire decision to it.

The explanation uses bounded categories to guide the customer. It does not need to reveal fraud thresholds, detailed signals, or verification material. This separation allows comprehension to improve without turning the interface into a complete description of defensive mechanisms.

In the lab, the HMAC proof is issued by the fixture itself and represents a fictional authority. The signature demonstrates the example's integrity, not the physical truth of the network. A real implementation would need to justify how it obtains trustworthy signals and which threats each signal can address.

ENGINEERING EXPLAINED

Validity means more than checking a signature

PYTHON / ORIGINAL PROPOSAL

```python
def decide(self, device, amount_cents, street_limit_cents, proof, now,
           permission=True):
    if type(permission) is not bool:
        raise ValueError("boolean_permission_required")
    if (type(amount_cents) is not int or amount_cents <= 0
            or type(street_limit_cents) is not int or street_limit_cents < 0):
        raise ValueError("invalid_money")
    reason = "context_missing"
    trusted = False
    if not permission:
        reason = "permission_missing"
    elif proof:
        body = proof["body"]
        expected = hmac.new(self.key, canonical(body), hashlib.sha256).hexdigest()
        valid = hmac.compare_digest(expected, proof["signature"])
        valid = valid and body["device"] == device
        valid = valid and body["epoch"] == self.epoch
        valid = valid and body["issued"] <= now < body["expires"]
        valid = valid and body["network"] in self.approved.get(device, set())
        trusted = valid
        reason = "verified_context" if valid else "context_invalid"
    # Contexto apenas decide este desafio; não autoriza pagamento.
    challenge = amount_cents > street_limit_cents and not trusted
    return dict(challenge=challenge, context_trusted=trusted, reason=reason,
                payment_authorized=False)
```

Excerpt from the synthetic lab; execution verified in the complete file.

The decision combines signature, device, generation, time, and an approved association. Failing any condition keeps the context untrusted. Even when the example no longer requires the challenge, payment_authorized remains false, making it explicit that the function neither moves money nor replaces financial controls.

Missing permission produces its own reason and retains the restrictive policy. The lab receives structured test objects; it does not implement comprehensive hostile-input validation or issuer authentication. The fixed key appears only in the synthetic test. In production, issuance, key protection, and schema validation would require another architecture and specialized review.

ENGINEERING EXPLAINED

Dart: explain state without exposing secret rules

DART / ORIGINAL PROPOSAL

```dart
enum ContextReason {
  verified, permissionMissing, unavailable, expired,
}

class ContextMessage {
  final String title, body;
  final bool offerSettings;
  const ContextMessage(this.title, this.body, this.offerSettings);
}

ContextMessage explain(ContextReason reason) {
  // Texto proposto: não informa critérios internos de risco.
  switch (reason) {
    case ContextReason.verified:
      return const ContextMessage('Contexto confirmado',
          'Outras verificações de segurança continuam valendo.', false);
    case ContextReason.permissionMissing:
      return const ContextMessage('Proteção mantida',
          'Não foi possível confirmar o contexto sem a permissão.', true);
    case ContextReason.unavailable:
      return const ContextMessage('Proteção mantida',
          'O contexto não está disponível neste momento.', false);
    case ContextReason.expired:
      return const ContextMessage('Precisamos verificar novamente',
          'A confirmação anterior perdeu a validade.', false);
  }
}
```

Proposed message mapping; Dart/Flutter not executed.

The example maps domain categories to stable messages. It avoids calling missing permission a “security error” and does not promise to unlock the account by opening settings. The copy explains that other controls remain in place even when context has been confirmed.

A Flutter interface would need to connect these messages to authorized navigation, focus, and accessible semantics. The settings option should provide guidance without pressuring customers to grant more access than necessary. This is a teaching proposal in Dart and was not executed. A final version would require comprehension research and joint review by product, security, and mobile platform teams.

EXTERNAL ANALYSIS AND PROPOSAL

Permission requirements depend on the API and platform version

Android documentation distinguishes nearby Wi-Fi device permissions from location permissions, with differences by version and API. This explains why a generic statement such as “Wi-Fi never needs location” may be wrong. It does not, however, establish which API Nubank uses or diagnose the report. Investigation would start with the operating-system version, the application target, the requested operation, and the actual call result.

I would build a capability matrix by platform and version, testing permissions granted, denied, revoked, and changed during a session. The adapter would return a domain category while retaining technical detail for controlled diagnosis. Security policy must not interpret a platform exception as positive confirmation. The interface, in turn, must not request permission without explaining the specific purpose for which the product actually uses it.

The contract must also acknowledge that capabilities change. An operating-system update may alter access to signals even when the customer has not changed networks. Conservative behavior must remain understandable, and the team must be able to identify regressions by version without unnecessarily collecting network names or location. This is a mobile integration and security responsibility, not merely a copy review.

Android - Wi-Fi permissions

ENGINEERING EXPLAINED

An intact proof may still be expired or out of context

PYTHON / ORIGINAL PROPOSAL

```python
def issue_demo_proof(self, device, network, now, ttl=30):
    body = dict(device=device, network=network, issued=now,
                expires=now + ttl, epoch=self.epoch)
    signature = hmac.new(self.key, canonical(body), hashlib.sha256).hexdigest()
    return dict(body=body, signature=signature)

def test_missing_permission_keeps_conservative_state(self):
    result = self.decide(permission=False)
    self.assertTrue(result["challenge"])
    self.assertEqual(result["reason"], "permission_missing")

def test_expired_and_future_proofs_are_rejected(self):
    for now in (99, 130, 1000):
        self.assertTrue(self.decide(now=now)["challenge"])

def test_tampered_signature_is_rejected(self):
    proof = json.loads(json.dumps(self.proof))
    proof["body"]["expires"] = 9999
    self.assertTrue(self.decide(proof=proof)["challenge"])
```

Excerpt from the synthetic lab; execution verified in the complete file.

The fixture issues a proof with a validity interval and generation. The tests change time, permission, and signed content. Tampering with expiration must invalidate the signature, while an intact proof must also fail outside the permitted interval.

These scenarios prevent a review from collapsing into “the HMAC passed, so trust it.” They do not validate the physical origin of the signal or key protection on a real device. The issuer is fictional and shares a key with the verifier only to make the rule executable. The chapter uses that limitation to distinguish message integrity from contextual truth.

ENGINEERING EXPLAINED

Recovery and device replacement invalidate old context

PYTHON / ORIGINAL PROPOSAL

```python
def test_valid_context_removes_only_this_challenge(self):
    result = self.decide()
    self.assertFalse(result["challenge"])
    self.assertFalse(result["payment_authorized"])

def test_proof_is_bound_to_device_and_approved_network(self):
    self.assertTrue(self.decide(device="other")["challenge"])
    other = self.p.issue_demo_proof("device-demo", "same-name-fake", 100)
    self.assertTrue(self.decide(proof=other)["challenge"])

def test_recovery_invalidates_old_context(self):
    self.p.epoch += 1
    self.assertTrue(self.decide()["challenge"])

def test_below_limit_is_not_automatic_payment_authorization(self):
    result = self.decide(amount_cents=500, proof=None)
    self.assertFalse(result["challenge"])
    self.assertFalse(result["payment_authorized"])
```

Excerpt from the synthetic lab; execution verified in the complete file.

The binding test prevents use of the proof on another device and rejects an unapproved fictional network even when the test issuer signed it. Changing the generation represents access recovery that invalidates previous contexts. The final scenario demonstrates that being below the limit does not authorize payment.

The file passed eight tests, including rejection of non-boolean permissions. There was no communication with networks, sensors, biometrics, or risk services. The evidence concerns consistency of the local policy under synthetic inputs. A real review would need to assess signal provenance, the adversary considered, and system behavior when those sources are no longer trustworthy.

EXTERNAL ANALYSIS AND PROPOSAL

Observe friction without rewarding the removal of protection

I would track how often context could not be confirmed and why: missing permission, unavailable signal, expired proof, or incompatible generation. This helps distinguish configuration difficulty from a technical regression. The metric should not be used to demand that every challenge disappear. An artificial reduction in challenges may indicate a permissive policy, while an increase may reflect protection working when signals are insufficient.

The product outcome would be understanding the state and legitimately completing the task with appropriate protection. Research and support contacts could reveal whether customers understand why biometrics are still requested. Disabling the feature after encountering difficulty is also a useful signal, but requires interpretation: it may reflect frustration, changed needs, or a conscious preference. The event alone must not be treated as proof of its cause.

In operational data, I would avoid network names, coordinates, and persistent device identifiers in aggregate metrics. Version classes and reasons can guide investigation with less exposure. Alerts would respond to unexpected changes, such as those following an operating-system update, and have a reproduction runbook. The dashboard would show security and usability side by side so that a conversion improvement could not conceal a loss of control.

EXTERNAL ANALYSIS AND PROPOSAL

Release a new explanation without silently changing policy

The first step would review messaging and diagnosis while retaining the current security policy. This allows testing whether people understand the difference between missing permission and a temporary failure without introducing a risk change. The team would compare observed technical categories with displayed copy and check whether the suggested action can actually resolve that condition. A settings button does not help when an API is unavailable for another reason.

Any change to the trust policy would follow a separate track with threat assessment, device validation, and stop criteria. Rollout could observe candidate decisions in parallel without using them to authorize operations, allowing discrepancies to be understood. That comparison would require data minimization and a clear purpose. Expansion would depend on technical evidence and security review, not merely fewer challenges.

Rollback would retain conservative behavior and the ability to explain the reason to the customer. Reusing old proofs to smooth the transition or erasing the recovery generation would be unacceptable. I would also include support and configuration documentation in the launch. A consistent policy that nobody can explain still creates friction and encourages improvisation in a journey where predictability matters.

EXTERNAL ANALYSIS AND PROPOSAL

The threat model defines what a signal can establish

A network with a familiar name can be imitated; a device may report incomplete data; a valid observation can become stale. The design must therefore state which threats it considers and which properties each signal actually provides. The lab does not attempt to resolve these physical threats. It assumes a fictional issuer and demonstrates that, even under that favorable assumption, binding, validity, and generation must still be checked before context is used.

I would avoid an experience that says “you are safe” based on a single indicator. The proposed message describes contextual confirmation and keeps other controls explicit. This reduces the likelihood of customers interpreting a convenience as a broad guarantee. The same care applies to performance: caching a contextual result can reduce latency but extends the window during which an environmental change is not reflected. That window should be a risk decision, not just a cache setting.

The cost of collecting signals matters too. The team should measure resource use, query frequency, and the actual need for each permission rather than assuming that a technical dependency justifies any collection. A better design may reduce calls, explain their purpose, and retain a conservative alternative. Understandable security requires precision about what is known, what is missing, and which action remains protected.

EXTERNAL ANALYSIS AND PROPOSAL

ADR: unknown remains unknown

The proposed decision keeps context unconfirmed when signals, permissions, or validity are missing. A permissive alternative would reduce challenges but treat absence of evidence as evidence of safety. Blocking all app use whenever context is missing could impose friction beyond the feature's purpose. I would choose a policy limited to the relevant challenge while preserving the independent authority of identity and payments.

The contract must return a reason the interface can explain without revealing sensitive rules. This improves support quality and avoids generic messages that blame the customer. The implementation must also distinguish an expected condition, such as denied permission, from an adapter failure. Both may retain protection, but they require different operational actions. Conservative state must not become an excuse to ignore technical regressions.

The accepted cost is maintaining a platform matrix and validity rules. The benefit is predictability: policy does not change meaning because an API returned nothing. I would revisit the ADR when new signals offer demonstrated guarantees, threats change, or evidence suggests that challenge scope is excessive. The review would need to show which properties improved and which tests still ensure that an old or out-of-context signal cannot gain inappropriate authority.

EXTERNAL ANALYSIS AND PROPOSAL

EM and Staff: review security and comprehension together

As an EM, I would bring together mobile platform, identity, security, product, and support owners to review a reproducible journey. The objective would be to map the observed condition, applied policy, and displayed message. Public evidence informs that question but does not replace authorized reproduction with controlled data. The first delivery should correct mismatches between state and explanation before considering deeper trust changes.

As a Staff engineer, I would examine signal provenance, device binding, and generation, along with the separation between a contextual challenge and financial authorization. The eight synthetic tests provide a starting point for making those decisions concrete. I would also require a device and version matrix for behavior Python does not simulate. The contribution is connecting a permission detail to trust architecture and experience without promising that a signature establishes physical truth.

Value would be measured through completed configuration, comprehension, and fewer support requests while preserving protection indicators. I would not equate feature adoption directly with fraud prevented. The interview question would be: “When a contextual signal disappears, what is the safe decision, and what explanation lets the customer continue without improvising?” Engineering, risk, and design must build that answer together.

EXTERNAL ANALYSIS AND PROPOSAL

A small amount, a substantial interoperability contract

07

Original source screenshot, accessed 23 October 2026. An individual report is not a reproduced incident.

The public report describes a negative one-cent amount represented incorrectly in an OFX statement and import difficulties after changes to number formatting. The original file was not obtained, and the behavior was not reproduced in the actual app. The source therefore cannot establish whether the exporter or importer caused the issue. It offers a concrete example for discussing monetary correctness and compatibility of external contracts.

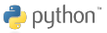
The potential impact does not depend on the one-cent amount. A consumer may reject the entire file, import it partially, or require manual reconciliation when a field violates the expected format. Investigation would need to distinguish an invalid format, a valid change unsupported by the consumer, and a content discrepancy. These situations require different responses; “the file opens” is not sufficient evidence that the integration is correct.

The lab demonstrates serialization of integer cents, parsing of a strict decimal representation, and idempotent import of an XML fragment inspired by OFX fields. It does not generate a complete OFX file or certify interoperability with financial applications. This choice makes small, relevant invariants testable while making clear that a real implementation still needs a version specification, mandatory metadata, consumer contracts, and file-processing security validation.

NuCommunity - OFX statement error

EXTERNAL ANALYSIS AND PROPOSAL

Money, representation, and identity are three contracts



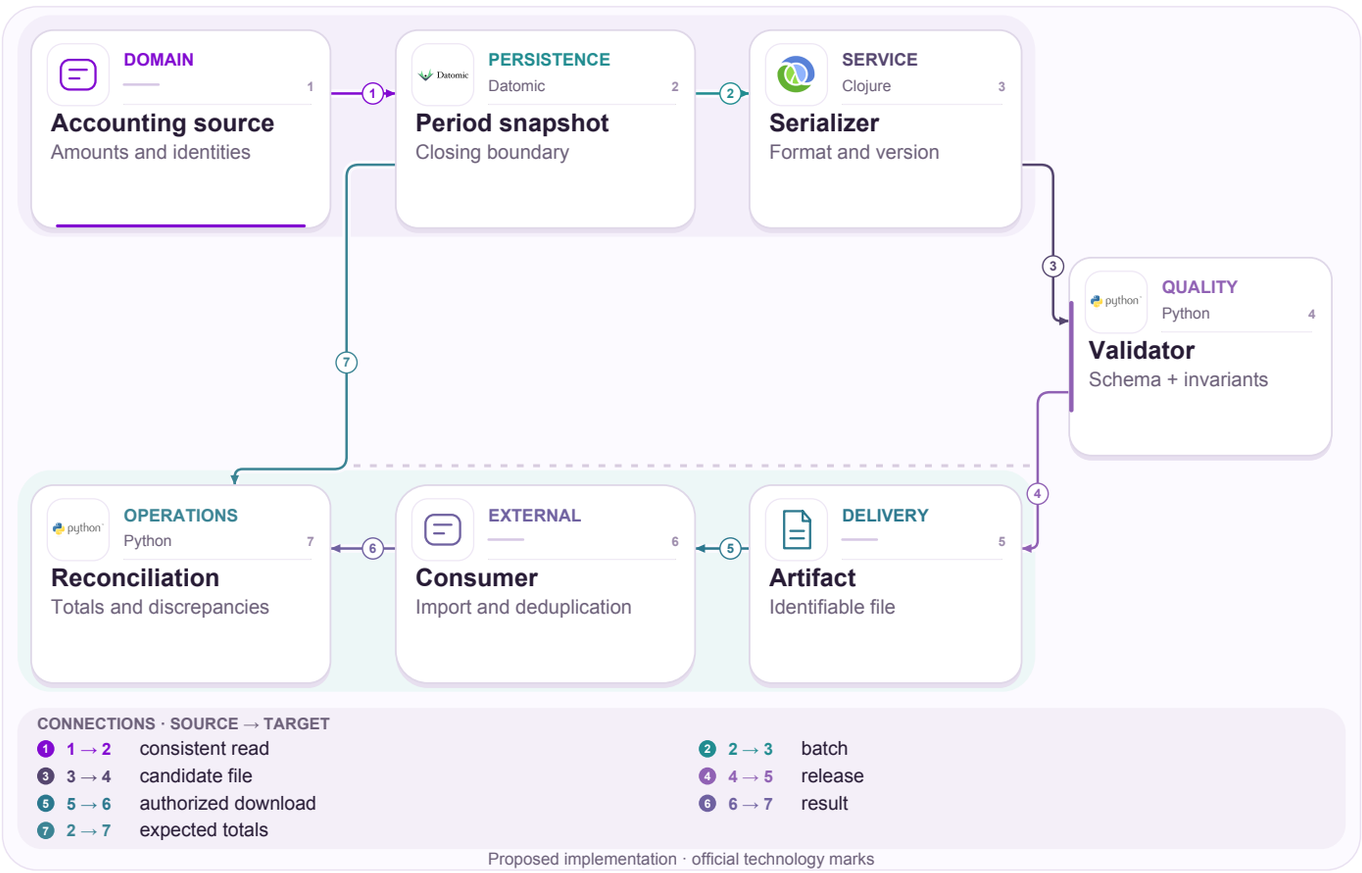
A monetary value exists independently of the string used to transport it. In the local model, negative one cent is the integer -1. The corresponding external representation is a string with a sign, an integer part, and two decimal places. Separating these layers avoids reliance on floating-point rounding and makes boundary cases explicit. The exporter must receive a known unit, not guess whether a number represents reais, cents, or another currency.

The third contract is transaction identity. Reissuing the same period should not turn previously imported entries into new transactions. Identity must therefore remain stable when content has not changed. If a correction changes the content associated with the same identity, the consumer needs a clear policy: reject the conflict, apply a recognized revision, or follow the format's correction mechanism. The lab chooses to reject conflicts and preserve the previous batch.

These rules are independent of document appearance. A visually correct PDF may coexist with an invalid integration file. The quality strategy must treat each format as its own contract, with tests for amounts, dates, identity, and completeness. The first investigation question would be which representation was generated and which consumer interpreted it, preserving the file and version as technical evidence.

PROPOSED ARCHITECTURE • EXPLICIT BOUNDARIES

Export with identity; import with reconciliation



Original proposed design; it does not represent Nubank's internal architecture.

The proposal first fixes the period boundary and the source of entries. The serializer transforms amounts and identities without redefining their semantics. A separate validator checks format and invariants before releasing the artifact. Reconciliation compares the consumed result with the expected snapshot, exposing problems that simple syntax validation cannot detect.

A reissue should identify its relationship to the previous file and preserve identities when the entries are unchanged. This reduces duplication risk in the consumer, but depends on the external contract selected. The lab demonstrates that principle with FITID in a simplified fragment without claiming full compliance with an OFX version.

Download is also an authorization boundary: an artifact identifier must not allow unrestricted access to the statement. Persistence, retention, and distribution would be additional decisions. The design focuses discussion on content correctness and the ability to reproduce its generation.

ENGINEERING EXPLAINED

Serialize the sign without converting money to float

PYTHON / ORIGINAL PROPOSAL

```python
# Entrada monetária: inteiro em centavos.
# Saída externa: decimal com ponto e duas casas.
# O parser rejeita formatos fora deste contrato sintético.
# Não existe etapa de arredondamento neste percurso.
def format_cents(cents):
    if type(cents) is not int:
        raise ValueError("integer_cents_required")
    sign = "-" if cents < 0 else ""
    units, remainder = divmod(abs(cents), 100)
    return f"{sign}{units}.{remainder:02d}"

def parse_cents(text):
    if not re.fullmatch(r"-?(?:0|[1-9][0-9]*)\.[0-9]{2}", text):
        raise ValueError("invalid_decimal_format")
    negative = text.startswith("-")
    units, fraction = text.lstrip("-").split(".")
    result = int(units) * 100 + int(fraction)
    return -result if negative else result
```

Excerpt from the synthetic lab; execution verified in the complete file.

Formatting uses the absolute value to separate whole units and cents, applying the sign only at the start. This keeps -1 as -0.01 without invalid combinations of decimal point and sign. The parser accepts a restricted format and reconstructs the integer through multiplication and addition without going through float.

The lab contract requires two decimal places and does not represent every variant accepted by actual OFX versions or consumers. This restriction is deliberate for testing round trips and invalid cases. An export implementation would need to align its format with the chosen specification and maintain compatibility tests with relevant consumers.

ENGINEERING EXPLAINED

Clojure: retain exactness up to the text boundary

CLOJURE / ORIGINAL PROPOSAL

```clojure
(require '[clojure.string :as str])

(defn money-text [cents]
  ;; Valor inteiro e unidade definidos pelo contrato proposto.
  (when-not (integer? cents)
    (throw (ex-info "integer-cents-required" {})))
  (let [value (bigint cents)
        negative? (neg? value)
        magnitude (if negative? (- value) value)
        units (quot magnitude 100)
        fraction (long (rem magnitude 100))]
    (str (when negative? "-")
         units "." (format "%02d" fraction))))

(defn parse-money [text]
  (when-not (re-matches #"-?(0|[1-9][0-9]*)\.[0-9]{2}" text)
    (throw (ex-info "invalid-decimal-format" {})))
  (let [negative? (.startsWith text "-")
        unsigned (if negative? (subs text 1) text)
        [units fraction] (str/split unsigned #"\.")
        cents (+ (* (bigint units) 100) (bigint fraction))]
    (if negative? (- cents) cents)))

;; Propriedade pretendida, a verificar no runtime de destino:
;; (= x (parse-money (money-text x)))
;; Nenhuma etapa exige conversão para Double.
```

Proposed serializer; Clojure not executed.

The example uses arbitrary-precision integers so that obtaining an absolute value does not depend on a fixed-width integer limit. Only the remainder between zero and ninety-nine is converted to long when formatting two decimal places. Text is constructed at the output boundary, preserving cents as the domain unit.

The excerpt is educational and was not executed in Clojure. It also does not define currency, variable scale, or limits accepted by the destination system. Those conditions belong to the export contract. The executed Python tests verify the corresponding property in the local model, not compilation of this variant.

EXTERNAL ANALYSIS AND PROPOSAL

The snapshot explains why two files differ

A statement must declare the period and information boundary it represents. Pending transactions, later postings, and reversals may change the view of an interval when the file is reissued. I would record the generation version and snapshot used, distinguishing a legitimate change in the dataset from a serializer regression. Without that reference, two different files may appear contradictory even when they were generated at different times.

The identity contract must also survive reissue. The proposal would not use row position or download time as a transaction identifier. A different ordering should not create new entries in the consumer. In the lab, id remains stable and is transported as FITID. If the same id arrives with a different amount or description, the synthetic importer rejects the batch, preserving the previous state rather than silently choosing a version.

This policy is conservative and does not replace correction mechanisms provided by an actual format. It highlights a question that must be answered before integration: how does the consumer recognize a legitimate correction without duplicating or deleting information? The answer belongs in documentation, compatibility tests, and support guidance, especially when a reissue is offered after a confirmed incident.

ENGINEERING EXPLAINED

Build XML as a structure, not by concatenation

PYTHON / ORIGINAL PROPOSAL

```python
# Fragmento didático: BANKTRANLIST não é um documento OFX completo.
# FITID preserva a identidade entre reemissões.
# O gerador XML cuida de escapar o texto da descrição.
# A ordem recebida é preservada neste fragmento.
# Identidades repetidas são recusadas antes da publicação.
# O consumidor de teste reconcilia o resultado do lote.
def export_transactions(transactions):
    root = ET.Element("BANKTRANLIST")
    seen = set()
    for tx in transactions:
        if tx["id"] in seen:
            raise ValueError("duplicate_transaction")
        seen.add(tx["id"])
        row = ET.SubElement(root, "STMTTRN")
        ET.SubElement(row, "FITID").text = tx["id"]
        ET.SubElement(row, "TRNAMT").text = format_cents(tx["cents"])
        ET.SubElement(row, "MEMO").text = tx["memo"]
    return ET.tostring(root, encoding="unicode")
```

Excerpt from the synthetic lab; execution verified in the complete file.

The generator creates elements and assigns text, allowing the library to escape characters such as ampersands and comparison signs. Concatenating descriptions directly into an XML string could break the structure. The seen set rejects duplicate source identities, preventing an ambiguous batch from being generated.

The result is only a fragment used by the tests. It omits headers, dates, and other elements required in a real OFX file. It also does not make the lab parser a secure service for hostile input: tests use controlled fixtures. A real integration would require limits, validation, and an appropriate file-processing review.

ENGINEERING EXPLAINED

Demonstrate round trips, reissues, and conflicts

PYTHON / ORIGINAL PROPOSAL

```python
def test_roundtrip_range_of_integer_cents(self):
    for cents in range(-10000, 10001):
        self.assertEqual(parse_cents(format_cents(cents)), cents)

def test_invalid_formats_and_float_are_rejected(self):
    for text in (".-1", "1,00", "1.0", "1e2", " 1.00", "01.00"):
        with self.subTest(text=text), self.assertRaises(ValueError):
            parse_cents(text)
    for cents in (0.1, True):
        with self.assertRaises(ValueError):
            format_cents(cents)

def test_reissue_does_not_duplicate_import(self):
    document = export_transactions([dict(id="tx1", cents=-123, memo="x")])
    consumer = Importer()
    self.assertEqual(consumer.load(document), -123)
    self.assertEqual(consumer.load(document), -123)
    self.assertEqual(len(consumer.entries), 1)

def test_conflicting_reissue_is_atomic(self):
    consumer = Importer()
    consumer.load(export_transactions([dict(id="tx1", cents=-123, memo="x")]))
    changed = [dict(id="tx2", cents=100, memo="new"),
               dict(id="tx1", cents=-124, memo="x")]
    with self.assertRaises(ValueError):
        consumer.load(export_transactions(changed))
    self.assertEqual(consumer.entries, {"tx1": (-123, "x")})
```

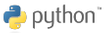
Excerpt from the synthetic lab; execution verified in the complete file.

The round trip covers 20,001 test integers, including negative values, zero, and positive values. Other scenarios reject invalid formats and floats, repeat import of the same document, and attempt to apply a conflicting reissue after a new entry in the same batch. The previous state must remain intact when that conflict occurs.

The file passed seven tests. This demonstrates properties of the fragment and synthetic importer, not compatibility with financial applications or every OFX version. For production, I would add authorized fixtures from real consumers and balance reconciliation by period, preserving the distinction between local testing and external certification.

EXTERNAL ANALYSIS AND PROPOSAL

Observe validity, completeness, and consumption separately



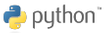
I would separate export operations into three outcomes: file generated, file validated, and file consumed correctly when evidence is available. The first measures generator availability; the second measures contract conformance; the third measures interoperability. Confusing these states allows success to be declared because a download occurred, even when the customer had to edit the file manually to import it.

Proposed metrics would include failures by validation category, discrepancies in entry counts, differences between expected and exported totals, and reissues associated with incidents. Categories such as invalid sign, duplicate identity, or missing mandatory field are useful for diagnosis. Individual amounts, purchase descriptions, and account numbers should not appear in aggregate metrics. A file sample used for investigation would require its own controls and minimization.

The most important alert would be a contract regression after a version change, even at low incidence. A single negative-cent case may reveal an entire class of affected values. Investigation should produce an authorized scope query rather than assume every file is wrong. Customer communication would depend on that confirmation and offer proportionate reissue or guidance, distinguishing corrected formatting from a change to financial content.

EXTERNAL ANALYSIS AND PROPOSAL

Migrate the serializer through semantic comparison



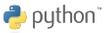
I would begin by generating candidate files from fixtures and authorized snapshots without distributing them to customers. Comparison between versions must be semantic: attribute order or whitespace may change without changing content, whereas a different sign or identity may change the financial interpretation. The diff should highlight amounts, dates, currencies, identities, and counts rather than treating every textual difference as equivalent.

Next, a representative set of consumers would import the files in a controlled environment. Acceptance would require checking the result, not merely the absence of an error. If a consumer silently imports only part of the file, reconciliation must detect the difference. Migration would also test reissue of the same period and coexistence between versions, ensuring that switching to a corrected file does not duplicate entries for the customer.

Rollout would be able to stop new generation while retaining access to already valid artifacts. A serializer rollback must not retroactively alter document identities or obscure which versions were affected. I would prepare a scope-assessment and reissue procedure before increasing traffic. This discipline treats the statement as an external engineering interface with compatibility obligations and ownership, not a low-risk download button.

EXTERNAL ANALYSIS AND PROPOSAL

Precision requires input and resource limits



Integers avoid one class of rounding errors, but do not solve file processing on their own. The system must still validate scale, currency, value range, and batch size. An arbitrarily large integer can be mathematically exact yet incompatible with the destination storage or consumer. The contract must state those limits and reject inputs outside them before producing an artifact that appears usable.

The lab reads XML generated by its own fixtures. It was not designed to analyze hostile documents submitted by third parties. A real application would need to consider maximum size, depth, processing time, parser capabilities, and unexpected fields. It must also control statement-download authorization and prevent error logs from including complete financial content. Monetary exactness and secure processing are complementary properties with different tests and owners.

For performance, I would measure memory and generation time by entry count while preserving a consistent view of the period. Batch processing may reduce memory, but requires care with ordering, file completion, and recovery from interrupted generation. A partially written file must not be released as complete. Artifact publication should occur only after validation and confirmation of completeness.

EXTERNAL ANALYSIS AND PROPOSAL

ADR: a central serializer and stable identity

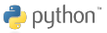
The proposed decision centralizes monetary conversion and identity validation at an export boundary, with contracts for each format. Letting each product format its own amounts may look fast but multiplies variations in sign, scale, and rounding. Converting everything into a generic string too early loses unit information and makes validation harder. I would retain typed values in the domain and format them only at the external boundary.

Stable identity is an invariant for reissue. Corrected content must follow an explicit revision policy instead of receiving unexplained new identifiers. The lab chooses atomic conflict rejection because it makes ambiguity visible. A complete implementation could adopt other semantics supported by the format and consumer, but would need to demonstrate that it neither duplicates nor improperly deletes entries.

The accepted cost is maintaining a compatibility suite and reviewing seemingly small serialization changes. The benefit is fewer silent failures and less manual reconciliation. I would revisit the ADR when adding currencies with different scales, new formats, or consumers with incompatible requirements. Those changes should create versioned contracts, not hidden conditionals scattered through the code. A clear boundary enables evolution without making every export a difficult-to-test exception.

EXTERNAL ANALYSIS AND PROPOSAL

Turn a small report into systemic quality



As an EM, I would give the investigation a defined scope: obtain authorized evidence, reproduce the representation, identify affected consumers, and separate content from format. The statements team, financial platform, and support would have clear owners. The first delivery would be an automated reproduction of a confirmed defect or, if it could not be reproduced, an honest classification of what still needs investigation. Pressure to respond quickly would not justify announcing a cause without evidence.

As a Staff engineer, I would look for the failure class: sign, scale, identity, temporal boundary, or compatibility. The one-cent case informs a broader suite, such as the lab's round trip across 20,001 values, but does not prove that the real implementation has the same problem. The contribution is turning a small example into reusable invariants, with serialization and reissue decisions other teams can review.

Value would be calculated from avoided manual reconciliation hours, fewer support requests, and lower reissue costs using measured internal volumes. I would not infer financial loss from a rejected file. The interview question would be: "If we reissue the corrected statement, how do we demonstrate that the consumer recovers the right content without duplicating what it already imported?" That connects code precision, external contracts, and customer trust.

ANALYSIS AND DECISION

08 · Delivery is part of the product

08

python kafka

Nubank: conta, cartão e mais

Original source screenshot, accessed 23 October 2026. An individual report is not a reproduced incident.

A public app store review describes difficulty replacing a card and limited clarity about its progress. This is a signal to investigate, without a confirmed diagnosis of an integration, carrier, or support issue. The outcome is unavailable. The useful question is what information would help the customer understand the situation and enable an operator to act without restarting the conversation at every contact.

The proposal addresses continuity across the physical journey. Order acceptance, production, dispatch, delivery attempt, exception, and receipt have different owners. Even if each participant fulfills its local contract, the customer may still lack concrete guidance. A dashboard that merely repeats the latest message received does not solve this problem. We need to distinguish an event, the interpreted state, the promised deadline, and the next authorized action.

Before building, I would select a sample of deliveries involving repeat contact, compare available events with the responses provided, and look for reproducible gaps. I would interview support and operations to understand decisions that currently depend on context outside their tools. The initial intervention might be better information, reconciliation, or an exception policy; I would not assume a new service is needed. Success would mean less time without an owner and fewer repeat contacts, while preserving delivery security. This chapter tests just one component: durable ingestion of synthetic events.

Google Play - Murilo Meyrelles review about card delivery

ANALYSIS AND DECISION

A shipment is not a sequence of notifications

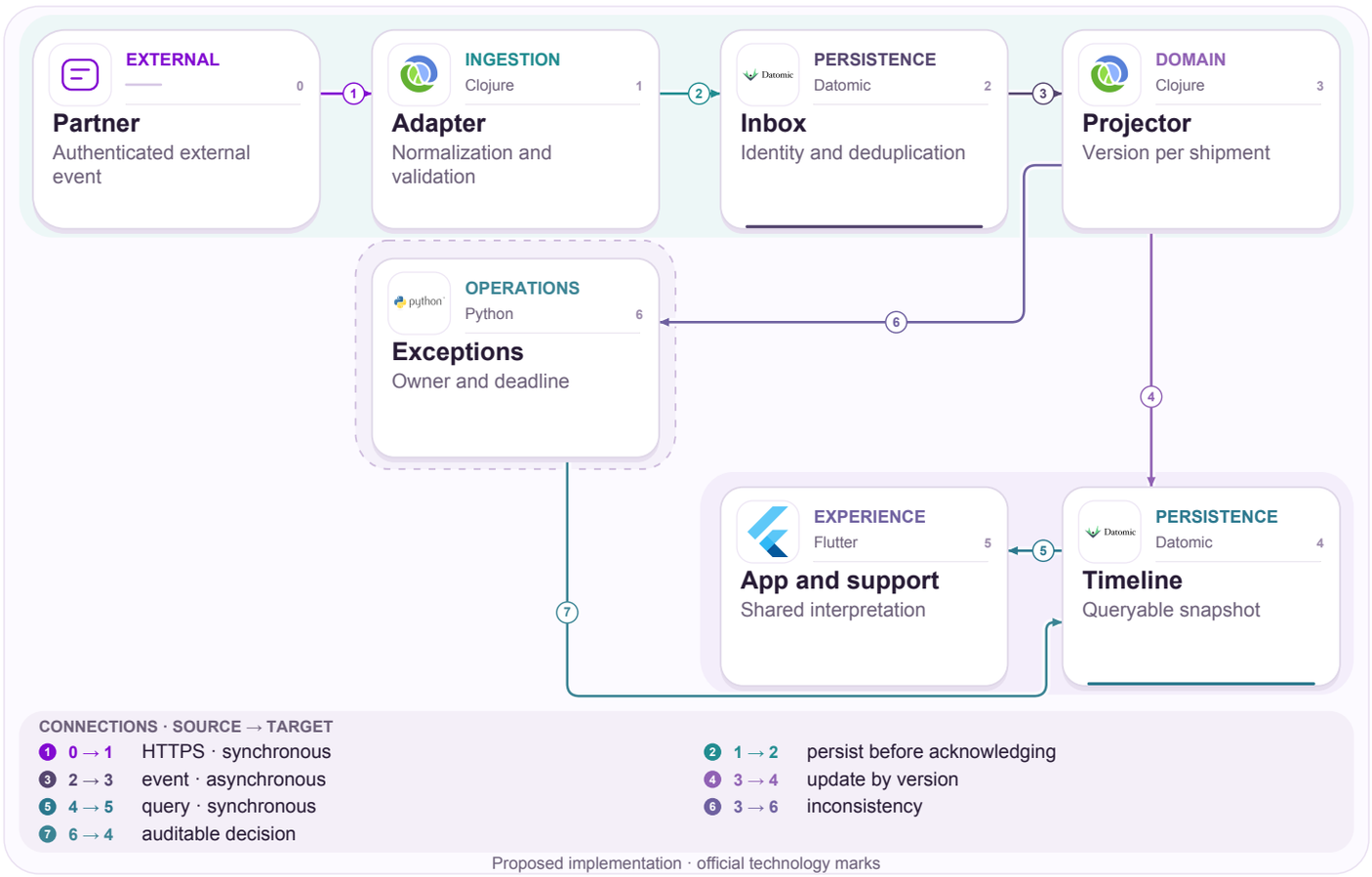
The order identifier links the journey to the customer; the shipment identifier represents a specific physical attempt. Reissue, return, and address changes should not be disguised as simple text changes on the same timeline. Without this distinction, a late event from the previous delivery can contaminate the view of the new card and trigger an incorrect operational action.

In the proposal, each normalized update has an identity, a monotonically increasing shipment version, a state, a source, and a next step. The lab reduces the vocabulary to in transit, exception, and delivered. This simplification is deliberate: it does not model every carrier state. A delivered shipment cannot return to in transit in this model; a subsequent return requires another process. An exception, however, can return to in transit after a legitimate resolution.

The domain must define who can assert each state. Receiving a webhook does not prove delivery to the correct recipient. A source may be trusted to record an attempt but insufficient to close a dispute. The displayed state and the decision to close a support case are therefore separate contracts. The map also includes situations with no update: silence is neither success nor proof of loss. The next step should specify whether to wait, investigate, or request customer action, with an operational deadline and an identifiable owner.

PROPOSED ARCHITECTURE · NOT AN INTERNAL DESIGN

From partner event to next action



Proposed external architecture. Boxes represent responsibilities; arrows make contracts explicit.

The adapter is the trust boundary: it verifies source, format, and identity before accepting an event. Acknowledgment to the partner depends on persistence, not on how quickly the interface updates. A consumer can restart and repeat work; the projector must recognize that repetition without advancing the same shipment twice. Kafka would be a transport option, not a guarantee of domain correctness.

The timeline is a derived read model. The normalized raw record remains available to explain how the state arose, subject to retention and minimization. Divergence should not be resolved by silently overwriting history. The exception path has an owner and a deadline, enabling reconciliation with the partner. App and support consult the same interpretation with different permissions: the customer receives sufficient guidance; the operator receives the context needed to act. This separation prevents internal details from becoming a confusing experience and allows each boundary to be tested with synthetic events.

CODE WITH EXPLICIT SCOPE

Ingestion: the decision fits in one transaction

PYTHON / ORIGINAL PROPOSAL

```python
def ingest(self, event, crash=False):
    payload = json.dumps(event, sort_keys=True)
    digest = hashlib.sha256(payload.encode()).hexdigest()
    if event['status'] not in {'transit', 'exception', 'delivered'}:
        raise ValueError('status desconhecido')
    if type(event['seq']) is not int or event['seq'] < 1:
        raise ValueError('sequência inválida')
    with self.db:
        # A decisão de versão deve usar a mesma transação da gravação.
        self.db.execute('BEGIN IMMEDIATE')
        old = self.db.execute('SELECT digest FROM events WHERE event_id=?',
                              (event['event_id'],)).fetchone()
        if old:
            if old[0] != digest:
                raise ValueError('evento conflitante')
            return 'duplicate'
        row = self.db.execute('SELECT seq,status FROM shipments '
            'WHERE shipment_id=?', (event['shipment_id'],)).fetchone()
        self.db.execute('INSERT INTO events VALUES (?,?)',
                        (event['event_id'], digest))
        if row and event['seq'] <= row[0]:
            return 'stale'
        if row and row[1] == 'delivered' and event['status'] != 'delivered':
            raise ValueError('regressão terminal')
        self.db.execute('INSERT OR REPLACE INTO shipments VALUES (?,?,?,?)',
            (event['shipment_id'], event['seq'], event['status'],
             event['next_step']))
        if crash:
            raise RuntimeError('falha antes do commit')
    return 'applied'
```

Literal excerpt from labs/case_08.py. Python executed locally; synthetic data and contracts.

This excerpt comes from the executed lab. BEGIN IMMEDIATE acquires the write transaction before reading identity or version, preventing concurrent connections from deciding on stale state and moving the projection backward. Identity and projection are written together. An exception before commit rolls both back. A repeat with identical content is recognized; reusing an identity with different content raises a conflict.

The sequence is an assumption of the synthetic contract. A real integration would need to verify whether the partner provides reliable ordering or establish another reconciliation rule. The code does not authenticate webhooks, operate Kafka, or decide card reissue. It isolates local consistency so that the behavior can be inspected.

CODE WITH EXPLICIT SCOPE

Clojure: normalize without guessing the partner contract

CLOJURE / ORIGINAL PROPOSAL

```clojure
(ns portfolio.delivery)

(def allowed-status
  #{:transit :exception :delivered})

(defn normalize-event [raw]
  (let [{:keys [event-id shipment-id sequence
                status next-step]} raw]
    (when-not (and (string? event-id)
                   (string? shipment-id)
                   (integer? sequence)
                   (pos? sequence)
                   (contains? allowed-status status)
                   (string? next-step))
      (throw (ex-info "Invalid partner contract"
                      {:kind :invalid-event})))
    {:event-id event-id
     :shipment-id shipment-id
     :sequence sequence
     :status status
     :next-step next-step}))

;; Autenticação ocorre antes desta função pura.
;; Persistência e confirmação ocorrem depois dela.
```

Unexecuted educational Clojure excerpt; independently proposed function and contract.

The proposed function returns a normalized value or a domain error. It makes no calls, acknowledges nothing, and does not interpret a missing field as success. This allows the contract to be exercised with small example sets before integrating any provider. Every field name belongs to this proposal; none was extracted from a Nubank API.

In a production adaptation, partner-specific fields would remain outside the core. The team would need to decide compatibility, authentication, and handling of invalid events. Sending everything to an error queue without an owner merely exchanges a visible failure for an invisible backlog.

CODE WITH EXPLICIT SCOPE

An event envelope with explicit responsibility

JSON / ORIGINAL PROPOSAL

```json
{
  "contract": "portfolio.delivery.v1",
  "event_id": "demo-event-42",
  "shipment_id": "demo-shipment-7",
  "sequence": 12,
  "status": "exception",
  "occurred_at": "2026-10-20T14:00:00Z",
  "received_at": "2026-10-20T14:01:00Z",
  "origin": {
    "partner": "synthetic-carrier",
    "schema_revision": 1
  },
  "next_action": {
    "owner": "delivery-operations",
    "kind": "review-attempt",
    "due_at": "2026-10-21T14:00:00Z"
  },
  "privacy": {
    "contains_full_address": false,
    "customer_visible": true
  }
}
```

Independent synthetic JSON contract, not integrated; additional fields are not implemented by the lab.

The envelope extends the lab to discuss timing and operations. Occurrence time and receipt time answer different questions: the former helps reconstruct the journey; the latter measures integration delay. Neither automatically replaces a contractual sequence. Clocks can differ, and partners can correct events.

The next step has an owner and a due time, but includes no full address or sample personal data. In a real product, visibility would derive from a policy rather than a boolean trusted from a partner. This JSON documents a proposed contract; it is not a payload from an existing integration.

ANALYSIS AND DECISION

Failures that must not trigger automatic reissue

A delayed delivery and a lost delivery can look identical when viewed only through the last recorded timestamp. The first relevant failure is a missing event: the consumer may be healthy while the partner has stopped sending updates. The system should track both consumption lag and silence for each eligible shipment. Recovery may require querying the partner, human investigation, or changing the communicated deadline.

Another failure is reversed order. Receipt confirmation may arrive before an earlier delivery attempt. The lab preserves the higher version, but the rule works only because the sequence is assumed reliable. Without that property, sorting by arrival time creates a false history. A robust integration retains evidence and escalates contradictions, especially when they would change an irreversible action.

Reissue deserves its own barrier. Before issuing another card, shipments, card state, and the operational decision must be linked. A duplicate event must not generate two requests, and a late confirmation must reach the same case. I would treat reissue as a separate idempotent command. A timeline consumer should never implicitly have permission to perform that action merely because it recognized a word such as lost. This separation limits impact and makes decisions easier to explain.

CODE WITH EXPLICIT SCOPE

Test replay and ordering through observable results

PYTHON / ORIGINAL PROPOSAL

```python
def test_duplicate(self):
    self.model.ingest(self.event())
    self.assertEqual(self.model.ingest(self.event()), 'duplicate')
    self.assertEqual(len(self.model.state()), 1)

def test_conflicting_identity(self):
    self.model.ingest(self.event())
    with self.assertRaises(ValueError):
        self.model.ingest(self.event(status='delivered'))
    self.assertEqual(self.model.state()[0][2], 'transit')

def test_out_of_order(self):
    self.model.ingest(self.event(seq=5, status='delivered'))
    result = self.model.ingest(self.event(event_id='e2', seq=2))
    self.assertEqual(result, 'stale')
    self.assertEqual(self.model.state()[0][2], 'delivered')

def test_terminal_guard(self):
    self.model.ingest(self.event(status='delivered'))
    with self.assertRaises(ValueError):
        self.model.ingest(self.event(event_id='e2', seq=2))
```

Test excerpt from labs/case_08.py. Result: 8 tests passed; see results_08.txt.

All eight tests in the complete file passed. The excerpt covers replay, conflict, ordering, and the terminal-state barrier. The others verify rollback, reopening, acquisition of the transaction before reads, and eight concurrent connections. The last check requires the projection to retain the highest sequence regardless of execution order. The assertions examine persisted state.

This result does not test a real partner or prove recovery from power loss. The injected failure is an exception before commit. The next validation layer would need to include the adapter, lost acknowledgments, reprocessing with a new version, and labeled operational cases. A small test is useful when its boundary remains clear.

ANALYSIS AND DECISION

The dashboard starts with time spent in uncertainty

I would organize observability around operational questions. How many shipments have no next step? For how long? Which owner can act? An events-per-second chart helps detect interruptions, but does not show whether a customer is stuck. An exception queue must support filtering by partner, stage, and age without putting customer identifiers into high-cardinality metric labels.

Technical metrics include duplication rate, identity conflicts, events discarded by version, and delay between occurrence and processing. Each requires interpretation. Duplication can be a normal result of retries; conflicting content is qualitatively different. Fewer old events may indicate a better integration or silent data loss. Comparing totals with a reconciliation source prevents celebrating an apparently clean dashboard.

At the experience level, I would measure contacts per delivery, time to a resolving action, and reopened cases. I would not mix an information request with a dispute over receipt. An alert would include a window, condition, affected partner, link to authorized cases, and an available action. If nobody can act when it fires, the operational contract is incomplete. The goal is to guide work and protect the journey; alert volume is not evidence of maturity.

ANALYSIS AND DECISION

A parallel migration before transferring trust

The first deployment would be a parallel read model fed by the same events as the existing experience. It would neither send messages nor initiate reissues. We would compare states across a stratified sample of partners and shipment types, recording discrepancies with an operations specialist. The sample must include delays and exceptions: evaluating only straightforward deliveries creates artificial confidence in the model.

After initial reconciliation, support could consult the new timeline as supplementary information. This stage tests vocabulary, usefulness of the next step, and the ability to find the owner. I would expose it to customers only after material differences were explained and the support path was ready. Communicated deadlines must follow operational policy, not a static value selected by the interface team.

Rollback separates reads from commands. Removing a new timeline from the screen is straightforward; undoing a reissue already accepted by another system is not. Commands would therefore be enabled later, with an idempotency key, audit trail, and exposure limit. The rollout plan would document which changes are reversible, which require compensation, and who authorizes each step. This prevents confusing a feature flag with a complete recovery strategy.

ANALYSIS AND DECISION

An address is sensitive data, not context for every event

A logistics integration may carry a name, address, phone number, and access instructions. These data do not need to circulate in every event used to update a timeline. I would separate the operational reference from the data required for physical fulfillment, restricting access by role. A shipment identifier may still be linkable to a person; calling it anonymous does not remove that responsibility.

At the external boundary, I would validate source, integrity, replay window, and size limits before processing content. A webhook signature and an event key solve different problems: one confirms the expected source; the other prevents repeating the same fact. The lab does not implement authentication. The mechanism must follow the actual partner contract and support credential rotation without interrupting the entire operation.

I would also avoid responses that disclose inappropriate details. A customer can receive information about an attempt without seeing internal notes or third-party data. Support needs sufficient context, with access logging and permission reviews. Tests should include unexpected free-text content, oversized attachments, and events for a different shipment. A well-formed payload does not authorize access to any object. These boundaries matter to supply-chain security as much as database consistency.

ANALYSIS AND DECISION

ADR 08 · Versioned projection, separate command

The proposed decision is to maintain a delivery projection versioned by shipment and reserve actions such as reissue for explicit commands. The context is a chain of independent participants, late events, and consequences beyond the screen. The objective is to make state explainable and prevent a tracking update from executing a financial or physical action without appropriate evaluation.

One alternative is to query the partner whenever the app opens. This reduces local storage needs but transfers external latency and availability into the experience, complicates history, and may increase cost. Another is to always accept the latest received message. It is simple, but turns transport order into business truth. A versioned projection provides predictable reads and supports reprocessing, at the cost of operating a consumer, retention, and reconciliation.

I would record two conditions for reconsidering the choice. The first is discovering that no partner provides a sequence or comparable snapshot: a richer precedence and investigation model would be needed. The second is reconciliation overhead exceeding the projection's benefit. In that case, we would reduce scope or revisit contracts rather than add silent exceptions. Architecture must remain accountable to information quality and the actual ability to operate difficult cases.

ANALYSIS AND DECISION

EM execution: an owner until the card arrives

As Engineering Manager, I would start with operations and support, not a technology list. I would assign an owner to the pilot and map the points where a case changes teams. The agreement would define completion, the deadline for investigating discrepancies, and availability of people familiar with the partners. Engineering cannot independently make a promise that depends on production, transportation, and card decisions.

Value would be calculated for an eligible population, without extrapolating one review to the customer base. Avoided contacts multiplied by incremental support cost and avoided reissues multiplied by unit cost are possible components. Integration, observability, exception handling, and maintenance would be deducted. Recovered support capacity does not automatically become cash savings. Additional activation would be included only with evidence of causality.

I would review decision quality and operator workload weekly. The team would need room to correct contracts and documentation as well as develop screens. The Staff question would be: what fact allows a shipment to be closed when the partner and customer disagree? A mature answer makes authority, evidence, and escalation explicit, recognizing that no algorithm eliminates every dispute. The expected result is less uncertainty, supported by a system that knows who should act.

ANALYSIS AND DECISION

09 · Preserve the task during visual evolution

09

Nubank: conta, cartão e mais

Original source screenshot, accessed 23 October 2026. An individual report is not a reproduced incident.

A public review questions changes to the statement view, including the relationship between hiding a chart, seeing the total, and navigating between months. The cited version has not been independently reproduced. The signal raises a product question: which information must remain available regardless of a presentation preference? This chapter turns that question into verifiable contracts without asserting a current defect in the app.

A statement is a financial view with established expectations. People return to find the amount, due date, and transactions, compare periods, and understand situations such as partial payment or a reversal. An interface can gain visual consistency while increasing the effort required for these tasks. Risk arises when the implementation makes essential data depend on the presence of a decorative component.

I would investigate tasks before choosing a new layout. I would observe regular users, including people using screen readers, enlarged text, and unstable connections. I would distinguish aesthetic preference from interpretation errors: both matter, but require different responses. The technical proposal keeps the financial snapshot, period selection, and visual preferences in separate states. The lab demonstrates that separation with synthetic data; real validation would need statement rules, research, and the components actually used.

Google Play - Fernando Cordeiro review about viewing the statement

ANALYSIS AND DECISION

Three states, three responsibilities

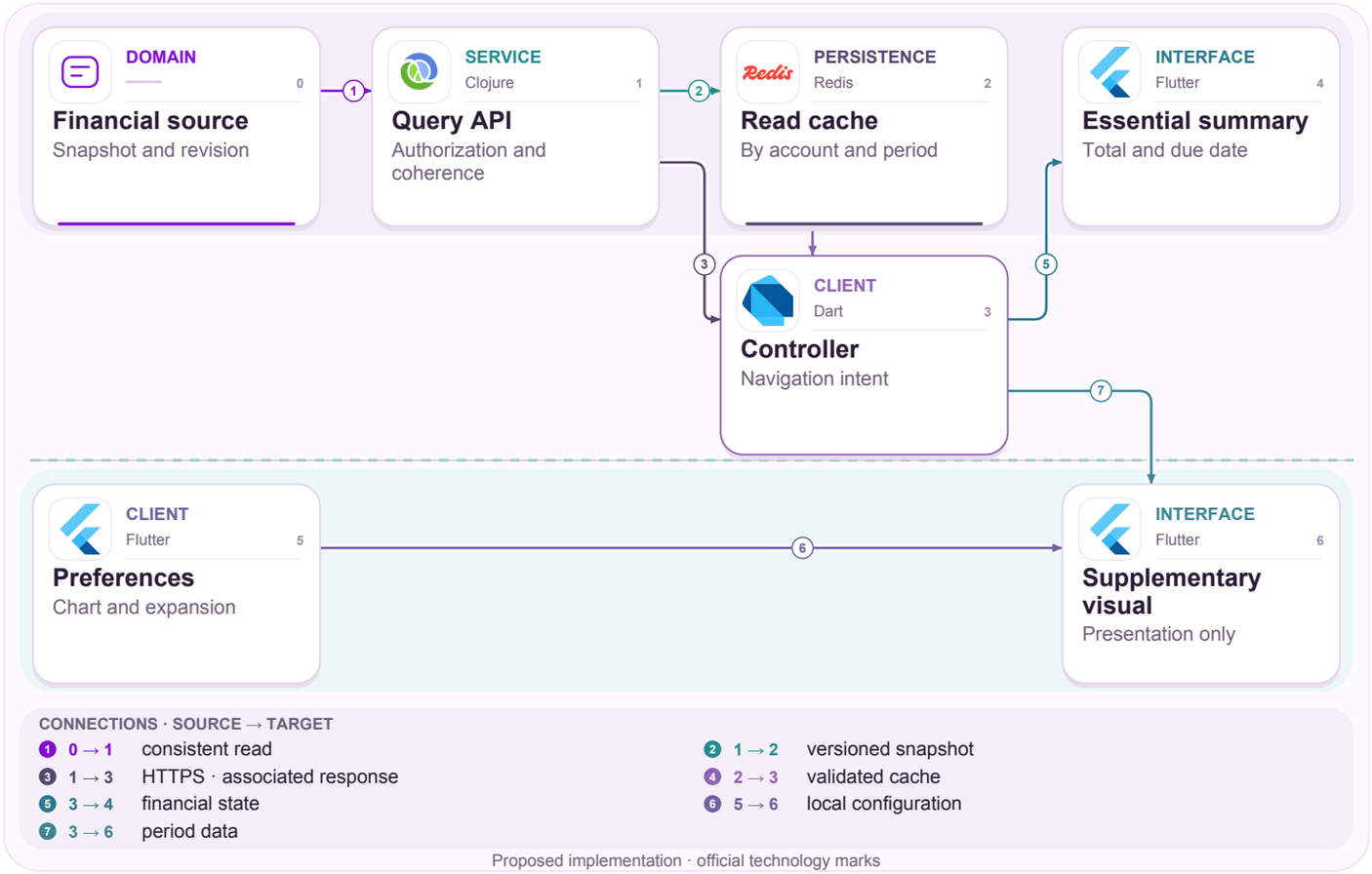
The first state is financial: period, revision, total, due date, and transactions in a coherent snapshot. The second is navigation: which period the person selected and which query is in progress. The third is presentation: chart visibility, sorting, or section expansion. Mixing them creates accidental dependencies, such as hiding the total when a visual preference is disabled or replacing the current month when an old response arrives.

In the lab, Invoice requires a tuple of entries, validates integer cents, and checks the sum. Rejecting mutable lists preserves the snapshot after validation. This equality is a local simplification, not the complete formula for a real statement. Fees, payments, and adjustments would need explicit entries or another reconcilable contract. A revision identifies content; two responses with the same revision and different due dates are a conflict, not an innocent update.

Month selection is explicit. A response for another period cannot silently be applied to the screen. When the user changes months, the controller retains navigation intent and associates the response with the relevant request. Preferences can survive the change, but do not alter the snapshot. This design helps test empty, delayed, and conflicting states without requiring a complete visual test for every domain rule.

PROPOSED ARCHITECTURE · NOT AN INTERNAL DESIGN

Financial data and preferences stay separate



Proposed external architecture. Boxes represent responsibilities; arrows make contracts explicit.

The proposal keeps the essential summary outside the condition that determines whether the chart renders. The controller associates each response with the requested period and prevents a delayed response from changing the selected context. A cache can improve opening time, provided it indicates freshness and never presents an old revision as confirmation of a recent payment.

The design does not give the client authority to calculate the debt. Local validation checks the consistency of a synthetic contract; in production, the financial source defines field semantics. Separating preferences allows visualization changes without migrating financial data. It also reduces impossible states: a chart can be hidden while the total and due date remain visible. The central architectural test is to trace an interaction to the dependency it modifies. If hiding a component changes the snapshot, the boundary is wrong or requires explicit product justification.

CODE WITH EXPLICIT SCOPE

A controller that rejects inconsistent responses

PYTHON / ORIGINAL PROPOSAL

```python
def apply(self, invoice):
    invoice.validate()
    if invoice.period != self.invoice.period:
        raise ValueError('seleção explícita de período necessária')
    if invoice.revision < self.invoice.revision:
        return False
    if invoice.revision == self.invoice.revision:
        if invoice != self.invoice:
            raise ValueError('mesma revisão com conteúdo diferente')
        return False
    self.invoice = invoice
    return True

def select_period(self, invoice):
    self.invoice = invoice.validate()

def view(self):
    return dict(period=self.invoice.period,
                total_cents=self.invoice.total_cents,
                due_date=self.invoice.due_date,
                chart=self.invoice.entries if self.show_chart else None)
```

Literal excerpt from labs/case_09.py. Executed local Python model; not an app implementation.

The lab excerpt applies revisions only to the currently selected period. A lower revision is ignored; an equal revision with different content is rejected. The chart preference affects only the projection's chart field. Total and due date derive from the snapshot, preserving the primary task when the chart is hidden.

The model includes neither asynchronous networking nor persistence across sessions. In a real application, the controller would need request identity, logical cancellation, and loading states. The exercise makes explicit the conditions that any state-management mechanism should preserve, regardless of the chosen library.

CODE WITH EXPLICIT SCOPE

Flutter: the summary exists without the chart

DART / ORIGINAL PROPOSAL

```dart
import 'package:flutter/material.dart';

class InvoiceSummary extends StatelessWidget {
  const InvoiceSummary({super.key, required this.total,
    required this.dueDate, required this.showChart,
    required this.chart});
  final String total;
  final String dueDate;
  final bool showChart;
  final Widget chart;

  @override
  Widget build(BuildContext context) {
    return Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Semantics(
          header: true,
          child: Text('Total da fatura: $total'),
        ),
        Text('Vencimento: $dueDate'),
        if (showChart) chart,
      ],
    );
  }
}
```

Educational Flutter/Dart excerpt, not executed. Independent component with no dependency on internal NuDS.

The Flutter example highlights the presentation boundary: showChart controls only the supplementary visual. Formatted values are component inputs; their source and authorization sit outside this tree. The Semantics widget provides structural information, but does not replace real evaluation with assistive technologies or independently solve contrast, focus, and navigation order.

A complete integration would need approved components and tokens, currency localization, and handling of unavailable values. I would avoid replacing absence with zero, because that communicates a different financial situation. The excerpt is illustrative and was not compiled in this environment; only the associated Python lab was executed.

Flutter · Semantics

CODE WITH EXPLICIT SCOPE

A snapshot contract, not a sum calculated on screen

JSON / ORIGINAL PROPOSAL

```json
{
  "contract": "portfolio.invoice.v1",
  "period": "2026-10",
  "revision": 4,
  "currency": "BRL",
  "total_cents": 900,
  "due_date": "2026-11-10",
  "status": "open",
  "entries": [
    {
      "id": "demo-purchase",
      "kind": "purchase",
      "amount_cents": 1000
    },
    {
      "id": "demo-adjustment",
      "kind": "adjustment",
      "amount_cents": -100
    }
  ],
  "complete": true,
  "updated_at": "2026-10-23T12:00:00Z"
}
```

Proposed JSON contract with synthetic data; only the Invoice subset is executed in the lab.

The proposed contract makes currency, period, revision, and completeness explicit. Negative values represent adjustments in this example; they are not validation errors by definition. A real API would need to specify exactly which components enter the total and how installments, payments, fees, and reversals are reflected. The client should not reconstruct an incomplete financial rule from partially loaded entries.

The complete field distinguishes a full list from a results page, but trusting it requires an authorized producer. The lab uses a complete tuple to check consistency. This JSON adds documentation of intent and does not claim to describe Nubank's format.

ANALYSIS AND DECISION

The race between months needs a defined response

Imagine two queries: someone opens October and selects September before the response arrives. September's response arrives first; October follows. A controller that applies any response to the current state changes the month without consent. Canceling the earlier request can save resources, but is not a sufficient guarantee: it may have finished or passed through a cache. The application rule must check the current intent.

Another race involves revisions. A payment or adjustment changes the snapshot while the app reads the period. Mixing a total from one revision with entries from another creates an inconsistency perceived as a financial error. The solution might be a complete snapshot, a read token, or a pagination contract tied to a revision. The mechanism depends on the source; the principle is not to combine data without knowing whether they belong to the same state.

Presentation failures deserve their own states. Truncated text, missing currency, and loading that erases the total are different experiences from an unavailable statement. I would preserve useful context during refresh and identify when it is older. I would neither show zero for a failed query nor hide the period to save space. Customers need to know what they are seeing and whether the information is ready to support a decision.

CODE WITH EXPLICIT SCOPE

The important assertion is not that the chart exists

PYTHON / ORIGINAL PROPOSAL

```python
def test_hide_chart_keeps_financial_information(self):
    before = self.screen.view()
    self.screen.show_chart = False
    after = self.screen.view()
    for key in ('period', 'total_cents', 'due_date'):
        self.assertEqual(before[key], after[key])
    self.assertIsNone(after['chart'])

def test_empty_period(self):
    self.screen.select_period(Invoice('2026-11', 1, 0, '2026-12-10', ()))
    self.assertEqual(self.screen.view()['total_cents'], 0)

def test_refund_is_signed(self):
    self.assertEqual(self.screen.view()['total_cents'], 900)

def test_stale_revision_ignored(self):
    self.screen.apply(replace(self.invoice, revision=2))
    self.assertFalse(self.screen.apply(self.invoice))
    self.assertEqual(self.screen.invoice.revision, 2)
```

Excerpt from labs/case_09.py. Local result: 8 tests passed; see results_09.txt.

Eight tests passed in the complete lab. The excerpt verifies independence from visual preference, an empty month, a signed adjustment, and a delayed revision. The others reject an inconsistent snapshot, a revision conflict, the wrong period, and entries in a mutable list. The last barrier prevents content from changing after validation. The properties examined are semantic.

This does not replace widget tests. They would need to confirm that the tree actually exposes the total, enlarged text remains usable, and month navigation is operable. The combination is valuable: fast rules isolate the domain, while interface evaluations verify how it translates into the experience.

ANALYSIS AND DECISION

Measure understanding, not just screen opens

Statement opens can increase when the experience gets worse: people return repeatedly to confirm the same amount. I would therefore not use views as the primary success metric. Measurement would begin with tasks: finding the total and due date, identifying the period, locating a transaction, and explaining statement status. Moderated research helps reveal errors that click events cannot.

In telemetry, I would collect completed navigation and loading failures with app revision, text size, and device class where permitted. I would not record financial amounts in analytics events to answer questions that can be measured in aggregate. Repeated period changes may indicate legitimate exploration; they need context before being labeled friction.

The health dashboard would combine snapshot inconsistencies, responses discarded because of context, read latency, and categorized contacts about comprehension. I would segment by accessibility usage and connection without assuming an average represents everyone. Better speed accompanied by more errors identifying the due date would be a regression. The team needs a comprehension metric and a harm guardrail, as well as the ability to inspect consented examples when the numbers conflict.

ANALYSIS AND DECISION

A visual migration also requires compatibility

Rollout begins with an inventory of tasks and states in the previous experience. Listing replaced components is insufficient. We need to know how a regular user finds previous months, recognizes a reversal, and resumes an interrupted query. The migration preserves those outcomes or offers a clearly better path validated with real people. A new design pattern alone does not demonstrate reduced effort.

I would separate changes to content, behavior, and style so the experiment remains interpretable. Changing monthly navigation, the chart, and total hierarchy simultaneously makes a decline in completion difficult to attribute. The first cohort would include diverse devices and access needs, with a defined rollback path. Local settings must remain valid; an old preference should not accidentally hide a new essential component.

Before expanding, I would review quantitative metrics alongside research observations and support contacts. An inconclusive result does not justify expansion merely because there were no crashes. Rollback should restore an experience compatible with current data without switching the financial source or deleting preferences. If maintaining variants becomes costly, I would set a decision window and explicit closure criteria, avoiding a permanent experiment nobody can operate.

ANALYSIS AND DECISION

Accessibility is financial semantics

An accessible tree must communicate more than visible text. Total, due date, period, and status need understandable structure and order. A button announced only as arrow does not explain whether it changes the month or expands a section. Focus should not jump to the beginning when an update arrives or disappear when the chart is hidden. These properties require tests on real devices and assistive technologies.

Enlarged text changes composition. Shrinking the font to fit is not the solution: layout must reflow, prioritize information, and avoid fixed heights for variable content. I would also check the distinction between positive amounts, negative adjustments, and unavailable states. Color can reinforce meaning, but must not be the only way to communicate debt, credit, or error. Currency formatting must keep sign and unit identifiable.

Security includes presentation privacy. Screenshots, previews, and error records can expose statement data if the team treats the screen as ordinary content. Telemetry should minimize those data and prevent exceptions from printing the entire snapshot. The goal is to preserve autonomy without turning observability into a parallel repository of financial information. This connects visual quality, semantics, and engineering discipline within a single product review.

ANALYSIS AND DECISION

ADR 09 · A financial snapshot independent of visuals

The decision is to model financial information as an immutable snapshot, keeping navigation and preferences in separate states. The context is an evolving interface that receives asynchronous responses and must preserve familiar tasks. The main benefit is not a specific library: it is fewer invalid states and predictable boundaries around which interaction can change which part of the experience.

One alternative is a single mutable object containing data, loading, and presentation fields. It simplifies the beginning but allows meaningless combinations: a new month with an old total, a hidden chart with no summary, a current revision with an earlier list. Another alternative is to derive the entire statement from visible transactions. That would be correct only if the contract guaranteed completeness and fully represented the financial rule-conditions a screen should not assume.

Separation requires an explicit controller and compatibility tests between snapshots and components. It also requires deciding how to display older information during refresh. I would record that policy as part of the experience contract, not leave it as a detail for each screen. I would reconsider the design if the source cannot supply a coherent revision or pagination prevents validating the whole. In that case, the right change is to strengthen the contract, not hide discrepancies behind more complex visual states.

ANALYSIS AND DECISION

EM execution: protect understanding through change

As Engineering Manager, I would establish a short initiative with Design, Research, Mobile, and statement owners. The first delivery would be agreement on essential tasks and financial states, backed by verifiable examples. I would distribute ownership across the data contract, behavior, and accessibility, avoiding reliance on one reviewer who knows the old experience. Planning would include regular users and time to address findings.

Value would begin with comprehension and effort: completed queries, time to find information, and demonstrated reduction in repeat contacts. To express support impact as operational value, I would use eligible volume, causal change, and an appropriate unit cost. I would not present fewer clicks as revenue. I would also track migration cost and temporary variant maintenance. A simple change can be a better investment than rebuilding the entire experience.

The Staff question would be: which invariant remains true when a customer toggles the chart, month, and text size while responses arrive out of order? The answer must connect domain, state, and rendering. The leadership question would be who stops expansion when the average improves but a relevant group loses the ability to understand the statement. That decision must be agreed before the experiment, with clear evidence and responsibilities.

ANALYSIS AND DECISION

10 · Define the right document first

10

python

Nubank: conta, cartão e mais

Original source screenshot, accessed 23 October 2026. An individual report is not a reproduced incident.

A public review asks to share proof of a credit-card purchase without depending on email support. The need is understandable, but the source does not clarify which document would resolve the situation. A purchase record, confirmation of statement payment, and a merchant document are not equivalent. Creating a button before resolving this ambiguity could simply accelerate sending inappropriate information.

The investigation would begin with reasons for requesting the document. Does the person want to acknowledge a purchase, dispute a charge, request reimbursement, or demonstrate payment? Each purpose requires different fields, language, and responsibility. I would review anonymized examples with support and product owners, separating requests eligible for self-service from situations that still require assistance. The sample should not include only documents that are easy to generate.

This chapter proposes a versioned, shareable record associated with the known transaction state. It states its scope and minimizes unnecessary data. The lab produces this record from an immutable object, checks synthetic authorization, and detects content changes using a local secret. It does not create a tax receipt, confirm external settlement, or replace a legally valid signature. Its contribution is to show that semantics, access, and integrity must be designed together before discussing appearance or file format.

Google Play - Neidslem Fernandes da Costa review about purchase records

ANALYSIS AND DECISION

Authorized, confirmed, and reversed are not synonyms

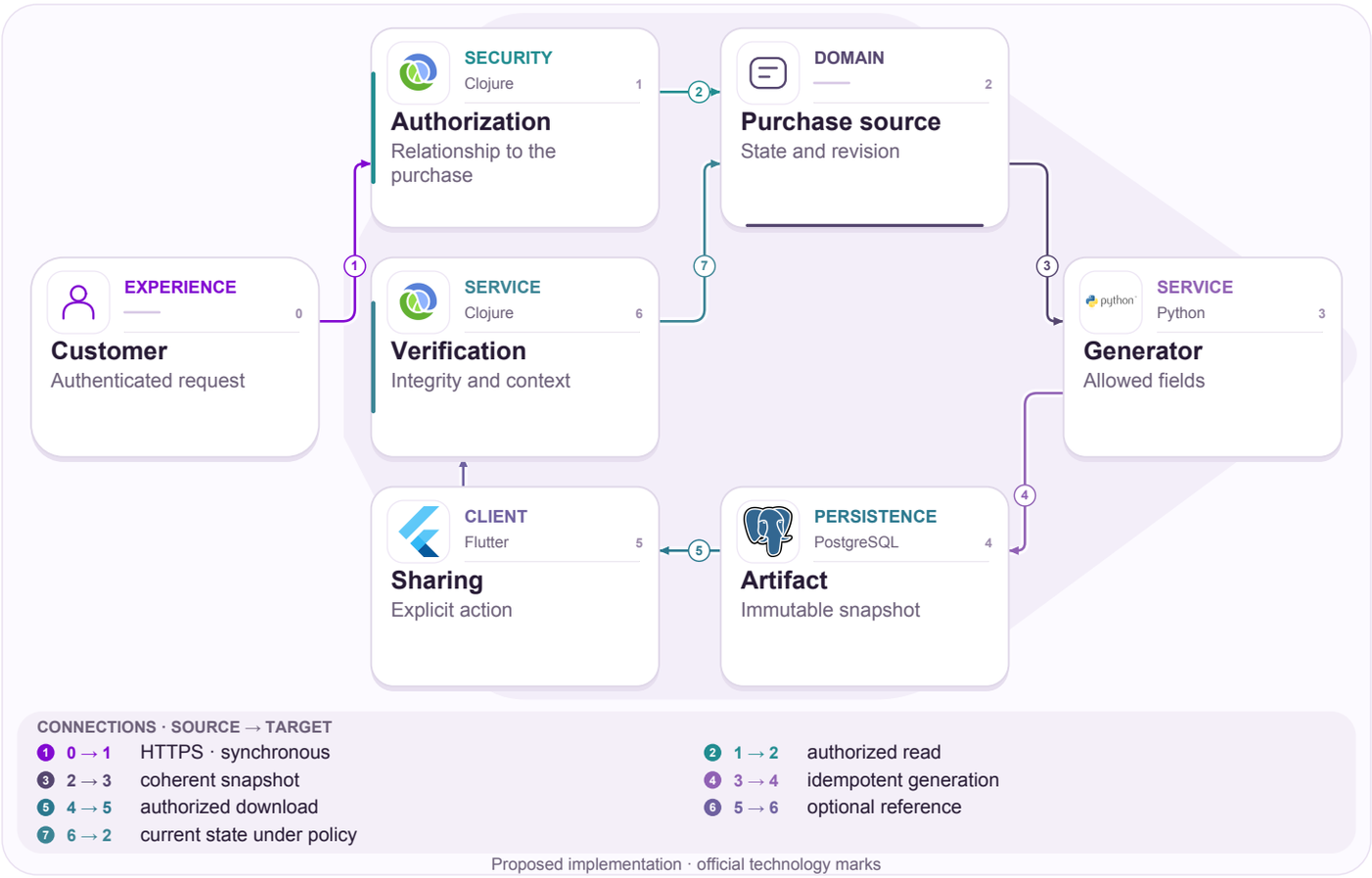
The document domain begins with the purchase lifecycle. A pending authorization may reserve capacity without representing the same situation as a confirmed purchase. A later reversal changes the record's meaning, but should not erase the fact that an earlier version was issued. The experience must state the status and reference date, avoiding a generic paid label when available information does not support it.

In the lab, Purchase contains an identifier, owner, revision, amount, merchant, and state. A closed set of labels translates the three synthetic states. The revision changes when relevant content changes. The document receives a snapshot copy rather than querying fields individually during assembly. This prevents mixing an amount from one revision with a status from another. A complete real-card rule would require a richer vocabulary.

An issued document and a current document are different objects. A shared file may continue to exist after a reversal, so copies already distributed cannot be promised automatic updates. A verifiable reference can provide the latest status, provided it does not expose data to unauthorized third parties. The product decision must explain that limitation. Customers need to know whether they are sending a historical snapshot or a live query, and recipients must be able to interpret its scope.

PROPOSED ARCHITECTURE · NOT AN INTERNAL DESIGN

Export an authorized snapshot



Proposed external architecture. Boxes represent responsibilities; arrows make contracts explicit.

Authorization belongs to object access, not merely to the existence of a session. The generator receives a coherent snapshot and applies a field allowlist. It does not automatically copy all source data into the document. The artifact is immutable; another revision produces another record and preserves the relationship between them.

The verification arrow is optional and requires a privacy decision. Making a public identifier queryable can turn convenience into exposure of financial activity. External verification could confirm only integrity and version, without returning the full history. The lab uses a local HMAC to detect tampering but does not implement public verification infrastructure. The architecture separates these responsibilities so that choosing PDF or an image does not conceal important decisions about access, temporal validity, and the meaning of what is shared.

CODE WITH EXPLICIT SCOPE

An explicit allowlist of shareable fields

PYTHON / ORIGINAL PROPOSAL

```python
def export_record(purchase, subject, secret):
    if purchase.owner != subject:
        raise PermissionError('objeto não autorizado')
    if purchase.status not in LABELS:
        raise ValueError('estado desconhecido')
    if type(purchase.amount_cents) is not int or purchase.amount_cents <= 0:
        raise ValueError('centavos inteiros positivos')
    if purchase.revision < 1:
        raise ValueError('revisão inválida')
    body = {
        'kind': 'synthetic-purchase-record',
        'purchase_id': purchase.purchase_id,
        'revision': purchase.revision,
        'amount_cents': purchase.amount_cents,
        'currency': 'BRL',
        'merchant': purchase.merchant,
        'status': purchase.status,
        'label': LABELS[purchase.status],
        'notice': 'Não é recibo fiscal nem prova de quitação da fatura.'
    }
    signature = hmac.new(secret, canonical(body), hashlib.sha256).hexdigest()
    return {'body': body, 'integrity_tag': signature}
```

Literal excerpt from labs/case_10.py. Local HMAC with a test key; neither a legally valid signature nor real infrastructure.

The executed generator compares the owner with the supplied subject and rejects invalid states or amounts. The document is assembled field by field; owner is not exported. The notice accompanies the content to prevent the record being mistaken for statement settlement or a tax receipt. The integrity tag covers the canonical body.

The subject and secret are controlled test inputs, not production authentication mechanisms. A real service would need trusted sources for both, key management, and consistent authorization. The example demonstrates content minimization and state preservation without claiming complete sharing security.

CODE WITH EXPLICIT SCOPE

Clojure: contract before rendering

CLOJURE / ORIGINAL PROPOSAL

```clojure
(ns portfolio.purchase-record)

(def labels
  {:pending "Autorização pendente"
   :confirmed "Registro de compra confirmada"
   :reversed "Compra estornada"})

(defn public-record [purchase subject]
  (when-not (= subject (:owner purchase))
    (throw (ex-info "Forbidden" {:kind :forbidden})))
  (let [status (:status purchase)
        label (get labels status)]
    (when-not label
      (throw (ex-info "Unsupported state"
                      {:kind :invalid-state})))
    (assoc
      (select-keys purchase
        [:purchase-id :revision :amount-cents
         :merchant :status])
      :label label
      :kind :purchase-record
      :notice "Não comprova quitação da fatura")))

;; Proposta pura: autenticação e assinatura são externas.
```

Illustrative Clojure excerpt, not executed; names and contracts proposed by the author.

This educational function expresses the same boundary in Clojure: verify object access and select fields before rendering. A visual template receives only the public model, reducing the risk of accidentally including internal information. Complete type and value handling would still be necessary; the excerpt emphasizes access and semantics alone.

The contract is independently authored and does not correspond to a known Nubank API. A pure function allows the transformation to be reviewed separately from PDF, storage, and transport. If the public model is wrong, changing the renderer does not fix it; it distributes the same incorrect interpretation in another form.

CODE WITH EXPLICIT SCOPE

Persist the artifact’s revision and purpose

SQL / ORIGINAL PROPOSAL

```sql
-- Esquema proposto; não executado pelo laboratório.
CREATE TABLE purchase_records (
  document_id TEXT PRIMARY KEY,
  purchase_id TEXT NOT NULL,
  purchase_revision BIGINT NOT NULL,
  template_revision INTEGER NOT NULL,
  document_kind TEXT NOT NULL,
  state_at_issue TEXT NOT NULL,
  payload_digest TEXT NOT NULL,
  storage_reference TEXT NOT NULL,
  issued_at TIMESTAMP NOT NULL,
  owner_reference TEXT NOT NULL,
  UNIQUE (purchase_id, purchase_revision,
          template_revision, document_kind),
  CHECK (purchase_revision > 0),
  CHECK (template_revision > 0)
);

CREATE INDEX records_by_purchase
  ON purchase_records (purchase_id, purchase_revision);

-- Acesso ao blob ainda exige autorização.
-- Digest não torna o arquivo anônimo ou público.
```

Proposed reference SQL. Not executed; the lab uses immutable Python objects.

The schema is a proposed option for storing artifact metadata, not a description of Nubank's persistence. Identity combines purchase revision, template revision, and document type. A visual change can therefore produce a different file without pretending the transaction changed. The storage reference must not be treated as a permanent public URL.

An implementation would need decisions on retention, encryption, access policy, and reconciliation between the record and blob. The lab keeps only in-memory objects and does not execute this SQL. This distinction prevents a small exercise from being used as proof of durable or secure document distribution.

ANALYSIS AND DECISION

A reversal can happen while the file is being generated

The main race is between reading the purchase and issuing the document. If the state changes during generation, canceling the file is not necessarily correct: it may represent a valid historical snapshot. What must not happen is mixing fields from different revisions or omitting the temporal reference. The decision depends on the document's purpose and must be communicated before sharing.

Another failure occurs after generation. The file exists, but the customer loses the response and retries. A key based on purchase, revision, template, and purpose can return the same artifact, avoiding duplicate cost and indistinguishable versions. Sharing remains an explicit device action; generating a file does not authorize sending it to a previously used recipient. These are different consents and effects.

There are also integrity and access failures. A valid tag does not mean the requester may read the object. An expired link should not leak metadata to explain the error. A defective template can produce an incorrect document with a correct hash, because integrity only shows that the bytes have not changed. Tests must therefore examine meaning, authorization, and rendering as well as cryptographic verification. None of these layers replaces the others.

CODE WITH EXPLICIT SCOPE

Tests distinguish integrity from meaning

PYTHON / ORIGINAL PROPOSAL

```python
def test_pending_has_no_paid_claim(self):
    self.assertEqual(self.make()['body']['label'], 'Autorização pendente')

def test_reversed_has_explicit_status(self):
    doc = self.make(status='reversed', revision=2)
    self.assertEqual(doc['body']['label'], 'Compra estornada')
    self.assertEqual(doc['body']['revision'], 2)

def test_owner_not_in_shareable_body(self):
    self.assertNotIn('owner', self.make()['body'])
    self.assertNotIn('alice', canonical(self.make()).decode())

def test_wrong_subject_is_denied(self):
    with self.assertRaises(PermissionError):
        export_record(self.purchase, 'bob', self.secret)

def test_same_snapshot_same_document(self):
    self.assertEqual(self.make(), self.make())
```

Excerpt from labs/case_10.py. Result: 7 tests passed; see results_10.txt.

Seven tests passed in the complete lab. The excerpt checks labels, revision, minimization, denied access, and determinism for the same snapshot. The remaining tests alter the amount after issuance and verify rejection of unsuitable values. A document can retain integrity while being inappropriate for its purpose, which is why labels are also asserted.

The tests do not validate authentication, secret management, or interpretation by third parties. For a pilot, I would add content review with Product and Legal, samples of installment purchases and reversals, and evaluation of document reading outside the app. External context changes what must be explicit.

ANALYSIS AND DECISION

Self-service that resolves needs, not just generates files

Counting downloads is insufficient to evaluate the product. Someone may download three documents because none meets the need. I would carefully measure the declared or inferred reason, successful generation, and subsequent contact about the same purchase. The goal is to determine whether the record replaces an eligible support request without pushing uncertainty to another channel.

For technical health, I would track failures by stage: authorization, reading, rendering, storage, and delivery. This separates unavailability from denied access and reveals templates that fail on specific content. A correlation identifier should link the request to stages without placing the entire document in logs. Amounts, merchants, and names are not necessary for every operational metric.

Quality requires sampling issued documents under appropriate access controls and purpose. The dashboard should highlight incorrect states, missing revisions, and discrepancies between artifact and source. I would also track cost per issuance and artifact reuse rate. Greater efficiency with semantically confusing documents is not success. Support needs a way to record that a customer received a file but still lacked suitable evidence for the situation that prompted the request.

ANALYSIS AND DECISION

Launch by purpose, then broaden coverage

The first pilot should choose a narrow purpose and a well-understood set of states. For example, it could provide a confirmed-purchase record for reference and informational sharing, with reviewed wording. Covering every situation initially creates an overly broad implicit contract and obscures which documents actually resolve requests. The initial restriction must be clear to customers and support.

Before exposure, I would compare generated examples with information currently provided by support, investigating differences in semantics and content. The team would test the full experience of obtaining the file, opening it on another device, and understanding its limits. People receiving it outside the app lack the context of the previous screen. The file must carry sufficient information without accumulating unnecessary personal data.

Expansion would be guided by resolution and repeat contact, with sampling of problematic cases. Disabling the button prevents new issuance but does not retrieve files already shared; that consequence belongs in the risk plan. Template revisions must be traceable, and communicating corrections requires an appropriate process. The feature flag controls future exposure; document governance addresses what has already been issued. Treating them as the same leaves an important part of the journey without an owner.

ANALYSIS AND DECISION

Sharing creates a new privacy boundary

Inside the app, access may depend on session and authorization. After sharing, the file can circulate beyond the original intent. The proposal minimizes information during generation: no full card number, secret, or internal identifier automatically enters the public model. Field selection must match the document's purpose, not the convenience of reusing an API object.

Verifiable links require additional care. A long token can reduce guessing, but remains a credential if it grants content access. Expiration, revocation, scope, and usage records must be designed for the case. A public page confirming a purchase exists can disclose information even without displaying an amount. Verification must separate integrity evidence from querying current financial state.

The lab HMAC uses a fixed test secret to make tampering observable. This is not production key management, a public signature, or legal validity. A real service would need rotation, secure storage, environment separation, and permission review. Malicious content in merchant names would also need testing to prevent renderer problems. Document security spans data, representation, and distribution; it does not end with selecting a cryptographic algorithm.

ANALYSIS AND DECISION

ADR 10 · Issue a verifiable purchase snapshot

The proposed decision is to issue an immutable snapshot with explicit state, revision, and purpose. The context is a self-service demand whose meaning varies between customers. The objective is to reduce support dependence for eligible requests without calling every record proof of payment. The public model is separate from the internal object and precedes the visual format decision.

One alternative is to export the screen directly. This preserves a familiar appearance, but may include unnecessary information, omit temporal context, and depend on device size. Another is to always generate a document with the latest data and no revision reference. That simplifies current reading but complicates explaining what someone shared before a reversal. An explicit snapshot offers traceability at the cost of retention and clear communication about updates.

I would reconsider the decision if the dominant purpose requires a document from another participant, such as the merchant, or if an informational record does not reduce repeat contact. The better product might then guide people toward the correct request rather than create more formats. I would also record that cryptographic integrity does not establish the commercial truth of the content. Architecture guarantees association with a source and revision; the domain remains responsible for the issued document's meaning and legitimacy.

ANALYSIS AND DECISION

EM execution: correct language prevents rework

As Engineering Manager, I would bring together Cards, Support, Design, and those responsible for document interpretation. The first decision would name the purpose and delimit covered states with approved examples. Technical work would then have concrete acceptance criteria: a coherent source, object-level authorization, minimization, traceability, and comprehension outside the app. I would avoid splitting the team into PDF generation and button creation before that alignment.

The value model would use eligible requests, effective adoption, resolution without repeat contact, and incremental cost per contact. It would deduct issuance, storage, maintenance, and investigation of problematic documents. A download is not an avoided support interaction, and freed capacity does not automatically become financial savings. The pilot must show that people achieve their purpose, not merely export a file.

The Staff question would be: what remains true when a purchase is reversed between generation, download, and the recipient reading it? The answer must separate historical snapshot, current state, and query authorization. The leadership question would be who responds when a document is intact but fails its expected purpose. A mature team does not use successful generation to close a product problem. It observes the outcome and adjusts the promise made to the customer.

ANALYSIS AND DECISION

11 · The published result is a starting point

11

python kafka Datomic aws

How we reduced critical path latency by 76% at Nubank

Original official publication, accessed 23 October 2026.

Nubank published a case reducing the P90 of a payment-conditions API from approximately 1,200 to 280 milliseconds. The account describes local improvements followed by event-based aggregation with persisted reads and fallback. That result belongs to the documented flow and period; it is neither the latency of the entire app nor a gain delivered by this portfolio.

The independent contribution begins with the next question: what criteria identify another read suitable for materialization? The answer must combine composition cost, dependency variability, and explicit tolerance for stale data. Observing a slow query is insufficient. A condition shown to a customer may tolerate a snapshot briefly, while financial authorization requires validation at another boundary.

I would begin with tracing, redundant-call identification, and latency distribution by dependency. I would propose new persistence only after exhausting simple changes and understanding data semantics. This chapter's lab models versioned components, freshness budgets, and fallback. It neither measures performance nor executes payments. Its purpose is to make a read decision verifiable: when the projection may be used and when it must yield to the source, without hiding the operational cost transferred to event processing.

How we reduced critical path latency by 76% at Nubank

ANALYSIS AND DECISION

Freshness is a contract per component

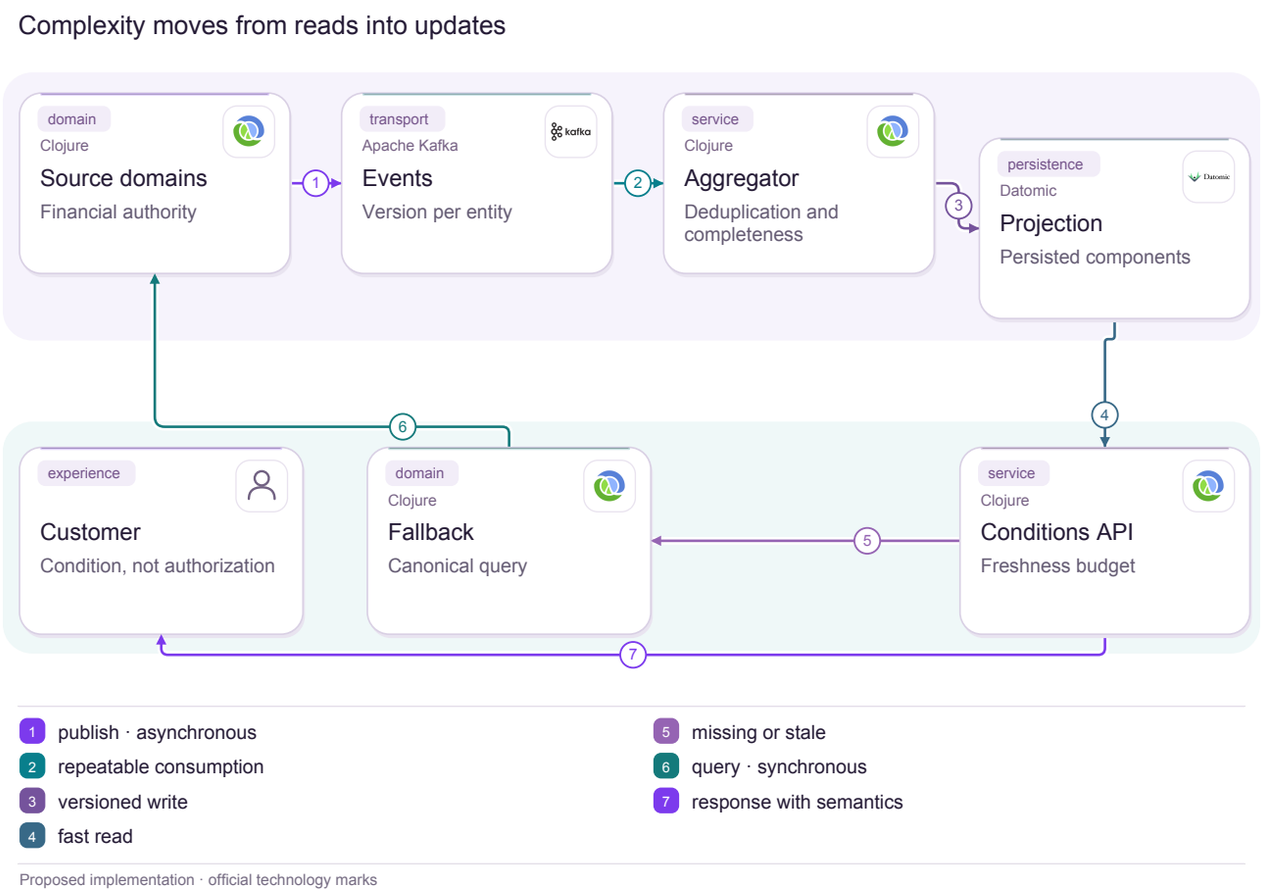
A composite projection combines data that can change at different rates. A single update timestamp can conceal that only one component was refreshed. The synthetic model therefore tracks version and observed time separately for balance and limit. A read uses the projection only when both are present and within the configured budget. Choosing that budget belongs to the domain, not the cache alone.

Version and time answer different questions. Version prevents an older update from replacing a newer one; time helps assess age. A recently transported event can carry old state, and a future clock can make data appear fresh indefinitely. The lab rejects future timestamps relative to the clock supplied by the test. A distributed deployment would need defined time sources and clock-skew tolerance.

Completeness must also be distinguished from validity. Having every field does not mean those fields may authorize an operation. The projection may serve an estimate or presentation condition while confirmation queries the responsible service. The contract must make this distinction explicit so another team does not reuse a fast read as financial authority. A fast interface creates incentives for broader usage; documented, verifiable limits prevent convenience from becoming dangerous coupling.

PROPOSED ARCHITECTURE · NOT AN INTERNAL DESIGN

Complexity moves from reads into updates



Proposed external architecture. Boxes represent responsibilities; arrows make contracts explicit.

The proposed architecture favors predictable reads but adds an update system that must be operated. A consumer can lag, receive duplicate events, or rebuild a projection. The API must not interpret projection-store availability as proof of freshness: the read contract checks completeness and age for each relevant component.

Fallback preserves a correctness path but can receive a traffic surge when the projection fails. Capacity, concurrency limits, and fallback priority therefore belong in the initial design. The arrow to the customer carries conditions with defined meaning; it does not transfer financial authorization authority. The lab uses local functions to represent these boundaries. A real proposal would require observability, degradation policies, and domain contracts. Materialization is worthwhile only when the new operational cost is justified by the variability removed from the critical path.

CODE WITH EXPLICIT SCOPE

Apply a version without accepting a regression

PYTHON / ORIGINAL PROPOSAL

```python
def apply(self, component, version, cents, observed_at, now):
    if component not in {'balance', 'limit'}:
        raise ValueError('componente desconhecido')
    if type(cents) is not int or cents < 0:
        raise ValueError('centavos inválidos')
    if type(version) is not int or version < 1:
        raise ValueError('versão inválida')
    if observed_at > now:
        raise ValueError('timestamp futuro')
    candidate = (version, cents, observed_at)
    previous = self.parts.get(component)
    if previous and version < previous[0]:
        return False
    if previous and version == previous[0]:
        if previous != candidate:
            raise ValueError('versão conflitante')
        return False
    self.parts[component] = candidate
    return True
```

Literal excerpt from labs/case_11.py. Executed Python; synthetic projection without financial authorization.

This executed core receives a component, version, value, and time. Older updates are ignored; a repeated version with different content raises a conflict. Comparison does not depend on time alone, because transport and clocks do not independently establish business order. The component set is closed to keep the contract inspectable.

Storage is in memory and does not demonstrate atomicity across processes. In production, comparison and update would need a consistent boundary with defined recovery and reprocessing. The excerpt supports discussion of version semantics before choosing Datomic, DynamoDB, or another persistence mechanism.

CODE WITH EXPLICIT SCOPE

Clojure: a read with an explicit alternative path

CLOJURE / ORIGINAL PROPOSAL

```clojure
(ns portfolio.conditions)

(def required-components #{:balance :limit})

(defn fresh? [now max-age parts]
  (and
    (every? #(contains? parts %) required-components)
    (every?
      (fn [{:keys [observed-at]}]
        (<= 0 (- now observed-at) max-age))
      (vals parts))))

(defn read-conditions
  [projection now max-age canonical-reader]
  (if (fresh? now max-age projection)
    {:source :projection
     :purpose :display-conditions
     :data projection}
    {:source :canonical
     :purpose :display-conditions
     :data (canonical-reader)}))

;; canonical-reader é contrato proposto, não API do Nu.
;; Autorizar pagamento pertence a outra fronteira.
```

Proposed Clojure excerpt, not executed; canonical-reader is an independent abstract dependency.

The function receives the canonical reader as an explicit dependency. This makes path selection testable without opening a real connection. The response carries source and purpose to aid diagnosis, but metadata does not replace access discipline. A consumer should not treat the declared purpose as authorization to bypass financial validation.

The excerpt is educational and includes no timeout, bulkhead, concurrency limit, or canonical-path failure handling. Those controls are needed so a projection failure does not take down the source. The design must specify behavior when both paths are unavailable, keeping the customer response honest.

CODE WITH EXPLICIT SCOPE

Kafka transports a versioned fact, not a global guarantee

JSON / ORIGINAL PROPOSAL

```json
{
  "topic_proposed": "portfolio.conditions-state.v1",
  "partition_key": "synthetic-customer-8",
  "event_id": "demo-limit-12",
  "schema_version": 1,
  "entity": {
    "type": "limit",
    "id": "synthetic-limit-8",
    "version": 12
  },
  "state": {
    "amount_cents": 2000,
    "currency": "BRL"
  },
  "time": {
    "observed_at": "2026-10-23T12:00:00Z",
    "emitted_at": "2026-10-23T12:00:01Z"
  },
  "purpose": "conditions-projection",
  "authorization_authority": false,
  "synthetic": true
}
```

Independently proposed envelope and topic name. Not executed in Kafka; the lab uses local calls.

The proposed envelope retains domain identity and version even when transport provides partition ordering. Different partitions, reprocessing, and consumers writing to external storage still require their own rules. Producer configuration must not become a promise of a single end-to-end effect.

The partition key is a distribution and affinity choice with consequences for volume and hotspots. The example contains no real data and creates no topic. The purpose field documents the contract; it is not sufficient security control. Deployment would require a compatible schema, permissions, retention, and a verifiable projection-rebuild plan.

Apache Kafka · Design

ANALYSIS AND DECISION

Fallback can become the next failure

An unavailable projection immediately changes load distribution. If all reads switch to the source simultaneously, the mechanism intended to protect correctness can saturate it and worsen the outage. The design must consider contingency capacity, prioritization, and concurrency limits. The correct response to excess load may be an explicit wait, not unlimited retries across multiple layers.

Late events create another failure category. Storage responds quickly, but content is stale. HTTP success rate can remain excellent while the experience uses unsuitable information. Freshness checks must be in the read path with component-specific criteria. Policy must also define what happens when time is untrustworthy or the source publishes no heartbeat for unchanged state.

Rebuilding deserves first-class treatment. Reprocessing history with new logic can produce results different from existing projections. I would compare parallel projections and record discrepancies before switching traffic. Older updates must not overwrite newer data during backfill. The solution may involve revision namespaces and a promotion barrier. These mechanisms add complexity, but make the true cost of removing synchronous composition from the critical path explicit.

CODE WITH EXPLICIT SCOPE

Freshness tests, not a latency benchmark

PYTHON / ORIGINAL PROPOSAL

```python
def test_fresh_projection_used(self):
    result = self.view.read(105, 10, self.origin)
    self.assertEqual(result['source'], 'projection')

def test_old_component_forces_fallback(self):
    self.view.apply('balance', 2, 900, 119, 119)
    result = self.view.read(120, 10, self.origin)
    self.assertEqual(result['source'], 'canonical')
    self.assertEqual(self.view.fallbacks, 1)

def test_old_version_does_not_replace(self):
    self.view.apply('balance', 2, 900, 101, 101)
    self.assertFalse(self.view.apply('balance', 1, 1000, 100, 101))
    self.assertEqual(self.view.parts['balance'][0], 2)

def test_conflicting_version_rejected(self):
    with self.assertRaises(ValueError):
        self.view.apply('limit', 1, 3000, 100, 100)
```

Excerpt from labs/case_11.py. Result: 7 tests passed; see results_11.txt. No benchmark.

All seven local tests passed. The excerpt verifies selection of a fresh projection, fallback for a stale component, and rejection of version regression. Other tests cover conflicting content at the same version, future timestamps, missing components, and explicit unavailability. The clock is an input, allowing boundaries to be examined without waiting in real time.

No result measures milliseconds or reproduces Nubank's published gain. Performance evaluation would require representative load, dependency distributions, and tail observation. The lab demonstrates only local read decisions. This separation protects the material's credibility: a functional property can be correct even when the architecture has not yet demonstrated economic benefit.

ANALYSIS AND DECISION

Observe writes to trust reads

The read dashboard must show latency and response source, but that is only half the story. Projection health depends on consumer lag, completeness, component age, and divergence from the source. I would avoid a single cache-hit indicator: a high rate can mean efficiency or persistent use of unsuitable content, depending on the freshness contract.

Tail metrics must be segmented by path. Combining materialized reads and fallback can conceal degradation of the latter until it receives more traffic. I would also track fallback frequency and reason, version conflicts, and rebuild time. Individual identifiers belong in access-controlled traces, not metric labels that unnecessarily multiply cardinality.

The business objective is journey completion with correct conditions and acceptable waiting. Improved P90 accompanied by more discrepancies or confirmation failures is not a win. I would define guardrails before rollout and maintain sampled comparison. Alerts must distinguish delayed transport, a silent producer, and an unavailable source because each requires a different action. Operating a fast read means understanding the path that feeds it, not merely the database that responds.

ANALYSIS AND DECISION

Silent comparison before serving conditions

Deployment would start in comparison mode: build the projection and query its result on a sample without changing the customer response. Differences would be classified as expected lag, transformation errors, incomplete sources, or incompatible semantics. An aggregate equality percentage is insufficient; critical fields need their own analysis and a set of explainable examples.

After validating freshness and correctness, a small cohort could receive materialized reads. Fallback would be deliberately exercised in a controlled environment to confirm capacity and behavior. The plan would include exposure limits, pause criteria, and a return to the previous path. I would also test the inverse situation: a degraded source while the projection looks healthy, since serving stale data without a limit can conceal a meaningful problem.

Expansion depends on net benefit, not architectural preference. I would measure perceived latency, journey completion, processing cost, and operational effort. If most queries require fallback, the staleness tolerance may be too small for the chosen pattern. I would then revisit composition or the contract rather than silently relax the freshness guarantee. The experiment must be allowed to conclude that materialization is unsuitable for that flow.

ANALYSIS AND DECISION

Fast financial data still need boundaries

The projection increases the number of places where relevant data exist. This requires explicit purpose, access, and retention. Storage designed for fast reads should not become a generic repository open to any service. Each consumer must demonstrate that its use is compatible with the offered semantics and freshness. Authorizing a payment with a presentation field would be a contract change, not a simple optimization.

For performance, the first question is where time is being spent. Reducing duplicate calls, propagating validated context, and removing layers with no domain role may be sufficient. Materialization is justified only when structural cost remains and asynchronous updates are acceptable. I would also consider object size, invalidation frequency, and traffic concentration: an overly broad projection can shift bottlenecks into writes and storage.

The lab maintains synthetic balance and limit values, without authenticated identity or customer isolation. A real implementation would need to preserve read authorization and prevent entity mixing in the aggregator. Tests would include incorrect keys, insufficient permissions, and backup restoration. Reliability and security meet at this boundary: a fast response to the wrong person is a serious failure even when version, age, and latency are within limits.

ANALYSIS AND DECISION

ADR 11 · Materialize only with freshness semantics

The proposed decision is to materialize a composite read only when documented freshness tolerance and an operable canonical contingency path exist. The context is variability from synchronous dependencies, not a rule that caches always improve architecture. The objective is to reduce tail latency and make reads predictable while preserving the meaning of data used by the journey.

One alternative is to retain synchronous composition while optimizing calls, parallelism, and context propagation. It reduces operational state but retains dependence on service availability and tail latency. Another is a cache with a single expiry and no component versions. It is simpler, but can hide inconsistency and cannot explain which data became stale. A versioned projection offers finer control at the cost of consumption, reconciliation, backfill, and observability.

I would record abandonment conditions: a low share of eligible reads, disproportionate operational cost, or inability to define safe freshness. I would also state what is outside the decision: financial authorization mechanisms and Nubank's internal architecture are not part of this exercise. Architectural review must compare alternatives using journey data. The published result inspires a way of reasoning; it does not relieve the team of demonstrating that the same pattern fits another context.

ANALYSIS AND DECISION

EM execution: distribute the new architecture's cost

As Engineering Manager, I would align Product, source domains, and Platform on the problem and guarantees. One team may gain latency while another inherits event load, support, and contract maintenance. Planning must recognize that transfer to avoid a local optimization that creates operational debt elsewhere. I would define owners for producers, projection, API, fallback, and reconciliation, with operational criteria before launch.

Value would be estimated from eligible attempts, causal change in completion, and margin per completion, minus infrastructure and operations. I would also measure capacity recovered through fewer calls. These components cannot be added without checking overlap. The 76% gain belongs to the official case; no lab number supports projecting revenue or availability. An investment decision requires an internal baseline and sensitivity analysis of assumptions.

The Staff question would be: how do we prove that a fast read remains acceptable when just one component stops updating and fallback receives all traffic? The answer must cover version, freshness, capacity, and failure experience. The leadership question would be who can stop rollout and which contingency cost was funded. An architecture is ready to expand only when the organization can operate it as clearly as it can draw it.

ANALYSIS AND DECISION

12 · The cloud has limits and lead times

12

Managing Cloud Limits

Original official publication, accessed 23 October 2026.

A Nubank engineering publication describes multiple accounts, sharding, and monitoring of utilization, quotas, and costs. The case presents capacity as a shared responsibility between platform and domains. The article's scale figures are historical; this chapter does not depend on treating them as a current inventory. The question is how to turn consumption signals into decisions before a journey hits a limit.

A quota is more than a configuration entry. It has a scope, unit, source, increase lead time, and relationship to dependencies. Limits may apply by region, account, resource, or service. An application can have CPU available yet be unable to create the connections, addresses, or components it needs. Planning only the most visible dimension creates a false sense of headroom. The capacity model must represent the bottleneck relevant to the journey.

The proposal combines an inventory of limits, explicit forecasting, and reservations before admitting planned changes. The lab tests multidimensional reservations and local concurrency without provisioning resources. It simulates neither AWS nor Kubernetes. Its role is to demonstrate that two teams cannot simultaneously consume the same promised headroom. Production design would require fresh sources, reconciliation of reservations against actual resources, and handling the delay between requesting capacity and having it available.

ANALYSIS AND DECISION

Observed, reserved, and available are different quantities

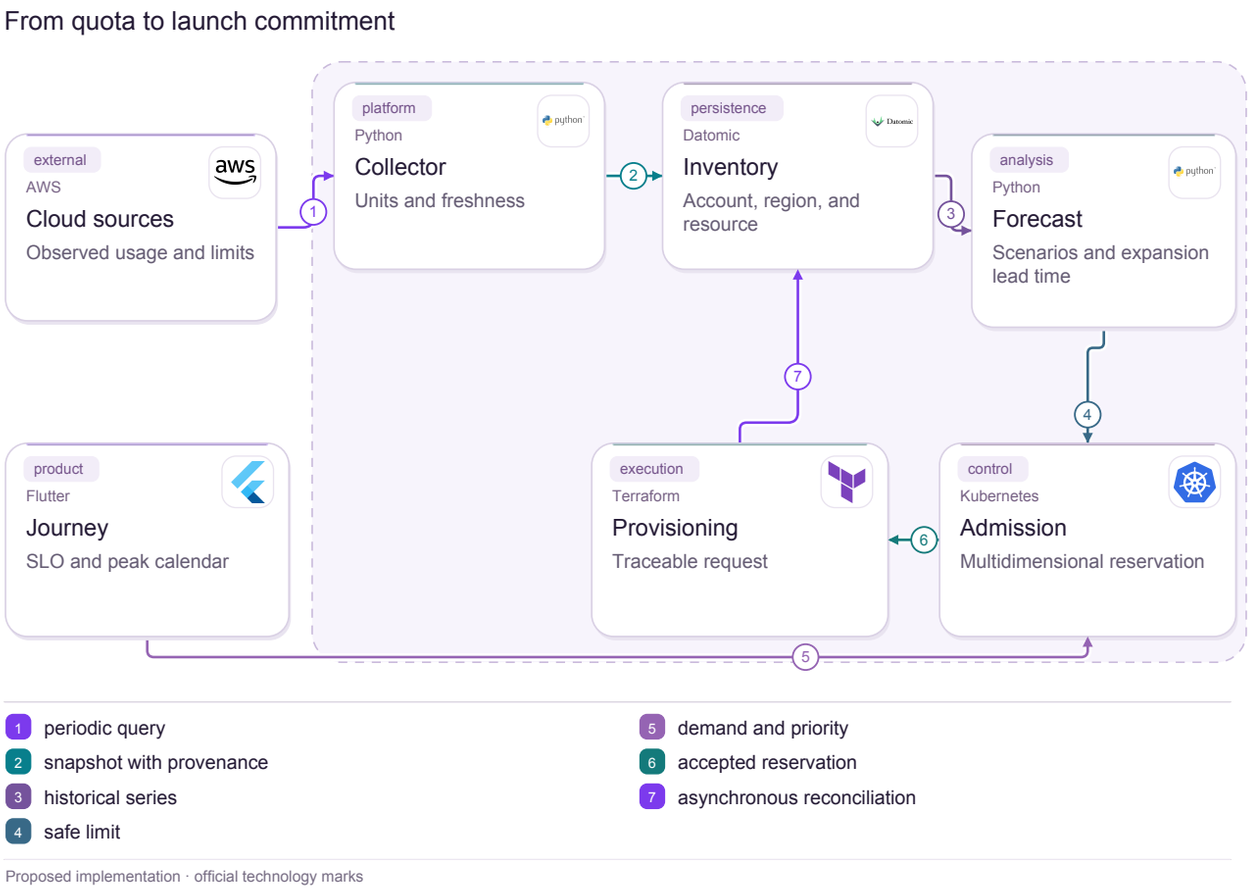
Observed consumption describes what a source measured. A reservation describes accepted intent not yet reflected in that measurement. Safety margin represents capacity we do not want to commit during normal operation. Safe availability is the limit minus those three components, respecting each resource's unit. Mixing reservations and observed consumption can count the same demand twice or, worse, not count it at all.

In the lab, baselines are constant and reservations are maintained under a lock. There is no provisioning, so no automatic transition from reservation to actual consumption exists. This limitation avoids hiding the hardest problem: reconciling the request identifier, created resource, and subsequent measurement. A real implementation would need states such as requested, reserved, provisioning, observed, and closed, with defined timeouts and recovery for each.

Capacity is multidimensional. Demand is accepted only when every required dimension fits; reserving CPU and failing on a network address would leave a useless fragment without atomicity. The model rejects the whole request, allowing the team to revise its size or schedule. Priority policy must also be explicit. A critical expansion may justify a different decision, but the exception needs an owner and evidence, not an informal path that ignores the margin without a record.

PROPOSED ARCHITECTURE · NOT AN INTERNAL DESIGN

From quota to launch commitment



Proposed external architecture. Boxes represent responsibilities; arrows make contracts explicit.

The collector normalizes units and records source freshness. A quota without recent data should not appear unlimited. The inventory separates scopes to avoid comparing consumption in one region with a limit in another. Forecasting produces scenarios, not certainties: linear growth can help anticipation, but does not automatically describe campaigns or concentrated events.

Admission links capacity to the launch commitment. It reserves every relevant dimension and tracks provisioning outcomes. Asynchronous feedback avoids holding a transaction open while a provider creates resources. Reconciliation must recognize completed operations whose responses were lost, as well as abandoned reservations. The journey supplies priority and schedule; the platform supplies headroom and required time. This contract enables evidence-based discussion of delay, reduced scope, or expansion without turning a technical estimate into a promise of unlimited availability.

CODE WITH EXPLICIT SCOPE

Multidimensional reservations without local overbooking

PYTHON / ORIGINAL PROPOSAL

```python
def allocate(self, token, demand):
    with self.lock:
        if token in self.requests:
            if self.requests[token] != demand:
                raise ValueError('token conflitante')
            return False
        if not demand or set(demand) - set(self.limits):
            raise ValueError('recurso desconhecido ou demanda vazia')
        if any(type(v) is not int or v <= 0 for v in demand.values()):
            raise ValueError('demanda inteira positiva obrigatória')
        available = {}
        for resource, limit in self.limits.items():
            pending = sum(x.get(resource, 0)
                          for x in self.allocations.values())
            available[resource] = (limit - self.used[resource]
                - self.reserve_floor[resource] - pending)
        if any(amount > available[key] for key, amount in demand.items()):
            raise RuntimeError('capacidade insuficiente')
        self.allocations[token] = dict(demand)
        self.requests[token] = dict(demand)
        return True
```

Literal excerpt from labs/case_12.py. Synthetic in-memory reservation; provisions neither AWS nor Kubernetes.

The executed code calculates and commits availability under the same lock. Two threads cannot promise the same headroom. Token history remains after release: replay recognizes the closed operation without recreating the reservation; different content raises a conflict. New intent requires a new token. Independent copies protect records from changes to the supplied dictionary.

The lock protects one process, and history is volatile with no retention policy. Distributed coordination would require persistence, atomic comparison, and expiry tied to the replay window. The lab specifies behavior; local memory neither governs real resources nor provides idempotency after restart.

CODE WITH EXPLICIT SCOPE

Prometheus: alert on headroom and stale data

YAML / ORIGINAL PROPOSAL

```yaml
groups:
  - name: portfolio-capacity
    rules:
      - alert: SafeCapacityExhausted
        expr: portfolio_safe_headroom < 0
        for: 5m
        labels:
          severity: warning
          owner: platform-demo
        annotations:
          summary: "Margem segura negativa"
          description: "Revisar reservas e demanda planejada."
      - alert: CapacitySourceStale
        expr: time() - portfolio_capacity_collected_at > 900
        for: 10m
        labels:
          severity: warning
          owner: platform-demo
        annotations:
          summary: "Fonte de capacidade desatualizada"
          description: "Confirmar coleta antes de admitir expansão."

# Métricas, limiares e responsável são exemplos propostos.
# Validar regras com dados e ferramentas do ambiente real.
```

Reference YAML, not executed. Metrics and names are proposed, not internal Nubank telemetry.

The reference rules distinguish insufficient headroom from insufficient information. If collection stops, a stable dashboard can look healthy while consumption keeps growing. Source age must inform operational decisions. The example thresholds are synthetic and require calibration against collection frequency and available reaction time.

Prometheus can evaluate these conditions, but an alert neither reserves capacity nor requests a quota increase. The owner needs a verifiable procedure and access to sources. Labels should use controlled dimensions such as account and quota type, avoiding per-request identifiers that would multiply time series without proportional benefit.

Prometheus · Alerting rules

CODE WITH EXPLICIT SCOPE

A forecast must carry its assumptions

JSON / ORIGINAL PROPOSAL

```json
{
  "scope": "synthetic-account-region",
  "resource": "demo-network-addresses",
  "unit": "count",
  "observed_used": 70,
  "provider_limit": 100,
  "safety_reserve": 10,
  "pending_reservations": 0,
  "forecast": {
    "method": "linear-scenario",
    "daily_growth": 5,
    "days_to_safe_threshold": 4,
    "assumption": "constant growth, no releases"
  },
  "expansion": {
    "lead_time_days": 7,
    "status": "not_requested"
  },
  "collected_at": "2026-10-23T12:00:00Z",
  "source": "synthetic-fixture",
  "decision": "review-before-new-demand"
}
```

Entirely synthetic contract and numbers; linear arithmetic covered by the forecast test.

The values illustrate twenty units of headroom before the safe limit and growth of five per day, resulting in four days. If expansion takes seven days, waiting for a saturation alert would be too late. This is an arithmetic scenario, not a forecast about Nubank or a guarantee of a provider's lead time.

The contract records method, unit, and assumption so the estimate can be challenged. Seasonal growth, campaigns, and architecture changes require other scenarios. An approved increase request must also be distinguished from usable capacity. A launch decision must consider when the resource will be available, not merely when a request was opened.

ANALYSIS AND DECISION

Abandoned reservations and delayed measurements

A controller can reserve capacity and fail before starting provisioning. Without expiry or reconciliation, the reservation occupies headroom indefinitely. Conversely, releasing solely because a timeout occurred can permit overbooking if the provider completed the operation. The solution requires traceable identity and outcome queries, not a universal choice between reserving forever and releasing immediately.

Delayed measurement creates a second difficulty. Once the resource appears in the source, its reservation must stop being counted as pending demand. This transition must rely on a verifiable link, not an approximate quantity. Otherwise, a partial update can understate or overstate availability. The lab does not model this lifecycle; that limitation matters because the production challenge lies in coordinating intent and observation.

Correlated failures also change the required margin. Losing a shard can shift traffic to others, and expansion may depend on a resource already at its limit. I would test contingency scenarios with multiple bottlenecks, including connection and call-rate limits. Normal reservation policy does not replace failure capacity. They are different commitments that must be discussed against the availability objectives of affected journeys.

CODE WITH EXPLICIT SCOPE

Concurrency turns forecasting into a verifiable rule

PYTHON / ORIGINAL PROPOSAL

```python
def test_multidimensional_reservation_is_atomic(self):
    with self.assertRaises(RuntimeError):
        self.capacity.allocate('a', {'cpu': 2, 'ip': 6})
    self.assertEqual(self.capacity.allocations, {})

def test_same_token_not_reserved_twice(self):
    self.assertTrue(self.capacity.allocate('a', {'cpu': 3}))
    self.assertFalse(self.capacity.allocate('a', {'cpu': 3}))
    self.assertEqual(len(self.capacity.allocations), 1)

def test_conflicting_token(self):
    self.capacity.allocate('a', {'cpu': 3})
    with self.assertRaises(ValueError):
        self.capacity.allocate('a', {'cpu': 4})

def test_concurrent_demands_cannot_overbook(self):
    barrier = threading.Barrier(2)
    def attempt(token):
        barrier.wait()
        try:
            self.capacity.allocate(token, {'cpu': 3})
            return 'accepted'
        except RuntimeError:
            return 'rejected'
    with ThreadPoolExecutor(2) as pool:
        results = list(pool.map(attempt, ['a', 'b']))
    self.assertEqual(sorted(results), ['accepted', 'rejected'])
```

Excerpt from labs/case_12.py. Result: 9 tests passed; see results_12.txt.

Nine tests passed. The excerpt covers multidimensional demand, replay, conflict, and concurrency. The others verify release, unknown resources, the scenario, and identity after release: replay does not recreate a reservation, and a different payload remains blocked. The concurrency test uses a barrier to bring two attempts together, observing one acceptance and one rejection. History remains in memory only.

This is neither a load test nor a provider simulation. Production validation would need independent controllers, persistence, lost responses, and resource reconciliation. The local result establishes one property: previously committed headroom cannot be promised twice. The coordination mechanism may change without abandoning that property.

ANALYSIS AND DECISION

Days to the limit matter only alongside action lead time

A utilization-percentage dashboard is easy to understand but can hide urgency. Eighty percent of a stable quota may be comfortable; sixty percent with accelerating growth and slow expansion may require immediate action. I would compare projected time to the safe limit with the time needed to obtain capacity, including approval, provisioning, validation, and possible load migration.

I would also track forecast uncertainty. The gap between base and peak scenarios indicates how much headroom we are buying against variability. I would not turn linear regression into an exact failure date. Sources must record unit, scope, and age; a definition change should appear as a metadata event so a series break is not mistaken for a real consumption decrease.

At the product level, I would connect capacity to relevant journeys and campaigns. The operational response may be increasing a quota, reducing consumption, redistributing load, or delaying a delivery. Each alternative has cost and lead time. The alert should expose these options and an owner rather than merely color a cell red. I would measure limit-related incidents, urgent requests, and resolution time, but avoid claiming incidents prevented without a reasonable counterfactual. Preparation has value even when its effect cannot be reduced to a simple count.

ANALYSIS AND DECISION

Deploy as advice first, control second

The initial inventory should inform decisions without automatically blocking provisioning. This allows quota comparison, unit corrections, and identification of ownership gaps before creating a new critical dependency. I would choose a few high-impact resources and retain access to original data. When classification differs from team observations, the team must be able to explain why and correct the source.

Once data stabilize, admission could operate within a controlled scope, with a clear policy for controller unavailability. Failing open preserves speed but may compromise headroom; failing closed protects limits but may block urgent recovery. The choice depends on the operation and must be discussed before failure. An exceptional path must be auditable and restricted, rather than silently becoming routine.

Rollout would include exercises for interrupted collection, orphaned reservations, and quotas changed outside the tool. Rollback must preserve decision records for later reconciliation. Disabling a check neither deletes resources already created nor restores consumed capacity. This is therefore an operational process, not merely a flag. I would expand only when teams understand the contract and can resolve exceptions without permanently depending on the system's author.

ANALYSIS AND DECISION

Isolation has a cost and needs a reason

Separate accounts and clusters can reduce failure scope and enable independent evolution, but increase inventory, configuration, and operational effort. Shared infrastructure improves utilization but creates contention and requires governance. The decision should consider workload profile, sensitivity, availability requirements, and team capacity. A single pattern applied to every domain simplifies the paper design and may worsen total cost.

In Kubernetes, requests, limits, and scheduling policies operate at a different layer from provider quotas. Permission to create a pod does not guarantee a network address, physical capacity, or an available external dependency. I would keep these scopes explicit and test the complete provisioning chain. I would also observe how retries, autoscaling, and failure recovery interact, since independent automation can amplify competition for resources.

Security requires minimal access to sources and change operations. A metrics collector does not necessarily need permission to increase quotas or create infrastructure. Separating reading, recommendation, and execution reduces error impact and simplifies auditing. The platform must protect against both excess consumption and inappropriate blocking. The goal is predictable growth with understood limits, without confusing centralized control with well-distributed operational responsibility.

ANALYSIS AND DECISION

ADR 12 · Capacity as a multidimensional contract

The decision is to represent capacity by scope and dimension, including consumption, reservations, margin, and source freshness. The context is a growing platform where independent teams compete for shared resources. The proposal anticipates limits and prevents incompatible promises, keeping recommendation, admission, and execution separate. The lab implements only the local reservation rule.

One alternative is to operate with percentage alerts and no reservations. It costs less but does not protect against simultaneous planned demand. Another is to centrally reserve all future capacity, reducing autonomy and potentially holding resources for uncertain intentions. A multidimensional contract offers a middle ground: commitments with identity and validity, backed by reconciliation. This requires operating a lifecycle and revisiting forecasts, not merely writing an availability algorithm.

I would reconsider the control when source quality is insufficient or imposed friction exceeds the risk reduction. I would also measure how many exceptions are opened and why. Many exceptions may indicate an unsuitable policy rather than undisciplined teams. Architecture should help decisions about time, cost, and reliability. If it becomes a bureaucratic barrier without trustworthy information, it will lose adoption and be bypassed precisely when risk is greatest.

ANALYSIS AND DECISION

EM execution: negotiate headroom before committing

As Engineering Manager, I would connect launch planning to the capacity calendar. The conversation must happen before announcing dates that depend on unavailable resources. I would assign an owner to each critical quota and establish a review channel with Platform, domains, and FinOps. Product contributes demand and priority; platform contributes limits, alternatives, and realistic expansion lead time.

Value may include fewer urgent requests, less waste, and lower outage exposure. These components require care: an increased quota is not savings, and an incident that never occurred cannot automatically be counted as protected revenue. I would use expected-loss scenarios, capacity cost, and operational effort while keeping assumptions visible. The decision may favor extra headroom without immediate returns when it protects a critical journey.

The Staff question would be: how do we avoid counting the same capacity as both a pending reservation and observed consumption during reconciliation? The answer must connect identity, state, and source. The leadership question would be who decides between delaying a delivery and consuming contingency margin. This trade-off must not be hidden in configuration. A prepared organization makes the choice explicit, records the risk, and gives teams the conditions to meet their commitment.

ANALYSIS AND DECISION

13 · The resource outlives the team change

13

python Datomic aws

How Nubank distributes infrastructure ownership to operate more than 21 thousand databases with a lean team

Original official publication, accessed 23 October 2026.

Nubank published a distributed-ownership case covering an operation with more than 21,000 databases. The account connects provisioning, inventory, and lifecycle to product-team responsibility. The cited central team does not represent the entire organizational effort. This chapter focuses on one transition: what happens when the structure responsible for a resource changes.

An owner field populated at creation can remain syntactically correct and become operationally wrong months later. The team may have dissolved, split, or been replaced while alerts, costs, and dependencies remain active. Updating a tag does not guarantee someone accepted responsibility, received access, or knows how to restore the service. Ownership must be a verifiable relationship between a resource, a journey, and a team capable of acting.

The proposal combines accepted transfers, revision control, and archival barriers. The lab uses a local immutable record and decision tests; it neither creates, migrates, nor removes databases. A Datomic excerpt illustrates transactional revision comparison without claiming it is the internal mechanism used in the official case. The Staff question is not how to delete more resources, but how to retain enough context to change responsibility or end usage without losing security, retention, and recovery capability.

ANALYSIS AND DECISION

Ownership means acceptance, capability, and context

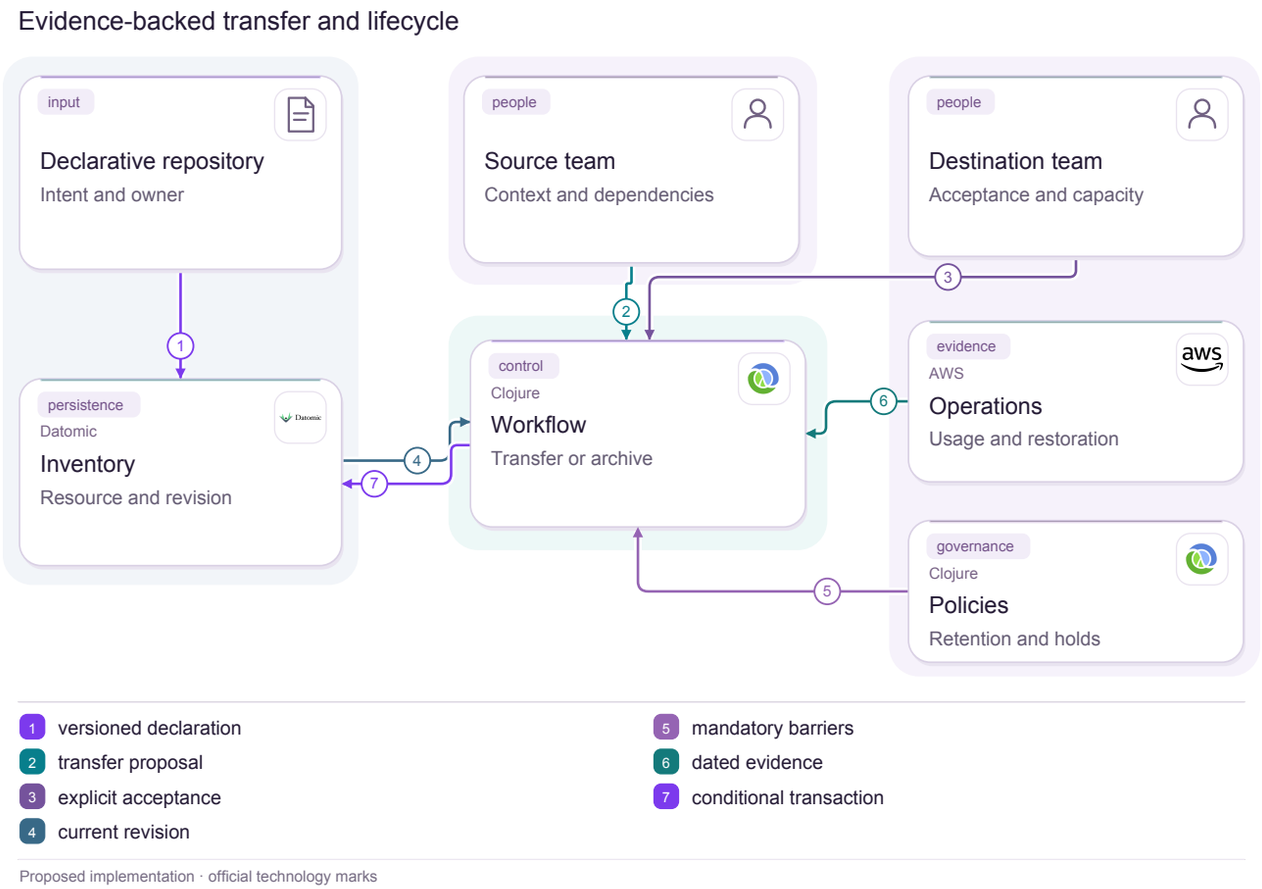
A resource's minimum state includes identity, owner, revision, operational status, and lifecycle constraints. For a transfer, the destination must explicitly accept the commitment. The source cannot simply write another team's name and consider the obligation resolved. Acceptance must be associated with the context received: runbook, dependencies, access, alerts, and relevant commitments.

Revision protects against decisions based on old state. A transfer approved before a new restriction or dependency change must not overwrite the current inventory without reassessment. The lab compares revisions within a lock. In a shared database, that rule must be atomic at the persistence boundary. Reading, checking in memory, and writing later creates a window for concurrent decisions.

Archival is another workflow. Inactivity is only a signal; it does not prove the absence of seasonal consumers, retention obligations, or recovery value. The model requires four synthetic pieces of evidence and no active hold. Even then, the action only marks the record archived. It does not demonstrate safe deletion. This distinction is essential: governance must separate identifying candidates, reviewing context, preserving recovery, and executing an action, each with appropriate authority and evidence.

PROPOSED ARCHITECTURE · NOT AN INTERNAL DESIGN

Evidence-backed transfer and lifecycle



Proposed external architecture. Boxes represent responsibilities; arrows make contracts explicit.

The proposal treats inventory as a set of relationships, not simply a spreadsheet of names. A decision uses the current revision and evidence with provenance. Transfer depends on destination acceptance; archival depends on policies and operational observations. The workflow must not turn a team's silence into automatic approval where loss or unavailability is possible.

The arrows represent responsibilities that different services and tools can implement. Datomic is consistent with the publicly documented stack, but the diagram requires no specific product. The central property is that the conditional change and its revision occur together. Usage data help identify candidates; retention and restoration determine whether an action is permitted and recoverable. Separating these questions prevents cost optimization from assuming authority over business commitments absent from traffic metrics.

CODE WITH EXPLICIT SCOPE

Archival requires more than absence of traffic

PYTHON / ORIGINAL PROPOSAL

```python
def set_hold(self):
    with self.lock:
        self.resource = replace(self.resource, hold=True,
                                revision=self.resource.revision + 1)

def archive(self, expected, *, inactive, retention_elapsed,
            no_dependencies, restore_verified):
    evidence = (inactive, retention_elapsed,
                no_dependencies, restore_verified)
    if any(type(value) is not bool for value in evidence):
        raise ValueError('evidências devem ser booleanas neste modelo')
    with self.lock:
        current = self.resource
        if expected != current.revision:
            raise ValueError('evidência de revisão antiga')
        if current.hold or not all((inactive, retention_elapsed,
                                    no_dependencies, restore_verified)):
            raise PermissionError('barreiras não satisfeitas')
        if current.archived:
            return current
        self.resource = replace(current, archived=True,
                                revision=current.revision + 1)
        return self.resource
```

Literal excerpt from labs/case_13.py. Logical in-memory archival only; no real database is changed.

The executed excerpt rejects incorrectly typed evidence, stale revisions, active holds, or any unmet barrier. The change preserves the record and increments its revision. The lab has no deletion command. This choice allows the rule to be tested without a destructive demonstration or pretending that four booleans suffice to remove real infrastructure.

In production, every piece of evidence would require a trusted source, temporal validity, and scope. A restore tested against another backup does not prove recovery of the current resource. The model reduces that richness to controlled inputs; its role is to expose the conjunction of conditions and the revision-comparison point.

CODE WITH EXPLICIT SCOPE

Datomic: compare revision and transfer in one transaction

CLOJURE / ORIGINAL PROPOSAL

```clojure
(ns portfolio.resource-transfer)

(defn transfer-tx
  [resource-eid expected-revision new-owner accepting-team]
  (when-not (= new-owner accepting-team)
    (throw (ex-info "Destination must accept"
                    {:kind :missing-acceptance})))
  (when-not (and (integer? expected-revision)
                 (pos? expected-revision))
    (throw (ex-info "Invalid revision"
                    {:kind :invalid-revision})))
  [[:db/cas resource-eid :resource/revision
    expected-revision (inc expected-revision)]
   [:db/add resource-eid :resource/owner new-owner]])

;; Proposta: :resource/revision é cardinalidade one.
;; Os atributos devem existir em um schema revisado.
;; Submeter o vetor em UMA transação Datomic.
;; Aceite deve vir de identidade autenticada.
;; A checagem local não substitui autorização.
;; Restrições adicionais pertencem à mesma decisão.
;; O trecho constrói dados; não conecta a uma base.
```

Reference Clojure/Datomic excerpt, not executed; proposed schema and function. :db/cas is a documented operation.

The example uses Datomic's documented compare-and-swap operation to bind the change to the expected revision. If another transaction changed the revision, comparison fails and the proposal must be reassessed. The owner change and increment belong to the same transaction data; executing them separately would lose the desired property.

The resource attributes are a proposed schema, not internal Nubank names. The acceptance check receives values as parameters and does not implement authentication. A real service would need trusted identity and all necessary restrictions. The excerpt was not executed in Datomic; the Python lab tests equivalent local semantics.

Datomic · Transaction Data

CODE WITH EXPLICIT SCOPE

Lifecycle evidence has validity and provenance

JSON / ORIGINAL PROPOSAL

```json
{
  "resource_id": "synthetic-db-17",
  "expected_revision": 8,
  "proposed_action": "archive",
  "owner": "demo-payments-team",
  "evidence": {
    "usage_window": {
      "start": "2026-09-01",
      "end": "2026-10-01",
      "source": "synthetic-observability"
    },
    "dependencies_review": "demo-review-12",
    "retention_policy": "demo-policy-v3",
    "restore_test": "demo-restore-4"
  },
  "holds": [],
  "approval": {
    "actor": "synthetic-authorized-owner",
    "expires_at": "2026-10-24T12:00:00Z"
  },
  "deletes_data": false
}
```

Proposed governance contract; neither Nubank's retention policy nor authorization to dispose of data.

The envelope shows what the lab's booleans conceal: evidence needs a reference, a window, and an applicable policy. No usage during one month may be insufficient for annual processing. Retention policy can vary by product and jurisdiction; it must not be inferred from a generic period in an article.

Approval expires because context changes. A new dependency or hold should invalidate an earlier decision through revision. The example provides no legal policy and authorizes no deletion. It documents a proposed logical-archival command with synthetic data to discuss what evidence a real implementation would need to obtain and verify.

ANALYSIS AND DECISION

Reorganization is a system change

When a team splits, distributing names in a spreadsheet is insufficient. Dependencies, alerts, access keys, and costs may follow different paths. A resource can keep working for months, hiding that nobody can restore it or interpret an incident. I would treat reorganization as a governance event that opens pending transfers with a temporary owner and deadline, without removing ownership before the destination accepts.

The most important race is between lifecycle review and a context change. An archival candidate may gain a new consumer after metrics are collected. A new restriction may arrive while approval is underway. The action must therefore check the current revision and barriers at execution time. Retaining an approved assessment does not justify ignoring later events that change its validity.

Another risk is the invisible consumer. Read metrics may miss seasonal access, indirect exports, or rare operational dependencies. The proposal combines usage signals with a consumer map and restoration capability. When confidence is insufficient, the response may be isolation or further observation rather than deletion. Reducing cost is desirable, but does not justify erasing context precisely where the organization admits it does not yet understand the resource.

CODE WITH EXPLICIT SCOPE

Test the barrier and revision, not just the happy path

PYTHON / ORIGINAL PROPOSAL

```python
def test_destination_must_accept(self):
    with self.assertRaises(PermissionError):
        self.registry.transfer(1, 'team-b', 'team-a')
    self.assertEqual(self.registry.resource.owner, 'team-a')

def test_stale_transfer_cannot_override(self):
    self.registry.transfer(1, 'team-b', 'team-b')
    with self.assertRaises(ValueError):
        self.registry.transfer(1, 'team-c', 'team-c')
    self.assertEqual(self.registry.resource.owner, 'team-b')

def test_hold_blocks_archive(self):
    self.registry.set_hold()
    with self.assertRaises(PermissionError):
        self.registry.archive(2, **self.evidence)

def test_each_missing_barrier_blocks(self):
    for key in self.evidence:
        args = dict(self.evidence, **{key: False})
        with self.subTest(key=key), self.assertRaises(PermissionError):
            self.registry.archive(1, **args)
```

Excerpt from labs/case_13.py. Result: 8 tests passed; see results_13.txt.

Eight tests passed in the complete lab. The excerpt examines destination acceptance, stale revisions, active holds, and each missing barrier. The others verify stale evidence after transfer, preservation of the archived record, prevention of subsequent transfer, and rejection of incorrectly typed evidence. These are local decisions about a single resource.

The result does not prove a complete inventory, a recoverable backup, or retention compliance. Those facts depend on external systems and owners. Real validation should rehearse transfers between teams, simulate context changes, and restore representative resources. A boolean test is useful for the decision rule provided it is not presented as proof of the supplied evidence.

ANALYSIS AND DECISION

Valid ownership must be observable

I would measure resources with active owners, transfers awaiting acceptance, and time under temporary responsibility. A populated tag does not equal a team capable of acting. Inventory should support verification of access, runbook, alert routing, and relevant dependencies without making each item a bureaucratic certification that becomes stale the next day. A recent operational exercise may be the best evidence.

For lifecycle, I would track identified candidates, decisions, reasons for holds, and restoration outcomes. A high archival rate may be positive or indicate superficial review; it needs interpretation alongside incidents and reversals. I would also distinguish removed cost, cost retained for retention obligations, and recovered capacity. Presenting every resource reduction as immediate savings ignores contracts, backups, and fixed costs.

Alerts should reach the current owner and have a path when none exists. Sending notifications to an abandoned channel satisfies a technical rule and fails operationally. I would test this flow periodically, including organizational changes. The goal is to detect lost context before an incident. Governance is useful when it reduces the time to find a decision-maker, preserves evidence, and helps the team operate-not when it merely increases required fields.

ANALYSIS AND DECISION

Automation begins with inventory quality

Initial deployment would be observational: gather declarations and discovered resources, identify discrepancies, and confirm ownership with teams. The system must distinguish an unknown resource from an unused one. This stage builds understanding of names, dependencies, and sources, avoiding automated decisions on an inventory that does not yet reflect reality. The central team needs domain support to interpret cases.

Next, I would automate reversible parts such as opening a transfer proposal, requesting acceptance, and updating metadata after validation. Logical archival would follow within a limited scope, with verifiable evidence and rehearsed restoration. I would not treat no response as approval for destructive actions. A process can be automatic without removing human responsibility: the system gathers evidence, applies barriers, and records who has authority to decide.

Expansion would depend on quality and operational load. If false candidates consume more time than the previous manual work, I would adjust signals and criteria. Rollback must preserve decision history and allow the resource to resume in the appropriate state. I would also verify who maintains the governance platform itself. An ownerless inventory designed to fix ownerless resources merely recreates the problem at another layer.

ANALYSIS AND DECISION

Datomic does not replace data policy

A model of facts and history can help explain changes, but does not independently define who may access, retain, or remove information. Choosing Datomic must be accompanied by schema, authorization, operations, and lifecycle understanding. This chapter's excerpt uses revision comparison; it does not implement physical disposal or guarantee regulatory obligations through a transaction.

Retention requires product, jurisdiction, and purpose context. I would avoid encoding a universal period from a public example. Policy needs an owner, version, and update mechanism, with appropriate holds when additional obligations exist. I would also separate operational archival from data deletion: reducing active capacity may retain necessary backups and references for a different period.

Security includes the automation platform. A service capable of changing ownership or executing lifecycle actions has significant power. I would separate inventory reads, proposals, and execution; limit scope; and record decisions with evidence. Emergency access must be reviewed, not used as a permanent shortcut. Automation's benefit is consistently repeating well-defined rules. When a rule depends on missing context, the correct behavior is to request investigation from the owner, not infer permission from silence or low utilization.

ANALYSIS AND DECISION

ADR 13 · Transfer with acceptance and revision

The proposed decision is to change ownership through a transfer accepted by the destination and conditional on the resource's current revision. The context is an evolving organization in which technical responsibilities outlive their originating teams. The intent is to preserve the ability to act, preventing a declarative change from hiding lost knowledge or assigning work to someone who never accepted the commitment.

One alternative is unrestricted editing of owner in a repository. It is fast and auditable as a text change, but guarantees neither acceptance nor readiness. Another is permanently centralizing all responsibility in Platform. This reduces apparent ambiguity but creates a bottleneck and distances operations from product context. Explicit transfer retains autonomy and distributes responsibility at the cost of workflow, follow-up, and review of pending items.

Archival uses the same revision discipline but not the same criteria. Retention, dependencies, and recovery belong to a separate decision. I would reconsider implementation if the process becomes so heavy that teams leave inventory stale. The response would be to simplify the experience and automate evidence collection while preserving essential barriers. Effective governance does not maximize approvals; it makes the correct decision easier to make and harder to lose.

ANALYSIS AND DECISION

EM execution: provide time to assume responsibility

As Engineering Manager, I would include ownership transfer in organizational and product-change planning. The receiving team needs time to understand dependencies, review alerts, and rehearse recovery. A handoff meeting without reserved capacity does not create readiness. I would define a temporary owner and acceptance criteria, avoiding an interval where each team assumes the other is handling the resource.

Value may come from safely decommissioned resource costs, recovered operational hours, and shorter incident response time. I would measure these separately. I would neither multiply one database's savings across the entire inventory nor confuse archival with elimination of all cost. Backups, retention, and platform maintenance remain. Review must consider net benefit and operating quality after the change.

The Staff question would be: which evidence becomes invalid when ownership or dependencies change during approval? The answer must connect revision, barrier, and execution time. The leadership question would be how to prevent a team accepting responsibility merely to close an administrative item. I would seek practical signs of capability: access, runbook, alert route, and a recovery exercise. Ownership is an operational commitment, not merely a name that satisfies a form.

ANALYSIS AND DECISION

14 · BDC expands speed and responsibility

14

Original official publication, accessed 23 October 2026.

Nubank documented BDC as a server-driven interface platform with Flutter components and a Clojure backend. The publication describes adoption and rollout control at the time of writing. It does not provide a complete public component contract or justify inferring current gaps. This chapter proposes change governance for such a platform, preserving the distinction between official reference and independent design.

Moving composition to the server shortens the path for certain changes, but does not remove compatibility requirements with installed binaries. A client can interpret only capabilities it possesses. There are also combinations of theme, text, accessibility, experiment, and journey state. A small content change can have broad effects if the platform distributes the same revision to millions of sessions without exposure criteria.

The objective is to publish experiences with a verifiable contract: immutable revision, action catalog, accessible semantics, capability negotiation, and rollback. The lab models those decisions in Python and tests selection and deterministic cohorts. It neither renders Flutter nor accesses real BDC or NuDS. The Dart excerpts are illustrative proposals. The main question is how to give squads autonomy while the platform prevents invalid combinations before someone discovers the error in their own app.

Backend Driven Content: why we developed a Server Driven UI framework at Nubank

ANALYSIS AND DECISION

Protocol, component, and content evolve separately

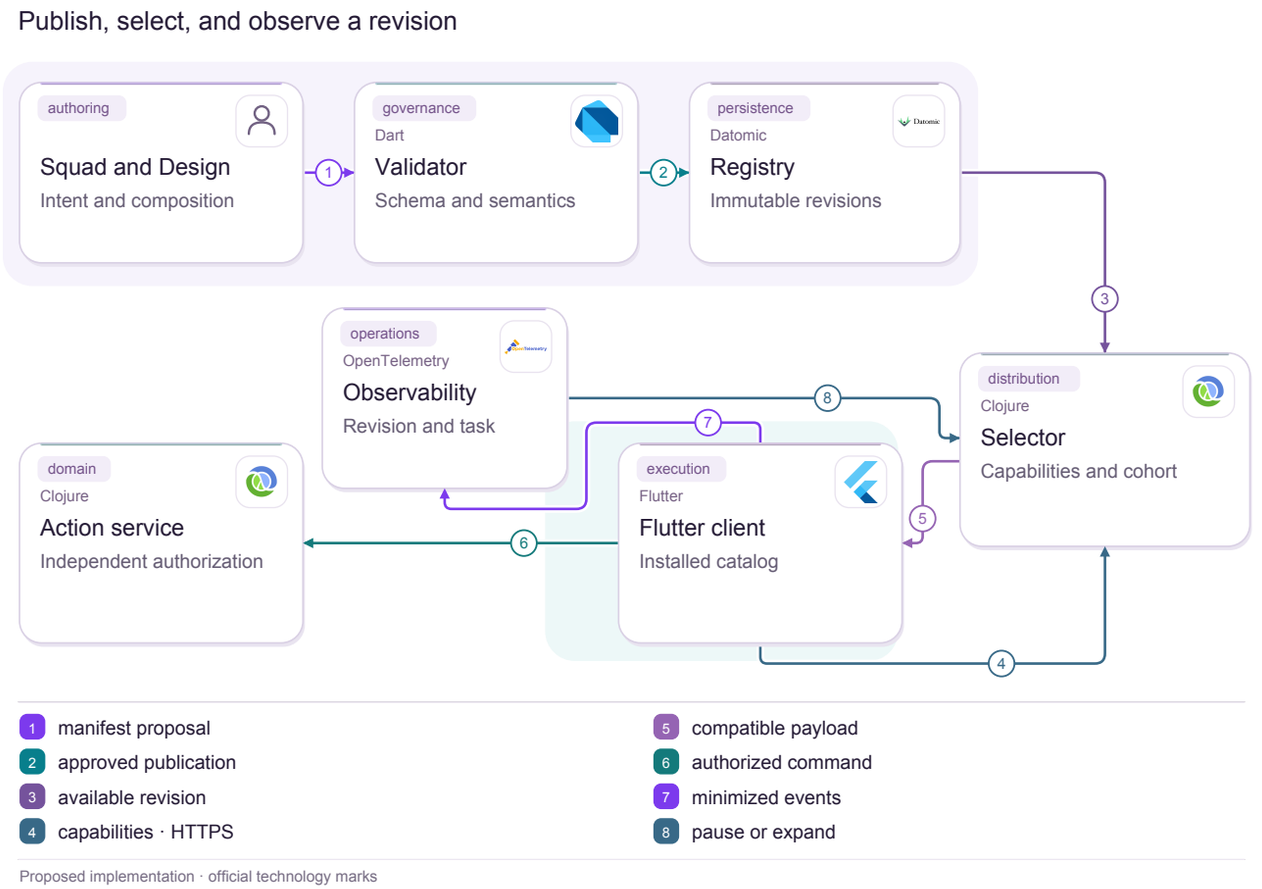
A protocol version defines how to interpret the manifest. A component catalog defines which elements and properties the client knows. A content revision defines the published composition. Using only the app's commercial version to represent all three concepts complicates compatibility and diagnosis. Two clients with apparently identical screens may have different capabilities, while several content revisions may share the same protocol.

In the lab, Manifest declares protocol, components, action, and accessible name. Validation requires a tuple of string identifiers to prevent component mutation after publication. The publisher rejects revision reuse with different content. Selection checks whether the client understands the protocol and has every required component. Otherwise, it tries a compatible fallback. If neither is suitable, it returns a controlled error instead of pretending an unknown composition can be interpreted safely.

The action catalog limits interface vocabulary but does not grant financial authorization. An action may open a journey that performs its own check in the responsible service. The manifest carries neither arbitrary code nor balance rules. This separation prevents the platform becoming a general execution language disguised as content. Evolution must provide enough expressiveness for the product while keeping boundaries testable, documented, and understandable to consumer teams.

PROPOSED ARCHITECTURE · NOT AN INTERNAL DESIGN

Publish, select, and observe a revision



Proposed external architecture. Boxes represent responsibilities; arrows make contracts explicit.

The pipeline validates a proposal before making it selectable. The registry keeps content immutable so the revision observed by a client can be reconstructed. The selector combines technical capability with planned exposure; cohort membership never overrides incompatibility. The client still validates input because failures and unexpected versions can pass through distribution.

An action goes to a service that applies domain authorization. This prevents an interface configuration from granting privilege simply by appearing on screen. Observability connects revision to task success, rendering failure, and fallback behavior without indiscriminately capturing financial data. Operational feedback can pause expansion but does not undo confirmed actions. The diagram proposes responsibilities, not specific internal services. A mature platform must make these boundaries easy for squads to use; if every team recreates validation, cohorts, and fallback, the promise of consistency is lost.

CODE WITH EXPLICIT SCOPE

Compatibility includes the fallback composition

PYTHON / ORIGINAL PROPOSAL

```python
def publish(self, manifest):
    validate(manifest)
    previous = self.revisions.get(manifest.revision)
    if previous is not None and previous != manifest:
        raise ValueError('revisão é imutável')
    self.revisions[manifest.revision] = manifest

def activate(self, revision):
    if revision not in self.revisions:
        raise ValueError('revisão não publicada')
    self.active = revision

def select(self, capabilities, protocol, fallback):
    if self.revisions.get(fallback.revision) != fallback:
        raise ValueError('fallback não publicado')
    current = self.revisions.get(self.active)
    compatible = current and current.protocol == protocol
    compatible = compatible and set(current.components) <= capabilities
    if compatible:
        return current
    if (fallback.protocol == protocol
            and set(fallback.components) <= capabilities):
        return fallback
    raise ValueError('nenhuma composição compatível')
```

Literal excerpt from labs/case_14.py. Executed Python model; does not implement real BDC/NuDS.

The executed publisher preserves revision identity and selects content compatible with the client. Checking fallback matters: returning an older composition does not guarantee every client understands it. When no compatible alternative exists, the model fails explicitly, allowing the experience to handle unavailability without attempting to render unknown data.

Storage is local and the catalog is synthetic. There is no manifest signature, distributed persistence, or guaranteed rollback propagation. Those mechanisms would need to be defined for deployment. The excerpt establishes a contract property that can remain true even with different infrastructure and transport formats.

CODE WITH EXPLICIT SCOPE

Flutter: interpret a closed catalog

DART / ORIGINAL PROPOSAL

```dart
import 'package:flutter/material.dart';

Widget renderProposedNode(
  Map<String, Object?> node,
  void Function(String) dispatch,
) {
  switch (node['type']) {
    case 'text':
      return Text(node['text'] as String);
    case 'action':
      final action = node['action'] as String;
      const allowed = {'view-invoice', 'open-support'};
      if (!allowed.contains(action)) {
        throw const FormatException('Unknown action');
      }
      return Semantics(
        button: true,
        onTap: () => dispatch(action),
        label: node['accessible_name'] as String,
        child: ExcludeSemantics(
          child: ElevatedButton(
            onPressed: () => dispatch(action),
            child: Text(node['text'] as String),
          ),
        ),
      );
    default:
      throw const FormatException('Unknown node type');
  }
}
```

Independent Flutter/Dart, not executed. dispatch and node types are proposed contracts, not Nubank APIs.

The illustrative renderer accepts known types and actions without interpreting scripts. ExcludeSemantics removes semantics from the descendant button; the outer node therefore explicitly provides onTap with the same dispatch as the visual interaction. The accessible name can differ from the text without losing the action. Schema validation before casts and error handling remain the responsibility of the enclosing composition.

In a real implementation, approved components and tokens would replace generic widgets where appropriate. Focus, navigation, and enlarged-text tests would also be needed. The Dart excerpt was not compiled; it expresses a design boundary. The only verified runtime in this chapter is the Python lab.

Flutter · Semantics
Flutter · ExcludeSemantics removes descendant semantics
Flutter · SemanticsProperties.onTap

CODE WITH EXPLICIT SCOPE

Manifest: stable identity and explicit requirements

JSON / ORIGINAL PROPOSAL

```json
{
  "contract": "portfolio.ui.v1",
  "revision": "demo-invoice-r17",
  "protocol": 1,
  "required_components": ["text", "action"],
  "purpose": "invoice-navigation",
  "nodes": [
    {
      "type": "text",
      "text": "Consulte sua fatura"
    },
    {
      "type": "action",
      "text": "Ver fatura",
      "accessible_name": "Abrir detalhes da fatura",
      "action": "view-invoice"
    }
  ],
  "rollout": {
    "experiment": "demo-navigation-v1",
    "maximum_percent": 5
  }
}
```

Independently proposed manifest and action names; synthetic data. Not the official BDC schema.

The proposed manifest declares requirements and composition. A revision must identify exactly the approved content; editing the payload under the same identifier would destroy traceability and make cohort comparison ambiguous. The rollout field expresses exposure intent, not permission for clients to place themselves in any group.

Selection must occur at a trusted boundary and consider compatibility before percentage. This example includes no financial data: the action opens a journey with its own contracts. The lab uses a reduced representation without a node tree. The JSON documents how this decision could be expanded without claiming to represent BDC's internal format.

ANALYSIS AND DECISION

A canary needs stable cohorts and exit criteria

Small initial exposure is useful when it allows behavioral comparison without unpredictably changing participants. The lab uses a deterministic hash of subject and salt to create a stable cohort. Increasing exposure from five to twenty percent retains those already included. This guarantees neither statistical balance nor experimental validity; it is merely an operational property that helps trace the experience received.

The cohort must be evaluated alongside capabilities. A selected client unable to render a component should receive a compatible alternative. This exclusion changes the observed population and must appear in analysis. Otherwise, the new version may look better because it was shown only on newer devices or clients with different characteristics. Comparison requires segments and criteria defined before results arrive.

Exit criteria include rendering stability, task completion, accessibility, and absence of material adverse effects. A canary without pause conditions merely delays full deployment. The team must also decide what happens to in-progress journeys when a revision is rolled back. The screen may change, but an accepted financial confirmation must not be repeated or erased to match new content. Presentation rollout and domain transitions must remain separate.

CODE WITH EXPLICIT SCOPE

The old client is part of the test too

PYTHON / ORIGINAL PROPOSAL

```python
def test_old_client_gets_fallback(self):
    self.assertEqual(self.publisher.select({'text'}, 1, self.old), self.old)

def test_capable_client_gets_active_revision(self):
    self.assertEqual(self.publisher.select({'text', 'chart'}, 1, self.old),
                     self.new)

def test_revision_cannot_change(self):
    changed = Manifest('r1', 1, ('chart',), 'view-invoice', 'Fatura')
    with self.assertRaises(ValueError):
        self.publisher.publish(changed)

def test_missing_accessibility_rejected(self):
    with self.assertRaises(ValueError):
        validate(Manifest('r3', 1, ('text',), 'view-invoice', ''))

def test_rollback_selects_old_revision(self):
    self.publisher.activate('r1')
    self.assertEqual(self.publisher.select({'text', 'chart'}, 1, self.old),
                     self.old)
```

Excerpt from labs/case_14.py. Result: 11 tests passed; see results_14.txt.

Eleven tests passed in the complete lab. The excerpt verifies fallback, active revision, immutable identity, semantics, and rollback. The others cover stable and monotonic cohorts, boundaries, invalid actions, and invalid fallback. A regression test rejects mutable lists and components without string identifiers, preventing later changes to published content. Assertions examine the manifest actually selected.

No Flutter widget, BDC integration, or distributed-propagation tests were executed. Those validations would need semantic trees, network errors, installed versions, and journeys started before rollback. The lab establishes small contracts a real platform could incorporate at different pipeline stages.

ANALYSIS AND DECISION

Observe the revision the person actually received

An error event without a content revision does not explain which experience was in use. I would record protocol, revision, capability class, and experiment in controlled dimensions, linking the journey to rendering failures and task completion. Binary version remains useful but does not independently identify the server-distributed composition. This distinction is central to investigating regressions on dynamic-content platforms.

I would also observe fallback rate and reason. Frequent fallback may mean the new catalog reaches few clients, negotiation is wrong, or the deprecation plan was optimistic. It should not automatically count as silent success. The old experience may remain correct while preventing the expected benefit or hiding that part of the customer base is excluded from the change.

For expansion decisions, I would combine technical metrics with task behavior and qualitative sampling. Counting clicks on a more prominent button does not demonstrate better understanding. Telemetry must minimize data and respect cardinality; capturing the entire tree with financial content would be an inappropriate shortcut. The dashboard should answer which revision, in which population, and with what consequence, without turning observability into an indiscriminate copy of a customer's private experience.

ANALYSIS AND DECISION

Content rollback is not time travel

An immutable revision allows distribution to point to an earlier composition, but does not guarantee instantaneous rollback for every client. Caches, active sessions, and limited connectivity remain. The plan must define expected propagation time, offline behavior, and content validity limits. If recovery depends on every client immediately receiving a remote command, the strategy has a significant weakness.

Rollback does not undo confirmed actions either. A journey started under one revision may continue after the switch. Operation identity, domain state, and confirmation remain valid regardless of screen appearance. The client must resume the current situation without repeating effects. This contract must be tested in relevant flows, especially when a new composition changes the number or order of steps.

I would begin deployment with manifest comparisons, matrix tests, and limited cohorts. The team would rehearse rollback before needing it, checking observability and support communication. The goal is not to slow every publication but to automate evidence that supports confident publishing. When a change needs a new binary capability, it follows that lifecycle explicitly; BDC cannot make an absent component executable on a device.

ANALYSIS AND DECISION

A declarative interface must not become remote authorization

A manifest can instruct presentation and navigation, but should not carry authority to execute financial operations. A button's existence does not prove permission. The action service checks identity, context, and domain rules even when content comes from a trusted platform. This separation reduces the impact of incorrect configuration and allows interface capability review without implicitly changing financial controls.

I would also limit expressiveness. Accepting arbitrary code or uncontrolled URLs increases behavioral surface and complicates validation. A known-action catalog, restricted schema, and size limits help keep the platform understandable. Validation must occur on server and client because distribution and versions can fail. Invalid content needs a safe, understandable response, not a generic attempt to continue at any cost.

Accessibility belongs in the contract but cannot be reduced to a required field. An empty accessible name can be detected automatically; a confusing name, unsuitable focus order, and inaccessible interaction require broader evaluation. I would combine automated rules with component tests and research on critical journeys. Transparency, animation, and visual effects must respect legibility and device preferences. An advanced platform increases repeatable quality; it does not use visual expressiveness to justify removing clarity or autonomy.

ANALYSIS AND DECISION

ADR 14 · Immutable content selected by capability

The proposed decision is to publish immutable revisions and select composition by protocol, capabilities, and cohort. The context is a platform that changes interfaces without distributing a binary for every change. The intent is traceability and compatibility while squads gain autonomy. The action catalog remains separate from domain authorization, and fallback must be validated like any other composition.

One alternative is to ship only static interfaces in the app. It reduces some content variability but retains update dependence for eligible changes. Another is a highly expressive remote layer approaching arbitrary execution. It increases freedom while expanding risk, combinations, and validation cost. A restricted declarative contract provides balance: it accelerates known compositions and makes explicit when a new capability requires platform or binary evolution.

I would reconsider the design if the compatibility matrix grows uncontrollably or squads bypass the catalog because suitable components are missing. The response may be better abstractions and deprecation rather than more exceptions. Architecture must measure adoption cost and delivered quality. Immutable revisions are a mechanism; the desired outcome is being able to explain what each client received, why it was compatible, and how to stop the change when evidence warrants it.

ANALYSIS AND DECISION

EM execution: a platform that multiplies quality

As Engineering Manager, I would organize the platform around consumer teams and the journeys they deliver. I would define contracts and tools that make correct publication simpler than parallel workarounds. Work would include documentation, diagnosis, accessible components, and adoption support as well as runtime. Leadership must reserve capacity for these activities because a platform with a poor user experience loses adoption even when its architecture is elegant.

Value would be measured through eligible delivery time, implementation and remediation effort, quality, and sustained adoption. I would not use the percentage of migrated screens as a standalone objective. Migration can increase cost without improving a task. Recovered-capacity calculations would deduct platform maintenance and squad support. Conversion gains would require their own experiment, separate from development savings, to avoid counting the same benefit twice.

The Staff question would be: how do we ensure a compatible, visually correct revision does not introduce an inappropriate action when two experiments combine? The answer involves composition, catalog, authorization, and observability. The leadership question would be how to prioritize a cross-team improvement whose benefits appear across several teams while its cost sits in Platform. I would seek clear commitments, consumer pilots, and observable results. Multiplying quality requires technical contracts and a sustainable way to decide and learn together.

INDEPENDENT ANALYSIS AND PROPOSAL

Perceived availability also depends on behavior

15

Platform engineering, Observability, and scaling Mobile Development

Original official publication, accessed 23 October 2026.

In April 2025, Nubank described a mobile platform with progressive releases, automation, and observability. The publication presents an existing capability, not a gap this study claims to uncover. My additional question would be how a decision to expand exposure connects technical stability with completed tasks. A crash-free session can still contain an unresponsive button, an unanswered wait, or a message the customer did not understand.

I would select one journey, such as viewing a bill, before discussing a gate for the entire app. I would define its start, completion, and unknown outcome with Product, Design, and the responsible team. A success event should mean more than a screen opening; it should represent the agreed task. I would also distinguish voluntary cancellation from failure and connectivity interruptions, avoiding attributing every abandonment to the release.

This chapter proposes a deterministic decision laboratory using invented data and deliberately simple rules. It demonstrates how a regression can prevent expansion even when the process has not crashed. It does not estimate Nubank's performance, test Flutter, or replace statistical analysis. Its value is to make the contract open to discussion: what information is missing, who decides in its absence, and which signals must remain stable before more people are exposed to the change.

INDEPENDENT ANALYSIS AND PROPOSAL

The unit is a journey attempt

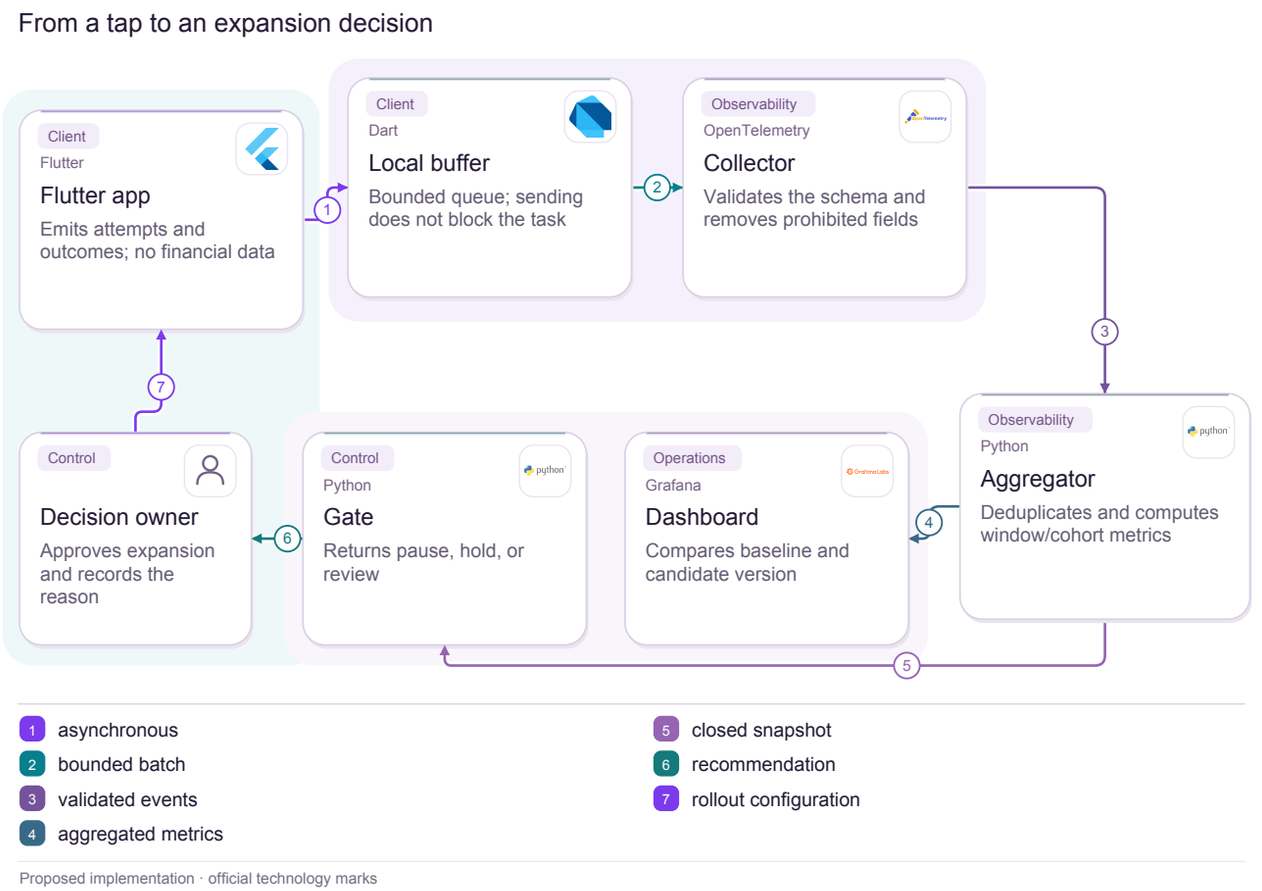
An attempt begins when the person expresses the intent defined for the task. A repeated tap during loading need not create another attempt; a new inquiry after returning to the screen might. This distinction depends on the product and must be recorded before collection begins. If every team counts differently, an apparent conversion decline may simply reflect an instrumentation change and consume days of investigation.

The local model associates a synthetic session, cohort, and version with a terminal outcome. This simplification demonstrates deduplication but does not represent customer identity. A real implementation would need an independent attempt identifier, an expiration policy, and intermediate states. The same person may perform several valid operations in a session; aggregating them incorrectly would erase problems and distort the denominator. Late events would also require reconciliation within an explicit window.

I would keep three states separate: completed, not completed, and incomplete observation. The third does not automatically become success or failure. It describes measurement quality and may suspend the expansion decision. Cohorts would be comparable in journey, device, and connection, with defined cardinality limits. The goal is not to profile people, but to avoid mistaking differences in the exposed population for the effect of the version under evaluation.

REFERENCE ARCHITECTURE

From a tap to an expansion decision



External proposal. Flutter and observability are publicly documented context; the names and boundaries in this design are original.

The main boundary separates the financial operation from observability collection. An unavailable collector must not prevent an inquiry or payment; the buffer needs predictable size and discard behavior. The decision path operates on an aggregated snapshot identified by version and interval, so an approval does not change meaning while someone reviews it. The gate recommends, and the owner decides within the established process. This design avoids granting release authority to an isolated chart. The laboratory implements only aggregation and decision logic, without networking, persistent storage, or remote app control. In a real integration, I would assess whether the components already exist and add only the missing contract. Approval to expand should record the exposed population, the baseline used, and the condition that requires a pause. Receiving telemetry does not directly authorize a customer's financial action.

CODE · PROPOSED CONTRACT

A gate that can say “not yet”

PYTHON / ORIGINAL PROPOSAL

```python
def gate(baseline, candidate, telemetry_complete=True, min_n=50):
    if type(min_n) is not int or min_n <= 0:
        raise ValueError("min_n must be positive integer")
    if type(telemetry_complete) is not bool:
        raise ValueError("telemetry_complete must be boolean")
    validate_stats(baseline)
    validate_stats(candidate)
    if not telemetry_complete:
        return "HOLD_TELEMETRY"
    if baseline["n"] < min_n or candidate["n"] < min_n:
        return "HOLD_SAMPLE"
    if candidate["rate"] < baseline["rate"] - 0.03:
        return "PAUSE_SUCCESS"
    if candidate["p95"] > baseline["p95"] * 1.25:
        return "PAUSE_LATENCY"
    return "ELIGIBLE_FOR_REVIEW"

baseline = {"n": 100, "rate": 0.98, "p95": 300}
candidate = {"n": 100, "rate": 0.90, "p95": 300}
assert gate(baseline, candidate) == "PAUSE_SUCCESS"
assert gate(baseline, baseline) == "ELIGIBLE_FOR_REVIEW"
assert gate(baseline, baseline, False) == "HOLD_TELEMETRY"
small = {"n": 10, "rate": 1.0, "p95": 100}
assert gate(baseline, small) == "HOLD_SAMPLE"
```

Excerpt from case_15.py; Python execution verified. Illustrative rule without statistical inference. Types, imports, and validators are in the complete file.

The function executed in the laboratory produces an explainable decision. Incomplete telemetry and small samples return a hold; success and latency regressions return a pause. The values of 50 observations, three percentage points, and 25% are synthetic assumptions for testing the mechanism, not recommended payment limits. A favorable result still requests review: it does not represent statistical significance, risk approval, or automatic authorization. In production, parameters would need versioning and calibration against journey variability, tolerance for harm, and experimental design. Changing a threshold is also a policy change and deserves a record. Invalid types and domains raise ValueError before any favorable result.

Python 3.12: unittest

CODE · PROPOSED CONTRACT

Decoupled instrumentation in Dart

DART / ORIGINAL PROPOSAL

```dart
enum JourneyOutcome { completed, failed, unknown }

class JourneySignal {
  final String attemptId;
  final String release;
  final JourneyOutcome outcome;
  final int elapsedMs;
  const JourneySignal(this.attemptId, this.release,
      this.outcome, this.elapsedMs);
}

abstract class BoundedTelemetry {
  bool offer(JourneySignal signal);
}

Future<T> observed<T>(
  Future<T> Function() task,
  BoundedTelemetry telemetry,
  String attemptId,
  String release,
) async {
  final watch = Stopwatch()..start();
  try {
    final result = await task();
    telemetry.offer(JourneySignal(attemptId, release,
        JourneyOutcome.completed, watch.elapsedMilliseconds));
    return result;
  } catch (_) {
    telemetry.offer(JourneySignal(attemptId, release,
        JourneyOutcome.unknown, watch.elapsedMilliseconds));
    rethrow;
  }
}
```

Illustrative Dart, not executed. Proposed interfaces and names; not an internal Nubank SDK.

This Dart fragment is a proposed interface and has not been compiled. BoundedTelemetry would be required not to throw exceptions and to return immediately when its queue is full; the implementation must test that obligation. The wrapper measures local duration but cannot know whether a server completed an operation whose response was lost. The error is therefore initially unknown, rather than a declined payment. Confirmation of completion must also reflect the actual meaning of task, rather than merely a request returning. Financial data, tokens, and free-form descriptions are excluded from the proposed contract.

Flutter: testing

INDEPENDENT ANALYSIS AND PROPOSAL

Persist the decision, not just the chart

A dashboard is a changing view. Its event set can grow, the rule can change, and the baseline can be recalculated. If expansion depends on those elements, I would record an immutable decision containing the release version, rule identifier, observation interval, population, and outcome. This record makes it possible to understand why a decision seemed reasonable at the time, including when late data subsequently reveals something different.

Raw events do not need to be retained indefinitely to support that traceability. Sufficient aggregates and quality metadata can be preserved under the retention policy. A correction would produce another snapshot linked to the previous one, rather than silently rewriting a historical approval. The decision owner should be able to distinguish a change in reality from a change in calculation. That distinction also helps investigate inconsistent alerts across teams and identify problems in the instrumentation itself.

In the laboratory, two identical observations with the same key count once, while conflicting outcomes raise an explicit error. This policy is conservative and local: in real systems, state-transition events are legitimate and need a sequence number or precedence rule. I would not move the in-memory dictionary into production. I would use the existing storage and semantics while preserving the requirement: a known conflict must not become apparently trustworthy statistics.

INDEPENDENT ANALYSIS AND PROPOSAL

When the experiment measures the wrong thing

Imagine that the candidate version starts emitting two completion events for the same attempt. The chart may improve even though no additional customer has completed a task. Deduplication reduces this risk but depends on stable identity. Changing the identifier midway through the journey or reusing it across operations produces opposite errors. I would treat instrumentation changes as part of the release contract, reviewed alongside visible behavior.

Another case is poor connectivity. A customer may complete the task and send telemetry much later, while the baseline transmits immediately. Comparing open windows would create an artificial regression. We need a closing window, a policy for late events, and a coverage indicator. If a collector failure disproportionately affects the candidate version, missing data cannot be interpreted as an absence of problems. The local gate demonstrates this refusal to decide.

There are also failures outside the application: a slow backend, a rule change, a campaign, or a holiday. I would avoid causal attribution based on a simple before-and-after comparison. The comparison would need to account for exposure, period, and cohort, with input from those designing experiments. If a version degrades an essential journey, however, containing plausible harm need not wait for a perfect statistical explanation. Policy must distinguish a conservative protective decision from subsequent evidence of benefit.

INDEPENDENT ANALYSIS AND PROPOSAL

Sixteen tests, three kinds of boundary

The case_15.py file successfully ran sixteen local tests. The checks cover healthy behavior, completion and latency regressions, missing telemetry, a small sample, an empty collection, duplication, conflict, negative duration, percentile calculation, exact threshold boundaries, and separation between versions. The result describes functions over in-memory structures; it does not measure battery use, rendering time, connection quality, or the stability of a real Flutter application. The review added rejection of NaN, infinity, invalid types, and inconsistent policy; an empty sample remains on hold.

In the main simulation, the baseline contains one hundred observations, 98% completion, and a P95 of 300 milliseconds. A candidate with 90% completion receives PAUSE_SUCCESS. These numbers were chosen to make the rule's branch obvious and must not be presented as business results. The healthy case returns ELIGIBLE_FOR_REVIEW, preserving the distinction between a passing test and a sufficiently justified release decision. The results file accompanies the code for inspection.

The next validation layer would require widget and integration tests, background and foreground scenarios, degraded networks, and representative devices. I would also rehearse event loss and collector recovery. Finally, I would validate how people interpret the states, because a technical test can prove that a message appeared without showing whether anyone understood what to do. Progression between layers would depend on journey risk, not on test count alone.

Python 3.12: unittest
Flutter: testing

INDEPENDENT ANALYSIS AND PROPOSAL

Observability that explains a decision

I would begin with a small set of metrics with a defined operational use: completion per eligible attempt, perceived duration, incomplete observation, and cumulative exposure. Each metric must state which decision it can change. An aggregate rate for the entire app may hide a regression on a particular device or connection; unlimited granularity, on the other hand, creates noise, cost, and identification risk. Selecting dimensions would be part of the privacy and operations review.

Traces can connect technical steps, but they do not replace the journey's business event. I would associate the aggregate outcome with diagnostic references under controlled access, without putting financial data in span names or unrestricted attributes. Troubleshooting sampling may differ from the collection used in the experiment's denominator. Mixing those populations without recording the distinction would produce attractive metrics and fragile decisions. Telemetry availability also deserves its own measurement.

A useful alert would state which version, cohort, journey, and rule changed state, and identify an owner and procedure. I would not send the same notification to every squad. I would track time to recognize the signal, contain exposure, and confirm recovery. If an alert changes no action or has no owner, its design needs adjustment. Observability succeeds through response speed and quality, not merely through the volume of data collected.

OpenTelemetry: traces

INDEPENDENT ANALYSIS AND PROPOSAL

Rollout as a reversible commitment

I would begin evaluation with a journey whose risk is bounded, without connecting the gate to automatic expansion. During the first stage, the rule would observe existing human decisions and surface disagreements for review. This would reveal poorly calibrated thresholds, late data, or different definitions of completion across teams. The observation phase also needs a deadline: remaining in shadow mode indefinitely creates cost without necessarily creating learning.

We would then choose a small exposed population using criteria defined before the release. The next window would begin only after evaluating the previous one, with explicit responsibility for pausing and communicating. Reverting a UI configuration is not the same as undoing completed data changes or operations. The containment plan must state what actually reverts, what remains in installed versions, and which effects require reconciliation. An update distributed through an app store cannot be removed from every device instantly.

As an EM, I would protect team time to respond to the first cohorts and align partners on availability. As a Staff engineer, I would review compatibility between versions and the separation of configuration, protocol, and authorization. I would not treat the absence of complaints as sufficient evidence of safety. Final release would require measurement quality, an acceptable outcome, and the ability to sustain operations. When those conditions are absent, keeping exposure limited may be the most responsible decision.

INDEPENDENT ANALYSIS AND PROPOSAL

Observing without burdening payment

Overly detailed collection can consume network capacity, battery, and CPU on already constrained devices. I would therefore define limits for events per attempt, bytes per batch, local retention time, and sending frequency. The queue could not grow without control during an incident. At the limit, it would discard data under a known policy and record aggregate loss while preserving journey functionality. Behavior under pressure must be specified as carefully as the healthy path.

On the server, version names, journeys, and device classes are controlled dimensions; personal identifiers should not become metric labels. Access to diagnostic data would be separate from access to the rollout dashboard. Someone authorized to monitor quality does not automatically gain access to transaction content. The design should also allow collection to be revoked or reduced when a privacy risk emerges, without requiring an emergency update across the installed base.

Protecting the decision requires additional controls. Signed configuration, change records, and scope limits prevent an operational error from exposing a version to a larger population than approved. None of those controls is implemented by the laboratory. They define the work remaining before a real integration. I would estimate collection cost per eligible journey and compare it with diagnostic usefulness, removing attributes that nobody can justify with a specific question.

INDEPENDENT ANALYSIS AND PROPOSAL

ADR: journey metrics as an additional gate

Proposed decision: complement release control with an explicit journey evaluation, retaining expansion authority within the team's process. Looking only at crashes is an inexpensive and useful alternative, but it misses silent task failures. Automating the entire release reduces decision time but requires much greater confidence in measurement and policy. For the first pilot, I would choose an explainable recommendation with human review and a defined evaluation period.

This decision adds work to define events, maintain contracts, and investigate incomplete data. It also risks blocking releases because of noise and delaying important fixes. Every pause therefore needs a reason and a path to resolution. Policy should not encourage teams to collect fewer errors to make outcomes look better. Coverage metrics and independent instrumentation review help avoid that distortion and must be included in scope.

I would reconsider the decision if the journey lacked sufficient volume, external factors dominated the effect, or instrumentation cost exceeded the plausible benefit. In that case, moderated research, integration tests, and existing controls might answer the question better. A useful ADR records those conditions for review. It does not turn the selected solution into a universal standard; it preserves why a team chose to experiment with this mechanism in that domain at that time.

INDEPENDENT ANALYSIS AND PROPOSAL

Team delivery, value, and the Staff conversation

The first increment could be one journey with an event contract, a validated baseline, a dashboard, and a gate in observation mode. I would distribute the work across mobile, backend, and observability, partnering with Product to define completion. Leadership would need to negotiate capacity for rapid fixes and prevent the pilot from becoming another unowned responsibility. Acceptance would include a missing-telemetry simulation and a pause exercise, not just a dashboard working in a demonstration.

The value model would start with exposure to failures actually avoided and associated support contacts, without assuming that every abandonment would have led to support. Investigation hours saved represent potential capacity; cash savings require a verifiable expense change. Instrumentation, storage, and maintenance costs belong in the calculation. I would keep those outcomes separate from conversion improvement, which requires a different causal design and may overlap the same customer population.

My Staff question would be: what guarantee are we relying on to promote a version, and how do we notice when that guarantee is no longer observable? My EM question would be: who can stop expansion, and do they have time to act when it happens? Both meet in the same responsibility. Improving the gate is worthwhile only if the team can interpret its result and protect the experience of whoever receives the next version.

INDEPENDENT ANALYSIS AND PROPOSAL

Routing is a hypothesis about a need 16

Presenting Precog, Nubank's Real Time Event AI

Original official publication, accessed 23 October 2026.

In June 2023, Nubank introduced Precog, which uses event representations to support intent models. The article reports an increase of more than 50% in the volume of calls correctly routed to specialists without customer input, compared with the previous model. That result belongs to the implementation and period described in the publication. It does not mean a gain of fifty percentage points, does not automatically measure resolution, and is not a target this laboratory can promise to reproduce.

The study opportunity lies in sustaining that quality as products, events, and reasons for contact change. A prediction can be technically confident and wrong about what the person needs now. It can also identify the product correctly and route the person to an unavailable queue. I would assess routing as an operational decision that can be corrected, rather than a classification that ends the effort to understand the customer. Support must be able to challenge the suggestion and preserve useful context.

This chapter's laboratory uses synthetic event weights, without embeddings or training. It tests time windows, deduplication, minimum support, abstention, and declared priority. The intention is to make those boundaries verifiable before discussing better models. A weight distribution is not a calibrated probability. The code demonstrates policy, not predictive quality, and grants no authority to block an account, transfer funds, or make any financial decision on someone's behalf.

Presenting Precog, Nubank's Real Time Event AI

INDEPENDENT ANALYSIS AND PROPOSAL

Intent, urgency, and availability are different things

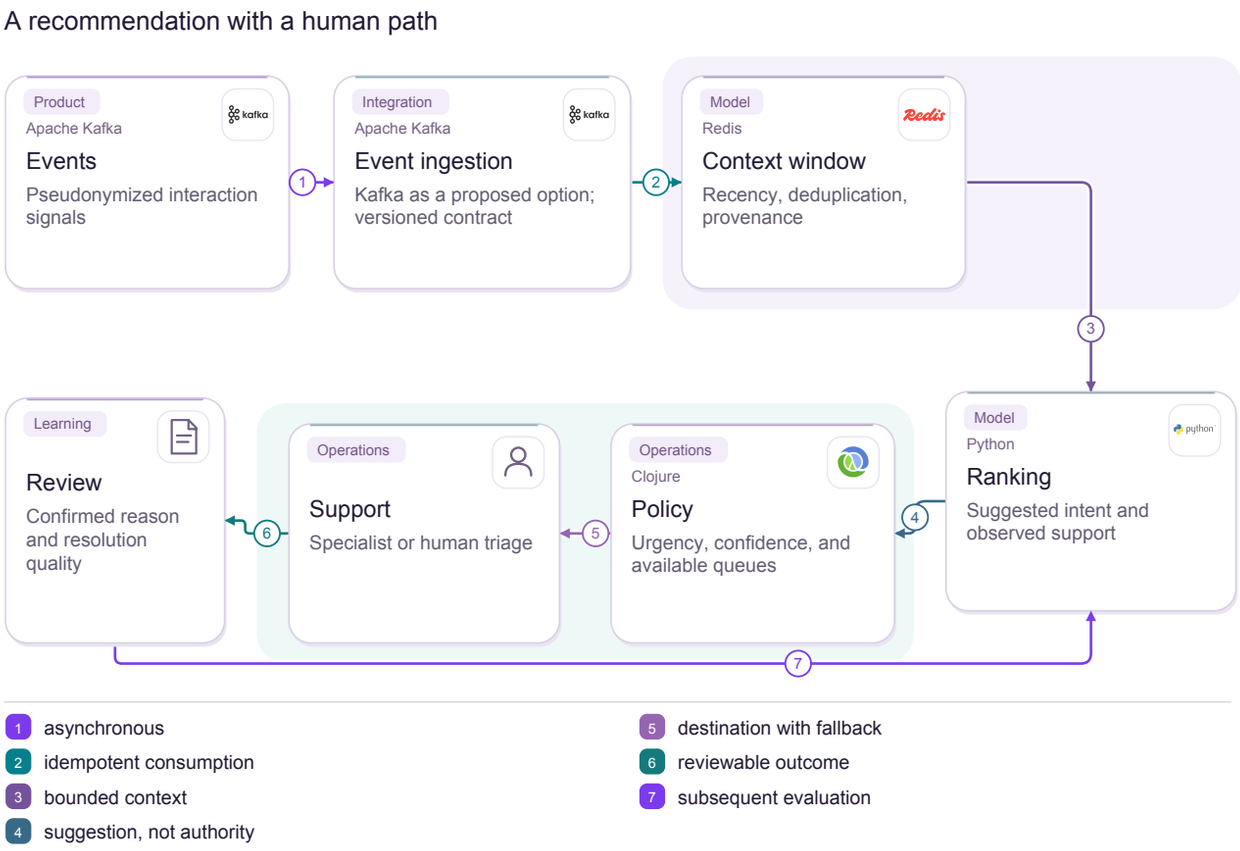
The domain needs to distinguish the probable reason for contact, declared urgency, and support capacity. Someone who has just attempted a payment may be calling for another reason. A risk signal may require specialist triage even when the likely product is a card. And a queue with capable specialists may be unavailable at that time. Combining everything into a single score makes the result harder to explain and would hide decisions governed by different policies.

I would model the recommendation as a record containing the suggested intent, evidence used, policy version, reason for abstention, and operational destination. The person and the support agent need a simple way to correct the reason. That correction should not automatically be treated as a perfect label: the customer may raise several topics, the agent may choose a convenient category, and another team may resolve the issue. Taxonomy quality requires joint work with Operations.

In the laboratory, declared urgency leads to a dedicated human queue, independently of the ranking. Insufficient events or an intent without an available queue lead to general triage. These are educational choices, not Nubank policies. A real implementation would need to validate capacity, schedules, languages, and accessibility needs. The important contract separates recognizing a pattern from deciding how to provide help; neither decision should remove the opportunity to explain the problem.

REFERENCE ARCHITECTURE

A recommendation with a human path



Original reference architecture with proposed queue names and boundaries. It does not reproduce internal systems.

The public article describes event consumption and Clojure/Python services, but Kafka's use in this design is a reference choice, not a claim about Precog. The boundary between ranking and policy prevents queue availability from being confused with model confidence. The chosen destination must carry a reason Operations can understand and allow correction without restarting the story. Outcomes return asynchronously for evaluation after considering how the contact ended. A corrected label does not rewrite the original decision: it records what was known and what was learned. The laboratory implements a window, weighted ranking, and in-memory policy. It includes no embeddings, telephony, or real queue. In production, I would prefer to integrate existing capabilities and measure the additional cost of a recommendation before introducing a new service into the support path.

CODE · PROPOSED CONTRACT

Abstaining is also a valid decision

PYTHON / ORIGINAL PROPOSAL

```python
def route(scores, available, urgent=False, threshold=0.75, min_weight=4):
    if type(urgent) is not bool:
        raise ValueError("urgent must be boolean")
    if urgent:
        return {"queue": "human_urgent", "reason": "declared_urgency"}
    validate_ranking(scores, threshold, min_weight)
    total = sum(scores.values())
    if total < min_weight:
        return {"queue": "human_general", "reason": "insufficient"}
    ranked = sorted(scores.items(), key=lambda x: (-x[1], x[0]))
    intent, weight = ranked[0]
    confidence = weight / total
    if confidence < threshold or intent not in available:
        return {"queue": "human_general", "reason": "abstention"}
    return {"queue": intent, "reason": "ranked", "share": confidence}

clear = route({"card": 8, "pix": 1}, {"card"})
uncertain = route({"card": 5, "pix": 5}, {"card"})
assert clear["queue"] == "card"
assert uncertain["queue"] == "human_general"
assert route({}, set())["reason"] == "insufficient"
assert route({}, set(), True)["queue"] == "human_urgent"
```

Executed excerpt from case_16.py. Ranking uses invented weights; it does not implement Precog. Types, imports, and validators are in the complete file.

The function applies urgency before ranking, requires minimum support, and avoids a nonexistent queue. The share field represents the winning intent's share of weight, not its probability of being correct. The 0.75 threshold is synthetic. An ML implementation would need confidence calibration by intent and monitoring of abstention quality, rather than copying these numbers. Human fallback is a functional path, not a hidden error. The rule also does not manage capacity: available is an already resolved input. If it is stale, the destination must reject the request with a defined recovery path. Invalid types and domains raise ValueError before any favorable result.

Python 3.12: unittest

CODE · PROPOSED CONTRACT

Pure policy in Clojure, effects at another boundary

CLOJURE / ORIGINAL PROPOSAL

```clojure
(ns independent.support-policy)

(defn choose-route
  [{:keys [urgent? available candidates min-support]}]
  (let [ordered (sort-by (juxt (comp - :weight) :intent)
                         candidates)
        total (reduce + (map :weight ordered))
        best (first ordered)
        share (if (pos? total)
                (/ (:weight best 0) total)
                0)]
    (cond
      urgent?
      {:queue :human-urgent :reason :declared-urgency}

      (< total min-support)
      {:queue :human-general :reason :insufficient}

      (or (< share 3/4)
          (not (contains? available (:intent best))))
      {:queue :human-general :reason :abstention}

      :else
      {:queue (:intent best)
       :reason :ranked
       :share share})))

;; O adaptador persiste e encaminha fora desta função.
;; Contrato proposto: candidate weights >= 0.
;; Empates recebem ordem determinística, não mais confiança.
```

Illustrative Clojure, not executed; namespaces, queues, and policy are external proposals.

The proposed function receives immutable data and returns a recommendation without calling telephony, recording effects, or updating shared state. This makes policy easier to test separately from integration. Clojure is part of Nubank's published technical context; this fragment is independent and was not executed. Weight validation and a consistent availability read belong to the adapter. Using an atom in a prototype does not provide persistence or a transaction with external systems. Functions passed to mechanisms that may repeat evaluation must remain free of effects. Routing needs its own identity and reconciliation outside the calculation.

Clojure: atoms

INDEPENDENT ANALYSIS AND PROPOSAL

Event time and knowledge time

An event may occur before a contact and arrive afterward. Reconstructing the decision with every event we know today would create an artificially favorable evaluation. I would record separately when the fact occurred and when it became available to the router. A replay must respect both. This boundary is particularly important on mobile networks, where signals may remain on the device and be transmitted in a batch after the person reconnects.

The local window discards future events, events outside the period, and old receipts, while rejecting conflicting duplicates. The limits of 180 and 30 units are synthetic parameters, not recommended minutes for Nubank. The experiment must declare its unit and clock. An inaccurate client clock may require additional validation or another reference; comparing numbers as if every machine shared exactly the same time is not enough.

A stable event key helps deduplicate delivery attempts but does not identify two different interactions of the same type. I would avoid discarding events simply because their attributes look similar. In the proposed Kafka integration, committing an offset does not confirm routing in the support system. Those effects belong to different boundaries. The design would need a durable decision record and reconciliation to determine whether an unknown routing attempt can be repeated without transferring the customer again.

Apache Kafka: design

INDEPENDENT ANALYSIS AND PROPOSAL

The costliest error may be a plausible queue

A model may route most contacts correctly while failing on the most urgent topics. An aggregate average would hide that pattern. I would stratify evaluation by intent, urgency, new product, and cases without history, always showing sample size and uncertainty. I would also observe when a specialist corrects the destination, how much time that takes, and whether the person has to repeat sensitive information. Quality does not end with selecting the first queue correctly.

A change in event naming can gradually reduce context without taking the service down. Another failure occurs when the predicted intent no longer has an available team. The contract must provide a general path and preserve the suggested reason as a hypothesis. Keeping a person in a transfer loop because each component considers its own decision correct is unacceptable. The system must recognize repeated routing and escalate to a human resolution path with clear responsibility.

I would also test manipulated or repeated events without turning this chapter into instructions about internal controls. The principle is straightforward: behavioral telemetry is limited evidence, never an access credential. Visiting particular screens does not grant financial authorization. Changes to destination, priority, or context must respect support and privacy policies. The laboratory demonstrates abstention and deduplication; abuse testing, telephony, and real operations remain outside its scope.

INDEPENDENT ANALYSIS AND PROPOSAL

Fifteen policy checks, no accuracy promise

The fifteen tests in case_16.py passed locally. They check a dominant intent, ambiguity, missing evidence, urgency, an unavailable queue, duplication, conflict, future events, an old occurrence received now, an old receipt, a negative weight, and insufficient support. Each test makes a claim about policy and windows, not about understanding human intent. The result does not validate embeddings, measure support, or prove the chosen threshold appropriate for any product. Additional regressions reject nonfinite, fractional, and boolean weights, as well as invalid policy and clocks.

In the demonstration, weights of eight for card and one for Pix route to card with a share of approximately 0.889. Equal weights produce abstention and general triage. These examples show why a decision can be reproduced, but they also expose the method's limits. Invented weights can produce a convincing recommendation that is completely wrong about a person. The correct interpretation is that the mechanism respected the synthetic contract, not that it achieved a particular level of predictive quality.

A real evaluation would require reviewed labels, a separate temporal dataset, treatment of multiple intents, and analysis of repeat contacts. I would involve support specialists in discussing errors rather than relying solely on a confusion matrix. We would need to measure resolution and repeat contact, because apparently fast routing may simply postpone the work. The decision to proceed would depend on those outcomes and on support's capacity to absorb fallback paths.

Python 3.12: unittest

INDEPENDENT ANALYSIS AND PROPOSAL

Metrics that follow through to resolution

The first metric would be correct routing under an agreed, reviewable definition. It must specify whether the destination was equipped to resolve the issue, whether a transfer occurred, and whether multiple topics make classification ambiguous. I would then track time to useful assistance, repeat contact, and first-contact resolution without claiming causality merely because these measures move together. Changes in team, staffing, or policy may affect the outcome independently of the model.

Abstention deserves its own dashboard. A low rate is not necessarily better: it may indicate overconfidence. A high rate may be a responsible choice for new cases, but it can also reveal missing events. I would show evidence coverage, recency, and fallback reasons by intent while preserving sufficiently aggregated groups. The dashboard should distinguish collection, model, policy, and operational-availability problems. Without that separation, every investigation ends up in the same data queue.

Each alert would need to point to an action. Loss of recency may trigger a pipeline investigation; increased transfers for an intent may suspend its automation; an unavailable queue requires an Operations decision. The model owner cannot resolve the third situation alone. I would record model and policy versions with every decision to compare changes in context. Logs should not contain free-form transcripts or identifiers unnecessary to answer the operational question that justified collecting them.

INDEPENDENT ANALYSIS AND PROPOSAL

Shadow evaluation with outcomes, then limited exposure

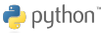
The first evaluation could run recommendations without changing routing. For each eligible contact, we would compare the suggestion with the observed destination and outcome, recognizing that the existing flow can also be wrong. The review design must include easy and difficult contacts, not just cases with a clear conclusion. A sample excluding abandonments, repeat contacts, or unknown categories can produce an optimistic evaluation precisely where the mechanism most needs oversight.

A controlled pilot could then automate only intents with sufficient evidence, preserving a human path and immediate correction. The population, limits, and pause conditions would be defined before exposure. I would not start with urgent topics or scenarios where an incorrect transfer could delay necessary protection. Those paths need specific policies developed with their owners, rather than an exception improvised after the first significant incident.

As an EM, I would align support capacity, training, and availability to review errors during the first weeks. As a Staff engineer, I would ensure decision replay, contract versions, and event compatibility. The pilot should lead to a choice: expand the intent, change the threshold, improve context, or return to triage. I would not treat increasing coverage as an isolated objective. Benefit depends on better routing and resolution with less effort, while preserving a person's right to explain a need the prediction does not capture.

INDEPENDENT ANALYSIS AND PROPOSAL

Privacy and separation of authority



Support context can reveal habits and financial difficulties. I would apply minimization from the definition of signals onward, avoiding free-form descriptions or transaction content in a pipeline that only needs to recognize interaction categories. Pseudonymizing identifiers reduces exposure but does not automatically anonymize the dataset. Access, retention, and purpose must remain explicit, with input from privacy and security owners. Assessing benefit does not remove these design obligations.

A queue recommendation must not change limits, release operations, or serve as proof of identity. That separation appears in the code and must also exist in service permissions. A component classifying intent may access limited signals, while performing a financial action requires a different contract and authority. Combining the two for convenience would create a path in which behavioral inference acquires power it should never possess.

Performance belongs at this boundary too. If ranking exceeds its time budget, support must continue through a known route. The general queue cannot depend on the same resource that just failed. I would estimate inference cost per eligible contact and monitor context storage without choosing retention solely to make experimentation easier. A sophisticated model that delays assistance or expands collection without proven benefit may be inferior to a simple, well-operated policy.

INDEPENDENT ANALYSIS AND PROPOSAL

ADR: automate with explicit abstention

The proposed decision separates intent calculation, routing policy, and operational execution, treating abstention as a normal outcome. A fixed rule based on the last screen is simple, but it may confuse recent navigation with the current need. A more expressive model may increase coverage, but it requires evaluation, maintenance, and interpretation of errors. The laboratory chooses a small rule to make boundaries examinable; it does not recommend replacing Precog or claim superiority over the published work.

The advantage of separation is being able to change a queue policy without retraining the model and evaluate a new model without changing the support integration. The cost is maintaining contracts and versions between components and preventing inconsistencies when a queue changes name or becomes unavailable. The persisted decision should record the destination catalog in effect. If this complexity does not solve an observed problem, the mechanism can remain within an existing service rather than becoming three new services.

I would revisit the ADR if human corrections suggested an unsuitable taxonomy, useful coverage were small, or maintaining context cost more than the gain. I would also consider a short question to the customer a legitimate alternative: sometimes asking is better than inferring. The choice should compare total effort and resolution quality, not just how many contacts avoid an initial interaction. The person seeking help remains the primary source about their own need.

INDEPENDENT ANALYSIS AND PROPOSAL

Useful capacity and responsibility for the queue

The first deliverable would be one intent with an event contract, an abstention policy, label review, and outcome tracking. Operations would participate from problem definition onward, because correct routing depends on how support works. As an EM, I would reserve time for joint error review and training, avoiding placing the entire cost of a technical experiment on support agents. I would also define who can suspend automation for an intent.

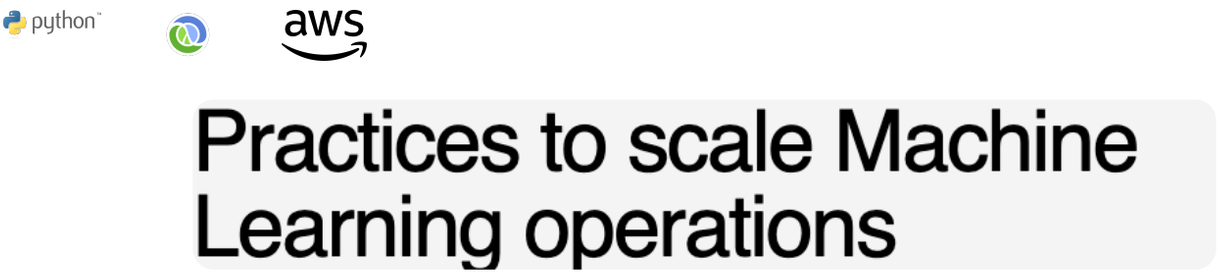
Economic value would be estimated from avoided transfers, work minutes that can actually be recovered, and the mechanism's incremental cost. A realization factor is essential: saving seconds on average does not automatically reduce expenses. Repeat contacts, longer handling time in another queue, and review work must be included in the comparison. Customer time saved would be reported separately as an experience outcome. I would not add transfer and contact reductions when they describe the same avoided effort.

My Staff question would be: can we reproduce what the router knew when it decided, including available queues and the policy version? My EM question would be: when the prediction is wrong, is a person or team responsible and able to resolve the issue without restarting the journey? The answer connects architecture and leadership. The goal is useful help with less repetition while preserving room to correct a hypothesis that seemed plausible.

INDEPENDENT ANALYSIS AND PROPOSAL

Protection within the time the experience allows

17



Original official publication, accessed 23 October 2026.

In April 2025, Nubank published practices for operating real-time models, including a Model Server with Clojure and Python, feature retrieval, optimization, and shadow mode. The article provides technical context but does not disclose decision policies, risk thresholds, or losses by product. This study does not identify a vulnerability or attempt to infer those controls. It starts with an engineering question: how do we preserve a responsible decision when context and time are limited?

In a financial journey, waiting indefinitely is also a choice with consequences. Allowing everything when a dependency fails is another choice, often an inappropriate one. Between those extremes are decisions based on context, recovery, and review. I would discuss with Risk which evidence is indispensable, what can be missing, and how the person continues when an operation cannot be authorized immediately. That conversation cannot be delegated to a score without explaining its purpose and limits.

The laboratory uses a logistic function with invented, fixed parameters. There is no training, calibration, or Nubank dataset. The objective is to demonstrate the separation of calculation, policy, and error cost, including false positives. Sixteen tests verify this small contract. They do not demonstrate fraud-detection capability, loss reduction, or regulatory suitability. The examples provide a basis for discussing how I would turn an ambiguous decision into a safe experiment with domain specialists.

Practices to scale Machine Learning operations

INDEPENDENT ANALYSIS AND PROPOSAL

The score is not the decision

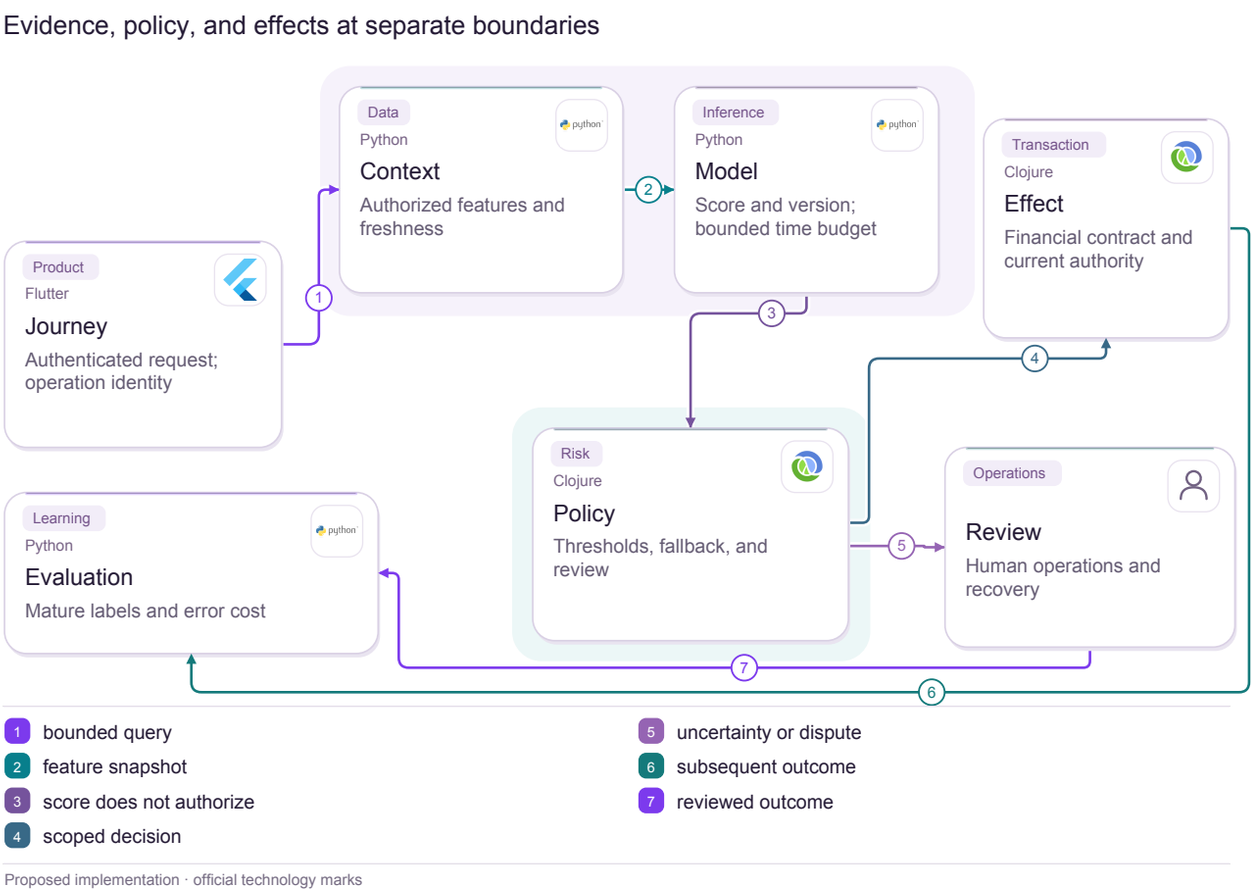
A number between zero and one may look like a probability, but that appearance does not guarantee calibration. Before using it, we need to know what it attempts to predict, over which horizon, with which data, and under what conditions. The local model merely transforms synthetic signals into a value that can be ranked. Calling it a probability of fraud would conceal a property that has never been measured. This distinction is central to preventing mathematical presentation from granting policy unwarranted authority.

The laboratory policy chooses allow, review, or block, with specific outcomes for missing and stale data. Thresholds are decision parameters, not natural characteristics of the model. A threshold change can alter review volume, friction, and losses while the model remains identical. I would version these elements separately so an incident could be attributed to the right change. I would also record the limits of each action: review does not mean condemnation, and blocking an attempt does not necessarily block the account.

Review must be a real capability. Without a team, deadline, or recovery path, an intermediate state becomes an invisible queue. In the proposal, each outcome would have an appropriate message, next step, and observability without revealing sensitive mechanisms. The technical contract must support an understandable experience. Model quality matters, but customer trust also depends on how the system handles uncertainty and allows an incorrect decision to be corrected.

REFERENCE ARCHITECTURE

Evidence, policy, and effects at separate boundaries



Independent architecture. It does not describe Nubank's internal thresholds, policies, or components.

A risk decision does not replace authentication, authorization, or financial consistency. The component producing the effect must validate appropriate authority at the operation boundary even when policy recommends allowing it. Features arrive with version and temporal information so absence is not confused with a normal value. The review path needs its own capacity and deadlines; it is not a repository for cases the model could not classify. Evaluation receives outcomes after labels mature, avoiding calling a transaction legitimate simply because no dispute has yet appeared. The laboratory implements only a score, policy, and synthetic error accounting. It performs no authorization, production detection, or support. Python and Clojure are technologies cited in the public source; the organization shown here is an original proposal that would need comparison with existing boundaries before any integration.

CODE · PROPOSED CONTRACT

An educational model and an explicit policy

PYTHON / ORIGINAL PROPOSAL

```python
def toy_score(features):
    validate_features(features)
    amount = min(features.amount_cents / 100000, 4)
    velocity = min(features.velocity, 10)
    z = -4 + 0.5 * amount + 0.7 * velocity
    z += 1.2 * int(features.device_change)
    return 1 / (1 + math.exp(-z))


def decide(features, allow_below=0.30, block_from=0.85):
    if not 0 <= allow_below < block_from <= 1:
        raise ValueError("invalid thresholds")
    if features is None:
        return "REVIEW_MISSING"
    validate_features(features)
    if features.age_seconds > 60:
        return "REVIEW_STALE"
    probability_like = toy_score(features)
    if probability_like >= block_from:
        return "BLOCK"
    if probability_like >= allow_below:
        return "REVIEW"
    return "ALLOW"
```

Executed excerpt from case_17.py. Fixed, untrained toy logistic model. Types, imports, and validators are in the complete file.

The coefficients were chosen to produce different test paths; they were not learned and do not represent real risk. The calculated value is named probability_like to emphasize that limit. Stale or missing data leads to synthetic review, while negative inputs are rejected. The 0.30 and 0.85 thresholds must not be transferred into operations. A real implementation requires evaluation of discrimination, calibration, stability, and error cost, plus a formal fallback decision. The code preserves the separation between producing a score and selecting an action. It contains no internal controls, actual fraud rules, or financial integration. Invalid types and domains raise ValueError before any favorable result.

Python 3.12: unittest

CODE · PROPOSED CONTRACT

Separate loss, false positives, and review

PYTHON / ORIGINAL PROPOSAL

```python
def realized_cost(rows, fp_cents=500, review_cents=100):
    if any(type(x) is not int or x < 0 for x in (fp_cents, review_cents)):
        raise ValueError("costs must be non-negative integer cents")
    totals = {"missed_fraud": 0, "false_positive": 0, "review": 0}
    for action, fraud, amount_cents in rows:
        if type(amount_cents) is not int or amount_cents < 0:
            raise ValueError("amount must be non-negative integer cents")
        if type(fraud) is not bool:
            raise ValueError("fraud label must be boolean")
        if action == "ALLOW" and fraud:
            totals["missed_fraud"] += amount_cents
        elif action == "BLOCK" and not fraud:
            totals["false_positive"] += fp_cents
        elif action in {"REVIEW", "REVIEW_MISSING", "REVIEW_STALE"}:
            totals["review"] += review_cents
        elif action not in {"ALLOW", "BLOCK"}:
            raise ValueError("unknown action")
    totals["total"] = sum(totals.values())
    return totals

rows = [("ALLOW", True, 10000),
        ("BLOCK", False, 10000),
        ("REVIEW", True, 10000)]
assert realized_cost(rows)["total"] == 10600
```

Function and scenario executed in case_17.py; synthetic costs, not a projection for Nubank. Types, imports, and validators are in the complete file.

Money remains in integer cents, and the components stay identifiable. In the example, allowed fraud costs 10,000 cents, an incorrect block costs 500, and a review costs 100, totaling 10,600. These are arbitrary simulation costs. The function does not model recovery, residual losses after review, reputation, or long-term effects. In particular, review does not guarantee fraud prevention; its full cost depends on the outcome. This decomposition avoids hiding false positives behind an aggregate accuracy metric, but it becomes useful for investment only when each component has its own definition and evidence.

Python 3.12: decimal

INDEPENDENT ANALYSIS AND PROPOSAL

A feature snapshot needs meaning

Two features named velocity may measure different things: attempts, confirmed operations, or observed devices. Even when both are integers, their semantics can differ. I would require a shared definition specifying the window, unit, source, and treatment of absence. The model should not receive a zero produced by a timeout as if it meant no activity. This confusion changes the interpretation of the customer without necessarily producing a visible technical service error.

Freshness is also domain-specific. Registration information may tolerate a different age from a signal associated with the current attempt. The laboratory's single age_seconds field is only a simplification for testing the stale-data branch. In production, I would assess freshness per feature and decision. Essential data needs an unavailability policy; optional data needs evidence that the model can operate without it under the evaluated conditions.

The inference snapshot should be traceable within privacy constraints, without requiring indiscriminate retention of sensitive data. Versioned definitions and metadata may suffice for some investigations; others require governed samples. The choice depends on purpose and the confirmation period. The team must be able to explain which information supported a decision and which was missing, without retrospectively inventing context that arrived only after the operation.

INDEPENDENT ANALYSIS AND PROPOSAL

A partial failure must not become blanket permission

A slow dependency can extend journey time and consume resources needed by other requests. Setting a timeout limits that harm but does not decide what happens next. I would separate the technical budget from risk policy: first, how long can we wait; then, what decision is acceptable without that context? The second answer belongs to the domain and may vary by operation, remaining evidence, and exposure. A generic permissive fallback is not an automatic engineering conclusion.

Reviewing everything also carries risk. A broad incident can exceed human-queue capacity, delaying recovery for legitimate customers. The plan must recognize saturation and define previously agreed actions, clear messaging, and appropriate priority. Review must not become a word that hides unavailability. Monitoring should show arrivals, departures, wait time, and case age, alongside the rate of degraded decisions creating the additional demand.

I would test missing, delayed, inconsistent, and subsequently recovered data without assuming all require the same response. I would also rehearse a new feature-producer version interacting with an old model. The laboratory covers only a small portion of these scenarios using local structures. It does not simulate queue capacity or distributed concurrency. Those limits matter: passing policy tests does not demonstrate that the system will remain safe during an operational failure at scale.

INDEPENDENT ANALYSIS AND PROPOSAL

Sixteen tests and a result that fits in a table

The laboratory successfully ran sixteen tests. They check low, high, and intermediate scores; missing, stale, and future data; negative inputs; invalid threshold ordering; velocity monotonicity; cost decomposition; correct actions with no modeled cost; and an unknown action. Monotonicity is a property of the chosen coefficients, not evidence that velocity must always increase real-world risk. The suite makes the demonstration consistent and reproducible. The review added rejection of nonfinite numbers, fractional cents, ambiguous flags, and unknown review actions.

The economic output adds 10,000 cents of allowed fraud, 500 of false-positive cost, and 100 of review cost. The 10,600-cent total does not represent a company's savings or losses. It is a fixture result, useful for checking that the implementation adds different categories without losing the unit. The correct-action scenario returns zero because the cost model excludes fixed expenses, infrastructure, and residual friction. This must remain explicit so the result does not look like perfect efficiency.

Subsequent validation would require authorized data, temporal separation of training and evaluation, label maturation, and baseline comparison. I would assess segments and degraded scenarios as well as the average. Review cost would depend on actual operations and what happens after triage. Only then could we discuss thresholds. The local artifact demonstrates contract discipline; the quality of a fraud-prevention policy requires different evidence and participation from the responsible specialists.

Python 3.12: unittest

INDEPENDENT ANALYSIS AND PROPOSAL

Measure protection and friction on the same dashboard

I would avoid choosing a single metric to declare success. Confirmed fraud, legitimate transactions blocked, review, and latency need to appear together with their denominators and maturation periods. A policy blocking everything may appear excellent at prevention while making the product unusable. A policy allowing everything may look excellent at immediate completion and accumulate losses later. The decision requires understanding the trade-off among these outcomes rather than celebrating one in isolation.

Label quality directly affects that interpretation. A transaction without a dispute may still be undiscovered fraud; a dispute may be reversed. I would record the window used and revise time series when updates occur while preserving earlier versions. I would also examine selection bias: blocked decisions may never reveal what would have happened if allowed. Comparing groups without considering that difference would produce weak causal conclusions and could lead to unsuitable thresholds.

Operationally, I would monitor time per dependency, missing features, and the proportion of degraded decisions. Correlation between those signals and customer outcomes guides investigation but does not establish cause on its own. Alerts should reach owners able to act on the model, data, or policy. An increase in false positives requires different context from an infrastructure failure. I would keep this responsibility map alongside the dashboard so observability helps resolve the identified problem.

INDEPENDENT ANALYSIS AND PROPOSAL

Shadow mode can teach without authorizing money

A candidate model can produce scores alongside the current flow without changing the real decision. This stage enables output comparison and detection of feature incompatibilities, but it does not eliminate every risk: additional collection consumes resources, processing can delay dependencies, and retention requires governance. I would define an execution budget and isolation so the experiment does not harm the operation it seeks to improve. Shadow mode is an evaluation method, not an exemption from operational responsibility.

Differences between the candidate and baseline would be analyzed by case type and outcome, with Risk and Operations involved. I would not assume disagreement means improvement. We might discover that a new model identifies useful patterns but performs worse for an underrepresented population, or that its gain comes from information unavailable at the actual decision time. Proceeding would require reproducing temporal conditions and validating policy cost, not merely comparing offline performance curves.

Limited exposure would require containment criteria, authority to pause, and a support plan for affected customers. I would also assess model and policy rollback separately. Restoring a version does not undo blocks or delays that have already happened; those effects may need specific recovery. As an EM, I would ensure capacity for that work. As a Staff engineer, I would maintain sufficient contracts and traceability to know what was decided, using which data and version.

INDEPENDENT ANALYSIS AND PROPOSAL

Less data, clearer boundaries

Access to a feature should not be granted merely because it might improve a model. Purpose, availability, and protection need justification, favoring representations that reduce exposure while preserving usefulness. Decision logs must not become an unrestricted copy of the customer's financial context. I would discuss with Security and Privacy what information is needed for investigation, for how long, and under which access controls. The design must support evaluation without distributing sensitive data across every team involved.

I would also separate permissions to read context, publish a model, change policy, and execute financial actions. A failure in one component should not give the same process all these capabilities. The laboratory has no identity, network, or credentials, so it does not demonstrate such isolation. In a real integration, I would use existing tools and standards while explicitly assessing who can change a threshold and how that change is reviewed, recorded, and reversed when necessary.

Technology choices must respect the environment. Python suits the educational core and appears in Nubank's public architecture, while Clojure appears in the described integration. This does not mean every proposal should create a new model server. Improving a feature contract, compatibility test, or metric on the current platform may be more useful. The investment should address an observed gap, with operational cost proportional to the expected benefit and the team's ability to sustain it.

INDEPENDENT ANALYSIS AND PROPOSAL

ADR: version policy separately from the score

The proposed decision keeps the model as a signal producer and policy responsible for the action, with independent versions. Embedding thresholds and fallback within the model artifact simplifies deployment but mixes different kinds of change. Distributing rules across consumers offers flexibility and may create divergences that are difficult to trace. I would start with an explicit contract where the decision already occurs, avoiding a new platform without a demonstrated need.

The benefit is explaining whether an outcome changed because of the signal, context, or policy. The cost is maintaining compatibility and preventing unevaluated combinations. A new model should not be promoted with just any old policy merely because its output format is identical. The version record must include conditions of use and expected data. Operations also needs to know which combinations are supported during migration and which forms the rollback path.

I would revisit the decision if separation added latency without useful benefit, or if the domain required a joint deployment to preserve consistency. Logical separation does not require a remote call between services. The same process can maintain distinct modules and contracts. The question is not how many boxes appear in the diagram, but whether a team can evaluate, change, and operate the decision without losing its history, limits, or responsibility for its effects.

INDEPENDENT ANALYSIS AND PROPOSAL

Economic outcomes with human responsibility

The first increment could be a critical-feature contract and a policy evaluation in shadow mode with discrepancy review. As an EM, I would align Engineering, Data Science, Risk, and Support around objectives covering protection and experience. I would also reserve capacity for false-positive analysis and recovery so the improvement backlog does not overlook affected customers. Leadership must turn this discussion into clear priorities, owners, and criteria for stopping the experiment.

The economic model would compare fraud losses under equivalent conditions, the cost of blocked legitimate transactions, review cost, and incremental infrastructure. I would not substitute accuracy for value. Friction costs need definition: additional support, operational time, and demonstrated lost usage are different measures. Benefits sharing the same event must not be counted twice. Without evidence, I would present a range of assumptions and sensitivity analysis rather than calling the number realized savings.

My Staff question would be: what information can be missing without invalidating the decision, and who approved that boundary? My EM question would be: do we have the capacity to correct the decision when a legitimate person cannot proceed? These questions connect the technical contract to teamwork. The goal is not an impressive classifier in isolation. It is a protection system that responds within the journey's time constraints and remains accountable for the errors it inevitably needs to recognize and address.

INDEPENDENT ANALYSIS AND PROPOSAL

The same name, different decisions

18

Dealing with Train-serve Skew in Real-time ML Models: A Short Guide

Original official publication, accessed 23 October 2026.

Nubank's public guide to train-serve skew, published in June 2023, discusses differences between data used for training and inference. Examples include different time windows, absence represented as zero, and incompatible sources. These are educational examples of a class of problem, not evidence of specific incidents or customer losses. This chapter uses that context to explore a feature contract that can be compared without reconstructing the past using future information.

A feature may have the correct name, type, and numeric range while still representing something else. Training may count confirmed purchases over sixty days, while the service counts attempts over thirty. Both produce plausible integers. No exception needs to be raised for the model to start receiving information outside the meaning on which it was built. The problem requires semantic observation, not just format validation or endpoint availability.

The laboratory implements a windowed sum with two clocks: occurrence and knowledge. It also generates a contract identity and compares values, absence, and version. Everything uses invented events and integer cents. Fifteen tests verify temporal boundaries, duplication, late information, and contract incompatibility. The objective is not to build a feature store, but to demonstrate the minimum needed for a precise question: can we reproduce the information available when the decision actually happened?

INDEPENDENT ANALYSIS AND PROPOSAL

The contract contains more than the type

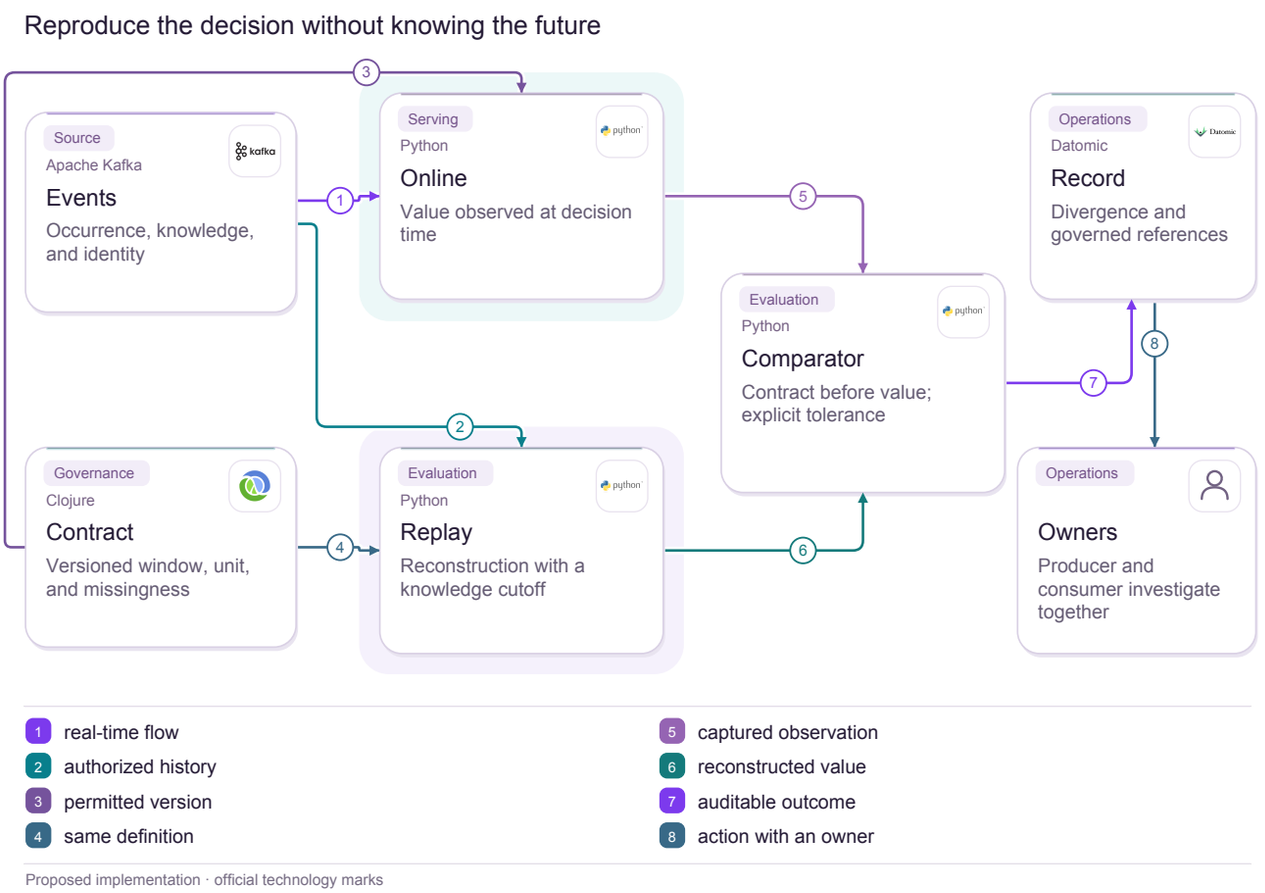
I would define a feature by name, unit, entity, window, temporal boundaries, the state of included events, source, and missingness rule. I would also record which subsequent corrections change the value and how they affect historical replays. Without these choices, two teams can correctly implement different specifications and discover the divergence only when model behavior changes. A textual description helps, but it needs executable examples that expose boundary cases.

In the laboratory, the window is open on the left and closed on the right: an event exactly at the start is excluded; an event at decision time can be included if it was already known. The unit is cents, and an unavailable source produces None. An available source with no events produces zero. This difference is small in code and important to the decision: zero describes an empty observation, while None describes missing observation. The model may need to treat these states differently.

Contract identity derives from a canonical representation of its parameters. It helps detect incompatible combinations but does not prove that the implementation follows the declared meaning. A service can publish the correct hash while calculating the wrong window. I would therefore combine contract verification with example comparison and replays. Documentation defines what should happen; tests and observations show whether producers and consumers are actually doing the same thing.

REFERENCE ARCHITECTURE

Reproduce the decision without knowing the future



Original design. Proposed contracts and names; it does not represent an internal feature store.

The comparator should compare only values representing the same entity, instant, and definition. Replay receives a knowledge cutoff to prevent later-arriving events from making the past artificially complete. The contract governs online and offline paths but does not replace implementation validation. The divergence record retains enough references to investigate, with retention and access proportionate to the data. A mismatch does not automatically imply an online error: it may reflect a historical correction, a legitimate availability difference, or a defect in the replay itself. The laboratory represents these concepts in memory and does not run a distributed pipeline. In production, I would look for existing capture points to reduce cost and avoid another complete event copy. Investigation needs to reach the owners of feature meaning, not just the team operating storage or the model.

CODE · PROPOSED CONTRACT

Information the system did not yet have

PYTHON / ORIGINAL PROPOSAL

```python
def window_sum(events, at, known_at, seconds=60, available=True):
    validate_window(at, known_at, seconds, available)
    if not available:
        return None
    unique = {}
    for event in events:
        values = (event.occurred, event.known, event.amount_cents)
        if any(type(value) is not int for value in values):
            raise ValueError("integer times and cents required")
        if event.known > known_at or event.occurred > at:
            continue
        if not at - seconds < event.occurred <= at:
            continue
        if event.id in unique and unique[event.id] != event:
            raise ValueError("conflicting event")
        unique[event.id] = event
    return sum(event.amount_cents for event in unique.values())

late = [Event("x", 95, 110, 900)]
assert window_sum(late, 100, 100) == 0
assert window_sum(late, 100, 120) == 900
```

Executed excerpt from case_18.py; synthetic events and clocks. Types, imports, and validators are in the complete file.

The occurred field represents when the event happened; known indicates when it entered the available set. Replay excludes data not known at the chosen cutoff, even if it occurred before the decision. The window boundary is explicit too. This structure does not resolve corrections, deletions, or multiple versions of an event: conflicting duplicates raise an error so the case is not silently aggregated. A real integration would need its own policy for those states. The demonstration uses integers as a synthetic clock; production timestamps require a unit, time zone, precision, and validation agreed among the components involved.

Python 3.12: datetime

CODE · PROPOSED CONTRACT

Compare meaning before comparing numbers

PYTHON / ORIGINAL PROPOSAL

```python
def contract_id(window_seconds=60, missing="null", unit="cents"):
    contract = {"window": window_seconds, "missing": missing,
                "unit": unit, "bound": "(at-window, at]", "version": 1}
    encoded = json.dumps(contract, sort_keys=True, separators=(",", ":"))
    return hashlib.sha256(encoded.encode()).hexdigest()[:16]


def compare(batch, online, batch_contract, online_contract, tolerance=0):
    validate_comparison(batch, online, tolerance)
    if batch_contract != online_contract:
        return "CONTRACT_MISMATCH"
    if batch is None or online is None:
        return "MATCH_MISSING" if batch is online else "MISSING_MISMATCH"
    if abs(batch - online) > tolerance:
        return "VALUE_MISMATCH"
    return "MATCH"

left = contract_id(60)
right = contract_id(30)
assert compare(0, 0, left, right) == "CONTRACT_MISMATCH"
assert compare(None, 0, left, left) == "MISSING_MISMATCH"
```

Functions and examples executed in case_18.py. The hash identifies parameters; it does not certify implementation. Types, imports, and validators are in the complete file.

Equal values can hide different contracts. The comparison therefore checks definition identity first, then missingness, and finally the numeric difference. The shortened digest is convenient for the demonstration and must not be treated as a security signature. In production, the version needs a link to a governed registry and the published implementation. Tolerance also belongs to the domain: an exact monetary sum may require equality, while another feature permits bounded numeric error. A large tolerance does not fix incompatible semantics; it merely turns a divergence into something monitoring stops displaying. Nonfinite values and invalid tolerance raise ValueError, preventing a false match.

Python 3.12: unittest

INDEPENDENT ANALYSIS AND PROPOSAL

Absence, zero, and delay are not synonyms

An available source may return no occurrences because the customer did not perform the activity. Under the local contract, that produces zero. An unavailable source provides no such information and produces absence. A late event may make the current value differ from what becomes known later without indicating a defect in the original implementation. Converting all these states to zero gives the model a simplification that erases precisely the context needed to interpret the decision.

I would carry this distinction into the schema and evaluation. An optional field should not exist merely to avoid serialization failure; its meaning must be documented and compatible with training. I would also avoid using a special string as a numeric value without governance. The consumer must know whether the feature is missing because of no history, technical unavailability, or an eligibility condition. In some domains, these causes may require different treatment, but that decision needs validation with specialists.

The laboratory uses only an available or unavailable source without classifying every possible cause. This keeps the example inspectable. For a real pilot, I would start with a critical feature and investigate which states actually occur rather than inventing a complete taxonomy in advance. Contract evolution would include old and new examples so richer expression does not silently break consumers still using earlier versions.

INDEPENDENT ANALYSIS AND PROPOSAL

Late events can mislead an otherwise sound evaluation

Consider a purchase that occurred before the decision but reached the platform afterward. A dataset reconstructed weeks later may include it as though it had always been available. The model evaluated offline appears to use complete context, while the online service never had that opportunity. This difference does not require an algorithmic bug. The problem is the question asked of history: are we reconstructing the world as it was, or the world as it later became known?

Another situation involves corrections and reprocessing. A value may be adjusted later, a duplicate removed, or an event reclassified. Replay must define whether the intended reference is the original or corrected view, preserving both when the purpose requires it. Rewriting every old report with current data destroys the ability to explain past decisions. Yet ignoring corrections forever prevents learning. The solution depends on versioning references and the purpose of comparison.

I would also test day and time-zone boundaries, window changes, and out-of-order events. The laboratory uses integer time to isolate the reasoning, so it does not demonstrate civil-time handling. That layer would require additional tests with time-zone-aware timestamps and agreed rules. The criterion is not having many calendar tests, but checking instants when two plausible implementations might include different event sets and produce incompatible decisions.

Python 3.12: datetime

INDEPENDENT ANALYSIS AND PROPOSAL

Fifteen tests to avoid a perfect past

The fifteen local tests passed. They cover equivalent values, future occurrence, late knowledge, an open left boundary, duplication, conflict, absence differing from zero, absence on both sides, an incompatible contract, tolerance, an unavailable source, and an invalid window. The suite checks the example's declared semantics. It does not demonstrate model quality, absence of skew in a real environment, or correctness of any Nubank implementation. These results belong only to the independent package accompanying the chapter. Additional regressions reject NaN, infinity, invalid tolerance, and events outside the integer time-and-cents contract.

In the demonstration, a decision at 100 knows about an event worth 200 cents. Another event occurred at 95 but became known only at 110. With a knowledge cutoff of 100, the sum is 200; with a cutoff of 120, the historical sum becomes 1,100. Both results can be correct for different questions. The mistake would be using the second as proof that the service should have calculated that value at the first instant without acknowledging the unavailable information.

The next step would compare online capture and replay over an authorized set, with sampling that includes missingness and version changes. I would examine divergences manually before choosing an alert threshold. Some represent expected delay; others reveal different definitions or a capture defect. The pilot's most important test is whether we can explain the divergence and assign responsibility for correction without assuming either side is always the truth.

Python 3.12: unittest

INDEPENDENT ANALYSIS AND PROPOSAL

Monitor divergence without creating another black box

I would separate three signal families: contract incompatibility, missingness incompatibility, and numeric differences under the same contract. Each leads to a different investigation. A different hash calls for deployment review; one-sided absence calls for checking availability and eligibility; different values call for examining events and transformations. Combining everything into one skew rate loses this information and makes it harder to prioritize a correction that could actually affect customer decisions.

Comparator coverage also needs to be visible. If we can reconstruct only simple features, the observed rate does not describe the whole system. I would show how many eligible decisions were compared, which versions were excluded, and why. Dimensions should be bounded to avoid unnecessary cost and exposure. A customer identifier must not become a time-series label. Individual investigation cases need a separate access and retention path with a defined purpose.

A useful alert would include the contract, window, magnitude, period, and owners of feature production and consumption. It would also indicate whether there is evidence of potentially affected decisions without automatically converting that evidence into financial loss. Where appropriate, I would evaluate whether divergence changes model output through a governed replay. This connects data quality to impact, but causal caution remains necessary: a changed score does not prove that a real decision was wrong.

INDEPENDENT ANALYSIS AND PROPOSAL

Compatibility as part of the data rollout

A window or source change can be as important as a model release. I would treat feature evolution as a shared contract with a new version, boundary examples, and identified consumers. The producer should not replace the meaning while keeping the same name merely because the format is unchanged. Migration must allow the two definitions to be evaluated in parallel or establish a coordinated switch, depending on cost and existing platform constraints.

The pilot would begin with comparison that does not block decisions. This makes it possible to measure cost, coverage, and the rate of expected divergences. We could then introduce gates for unequivocal incompatibilities, such as a consumer requesting an unsupported contract. I would be more cautious about blocking on value differences because legitimate delays and incomplete capture may cause false alarms. Automation should follow the quality of divergence classification, not merely the technical availability of a blocking control.

As an EM, I would align data and service owners and reserve capacity to resolve cross-team problems. As a Staff engineer, I would define compatibility and how to preserve historical references. The return plan would cover not only code but also available data and versions. If migration removes the ability to reconstruct the previous definition, rollback may exist in name only. The contract must state how long old combinations remain operable and what evidence supports retiring them.

INDEPENDENT ANALYSIS AND PROPOSAL

Useful replays without indiscriminate copies

A comparator can become a powerful data consumer. I would avoid centralizing the entire history simply because it makes debugging easier. I would start with references, governed samples, and aggregates, choosing the minimum needed to reproduce the feature. Access must respect the investigation's purpose, with appropriate retention and query auditing. Explaining a decision does not require everyone with dashboard access to explore every financial event that preceded it.

Computational cost also needs limits. Recalculating long windows for every decision can multiply queries and harm source systems. I would assess sampling, snapshots, and aggregate reuse without losing temporal semantics. An optimization that conveniently uses future data is a correctness regression even if it reduces job duration. Evaluation must consider both performance and reconstruction fidelity, because both form part of the replay contract.

In the laboratory, the sum traverses in-memory events and is suitable only for fixture-sized inputs. There is no index, distributed storage, or query budget. A real implementation would need an access design consistent with volume and the existing platform. I would prefer demonstrating usefulness on a few critical features before expanding coverage. If the comparator creates high cost and unactionable divergences, the first adjustment should be to the question it is trying to answer, not the purchase of more capacity.

INDEPENDENT ANALYSIS AND PROPOSAL

ADR: compare by contract and instant

The proposed decision compares equivalent feature versions at the same instant and with the same knowledge cutoff. Comparing daily averages is inexpensive and may detect broad changes, but it does not explain decision-level divergence. Requiring exact equality for every feature ignores numeric characteristics and legitimate delays. I would combine an explicit contract, sampled point-in-time comparison, and aggregate metrics, calibrating each part to the chosen feature's nature.

The cost lies in maintaining contract identity, temporal references, and replay capability. This may require producer and consumer changes as well as governance over historical data. Not every feature warrants that investment. I would prioritize those with high potential impact, complex transformations, or a history of hard-to-explain divergence. The decision should consider the expected benefit of detecting an error and the cost of investigating false alarms, avoiding treating every variation as an operational incident.

I would revisit the ADR when the definition cannot be faithfully reconstructed or comparison yields little useful signal. In those cases, producer contract tests, distribution monitoring, or governed capture at decision time may be better alternatives. The goal is not to prove batch and online are always identical. It is to make explicit when they should be, which differences are acceptable, and what action to take when reality exceeds those agreed limits.

INDEPENDENT ANALYSIS AND PROPOSAL

A shared contract needs owners

The first increment could cover one feature with a versioned definition, executable examples, online capture, and limited replay. Acceptance would include explaining at least one missingness case, one late event, and one contract change without rewriting the past. As an EM, I would bring together those producing, transforming, and consuming the data, reserving time to correct the divergences found. A platform alone cannot resolve semantic disagreement; someone must decide which meaning serves the domain and own its maintenance.

The value model would begin with recoverable investigation time and incompatibility incidents actually avoided. Reducing inappropriate decisions would require further evaluation without attributing every loss change to the comparator. I would separate storage, processing, and human-review costs. Hours saved represent potential capacity, not automatic savings. I would also record traceability benefits that have no simple monetary conversion, such as explaining a behavioral change and reducing uncertainty before a significant release.

My Staff question would be: does this feature describe what happened, what was known, or what was corrected later? My EM question would be: when two teams disagree on the answer, is there a fast, respectful process to decide and update the contract? Quality emerges from the combination. A hash and test suite make the definition verifiable; collaboration and responsibility keep it correct as the product and organization change.

INDEPENDENT ANALYSIS AND PROPOSAL

Dark Mode as the evolution of a living product

19

The birth of the Dark Mode: a journey into Nubank's app evolution

Original official publication, accessed 23 October 2026.

The official account published in September 2023 describes the evolution of Nubank's Dark Mode through design tokens, screen adaptation, and collaboration between Design and Engineering. It also discusses fallbacks during the transition and contrast adjustments. This is a development story; it does not demonstrate a specific defect in the current app. The opportunity in this chapter is to derive criteria for evolving a financial experience without sacrificing understanding.

The relevant problem goes beyond choosing a dark background. An available amount, a pending payment, and a failed transfer must retain their meaning across themes, changing light conditions, and enlarged text. An interface can look sophisticated in a presentation while demanding excessive effort in daily use. Transparency, glow, and gradients should support hierarchy while essential information remains distinguishable.

My independent proposal organizes this evolution around journeys and states, combining automated checks with human evaluation. The laboratory measures synthetic color pairs and a few semantic cues; it does not execute Flutter or certify accessibility. When hiring a technical leader, the productive question is how to turn a visual improvement into a sustainable contract between components, products, and customer support. The answer must combine a design system, accountability for financial states, and evidence that people can complete their tasks.

The birth of the Dark Mode: a journey into Nubank's app evolution

INDEPENDENT ANALYSIS AND PROPOSAL

A token’s contract is its meaning

A token named purple-500 describes an appearance. A role named content-on-surface describes a relationship. In a financial experience, this distinction allows a theme to change without requiring every squad to relearn color pairings. The contract must specify the surface on which content appears, its position in the hierarchy, and the states that must not share ambiguous cues.

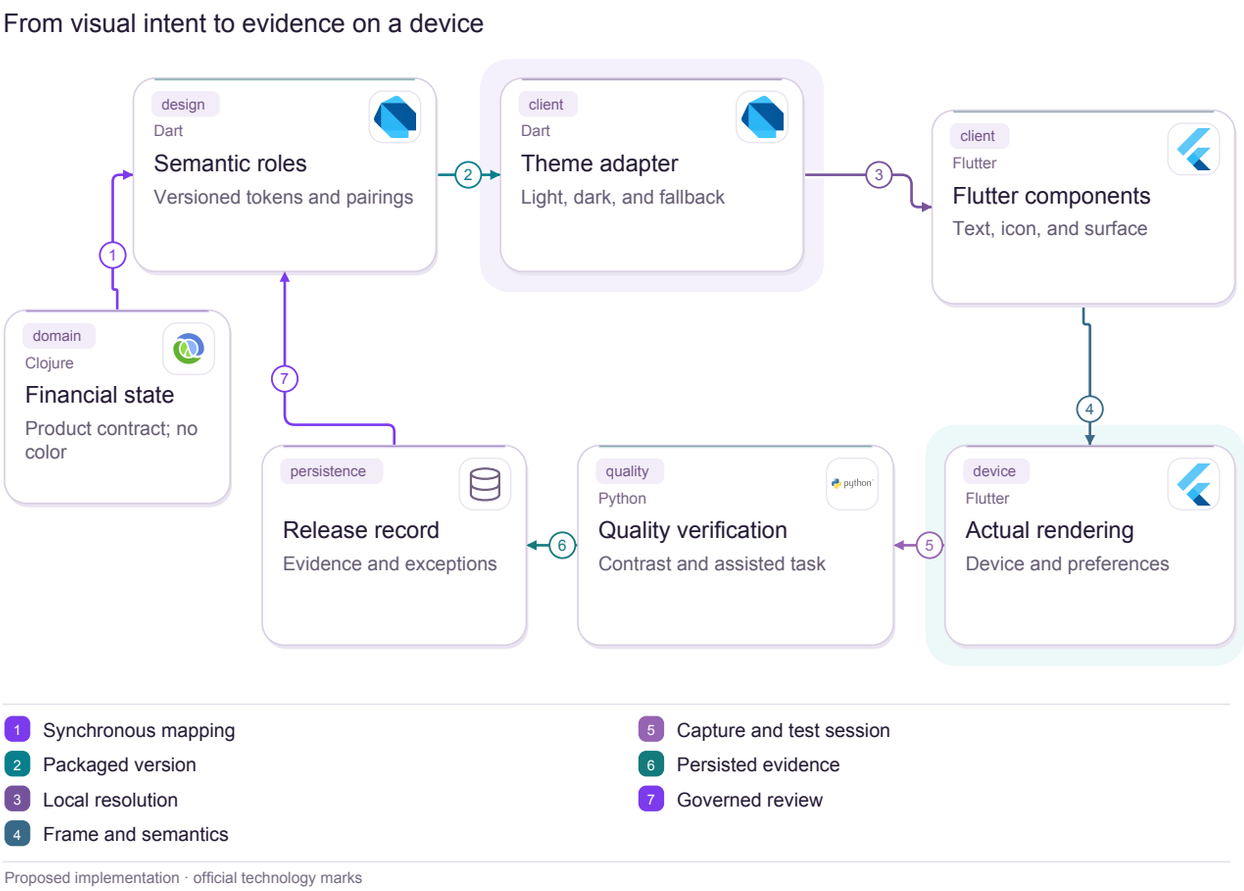
I would propose a matrix of components and states: available, processing, completed, declined, and temporarily unavailable. Each cell would connect text, icon, color, permitted action, and accessible message. An accent color would not mean success simply because it is visually convenient. The domain state would reach the interface as explicit data, and the presentation would choose a compatible semantic role. This separation prevents a brand adjustment from changing how a transfer is interpreted.

The migration also needs rules for legacy content. A component without a new token can use a known opaque surface, provided that text and actions remain readable and the exception has an owner. A fallback is a continuity strategy with a review deadline, not a magic color that fixes every composition. The Staff engineer’s focus would be the stability of this contract across products. The EM would organize the adoption sequence so that shared components reduce repeated work and critical journeys receive verification first.

Flutter API: ColorScheme

REFERENCE ARCHITECTURE

From visual intent to evidence on a device



Independent reference architecture; no internal API or tool has been assumed.

This proposed architecture separates the meaning of a financial state from its visual presentation. Tokens are versioned artifacts; a payment decision does not depend on them. The adapter resolves the theme locally, and the component produces pixels and semantics for evaluation. The release record retains the configuration examined, including device, theme, text scale, and accepted exceptions. The main trust boundary lies between the received state and its representation: a color change must not manufacture success or hide a pending operation. Another boundary separates simulation from actual experience. Contrast calculations on declared colors are useful, but transparency, images, and overlays require inspection of the rendered result. Feedback from the evaluation into the catalog is not an automatic production update. It opens a review involving Design, Engineering, and Product owners. This discipline allows a shared component to be corrected and its consumers verified before the change expands. The diagram does not represent Nubank's internal architecture.

CODE · PROPOSED CONTRACT

Calculate contrast before visual inspection

PYTHON / ORIGINAL PROPOSAL

```python
def luminance(color):
    if len(color) != 3 or any(not 0 <= x <= 1 for x in color):
        raise ValueError('Expected three channels in [0, 1]')
    linear = [x / 12.92 if x <= .04045 else
              ((x + .055) / 1.055) ** 2.4 for x in color]
    return sum(x * w for x, w in zip(linear, (.2126, .7152, .0722)))


def contrast(first, second):
    high, low = sorted((luminance(first), luminance(second)), reverse=True)
    return (high + .05) / (low + .05)

black = rgb('#000000')
white = rgb('#FFFFFF')
assert contrast(black, white) == 21
assert contrast(white, black) == 21
assert contrast(black, black) == 1
assert text_check('#777777', '#FFFFFF')['passes'] is False
assert text_check('#767676', '#FFFFFF')['passes'] is True
```

Excerpt from case_19.py executed with Python; synthetic pairings, with no app screenshots. Types, imports, and validators are in the complete file.

The excerpt executed in the laboratory converts sRGB channels into relative luminance and calculates the contrast ratio between two opaque colors. It rejects channels outside the accepted domain. The comparison is symmetric: swapping background and text preserves the ratio, although the visual experience and the function of each role remain distinct. WCAG applies different criteria to normal and large text; the laboratory makes that choice explicit in the text_check function. This calculation is a deterministic screening step. It does not verify font, physical size, device brightness, content comprehension, or the full compliance of a financial screen.

W3C: Understanding Contrast (Minimum)

Python 3.12: unittest

CODE · PROPOSED CONTRACT

A state that makes sense in both themes

DART / ORIGINAL PROPOSAL

```dart
import 'package:flutter/material.dart';

class PendingPayment extends StatelessWidget {
  const PendingPayment({super.key, required this.onDetails});
  final VoidCallback onDetails;

  @override
  Widget build(BuildContext context) {
    final colors = Theme.of(context).colorScheme;
    return Card(
      color: colors.surface,
      child: Padding(
        padding: const EdgeInsets.all(16),
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          children: [
            Icon(Icons.schedule, color: colors.onSurface),
            const SizedBox(height: 8),
            Text('Pagamento em processamento',
                style: TextStyle(color: colors.onSurface)),
            const SizedBox(height: 8),
            Text('Confira o andamento antes de tentar novamente.',
                style: TextStyle(color: colors.onSurface)),
            TextButton(onPressed: onDetails,
                child: const Text('Ver andamento')),
          ],
        ),
      ),
    );
  }
}
```

Proposed Flutter component; this is not Nubank code and was not executed in this environment.

This reference component receives an explicit action and uses the theme's surface and onSurface roles. The processing message appears through text and an icon, without relying on a shade of yellow. The column layout allows vertical growth, but still needs testing with enlarged fonts, translations, and narrow widths. The button should lead to a safe status inquiry, under a product contract yet to be defined. The example does not declare a balance, initiate a new transaction, or query services. The Dart is illustrative and was not compiled; the verified execution in this chapter is in the Python laboratory.

Flutter API: ColorScheme

Flutter: testing

INDEPENDENT ANALYSIS AND PROPOSAL

Glass and transparency need a predictable surface

A glass aesthetic depends on what sits behind the layer. The same text can cross light, dark, and saturated areas when the background changes, an image loads, or a panel moves. Approving only the token's hexadecimal value therefore does not establish sufficient contrast. The unit of evaluation must be the color pair actually composited at the content's position.

For financial values, notices, and actions, my proposal favors a minimum predictable surface beneath the text. Blur and glow can remain at the edges, in decorative elements, and in transitions, provided they do not carry essential information. If the composition fails to maintain the defined criteria, the fallback should increase opacity or use a solid surface. Users should not need to adjust the theme to discover whether an operation has finished.

The laboratory includes simplified alpha compositing in sRGB to demonstrate why an initially strong color loses contrast against a background. It does not reproduce every color space and rendering effect. Final validation requires an actual capture, loading and error states, long content, and system preferences. This constraint can improve the design itself: by reserving effects for suitable areas, the interface retains its visual character without forcing reading to compete with the background. Elegance also means predictability during an important task.

W3C: Understanding Contrast (Minimum)

INDEPENDENT ANALYSIS AND PROPOSAL

Change the theme without changing the meaning

A theme change can happen while someone is reading a receipt, filling out a form, or following an operation. The risk extends beyond a frame displaying an old color. Restarting the screen, losing focus, hiding a validation message, or clearing entered text turns a visual preference into a journey interruption. Domain state and editing state must survive the presentation update.

I would define a stable theme-resolution strategy for the component tree and test changes at critical points. The system must not reinterpret a pending operation as a new action because a component was rebuilt. Transition animations would be subtle and compatible with motion preferences, while essential messages would remain available in the semantics tree. A visual update should not trigger repeated announcements of the same financial state.

Compatibility with older components deserves separate monitoring. If one part of a screen receives the new theme while another keeps the old one, the boundary must preserve hierarchy and readability. A temporary exception may be acceptable; a button that disappears against its surface is not. To investigate regressions, I would record the app version, visual catalog version, and presentation context, without capturing private values. The objective is to reproduce the failure with equivalent synthetic data and correct the responsible boundary, instead of accumulating local adjustments without a contract.

Flutter: accessibility

INDEPENDENT ANALYSIS AND PROPOSAL

Twelve tests: what the numbers actually demonstrate

The laboratory's twelve tests verify bounded properties. Black and white produce a 21:1 ratio; identical colors produce 1:1; the ratio remains symmetric. Cases cover text thresholds, alpha compositing at its extremes, contrast loss in a selected transparency, invalid input, and the presence of cues beyond color. All passed in the local execution recorded in results_19.txt.

The result for #777777 on white was 4.4781, below the threshold used for normal text. The #767676-on-white pair produced 4.5422 and passed that numerical criterion. The small distance between them illustrates why rounding the ratio before comparison would be incorrect. The presentation rounds for readability, but the decision uses the calculated value. A synthetic dark example, #F6F1FA on #160B22, produced 17.0857.

These numbers are not measurements of the Nubank app. They also do not demonstrate that a person understands the content, perceives focus, or can operate the component. The redundant-cues test requires text and an icon identifier, but does not judge whether the icon makes sense. An appropriate suite would combine this inexpensive layer with widget tests, semantic inspection, visual comparison, and observed tasks on real devices. The technical conclusion is modest and useful: certain regressions can be found before they reach human evaluation.

W3C: Understanding Contrast (Minimum)
Python 3.12: unittest

INDEPENDENT ANALYSIS AND PROPOSAL

Observability without turning the screen into surveillance

Visual quality often produces evidence that differs from service observability. An API error rate can remain stable while a button becomes difficult to recognize. Evaluation should therefore relate the visual version to task completion, with a clear description of what each measure can indicate. Abandonment, duration, and support contact are signals for investigation; none independently identifies a contrast failure.

My proposal would monitor selected journeys and record only necessary events, such as start, completion, exit, and error category. Analysis could compare themes and versions when collection is appropriate and the sample is sufficient. Accessibility preferences require additional care: they must not become inferences about disability, commercial segmentation, or sensitive log content. Many questions can be answered through consented sessions and test scenarios, without permanent individual telemetry.

The quality dashboard would bring together known regressions, coverage of essential components, correction time, and research results. An observed difference between groups would prompt reproduction and sample-composition analysis, not an immediate causal claim. Staff and EM should agree in advance on what makes a change stop. A confirmed critical barrier may justify suspension even without a large statistical sample. This approach keeps the indicator in service of the experience, instead of treating the absence of a backend alarm as proof that everything is fine.

INDEPENDENT ANALYSIS AND PROPOSAL

A theme migration is a contract migration

Token adoption works better when dependencies are visible. Before changing a shared role, the team needs to know its consumers, expected surfaces, and exceptions. A catalog of examples with financial states makes this review concrete. The change should include compatibility evidence, especially for components that appear in different products and composite screens.

I would start with frequent components and essential journeys, choosing a sequence that produces learning without expanding risk all at once. Each batch would have an owner, acceptance criteria, reference captures, and a rollback path. Incompatible changes would receive an explicit version and a coexistence period. The platform team would support the migration with tools and documentation; consuming squads would verify the actual states of their domains.

Expansion criteria would combine readability, preservation of actions, message comprehension, and the absence of critical barriers on the devices evaluated. If a serious focus failure appears or essential information becomes imperceptible, expansion stops and the affected component returns to its safe configuration. The plan does not assume remote configuration can repair every installed version. When a fix requires a new binary, the fallback must already exist. This distinction between changing visual data and changing behavior is an architectural decision with direct operational consequences.

INDEPENDENT ANALYSIS AND PROPOSAL

Enlarged text, motion, and rendering cost

An accessible component must remain operable when text grows. Fixed height limits, truncated values, and buttons that require a single line can prevent task completion. The proposal is to allow vertical growth, preserve the association between label and value, and review reading order when elements wrap. Critical information should not depend on an ellipsis that the person cannot expand.

Visual effects also consume resources. Blurs, shadows, and overlays can increase compositing work on less capable devices. The performance budget must be verified with the final configuration, including animations and loading states. A screen that responds slowly can prompt repeated taps, uncertainty about an action, and contact with support, even when its colors are correct.

I would include these scenarios in the acceptance matrix: light and dark themes, enlarged text, narrow width, reduced motion, and a representative lower-capability device. Testing every possible combination for every change is unnecessary; understanding the risks of changed components and maintaining a relevant sample is essential. The EM would protect time for this verification, while the Staff engineer would define layout and instrumentation contracts that make it easier to perform. The desired result is a consistent appearance that continues to work when the user’s context is less convenient than the demonstration environment.

Flutter: accessibility
Flutter: testing

INDEPENDENT ANALYSIS AND PROPOSAL

ADR: semantic roles and an opaque fallback

The proposed decision is to use versioned semantic roles for financial components and provide an opaque surface when a translucent composition fails readability criteria. The context is an interface that must evolve visually without making every squad rediscover contrast, hierarchy, and the meaning of states. The accepted consequence is to limit some effects in areas containing essential information.

One alternative would be to define colors locally on each screen. That accelerates an isolated adjustment, but makes it difficult to propagate corrections and distinguish a product choice from a technical exception. Another alternative would be to block all change until the entire legacy interface has migrated. This strategy offers apparent uniformity, but may delay useful improvements and conceal the absence of a compatibility plan. Gradual adoption requires more discipline, but creates explicit boundaries.

The decision would be revisited if the catalog stopped representing the products or the fallback introduced barriers of its own. The record would include before-and-after examples, verified limitations, owners, and a review date. The choice does not claim that WCAG alone covers the entire mobile experience. It establishes a set of checks within a broader approach. For the Staff engineer, the architectural test is whether a new component can follow the contract without copying fragile adjustments. For the EM, it is whether adoption has capacity, an owner, and completion criteria.

INDEPENDENT ANALYSIS AND PROPOSAL

The value of an interface that demands less effort

The value hypothesis is that a more readable and consistent presentation reduces interpretation effort and supports task completion. It needs to be measured against a baseline and through research; this chapter attributes no financial result to Nubank. Task duration, misunderstandings, need for help, and recurring regressions are complementary measures. None should turn inclusion into a competition around a single number.

As an EM, I would bring Design System, product squads, accessibility, and customer support together around a concrete journey. The first commitment would be to clarify states, observe difficulties, and define what must improve. The agenda would include participants with diverse needs and a budget for corrections. Tracking would distinguish component delivery, product adoption, and observed task outcomes, avoiding a declaration of success simply because the library gained new tokens.

As a Staff engineer, I would start with the contracts between domain, presentation, and semantics. The contribution would be to make the right way to build easier to repeat: known color pairs, explicit states, testable examples, and a documented fallback. The interview question would be: how do you change a token used across dozens of journeys without breaking essential information? A good answer combines an inventory, compatibility, risk-based testing, release, and rollback. This is the connection between visual quality and systems engineering: preserving trust while the product continues to change.

INDEPENDENT ANALYSIS AND PROPOSAL

NuBraille: a physical initiative with a digital journey

20

Bridging the gap: Nubank’s commitment in serving the visually impaired

Original official publication, accessed 23 October 2026.

The official case published in July 2023 presents the development of NuBraille with participation from blind people, prototype evaluation, and collaboration with production and customer support. The card incorporates a name and the last four digits in braille to support identification and autonomy. The publication is evidence of a specific initiative; it does not certify accessibility across the entire experience or guarantee universal product availability.

The main engineering opportunity is continuity. Identifying the card does not complete the journey if instructions are difficult to access, activation requires a step without an operational alternative, or assistance loses the context. Success depends on a sequence that crosses a physical object, the app, authentication, and customer support. Every boundary can introduce a barrier even when its components appear adequate in isolation.

This chapter proposes a contract for an assistive journey, with clear states and recovery paths that preserve security requirements. The synthetic laboratory verifies transitions and some properties of labels and ordering. It does not emulate a screen reader, implement authentication, or measure a blind person’s experience. My contribution as a candidate would be to connect research, architecture, and operations so that autonomy observed at one step continues into the next. The question moves from whether an accessible feature exists to whether the person can complete what they came to do.

Bridging the gap: Nubank’s commitment in serving the visually impaired

INDEPENDENT ANALYSIS AND PROPOSAL

Journey states before screen components

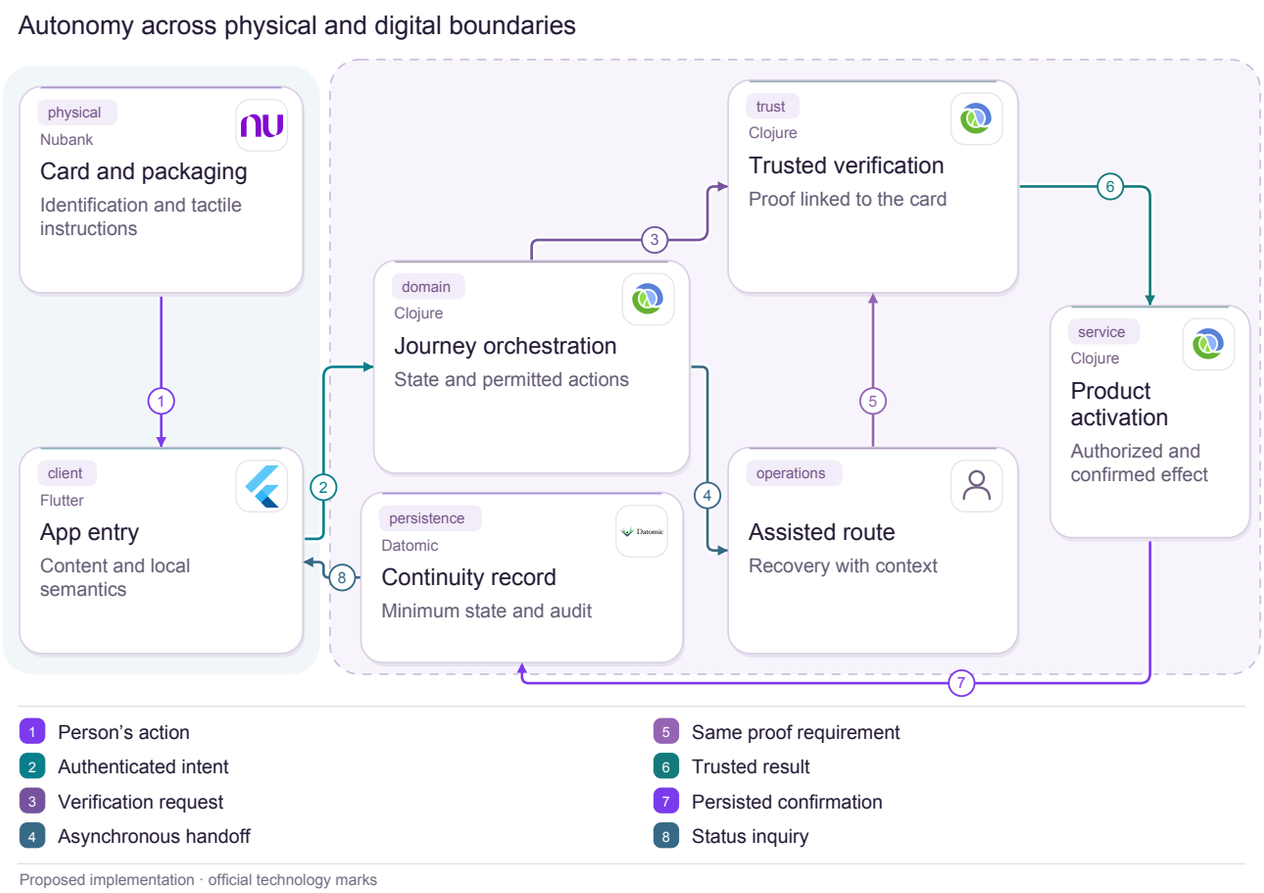
The proposed journey begins with card identification and proceeds through instructions, identity verification, activation, and confirmation. When the camera is unavailable, the person receives an assisted route with information about how to continue. This route is not equivalent to financial authorization. Activation remains conditional on the required proof, produced by a trusted mechanism to be defined with Security and Operations.

The contract must distinguish technical unavailability, the need for help, and verified identity. If they all become a generic error state, the app cannot explain the next step, and support tends to restart the conversation. If assistance is confused with proof, the solution weakens security. Explicit modeling enables an inclusive path while preserving this boundary.

Each state would have an understandable message, available actions, a recovery method, and the minimum context needed for continuity. Waiting time also deserves an explanation: people should know whether their request was received and what remains outstanding, without incessant announcements. The laboratory uses simple internal names to make these transitions verifiable. In the product, language would be validated with participants and content professionals. The Staff engineer's work is to keep contracts coherent across channels. The EM's is to ensure that owners of each step have capacity and a shared objective of journey completion.

REFERENCE ARCHITECTURE

Autonomy across physical and digital boundaries



Independent journey proposal; arrows do not describe existing Nubank integrations.

The reference architecture shows a journey with multiple owners. The physical artifact initiates identification but does not authorize activation. The interface presents instructions and sends an intent; identity proof crosses a separate trust boundary. The assisted path must meet the same authorization requirement, through an accessible procedure defined by the responsible teams. The continuity record preserves the minimum needed to resume the process. It must not contain indiscriminate copies of documents, audio, or card data. The person receives confirmation of the actual state, including when assistance continues asynchronously. Distinguishing the visible state from the financial effect prevents a premature promise of completion. A network failure after activation requires a result inquiry, not a blind repetition of the operation. The laboratory tests only a small local state machine. The complete design would require idempotency, authorization, retention, and recovery contracts that have not been inferred from Nubank's internal system.

CODE · PROPOSED CONTRACT

The assisted route preserves the proof requirement

PYTHON / ORIGINAL PROPOSAL

```python
def accept_proof(session, receipt, consumed):
    if session.stage not in ('VERIFYING', 'ASSISTED'):
        raise ValueError('Verification is not expected')
    if receipt.get('verified') is not True or receipt.get('card') != session.card:
        raise ValueError('Proof does not authorize this card')
    if not isinstance(receipt.get('id'), str) or not receipt['id'].strip():
        raise ValueError('Non-empty receipt identifier required')
    if receipt['id'] in consumed:
        raise ValueError('Receipt already consumed')
    consumed.add(receipt['id'])
    return replace(session, stage='VERIFIED')


def activate(session):
    if session.stage != 'VERIFIED':
        raise ValueError('Verified identity required')
    return replace(session, stage='ACTIVE')

session = advance(ready(), 'camera_unavailable')
assert session.stage == 'ASSISTED'
consumed = set()
verified = accept_proof(session, proof(), consumed)
assert verified.stage == 'VERIFIED'
assert activate(verified).stage == 'ACTIVE'
assert 'fixture-1' in consumed
```

Excerpt from case_20.py executed locally; an educational model, not an authentication implementation. Types, imports, and validators are in the complete file.

The laboratory accepts proof only in the expected verification states, linked to the synthetic card and not yet consumed. Activation requires the VERIFIED state. A camera failure allows access to support, but does not satisfy that condition. The receipt is a trusted test fixture: the code does not verify a signature, identity, or real authorization. It also does not provide transactional persistence for concurrency. These deliberate limitations make explicit the contract that a production integration would need to fulfill. The tests reject invalid proof, a different card, and resubmission of the same identifier. Invalid types and domains raise ValueError before any favorable outcome.

Python 3.12: unittest

CODE · PROPOSED CONTRACT

Semantics that explain the state and the next step

DART / ORIGINAL PROPOSAL

```dart
import 'package:flutter/material.dart';

class AssistedRecovery extends StatelessWidget {
  const AssistedRecovery({super.key, required this.onHelp});
  final VoidCallback onHelp;

  @override
  Widget build(BuildContext context) {
    return FocusTraversalGroup(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Semantics(
            header: true,
            child: const Text('Continue a ativação com ajuda'),
          ),
          const SizedBox(height: 12),
          Semantics(
            liveRegion: true,
            child: const Text(
              'A câmera está indisponível. Seu cartão ainda não '
              'foi ativado. Há outra forma de continuar.',
            ),
          ),
          const SizedBox(height: 12),
          FilledButton(
            onPressed: onHelp,
            child: const Text('Obter ajuda para continuar'),
          ),
        ],
      ),
    );
  }
}
```

Independent, illustrative Flutter/Dart; verify with TalkBack, VoiceOver, and users before any use.

The example proposes a recognizable heading, a status message, and an explicit help action. The standard button supplies its own semantics; adding a redundant label around it could duplicate the announcement. The dynamic region must be evaluated to avoid repeating announcements during rebuilds. FocusTraversalGroup organizes focus navigation, but does not establish the exploration order of every screen reader. The fragment needs integration into a container that supports enlarged text and scrolling. It was neither compiled nor validated on a device. Availability of the assisted route is a product contract, not a ready-made integration.

Flutter API: Semantics

Flutter API: FocusTraversalGroup

INDEPENDENT ANALYSIS AND PROPOSAL

The interface speaks in an order that must make sense

The semantics tree represents what the interface means to assistive technologies. It should provide structure, names, and states consistent with the available actions. An unlabeled help icon may look familiar while remaining indecipherable during spoken navigation. Conversely, labeling every decorative fragment creates noise and makes important information harder to find.

I would start with the task sequence: recognize the context, understand the state, choose an action, and receive confirmation. Headings orient the person, groups connect information and controls, and error messages explain how to proceed. Focus should remain predictable when a dialog opens, closes, or receives a result. An update should not unnecessarily return the person to the beginning, particularly after they have entered data or listened to long instructions.

In the laboratory, the checker identifies missing labels, repeated identifiers, duplicate ordering, and screens with no available action. These are useful syntactic rules for an example contract. They do not determine whether text is clear, the screen reader's actual order matches the list, or the required gesture is accessible. Those questions depend on device execution and human participation. The technical contribution is to combine quick checks with evaluations that reach meaning. Automating what can be verified frees time to observe what a list of properties cannot decide.

Flutter API: Semantics
Flutter: accessibility

INDEPENDENT ANALYSIS AND PROPOSAL

Recover access without erasing the path already taken

Interruptions are part of the journey: an unavailable camera, an expired session, an unstable connection, a device change, or a need for support. Recovery must explain what has been preserved and what must be repeated. Requiring a sensitive step again may be necessary for security, but the explanation must be understandable and the execution path must remain available.

My proposal separates guidance state from authorization state. Instructions already read and local preferences can help resume a task, while expired proof requires new validation. The app should not continue to appear complete if the server has not yet confirmed the effect. When uncertainty follows submission, querying the state with a safe reference is preferable to initiating a new activation without context.

Support would receive only what is necessary to understand the current step, under defined permissions and retention rules. People should not have to repeatedly describe a barrier already recorded or share credentials to obtain help. The laboratory demonstrates that the assisted alternative preserves the proof requirement, but it does not implement identity, sessions, or persistence. In production, these boundaries would require joint review. The quality measure would be the ability to resume safely and autonomously, not merely the existence of a button that transfers the difficulty to another channel.

INDEPENDENT ANALYSIS AND PROPOSAL

Fourteen tests protect boundaries; they do not certify the journey

The fourteen executed tests cover the direct path, the assisted path, the proof requirement, card matching, and rejection of receipt reuse. They also verify invalid transitions, resuming the camera path, and four classes of semantic properties. All passed in the recorded local execution. The result shows ASSISTED before the proof fixture and ACTIVE only after acceptance of that synthetic proof. The additional regression tests require verified to be exactly true and the identifier to be nonempty; rejected inputs do not consume receipts.

Another result is deliberately negative: a screen with the message “Camera unavailable” but no action produces screen:no_action. This exposes a structural barrier that can be found early. Even so, the existence of a button does not demonstrate that support is available, the label makes sense, or the person can activate it. The distinction between an interface property and a journey outcome remains explicit.

The laboratory uses an in-memory set for consumed receipts. It does not solve cross-process concurrency, failures during writes, or message authentication. A real system would require appropriate persistence and atomic operations. The tests do not use screen readers, simulate disability, or replace research. Their value is to make some contract boundaries verifiable before investing in integration. This technical honesty allows discussion of what remains missing without diminishing what has already been demonstrated reproducibly.

Python 3.12: unittest

INDEPENDENT ANALYSIS AND PROPOSAL

Research autonomy with participants, not assumptions

An assistive-journey evaluation needs to include people who use different technologies and strategies. Blind and low-vision people do not form a uniform group: experience with the app, familiarity with braille, screen reader, device, and usage conditions all change the task. Recruitment should seek diversity relevant to the product and clearly explain the research purpose.

I would propose compensated sessions with consent, synthetic data, and the option to stop the activity. The script would include identifying the card, locating instructions, starting activation, handling a failure, and resuming the process. The observer would record barriers and assistance needs without treating speed as the sole measure of competence. Participants could use their usual configuration whenever feasible and safe for the session.

The results should separate observed difficulty, a hypothesis about its cause, and the proposed change. If a participant becomes lost after an announcement, the immediate conclusion is a barrier in that context; the cause may involve content, focus, repetition, or structure. The team would reproduce the scenario before selecting a fix. The EM would ensure time and resources to incorporate the learning. The Staff engineer would turn recurring patterns into reusable contracts. Research is not a final ceremony to approve completed work: it helps decide what to build and which assumptions must be abandoned.

INDEPENDENT ANALYSIS AND PROPOSAL

Measure continuity without collecting a sensitive identity

The purpose of measurement is to understand where the journey stops working and whether the correction produces autonomy. Unassisted completion, resumption after failure, repeat contact, and observed critical barriers can form part of the evaluation. Every indicator needs a definition: a task completed after researcher intervention does not belong in the same category as independent completion, although both provide learning.

Collection should avoid turning assistive-technology use into a permanent personal profile. Where possible, consented research and quality testing answer questions with less exposure. Operational events can record the step and failure category without document content, audio, full card numbers, or credentials. Analysis must respect purpose, access, and retention, with clear owners for any additional data.

Customer support must also be observed. A route available in the interface may fail if the queue does not recognize the context or the procedure requires an inaccessible channel. The proposed measurement follows continuity through resolution, including waiting and repeated steps. Small research samples do not support universal projections; they reveal problems and guide new tests. A reproduced severe barrier may justify immediate correction even without a prevalence estimate. The primary criterion is to provide a functional, safe path supported by evidence proportional to the reach of the change.

INDEPENDENT ANALYSIS AND PROPOSAL

Launch a journey that includes operations

Launching an assistive improvement must cover the channels involved. Updating the app component without preparing instructions, physical production, and customer support creates an inconsistent experience. The proposal is to build a journey-specific dependency checklist, with owners and evidence of readiness. This document does not replace the actual experience; it prevents an essential step from being overlooked between teams.

Before expanding the change, I would verify the complete route with participants and the professionals who will provide support. The pilot would need operational capacity, clear availability messages, and a safe path when the alternative cannot be completed at that moment. The plan would include session recovery, representative devices, and app versions still supported. A visual rollback cannot remove a route people already depend on without providing continuity.

The stop criterion would be a reproduced critical barrier, an incorrect activation confirmation, or a breach of authorization requirements. Expansion criteria would combine resolution of those failures, experience evaluation, and support capacity. The EM would coordinate readiness among owners; the Staff engineer would review the contracts between state, confirmation, and recovery. The work does not end when code reaches production. The team must monitor whether people cross the boundaries with the same clarity they encountered in the first step.

INDEPENDENT ANALYSIS AND PROPOSAL

ADR: an assisted alternative with equivalent authorization

The proposed decision is to offer an accessible operational path to continue the journey when camera capture is unavailable, preserving the identity-proof requirement. The context is a task spanning the app and customer support that should not depend on a single technical interaction without recovery. The exact proof mechanism would be defined and reviewed by the competent teams; this chapter does not invent a Nubank security procedure.

One alternative would be to treat every unavailability as an attempt to repeat. This simplifies the flow, but can make the task impossible for some people. Another would allow activation after any support contact. That option confuses assistance with authorization and is rejected in the model. The decision accepts greater coordination effort across channels to preserve both access and security.

Consequences include persistent state, minimum shareable context, training, and support capacity. The implementation would need to establish that proof belongs to the correct person and product, cannot be improperly reused, and produces consistent confirmation. The laboratory verifies only some of these conditions in memory. The decision would be revisited if the alternative failed to provide continuity or required unnecessary data exposure. A useful ADR records these limits and the owners responsible for resolving them, allowing the team to discuss concrete choices instead of using accessibility or security as words that end the conversation.

INDEPENDENT ANALYSIS AND PROPOSAL

Hire to connect autonomy, engineering, and responsibility

The expected benefit needs to be divided into understandable dimensions. For the person, what matters is completing the task autonomously, recovering from interruptions, and understanding the product's state. For operations, there may be fewer repeat contacts and repeated steps. Any financial estimate would use avoided contacts and a validated incremental cost, subtracting the solution's cost. Customer time recovered would remain a separate measure, without an arbitrary conversion into money.

As an EM, I would begin by bringing Product, Engineering, Research, Security, and Operations together around an observable journey. I would establish who decides, who executes, and who monitors barriers. The plan would reserve capacity for participants, corrections, and training. A delivery would be considered complete only when the relevant flow could be demonstrated and operational dependencies were ready. This gives practical consequences to a commitment to inclusion.

As a Staff engineer, I would work on continuity across contracts: state, semantics, authorization, idempotency, and recovery. My experience in financial and antifraud systems would inform the discussion of boundaries; I would learn from specialists and users where my assumptions fall short of the actual experience. The interview question would be: how do you provide an accessible path without relaxing authorization? The expected answer connects journey design, equivalent alternatives, trusted proof, testing, and observation. That is the value of leadership that turns product intent into an experience sustained by engineering and operations.

LEADERSHIP AND EXECUTION

The first 30 days: learn the system.

I would begin by following a complete journey, including customer support and operations. The objective would not be to produce a perfect map of the organization, but to identify where the team encounters recurring decisions, which risks concern people most, and which commitments already exist with customers and partners. The first deliverable would be a bounded problem described in the language of the people experiencing it.

In individual conversations, I would explore expectations, friction points, perceived autonomy, and development opportunities. With Product and Design, I would understand the metrics in use and the reasoning behind the current roadmap. With Security, Risk, and Customer Support, I would investigate how exceptions reach engineering and how much context is lost along the way. The diagnosis should be shared and open to correction.

Deliverable	How to produce it	Sign of usefulness
Journey map	Follow the app, service, and support	The team recognizes boundaries and gaps.
Baseline	Define events and denominators	The number can be reproduced.
First problem	Choose it with the team	Clear owner and completion criterion.

Progression criterion

Propose a structural change only when its cause is sufficiently understood and the affected people have participated in defining the problem.

LEADERSHIP AND EXECUTION

Days 31-60: make the hypothesis testable.

With initial context established, I would choose a pilot that produces evidence within a few weeks and can be rolled back. For example, improving a status inquiry for an existing operation may be more controllable than changing an entire payment flow. The scope needs to include observability, support, and communication, because a change that is invisible to operations transfers work elsewhere.

The team would define one primary hypothesis, two or three guardrail metrics, and an exposure limit. The plan must specify which events end the pilot, who can pause it, and what happens to customers who have already entered the journey. Before launch, I would ask for a failure rehearsal with the rollback path, including compatibility between versions.

Decision	Artifact	Proposed owner
Can we start?	Reviewed contract and baseline	Tech lead + Product + Risk.
Can we expand?	Sample and guardrails	Journey owner.
Do we need to stop?	Observable trigger and runbook	Operational owner.

How I would contribute

As an EM, I would remove dependencies and protect focus. As a Staff engineer, I would examine contracts and failure modes in depth. In both roles, I would aim for the team to be able to explain the decision without depending on me.

LEADERSHIP AND EXECUTION

Days 61-90: decide what deserves to scale.

At the end of the pilot, the work does not finish with a results presentation. We need to decide whether the hypothesis was supported, whether operations can sustain the solution, and whether the cost of expansion still makes sense. A localized improvement can reveal data problems, unserved segments, or dependencies that prevent replication without additional investment.

I would present the decision as three alternatives: expand, revise, or end. Expansion requires a plan for capacity, ownership, support, and compatibility. Revision requires a specific uncertainty to resolve. Ending preserves the learning and removes the ongoing cost of a hypothesis that did not hold. None of these outcomes should be treated as a personal failure.

Alternative	Required evidence	Next action
Expand	Consistent benefit and controlled risks	Expand by cohort with observation.
Revise	Promising signal, bounded uncertainty	A new time-bound experiment.
End	Insufficient benefit or excessive cost	Remove the experiment and record the learning.

The 90-day outcome

A team with more shared context, a verifiable improvement or an informed decision not to expand, and practices that make the next cycle more efficient.

LEADERSHIP AND EXECUTION

Capacity is a design constraint.

Planning as if all team time were available for features makes every incident an unexpected delay. I would separate capacity for evolution, operations, consequential debt, and learning, using data from the actual flow of work to adjust the allocation. The initial proportion is a management hypothesis, not a fixed rule imposed on every team.

The capacity discussion needs to account for knowledge concentration. If only one person can operate a critical integration, the team has a risk that does not appear in story points. Pairing, documentation, and progressive rotation can reduce that dependency, but they need space in the plan and quality monitoring, not just a ceremony.

Signal	Interpretation to investigate	Possible action
Many interruptions	Support requests arriving without triage	A channel with context and severity.
Aging work	Unresolved dependency or scope	Reduce batch size and decide how to address the blocker.
One person involved in everything	Concentrated knowledge	Pairing on an actual delivery.

As an EM, I would make these constraints visible when negotiating timelines. As a Staff engineer, I would help reduce the complexity that generates interruptions. Promising speed without addressing the system of work tends to exhaust the most experienced people and leave the team less capable in the next cycle.

LEADERSHIP AND EXECUTION

Delegation with explicit boundaries.

Good delegation does not mean handing over a problem and disappearing. It means agreeing on the expected outcome, decision space, risks that require discussion, and check-in points. Someone who is developing can lead an important change when scope and support are appropriate. The leader remains responsible for creating the conditions for learning without invisible risks.

I would use real decisions as growth opportunities: one person leads the contract, another conducts the failure test, and another presents alternatives to stakeholders. Feedback should identify observable behavior and its effect. “Be more strategic” is vague; “explain the consequence of each alternative before choosing” allows practice and evaluation.

Delegation	Autonomy	When to escalate
Local component	Implementation within the contract	State change or sensitive data.
Cross-team contract	Proposal and negotiation	Incompatibility or disputed authority.
Rollout	Execution within the limits	Guardrail or insufficient sample.

Evidence of leadership

Success appears when more people can formulate, explain, and execute important decisions well, not when every decision goes through the manager.

LEADERSHIP AND EXECUTION

An ADR resolves a concrete uncertainty.

An architecture decision record should preserve the reasoning that will be needed later. I would avoid documents that merely describe the selected solution and omit the problem. The first page needs to explain context, constraints, alternatives, and the consequence of being wrong. Readers should be able to disagree with a premise before discussing the tool.

Part	Question	Example in the book
Context	Which decision is pending?	How to represent an unknown outcome.
Alternatives	What could we do?	Authoritative query, projection, or waiting.
Choice	Why this option now?	Lower risk with current capacity.
Review	When should we reopen the decision?	Delay, cost, or load beyond the limit.

The ADR should have an owner and an event-triggered review date, rather than relying only on the calendar. If the decision was made under deadline pressure, that should be visible. If an alternative was rejected because operational capacity was lacking, the rationale must survive the departure of the people who participated in the conversation.

As a Staff engineer, I would use the document to create alignment and expose trade-offs. As an EM, I would protect the time needed to decide and prevent the ADR from becoming a mandatory bureaucratic step for trivial changes. The degree of formality should match reach, reversibility, and risk.

LEADERSHIP AND EXECUTION

Incidents: restore, understand, improve.

During an incident, the priority is to reduce impact through clear coordination. I would separate incident command, technical investigation, and communication so that the same person does not have to do everything. Updates should state what is known, what remains uncertain, and which action is underway, without promising a timeline unsupported by evidence.

After stabilization, the review needs to examine system conditions: missing signals, reasonable decisions made with incomplete information, dependencies, and recovery mechanisms. Searching for someone to blame produces silence; ignoring accountability prevents improvement. The useful path is to turn what happened into verifiable changes, with owners, priority, and an effectiveness test.

Phase	Main question	Output
Response	How can we reduce harm now?	Coordinated mitigation and communication.
Reconciliation	Are there pending or divergent effects?	Verified operations and handled cases.
Learning	What makes recurrence less likely?	A testable action with an owner.

Effectiveness test

A postmortem action is complete only when we can demonstrate that it detects, prevents, or reduces the impact of the scenario that motivated it.

LEADERSHIP AND EXECUTION

Influence across teams without centralizing everything.

Journey problems rarely fit within a single repository. The interface may depend on another domain's data, while support absorbs the impact of an unclear contract. I would begin by making each team's incentives and boundaries visible. A solution that improves one team's metric while transferring cost to another needs renegotiation.

Technical influence comes from reducing the cost of deciding: a small prototype, reproducible data, comparable alternatives, and a migration proposal that respects the capacity of those involved. In disagreement, I would distinguish facts, preferences, and constraints. Not every conflict requires consensus; it needs a legitimate decision-maker and an understandable explanation.

Boundary	Required agreement	How to verify
Producer / consumer	Version and compatibility	Fixtures and a contract test.
Engineering / customer support	State and next action	Simulation of a realistic case.
Product / risk	Benefit and exposure limit	Guardrails and cohort review.

The outcome I seek is an organization that can change with fewer surprises. That depends on useful standards, clear ownership, and channels for challenging decisions when new evidence emerges. The Staff engineer facilitates this system; the EM ensures that people and capacity keep pace with the ambition.

ENGINEERING ECONOMICS

Value: start with the denominator.

A useful estimate connects eligible volume, occurrence rate, expected change, and unit value. If any of these elements is unknown, the model should expose the uncertainty. A public complaint helps formulate a hypothesis, but does not automatically establish how many customers are affected or how much a fix would yield.

Variable	Synthetic scenario	Interpretation
Eligible journeys / month	1,000,000	Hypothetical volume, not Nubank data.
Contacts per 1,000 journeys	8 → 6	Assumed reduction of 2 contacts.
Marginal cost per contact	R$ 6	An assumption that requires validation.
Estimated released capacity	R$ 12,000 / month	2,000 contacts × R$ 6.

Multiplication does not turn released capacity into cash savings. If the support structure remains unchanged, the benefit may be time available for more complex cases, shorter queues, or better quality. To claim financial savings, we would need to demonstrate that the cost actually ceased to exist or that planned expansion was avoided.

How to use this in an interview

I would present ranges and sensitivity, ask which assumptions are already known, and propose the smallest measurement capable of reducing the uncertainty that most affects the decision.

ENGINEERING ECONOMICS

Opportunity cost is also a cost.

The choice between two proposals should not rely solely on projected gross benefit. One initiative may require a broad migration, partner coordination, and ongoing maintenance. Another may produce less benefit in isolation and still be the better first delivery because it reduces risk, validates an assumption, or frees capacity for the next step.

Synthetic alternative	Expected benefit	Operational commitment
New global projection	Faster reads	Ongoing pipeline, reconciliation, and support.
Contract correction	Less ambiguity	Coordinated change and tests.
Content improvement	Clearer guidance	Editorial governance and measurement.

I would compare alternatives using the same time horizon and the same journey outcome. I would also separate reversible costs from commitments that are difficult to undo. Introducing another service creates responsibility for availability, updates, security, observability, and knowledge transfer. The architecture needs to bring that cost into the initial discussion.

As an EM, I would make the trade-off with Product and stakeholders using actual capacity. As a Staff engineer, I would seek to reduce the technical cost of the selected alternative. The best solution may be a new component, a small change to an existing contract, or a decision not to build until there is sufficient evidence.

ENGINEERING ECONOMICS

Performance: measure the tail and the task.

Average latency can conceal the experience of the most affected people. A fast endpoint may also fail to improve the journey if the app repeats calls, waits on a slow dependency, or presents a confusing state. I would start with a complete timeline and measure the step at which users perceive waiting, failure, or a need to repeat work.

Measure	Question it answers	Limit
P50 / P95 / P99	How is waiting time distributed?	Does not measure completion by itself.
Confirmation rate	Did the task finish?	Depends on complete events.
Subsequent contacts	Did the person remain uncertain?	There is a delay and there are other causes.
Cost per journey	Which resource was consumed?	Requires consistent attribution.

The comparison should use similar cohorts, sufficient volume, and a window that includes relevant peaks. Cache optimization can reduce latency while worsening freshness; parallelism can improve time while increasing saturation; retries can increase completion while amplifying load. Every gain must be considered alongside its guardrail metrics.

Decision criterion

I would expand an optimization when the task benefit is supported and correctness, data freshness, and operating cost remain within the agreed limits.

ENGINEERING ECONOMICS

Risk: measure both types of error.

A fraud-prevention decision has asymmetric effects. Allowing an improper action can generate a loss; blocking a legitimate action can prevent a real need from being met and create recovery costs. The discussion cannot treat every reduction in approval as a security improvement, or every reduction in friction as a product improvement.

Dimension	Proposed measure	Considerations
Protection	Loss in the eligible population	Delayed labels and underreporting.
Friction	Legitimate actions blocked / legitimate actions evaluated	Ground truth and review.
Recovery	Time to a safe resolution	Do not exclude silent abandonment.
Operational fairness	Performance by relevant cohort	Sample and data access.

The model needs a challenge pathway and human review proportionate to the risk. Review outcomes feed learning, but may also contain selection bias: people who give up do not appear as resolved cases. I would seek to measure that absence and discuss decision criteria with risk and product specialists.

My fraud-prevention experience makes this balance central to my candidacy. In the ML and access-recovery cases, the book separates signal, policy, effect, and review. The objective is to protect the person and the institution through decisions that can be explained and corrected.

LABORATORY GUIDE

How to run the laboratories.

The companion package contains one independent laboratory per case. They use Python's standard library and synthetic data, with no credentials, calls to Nubank, or account access. The purpose is to make rules and failures examinable on a local machine. Every file includes its own tests and can be run independently.

BASH / ORIGINAL PROPOSAL

```bash
# Dentro da pasta laboratorios do pacote extraído:
python3 case_01.py
python3 case_17.py

# Todos os casos em uma única execução:
python3 -m unittest discover -p "case_*.py" -v

# Núcleo Clojure, com Clojure 1.12.0 instalado:
clojure -M decision_core.clj
```

Local commands; they do not access banking services. Consult LEIA-ME.html for versions and scope.

The Clojure laboratory was executed with Java 17 and Clojure 1.12.0. The package provides the original code, the reproduction command, and the execution record. Flutter/Dart, SQL, YAML, and cloud integration examples are identified as references where they were not executed. The book does not present absent compilation as validation.

What to test next

Integration with real adapters, concurrency across processes, durable persistence, load, network failures, authorization, and operations belong to a subsequent stage in an appropriate environment.

LABORATORY GUIDE

From local proof to an integrated pilot.

A passing unit test is evidence of a property in the context where it was executed. Moving to a pilot requires new questions. Does the adapter preserve the core's semantics? Does persistence perform the version comparison and write atomically? Does a retry arrive with the same identity? Does the consumer remain correct when the process restarts?

Level	Validation scenario	Evidence to retain
Contract	An old consumer receives a new event	Fixture, version, and result.
Integration	Failure after commit and before ACK	Final state and recovery trail.
Load	A peak with a slow dependency	Latency, saturation, and losses.
Operations	Rollback with customers in progress	Procedure and reconciliation.

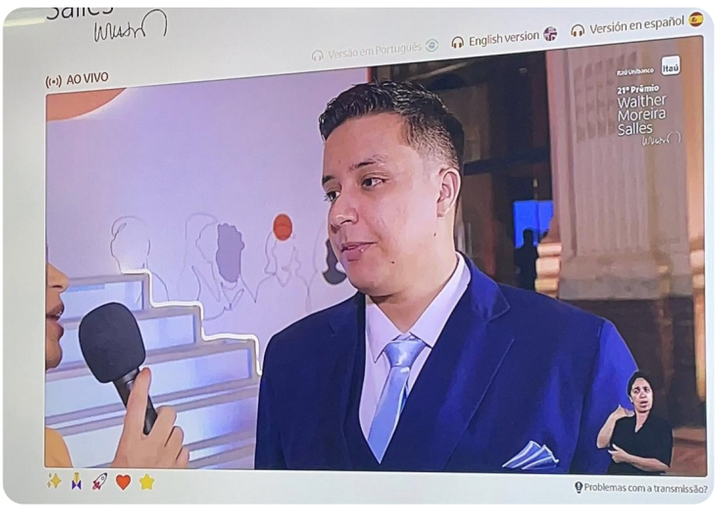
I would maintain a matrix connecting risk, testing, and the release decision. We do not need to test every imaginable combination before learning something; we need to cover the properties that prevent harm within the chosen exposure scope. The pilot should be smaller when the evidence is weaker or rollback is more difficult.

Integrity criterion

No local result in this book is extrapolated to Nubank's actual availability, security, or performance. The proposals become stronger when their limits are known and can guide the next validation.

CLOSING

A proposal for contribution, in my own words.



Public record of the candidate being interviewed during a recognition event, published in his portfolio.

I want to work on problems that require depth and collaboration. I enjoy getting down to the contract and the code, while also building context so that other people can make decisions. My experience in fraud, credit, and technical product work has taught me that quality appears in exceptions as much as in the happy path.

At Nubank, I want to contribute to teams that turn financial complexity into understandable experiences. As an EM, my focus will be developing people and delivery capacity. As a Staff engineer, it will be improving technical decisions that cross domains. In both roles, I want to take responsibility, learn from the existing context, and make the outcome verifiable.

Let’s discuss the real problem.

Wesley Vieira · LinkedIn: /in/techvieira · Portfolio: wesleyvieira99.github.io/WVPortfolio · GitHub: wesleyvieira99

TRACEABLE BIBLIOGRAPHY

Sources for cases 01-05

01 · When Pix fails, help fails too

An individual public report in a community hosted by Nubank. · 15/07/2026 NuCommunity - Is Pix down again?

The record establishes that the complaint was published. It does not establish a general outage, its duration, a financial loss or a technical cause. The issue may have been resolved since then.

02 · Recovering access when the front camera does not work

An individual public report, based on the available excerpt. · undated NuCommunity - Unable to access my account; facial recognition required

The excerpt does not establish which alternatives support offered, the outcome or current behavior. It does not demonstrate a failure of the biometric mechanism.

03 · A notification should help someone decide

A participant's report and suggestions, with screenshots they published. · undated NuCommunity - Nubank app push notifications omit the amount (suggestions)

The images document the author's claim, without establishing privacy settings, app version or current behavior. Comparisons with competitors were not audited.

04 · Open Finance must show what is still incomplete

An individual public report distinguishing visible data from missing data. · 03/10/2026 NuCommunity - Open Finance: Mercado Pago Cofrinhos do not appear in Nubank

The author does not identify which institution has the limitation. The source does not establish a coverage obligation, an integration defect or loss of money.

05 · The next due date, without searching across months

A detailed public suggestion, with screenshots supplied by the participant. · undated NuCommunity - Nubank's payment assistant needs improvements

The record describes the author's experience at that time. It does not establish the coverage of current features or demonstrate that reminders caused late payments.

01 · NuCommunity - Is Pix down again?
02 · NuCommunity - Unable to access my account; facial recognition required
03 · NuCommunity - Nubank app push notifications omit the amount (suggestions)
04 · NuCommunity - Open Finance: Mercado Pago Cofrinhos do not appear in Nubank
05 · NuCommunity - Nubank's payment assistant needs improvements

TRACEABLE BIBLIOGRAPHY

Sources for cases 06-10

06 · Modo Rua: explain permissions without weakening protection

A public report and suggestion with two screenshots supplied by the author. · undated NuCommunity - Modo Rua needs a small adjustment

The author's interpretation of GPS and battery behavior was not validated. There is no platform diagnosis, confirmed defect or evidence of a vulnerability.

07 · One cent can break an entire integration

An individual public report with a textual example of the expected and received value. · undated NuCommunity - OFX statement error

The original file was not obtained and the error was not reproduced. The source does not establish whether the defect was in the exporter, the importer or a particular version.

08 · The digital experience also ends with card delivery

A public app-store review accompanied by a developer response. · 08/04/2026 Google Play - Murilo Meyrelles review about card delivery

The source does not state the outcome, the carrier's delivery attempts or the cause. It cannot establish frequency or responsibility for each stage.

09 · Evolve the statement without losing the primary task

An individual public app-store review. · 24/02/2026 Google Play - Fernando Cordeiro review about viewing the statement

There is no independent reproduction, identified version or confirmation of current behavior. One opinion about a layout does not represent the preferences of the customer base.

10 · A purchase record in context, with the right meaning

An individual public app-store review. · 12/02/2025 Google Play - Neidslem Fernandes da Costa review about purchase records

The exact type of document requested and current coverage are not confirmed. A purchase record, payment confirmation and merchant receipt serve different purposes.

06 · NuCommunity - Modo Rua needs a small adjustment

07 · NuCommunity - OFX statement error

08 · Google Play - Murilo Meyrelles review about card delivery

TRACEABLE BIBLIOGRAPHY

Sources for cases 11-15

11 · Fast payments without trading correctness for speed

An official technical account with a published architecture and result. · 2026-01-12 How we reduced critical path latency by 76% at Nubank

The result concerns that flow and period. It does not represent latency across the whole app or establish current performance.

12 · Cloud capacity as a product commitment

An official engineering publication about capacity and isolation. · 2025-04-09 Managing Cloud Limits

The scale stated in April 2025 is historical, not a current count. The publication does not provide a rate of incidents prevented.

13 · Ownership that survives reorganization

An official account of operations, governance and automation. · 2026-07-01 How Nubank distributes infrastructure ownership to operate more than 21 thousand databases with a lean team

Five engineers do not represent the entire organizational effort. Data removal depends on retention, dependencies and business context.

14 · BDC: speed with an experience contract

An official description of the platform and adoption at publication time. · 2023-01-20 Backend Driven Content: why we developed a Server Driven UI framework at Nubank

The percentages are historical. BDC cannot invent components absent from the binary or eliminate every dependency on app updates.

15 · Mobile releases guided by the actual customer journey

An official recap of a meetup on platform engineering and mobile development. · 2025-04-16 Platform engineering, Observability, and scaling Mobile Development

This is a public account of practices, without access to the implementation or current coverage of every journey.

11 · How we reduced critical path latency by 76% at Nubank
12 · Managing Cloud Limits
13 · How Nubank distributes infrastructure ownership to operate more than 21 thousand databases with a lean team
14 · Backend Driven Content: why we developed a Server Driven UI framework at Nubank
15 · Platform engineering, Observability, and scaling Mobile Development

TRACEABLE BIBLIOGRAPHY

Sources for cases 16-20

16 · Precog: reaching the right specialist

An official technical account of a model, application and result. · 2023-06-21 Presenting Precog, Nubank's Real Time Event AI

The increase is relative to the number of correct routings in that comparison. It does not mean 50 percentage points, guaranteed resolution or a current effect.

17 · Fraud prevention that responds within the journey

An engineering practice published by Nubank itself. · 2025-04-07 Practices to scale Machine Learning operations

It does not disclose decision policies, thresholds, losses or false positives. It does not support an inference of an exploitable vulnerability or current fraud-prevention effectiveness.

18 · The same feature in training and at decision time

An official technical guide based on the team's experience. · 2023-06-27 Dealing with Train-serve Skew in Real-time ML Models: A Short Guide

The illustrative examples are not incidents attributable to a specific product or customer. No magnitude of loss is published.

19 · Dark Mode: consistency that includes readability

An official account of the app and design system's development. · 2023-09-26 The birth of the Dark Mode: a journey into Nubank's app evolution

Dark mode alone does not solve accessibility. The account does not establish a contrast failure in the current app version.

20 · NuBraille: end-to-end autonomy

An official inclusive-product case, with an identified process and participants. · 2023-07-18 Bridging the gap: Nubank's commitment in serving the visually impaired

This is evidence of a specific initiative, not certification of accessibility across the entire experience or a guarantee of universal availability.

16 · Presenting Precog, Nubank's Real Time Event AI

17 · Practices to scale Machine Learning operations

18 · Dealing with Train-serve Skew in Real-time ML Models: A Short Guide

19 · The birth of the Dark Mode: a journey into Nubank's app evolution

20 · Bridging the gap: Nubank's commitment in serving the visually impaired

VISUAL PROVENANCE

Image and brand credits

Logos and photographs were obtained from the official project pages, Nubank publications or the candidate’s public portfolio. Trademarks identify the technologies discussed. Editorial use does not imply an affiliation or endorsement. The package includes a provenance manifest with file URLs, source pages and asset hashes. Source screenshots are kept in their original language to preserve the evidence.

Asset	Source	Use
nubank	www.datomic.com	Technology identification
clojure	clojure.org	Technology identification
datomic	www.datomic.com	Technology identification
kafka	kafka.apache.org	Technology identification
prometheus	prometheus.io	Technology identification
flutter	flutter.dev	Technology identification
dart	dart.dev	Technology identification
python	www.python.org	Technology identification
kubernetes	github.com	Technology identification

nubank · source
clojure · source
kafka · source
prometheus · source
flutter · source
dart · source
python · source
kubernetes · source

VISUAL PROVENANCE

Image and brand credits · continued

Logos and photographs were obtained from the official project pages, Nubank publications or the candidate's public portfolio. Trademarks identify the technologies discussed. Editorial use does not imply an affiliation or endorsement. The package includes a provenance manifest with file URLs, source pages and asset hashes. Source screenshots are kept in their original language to preserve the evidence.

Asset	Source	Use
opentelemetry	opentelemetry.io	Technology identification
postgresql	www.postgresql.org	Technology identification
grafana	grafana.com	Technology identification
java	dev.java	Technology identification
portrait	wesleyvieira99.github.io	Editorial / career photograph
award-0	wesleyvieira99.github.io	Editorial / career photograph
award-1	wesleyvieira99.github.io	Editorial / career photograph
award-2	wesleyvieira99.github.io	Editorial / career photograph
aws	aws.amazon.com	Technology identification

opentelemetry · source
postgresql · source
grafana · source
java · source
portrait · source
aws · source

VISUAL PROVENANCE

Image and brand credits · continued

Logos and photographs were obtained from the official project pages, Nubank publications or the candidate's public portfolio. Trademarks identify the technologies discussed. Editorial use does not imply an affiliation or endorsement. The package includes a provenance manifest with file URLs, source pages and asset hashes. Source screenshots are kept in their original language to preserve the evidence.

Asset	Source	Use
redis	redis.io	Technology identification
typescript	www.typescriptlang.org	Technology identification
nu-app-life	international.nubank.com.br	Editorial / career photograph
nu-engineer	international.nubank.com.br	Editorial / career photograph
nu-meeting	international.nubank.com.br	Editorial / career photograph
nu-office	international.nubank.com.br	Editorial / career photograph
nu-collaboration	international.nubank.com.br	Editorial / career photograph
terraform	github.com	Technology identification

redis · source
typescript · source
nu-app-life · source
terraform · source